



US 20250280124A1

(19) **United States**

(12) **Patent Application Publication**
Kato et al.

(10) **Pub. No.: US 2025/0280124 A1**

(43) **Pub. Date: Sep. 4, 2025**

(54) **ENCODER, DECODER, ENCODING METHOD, AND DECODING METHOD**

H04N 19/46 (2014.01)

H04N 19/91 (2014.01)

(71) Applicant: **Panasonic Intellectual Property Corporation of America**, Torrance, CA (US)

(52) **U.S. Cl.**
CPC *H04N 19/13* (2014.11); *H04N 19/105* (2014.11); *H04N 19/176* (2014.11); *H04N 19/46* (2014.11); *H04N 19/91* (2014.11)

(72) Inventors: **Yusuke Kato**, Osaka (JP); **Kiyofumi Abe**, Osaka (JP); **Tadamasa Toma**, Osaka (JP); **Takahiro Nishi**, Nara (JP)

(57) **ABSTRACT**

(21) Appl. No.: **19/213,401**

(22) Filed: **May 20, 2025**

Related U.S. Application Data

(63) Continuation of application No. 18/441,617, filed on Feb. 14, 2024, now Pat. No. 12,341,964, which is a continuation of application No. 17/691,623, filed on Mar. 10, 2022, now Pat. No. 12,069,266, which is a continuation of application No. PCT/JP2020/026138, filed on Jul. 3, 2020.

(60) Provisional application No. 62/905,760, filed on Sep. 25, 2019.

Publication Classification

(51) **Int. Cl.**
H04N 19/13 (2014.01)
H04N 19/105 (2014.01)
H04N 19/176 (2014.01)

An encoder includes circuitry and memory coupled to the circuitry. In residual coding of a current block to which an orthogonal transform is not applied, when coefficient information flags relating to a coefficient in the current block are encoded, the circuitry: performs a level mapping process to transform the coefficient to a second coefficient by using a predicted value, in which the predicted value is determined based on neighboring coefficients of the coefficient within the current block; encodes second coefficient information flags by Context-based Adaptive Binary Arithmetic Coding (CABAC), each of the second coefficient information flags relating to the second coefficient; and encodes a remainder value of the coefficient with Golomb-Rice code, and when the coefficient information flags are not encoded, in operation, the circuitry: skips the level mapping process; and encodes a value of the coefficient with the Golomb-Rice code.

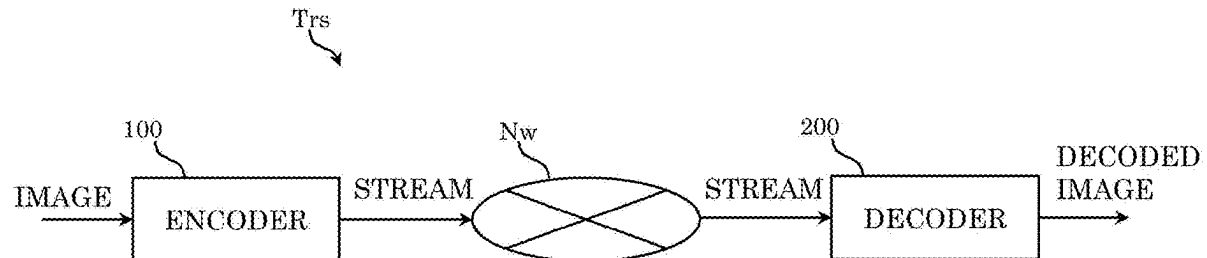


FIG. 1

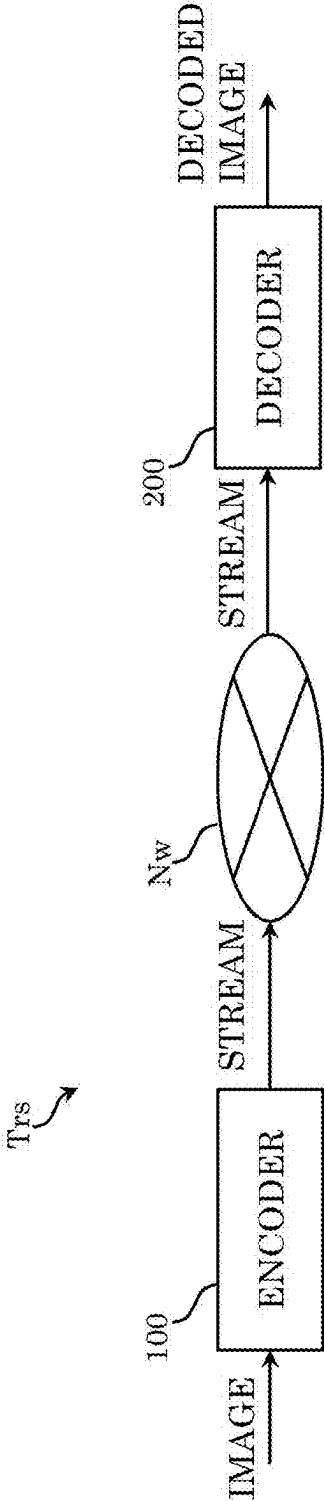


FIG. 2

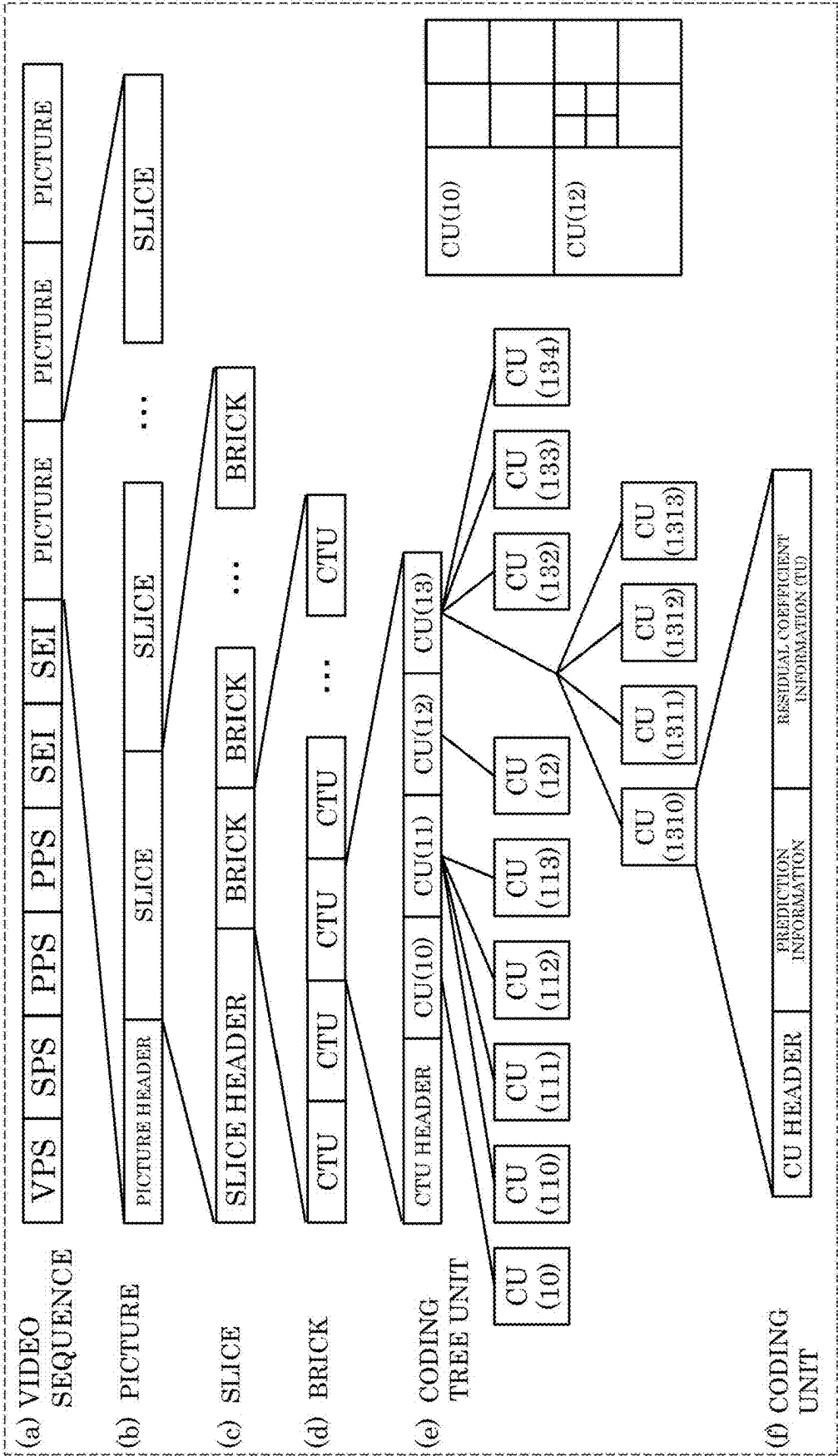


FIG. 3

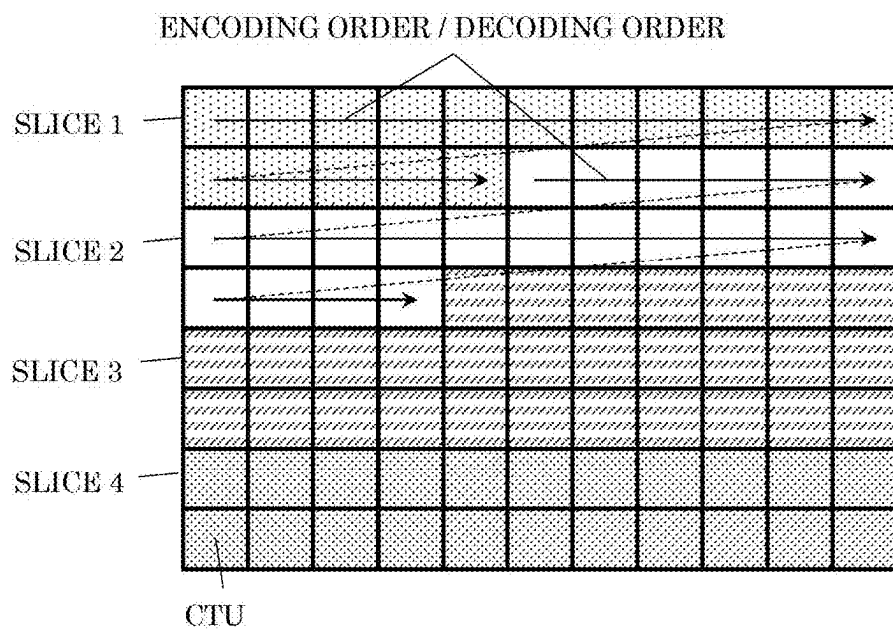
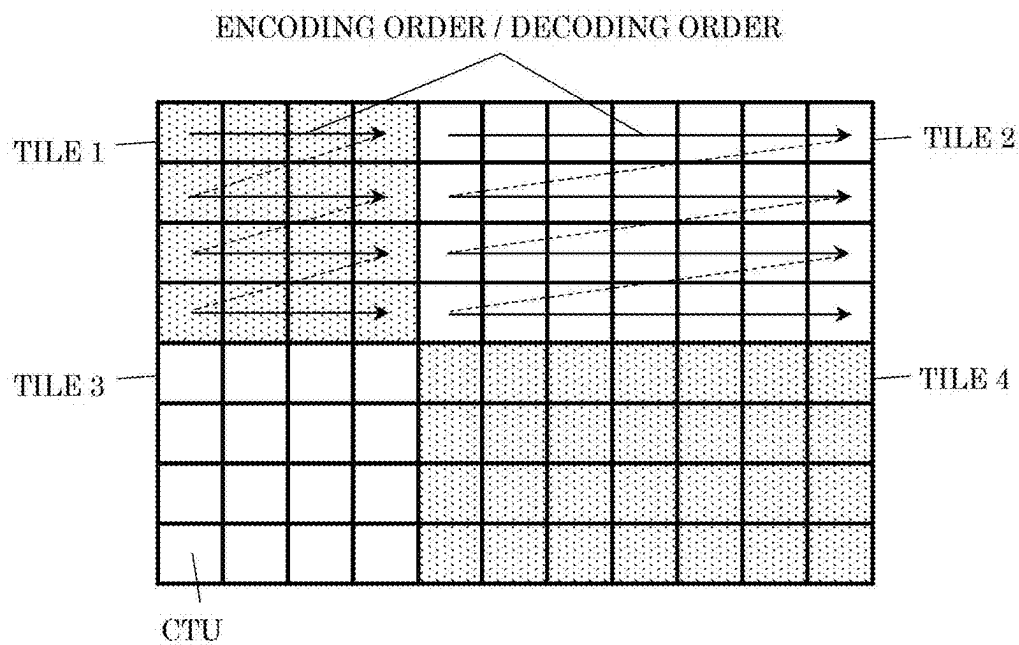


FIG. 4



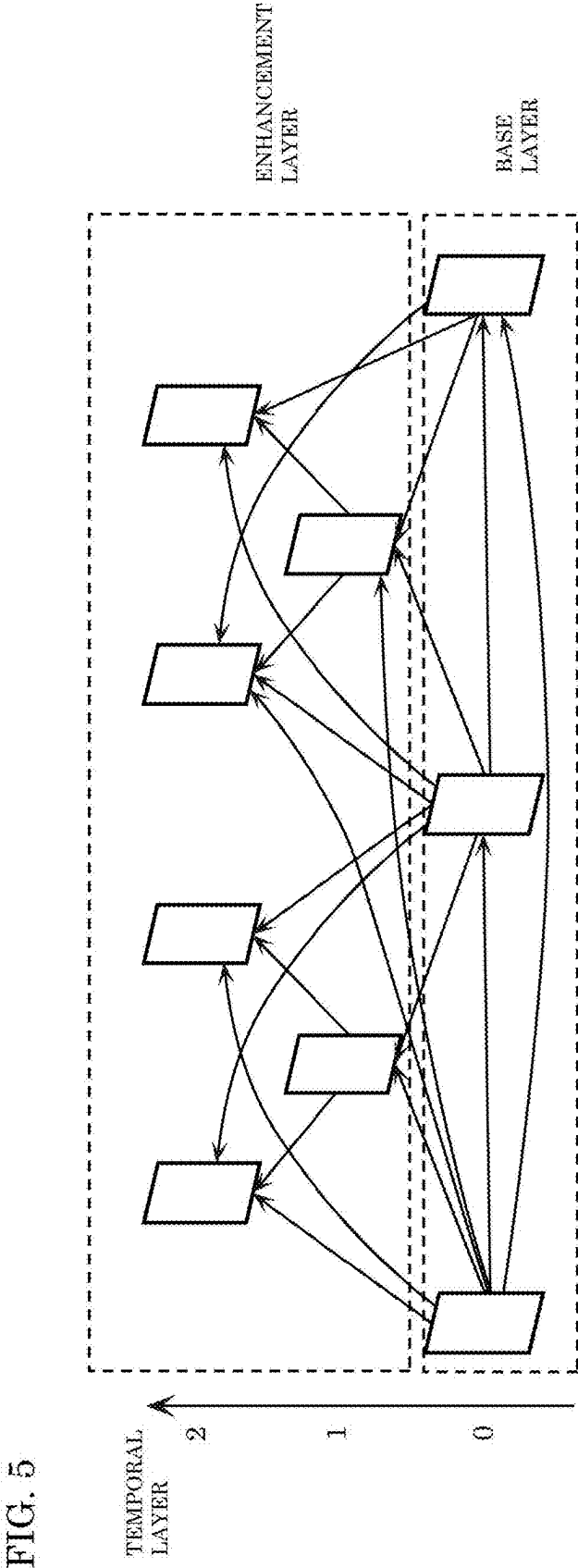


FIG. 6

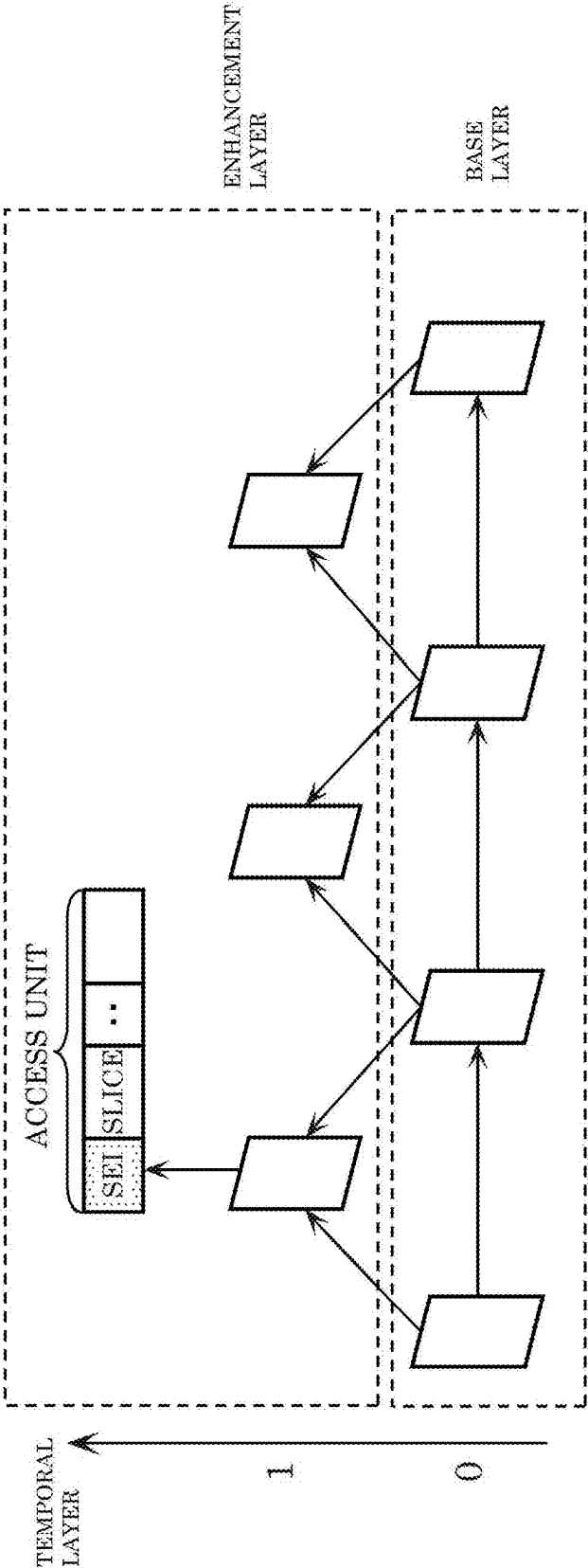


FIG. 8

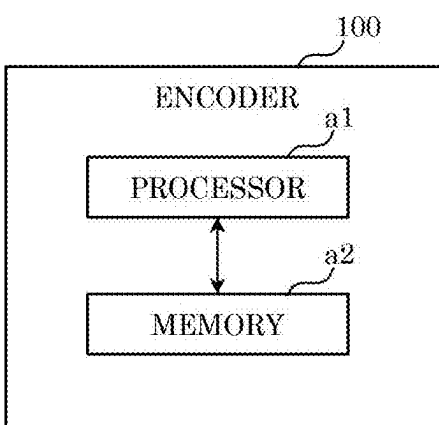


FIG. 9

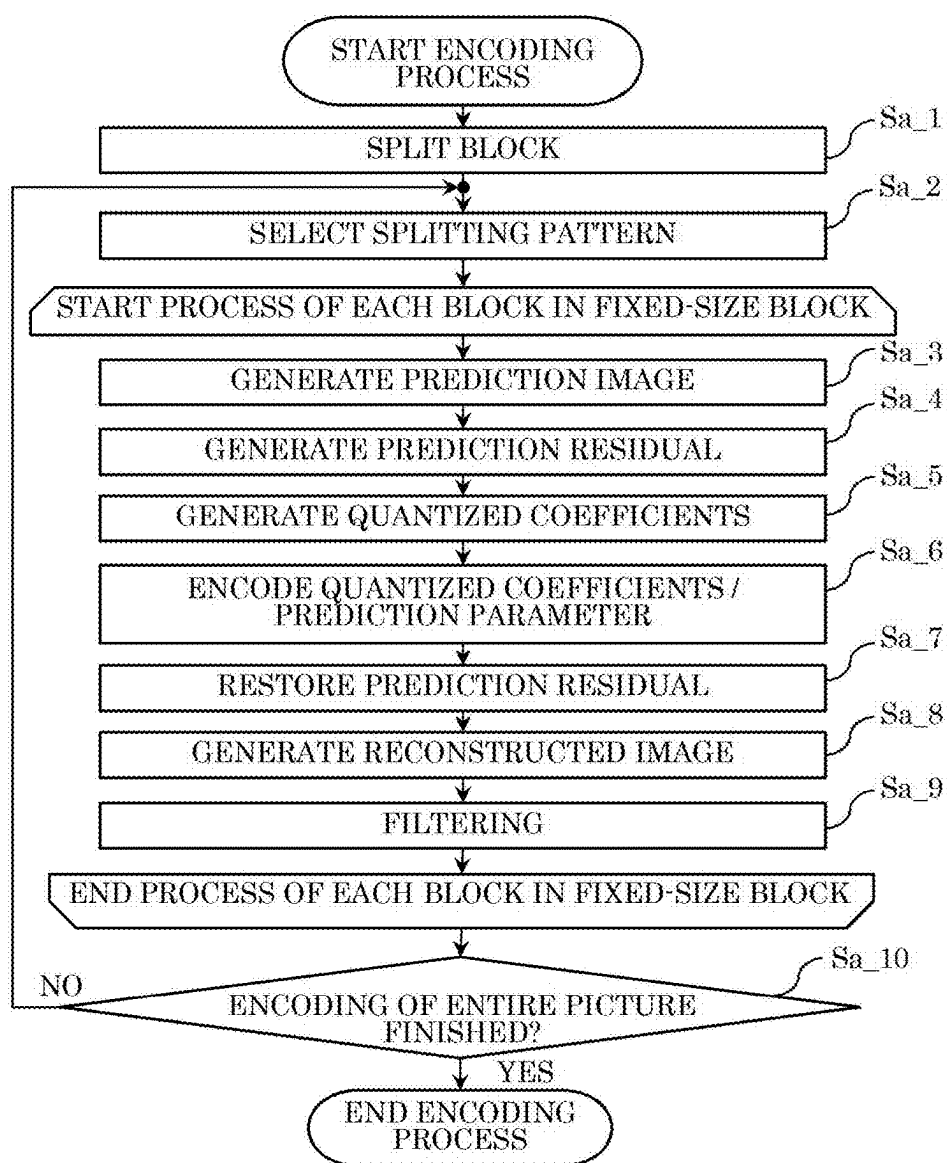


FIG. 10

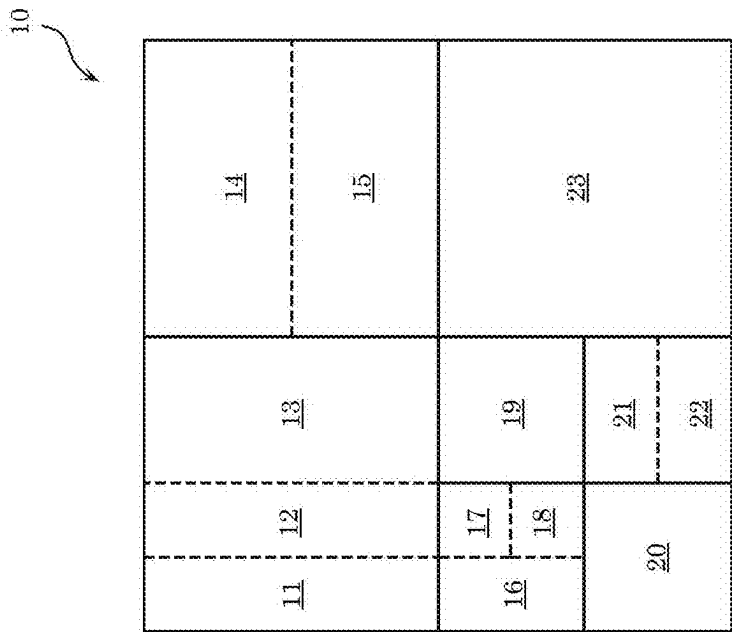


FIG. 11

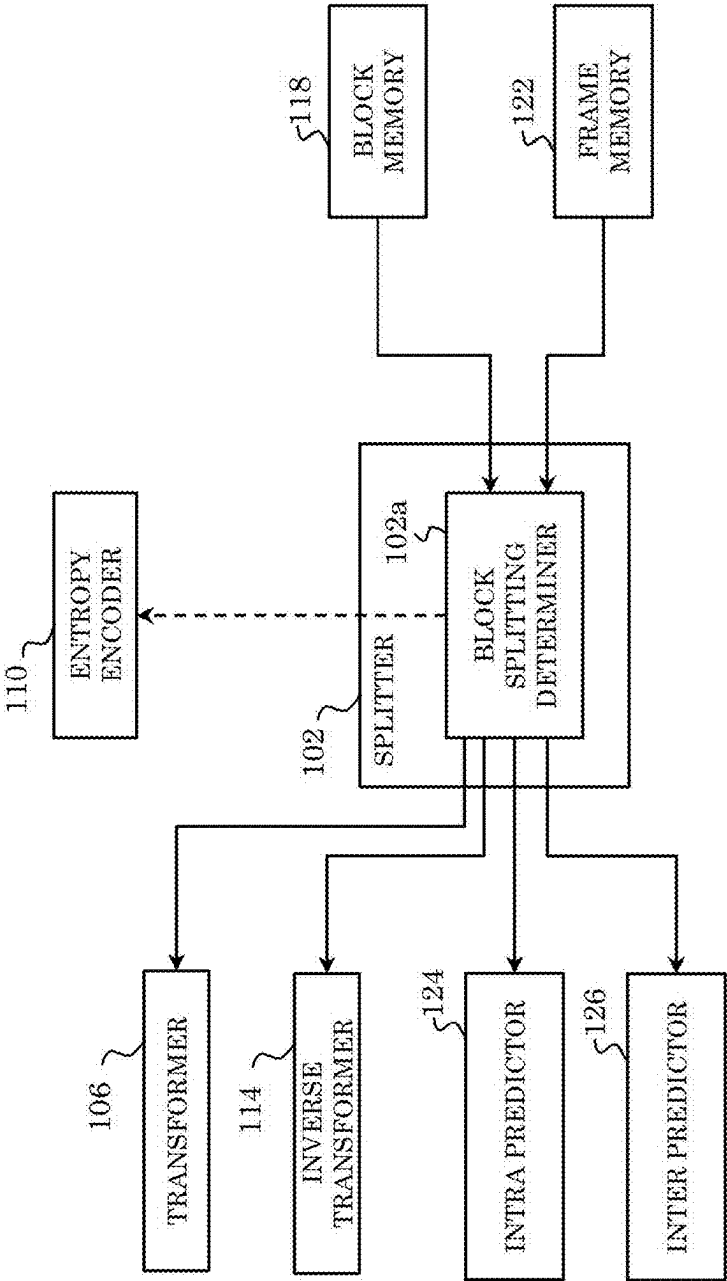


FIG. 12

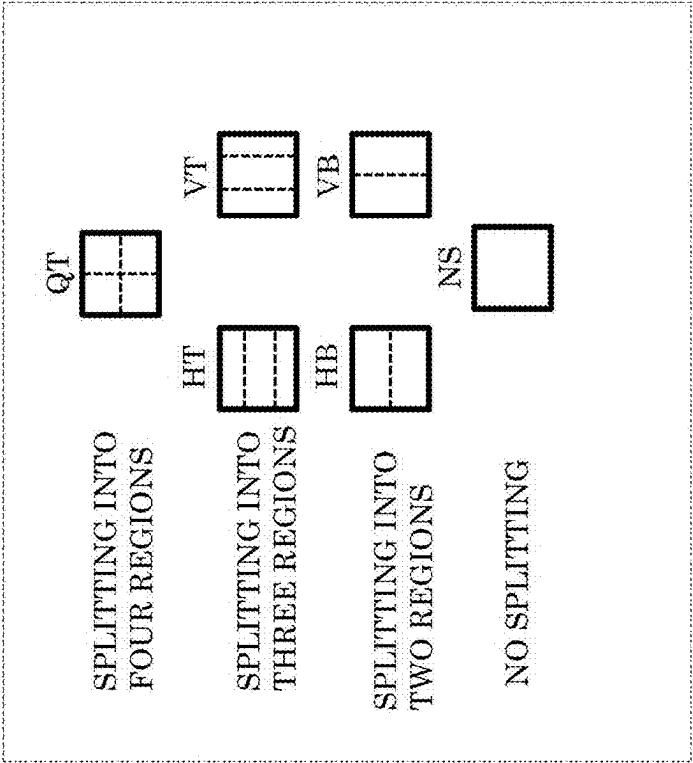


FIG. 13A

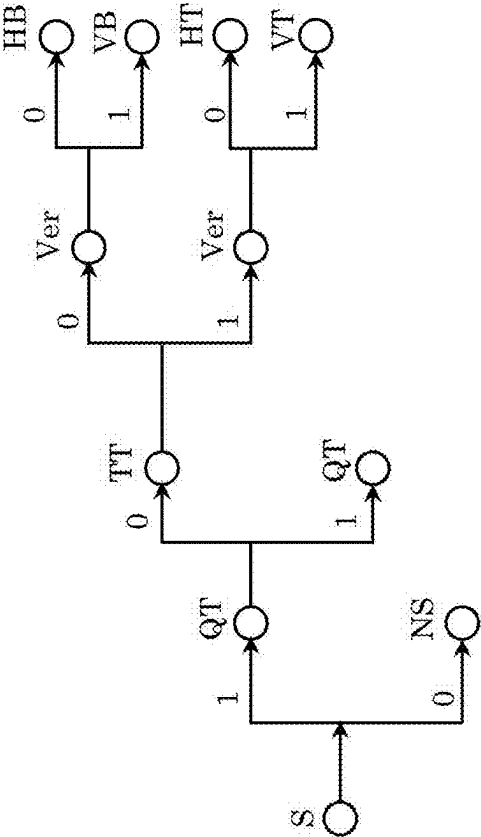


FIG. 13B

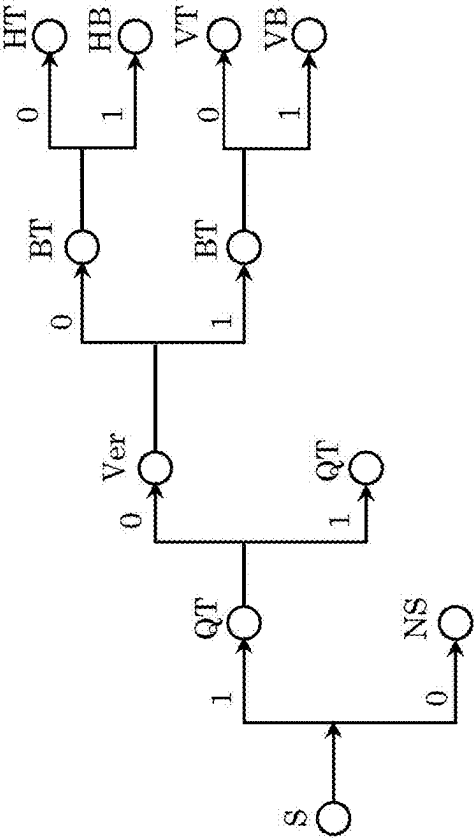


FIG. 14

TRANSFORM TYPE	BASIS FUNCTION $T_i(j)$, $i, j = 0, 1, \dots, N-1$
DCT - II	$T_i(j) = \omega_0 \cdot \sqrt{\frac{2}{N}} \cdot \cos\left(\frac{\pi \cdot i \cdot (2j+1)}{2N}\right)$ $\text{WHERE } \omega_0 = \begin{cases} \sqrt{\frac{2}{N}} & i=0 \\ 1 & i \neq 0 \end{cases}$
DCT - V	$T_i(j) = \omega_0 \cdot \omega_1 \cdot \sqrt{\frac{2}{2N+1}} \cdot \cos\left(\frac{2\pi \cdot i \cdot j}{2N+1}\right)$ $\text{WHERE } \omega_0 = \begin{cases} \sqrt{\frac{2}{N}} & i=0 \\ 1 & i \neq 0 \end{cases}, \omega_1 = \begin{cases} \sqrt{\frac{2}{N}} & j=0 \\ 1 & j \neq 0 \end{cases}$
DCT - VIII	$T_i(j) = \sqrt{\frac{4}{2N+1}} \cdot \cos\left(\frac{\pi \cdot (2i+1) \cdot (2j+1)}{4N+2}\right)$
DST - I	$T_i(j) = \sqrt{\frac{2}{N+1}} \cdot \sin\left(\frac{\pi \cdot (i+1) \cdot (j+1)}{N+1}\right)$
DST - VII	$T_i(j) = \sqrt{\frac{4}{2N+1}} \cdot \sin\left(\frac{\pi \cdot (2i+1) \cdot (2j+1)}{2N+1}\right)$

FIG. 15

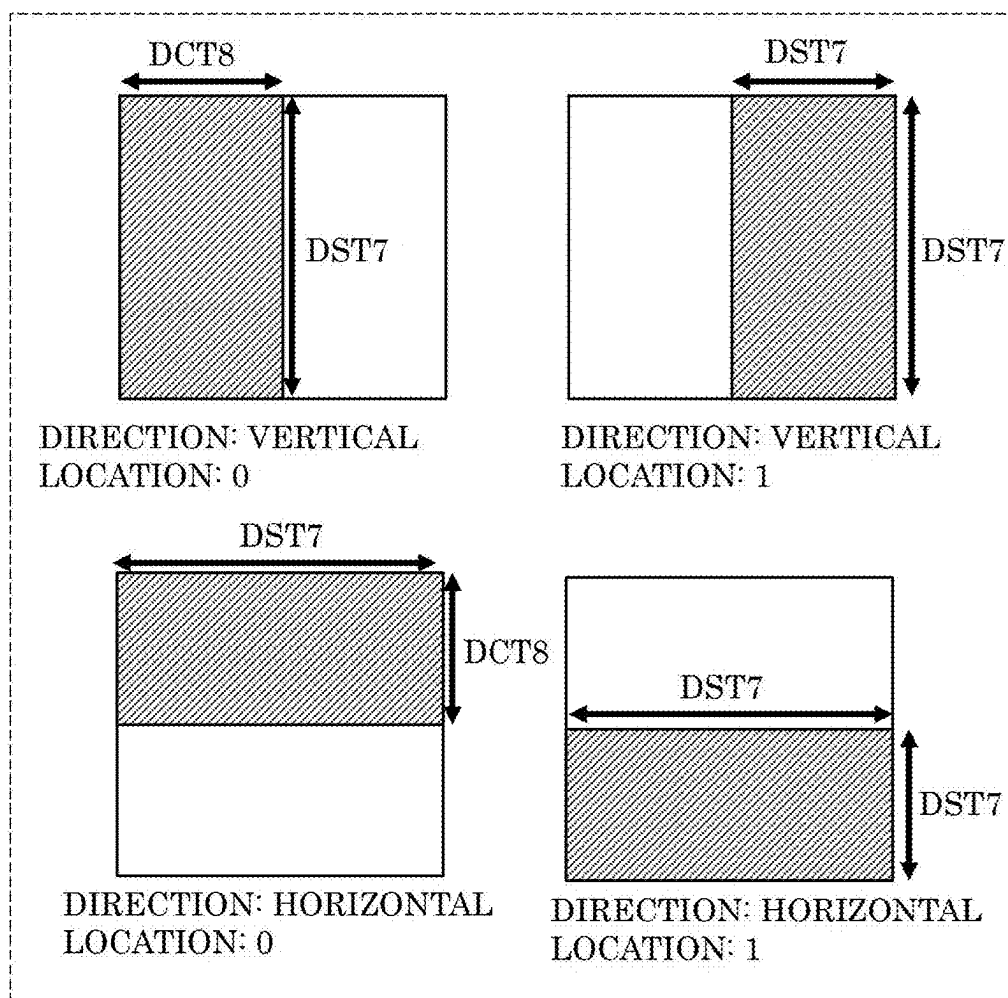


FIG. 16

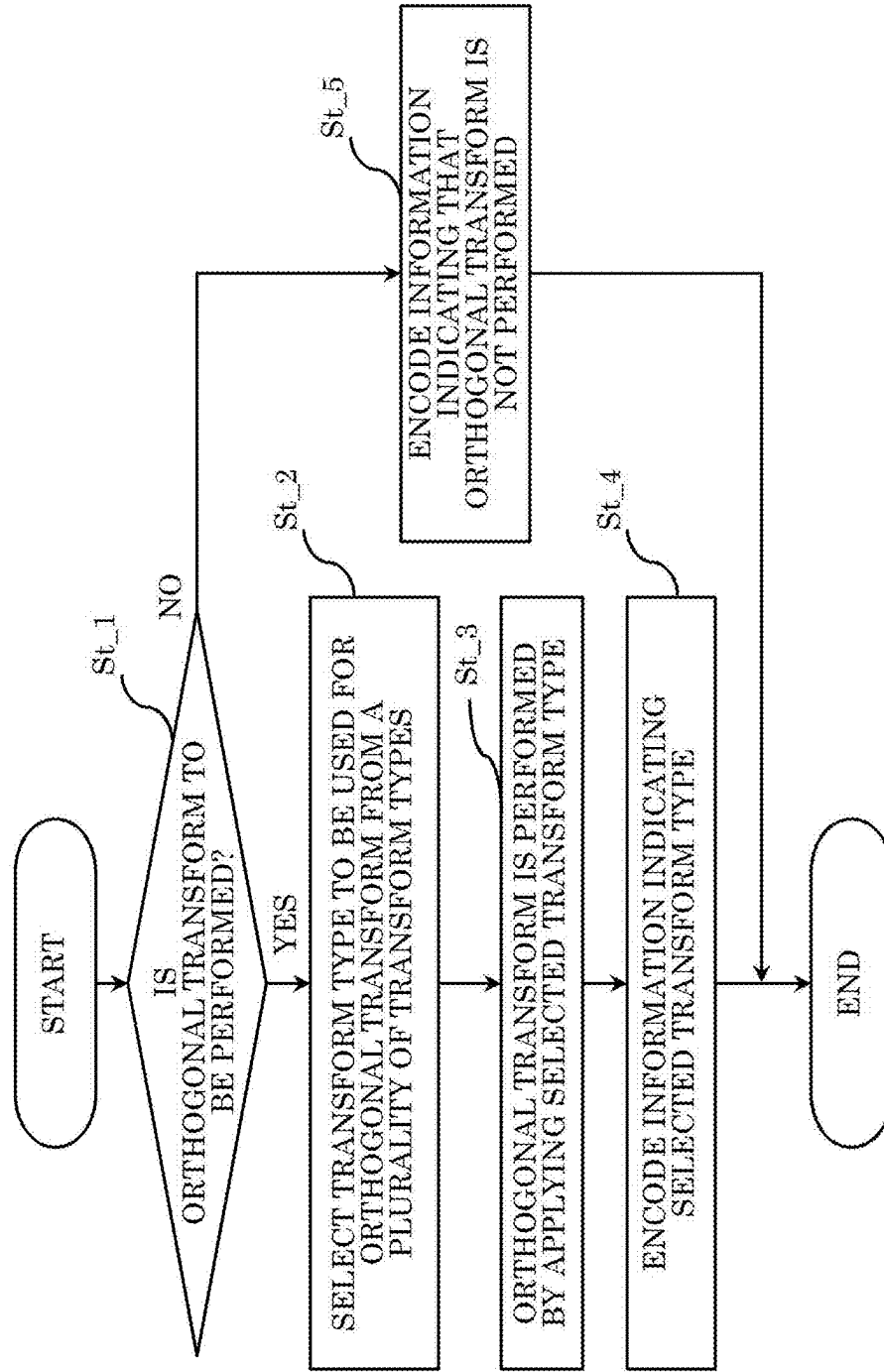


FIG. 17

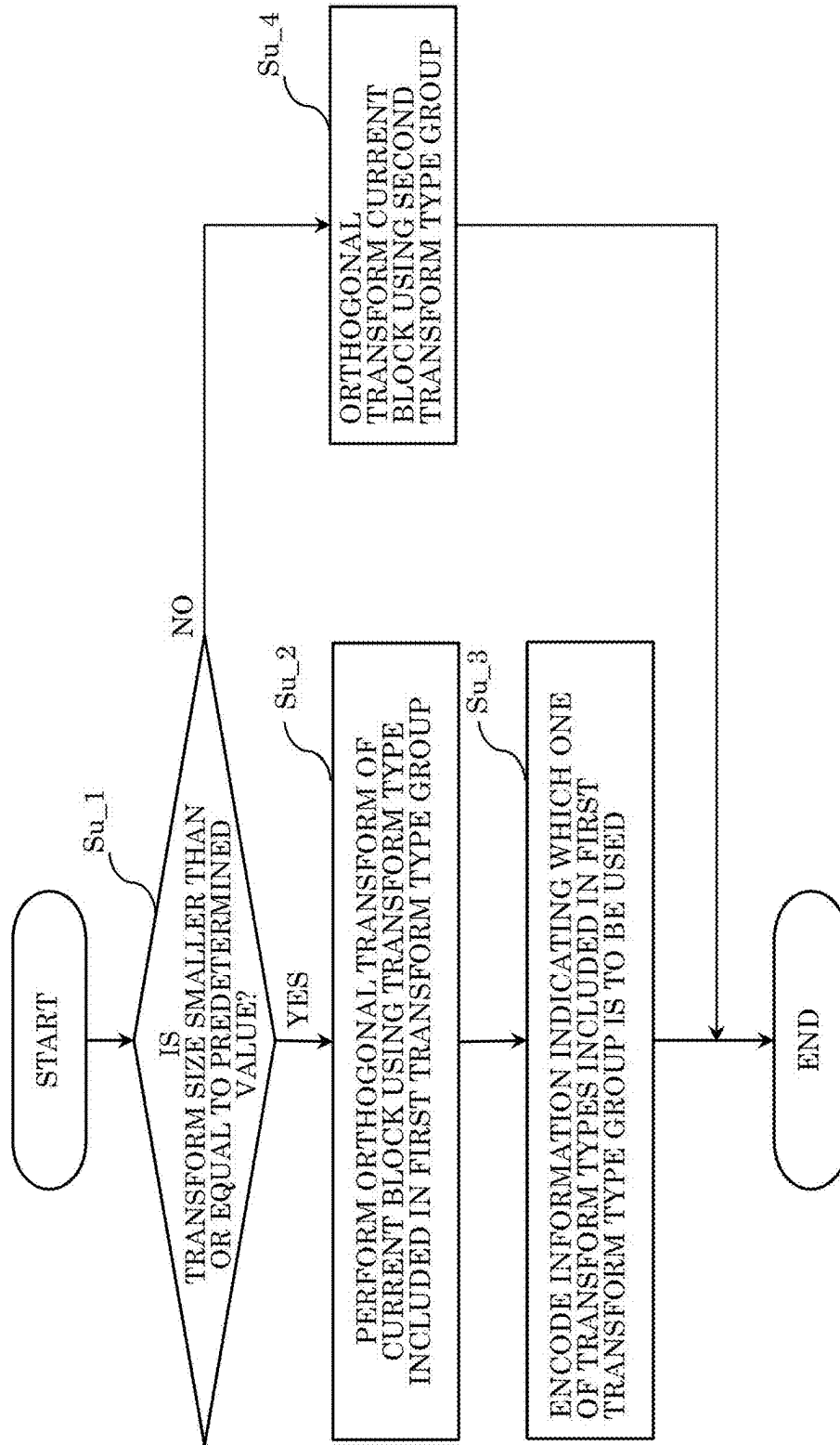


FIG. 18

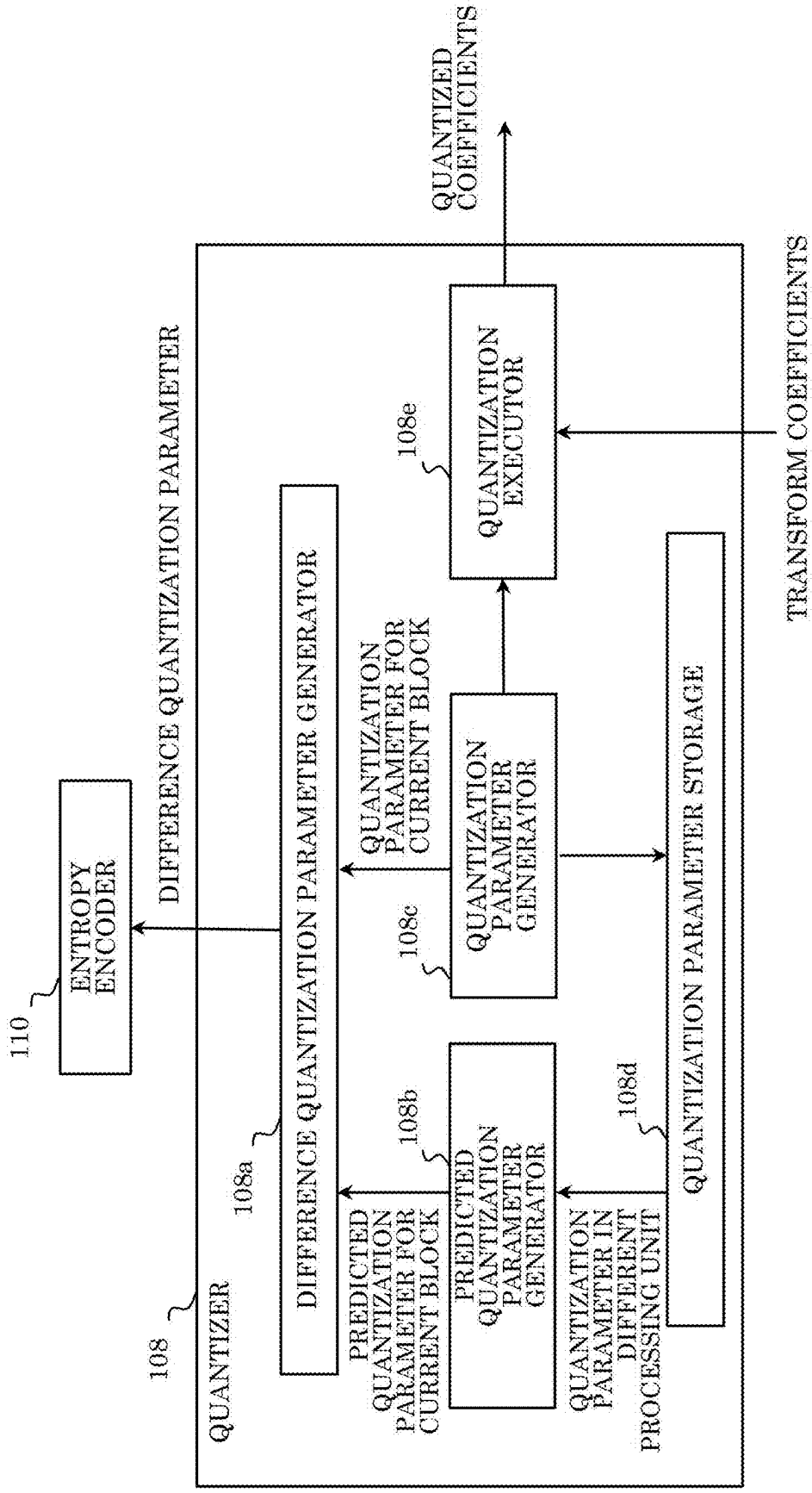


FIG. 19

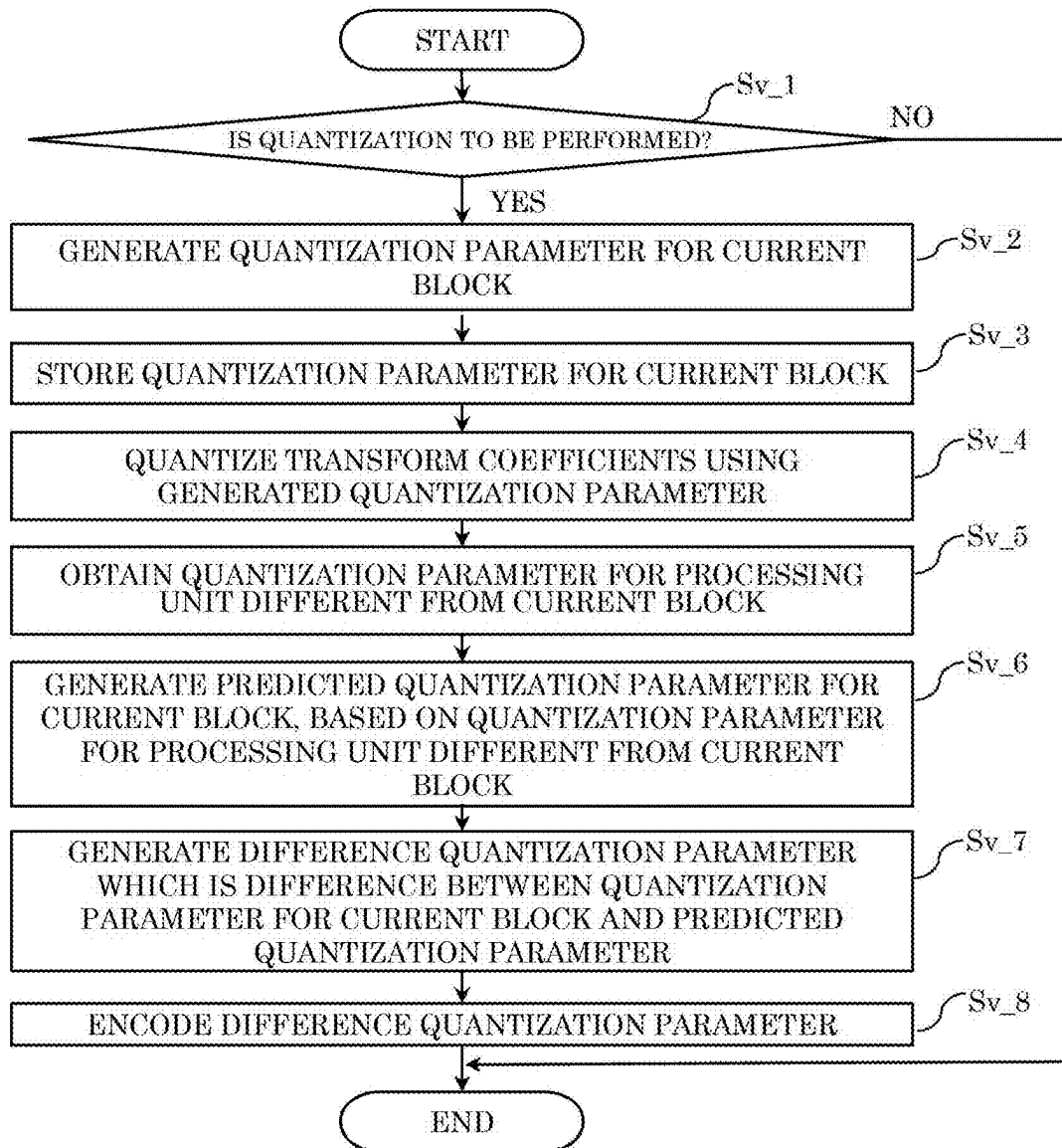


FIG. 20

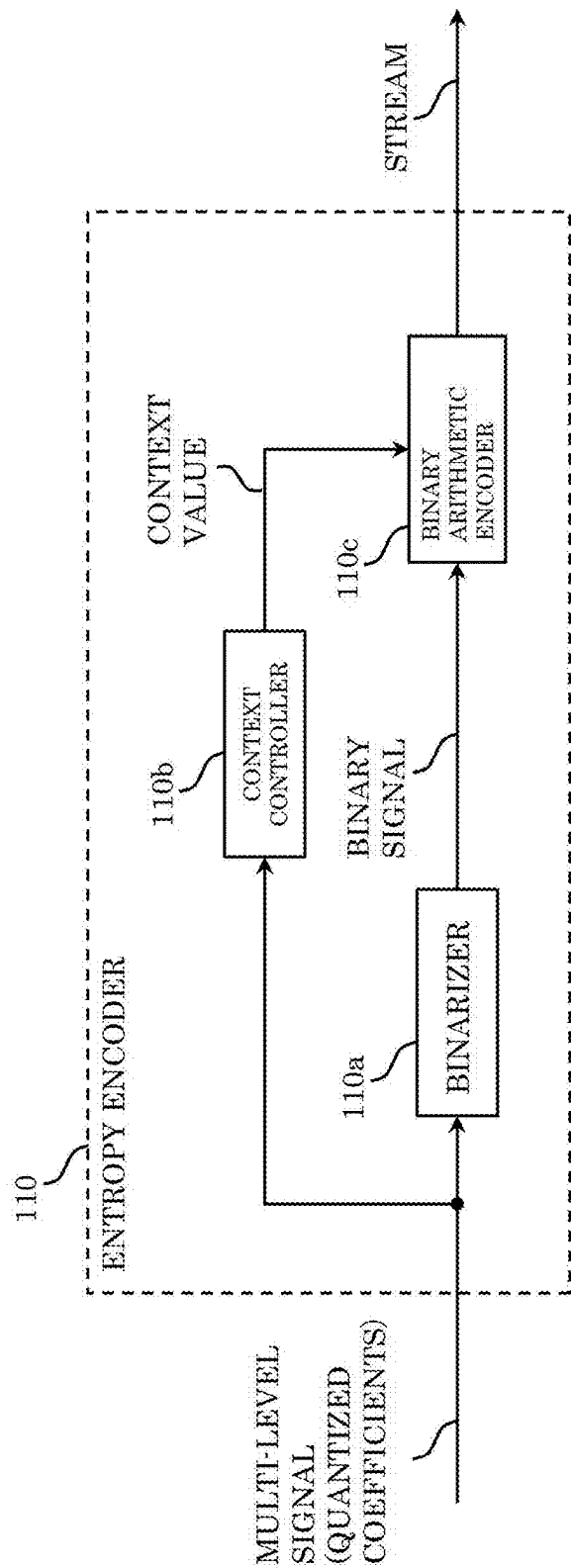


FIG. 21

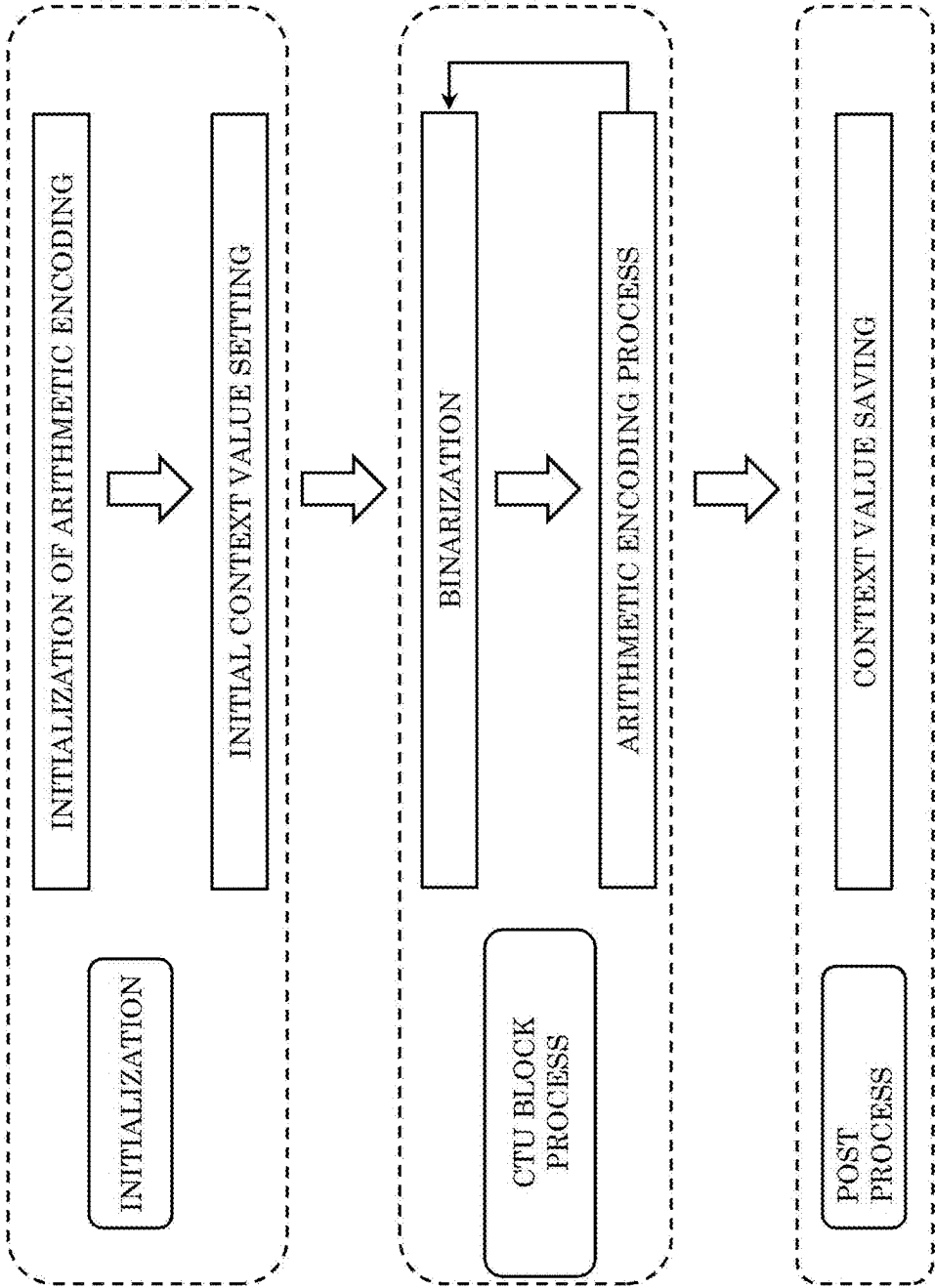


FIG. 22

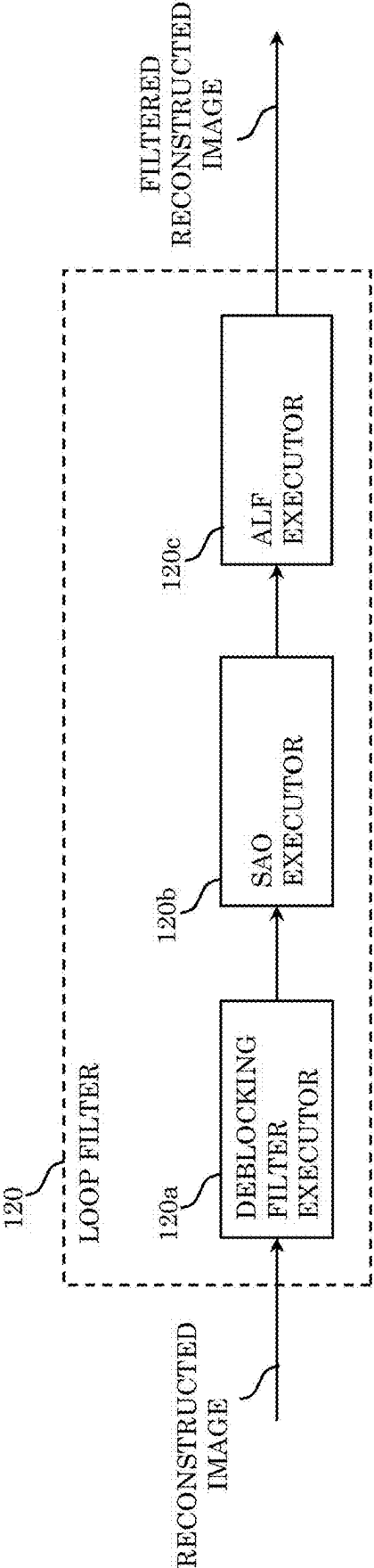


FIG. 23A

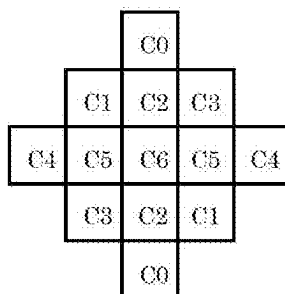


FIG. 23B

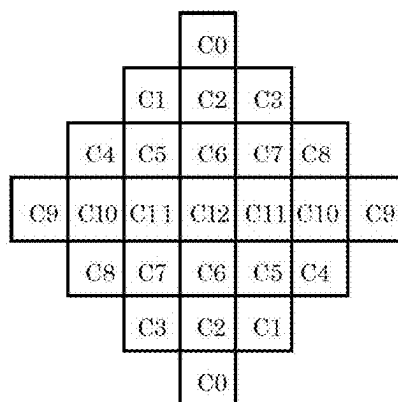


FIG. 23C

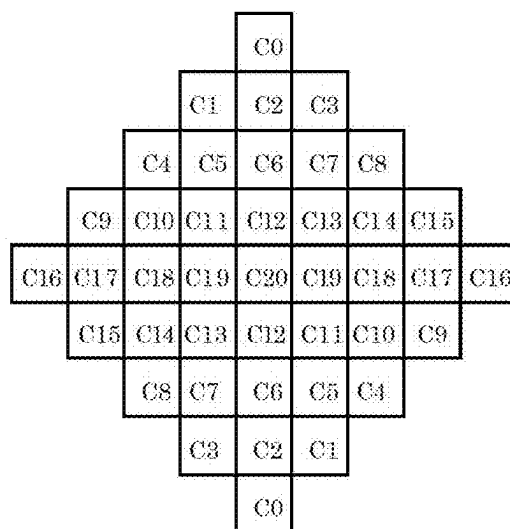


FIG. 23D

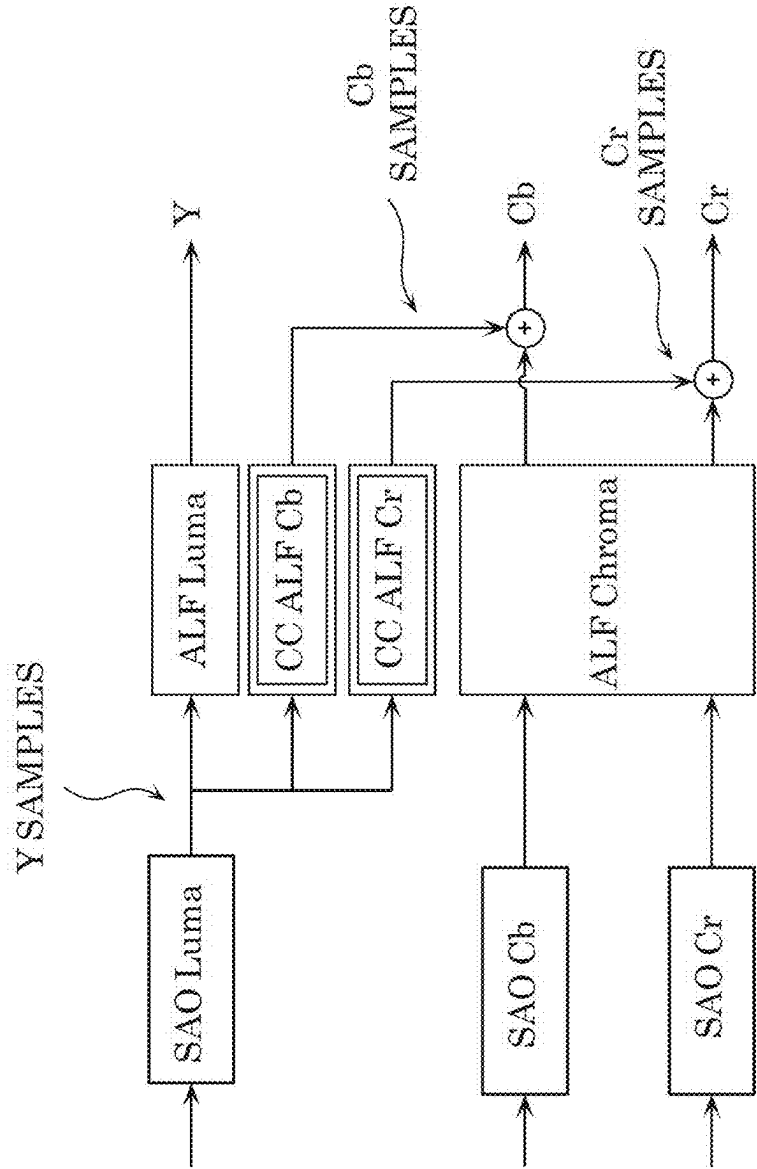
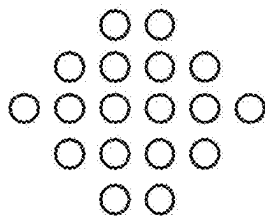


FIG. 23E



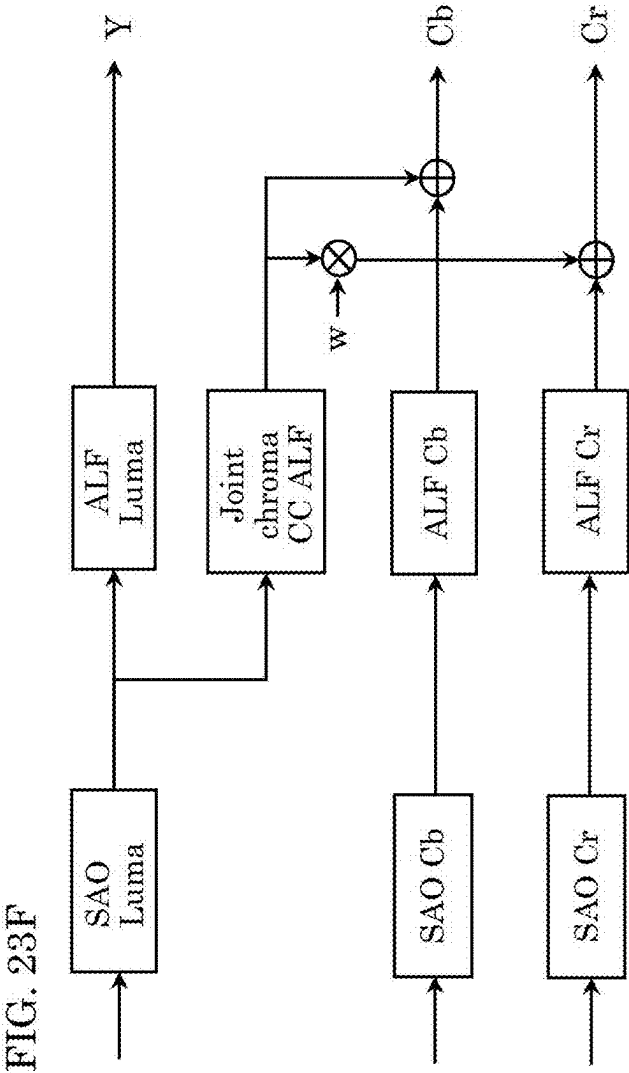


FIG. 23G

Index	1	2	3	4	5	6	7
w	1/4	1/2	3/4	1	4	2	4/3

FIG. 24

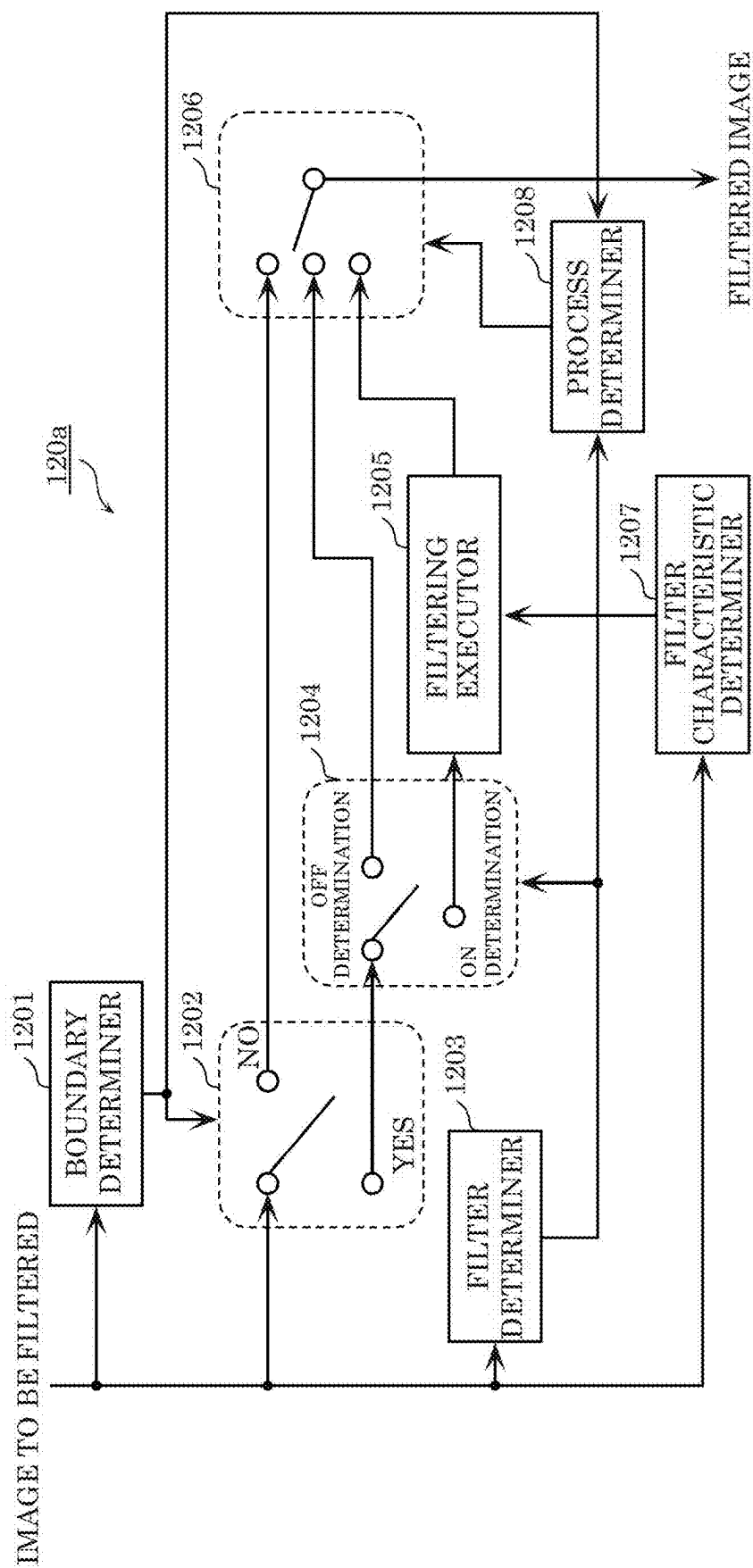


FIG. 25

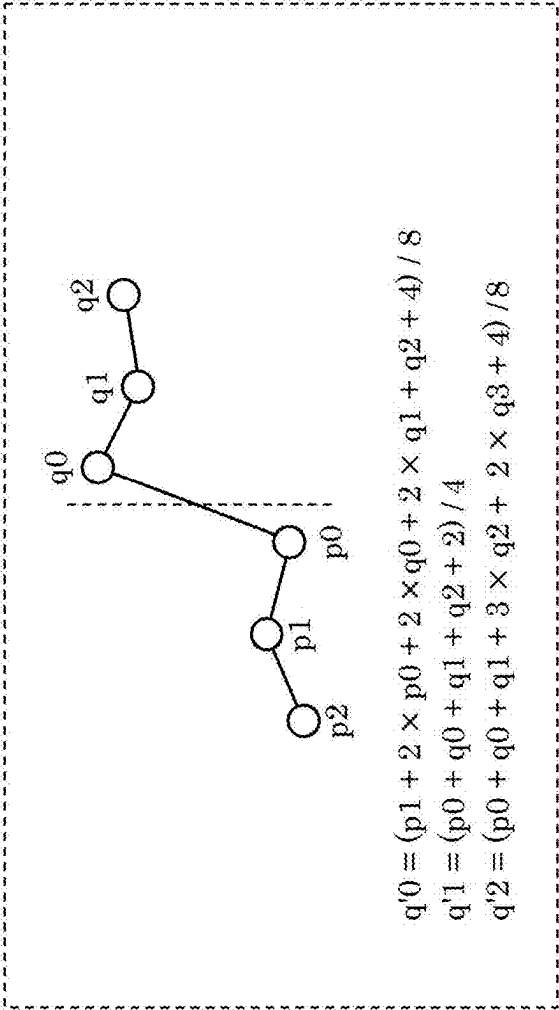


FIG. 26

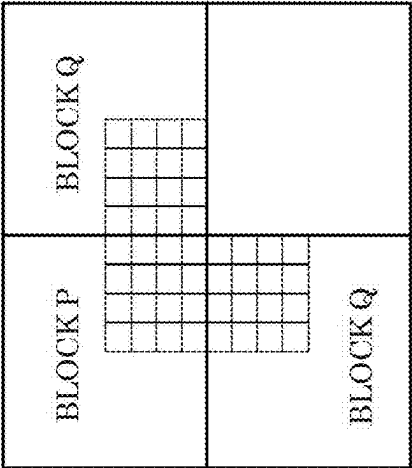


FIG. 27

CONDITIONS FOR BLOCKS P AND Q LOCATED ACROSS BOUNDARY FOR WHICH Bs VALUE IS TO BE DETERMINED	Bs VALUES
- AT LEAST ONE OF THE BLOCKS IS INTRA PREDICTION BLOCK	2
- AT LEAST ONE OF THE BLOCKS INCLUDES NON-ZERO TRANSFORM COEFFICIENT AND HAS A SIDE WHICH COINCIDES WITH BOUNDARY BETWEEN ORTHOGONAL TRANSFORM BLOCKS	1
- ABSOLUTE VALUE OF DIFFERENCE BETWEEN MOTION VECTORS OF THE TWO BLOCKS ACROSS BOUNDARY CORRESPONDS TO ONE OR MORE PIXELS	1
- REFERENCE IMAGES FOR MOTION COMPENSATION OF THE TWO BLOCKS ACROSS BOUNDARY OR THE NUMBERS OF MOTION VECTORS OF THE TWO BLOCKS ARE DIFFERENT	1
- THE OTHERS	0

FIG. 28

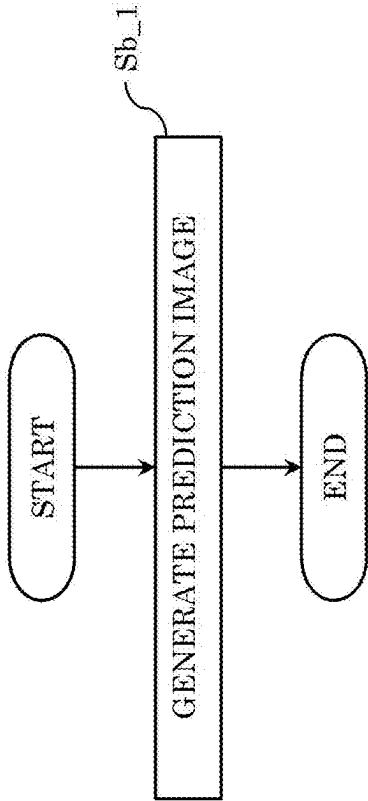


FIG. 29

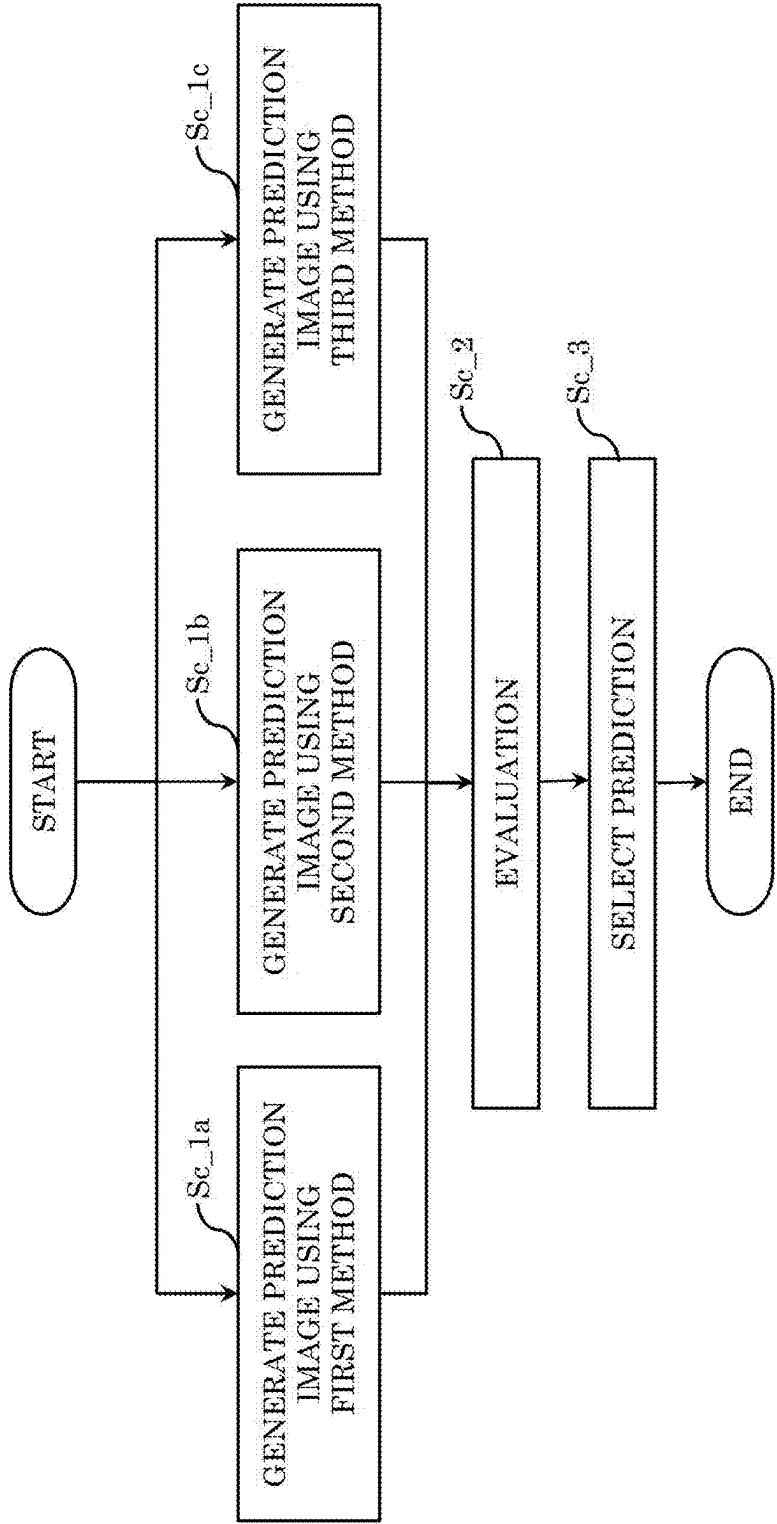


FIG. 30

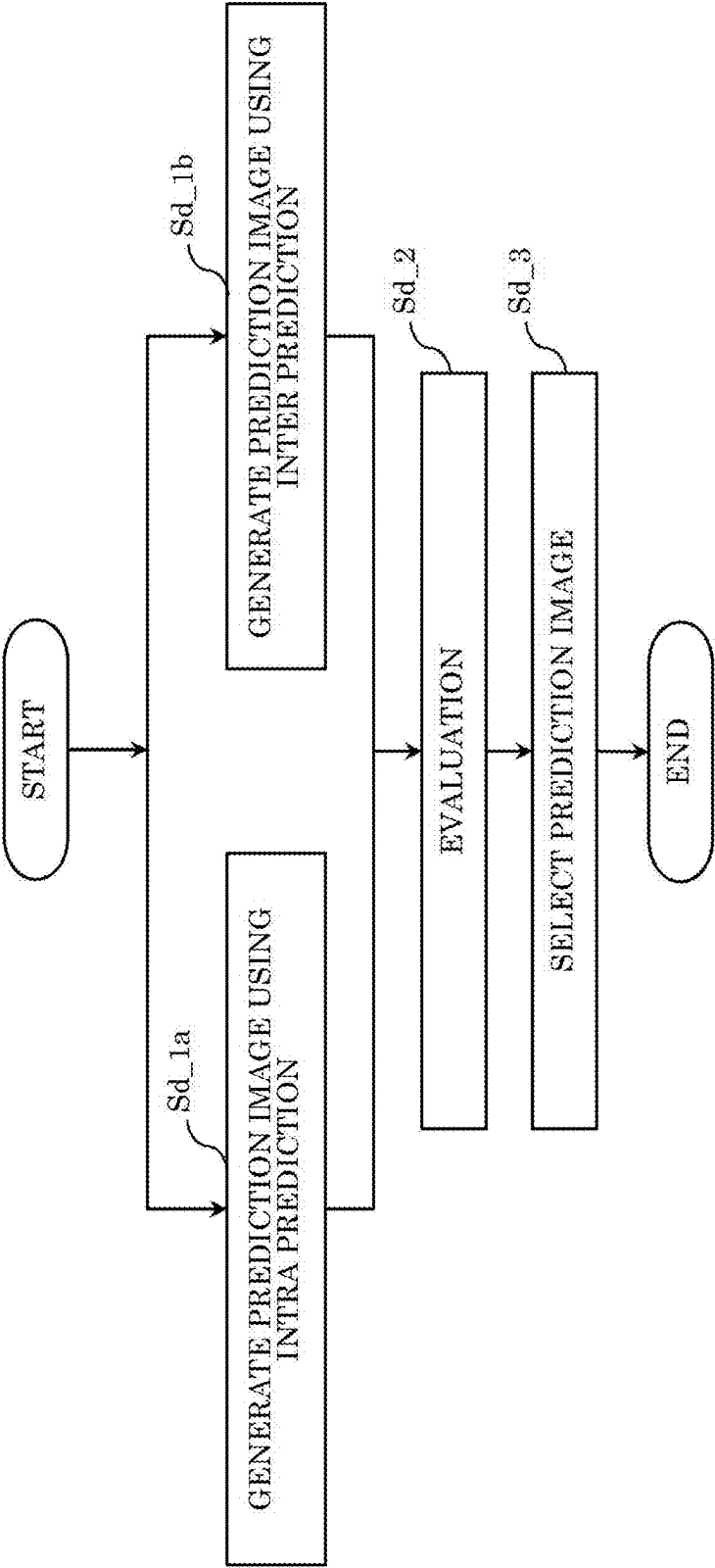


FIG. 31

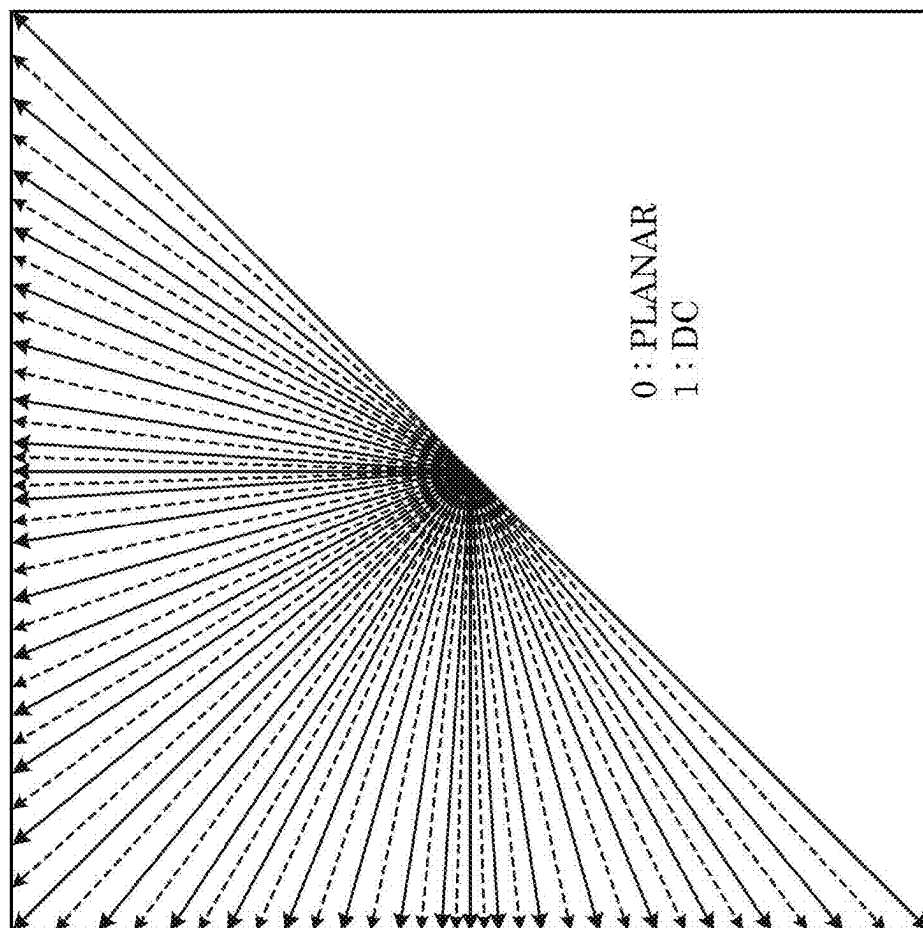


FIG. 32

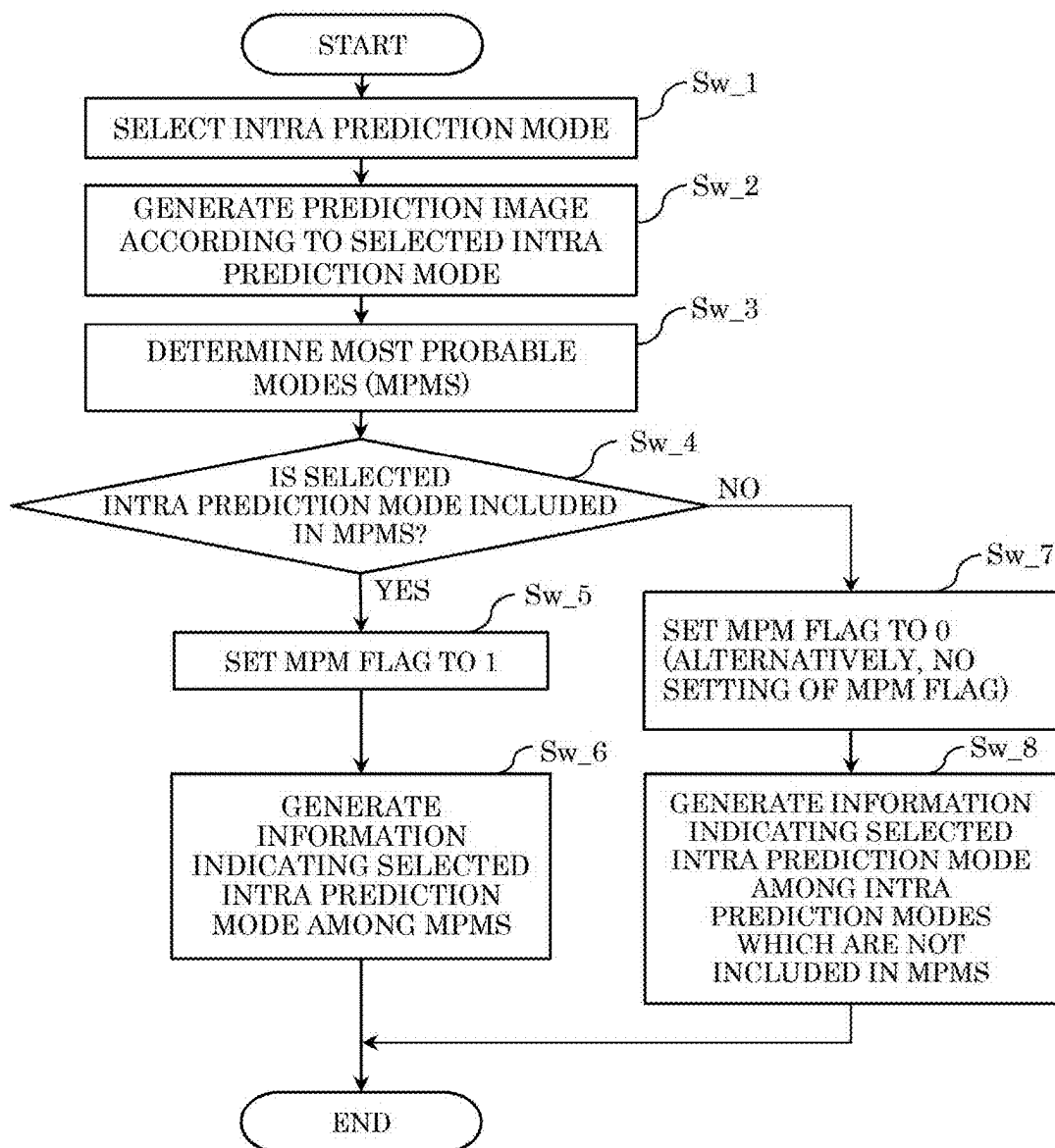
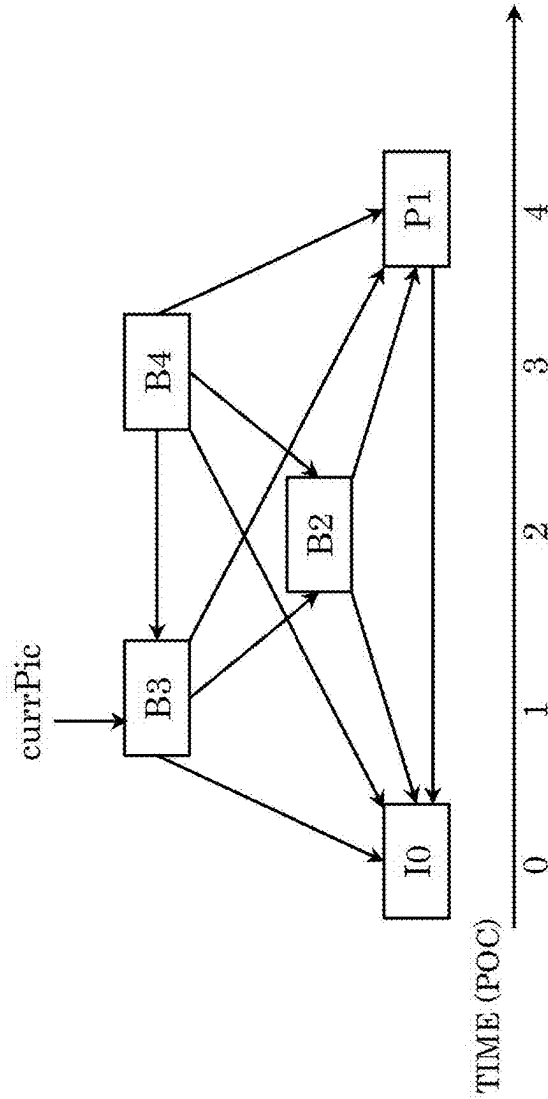


FIG. 33



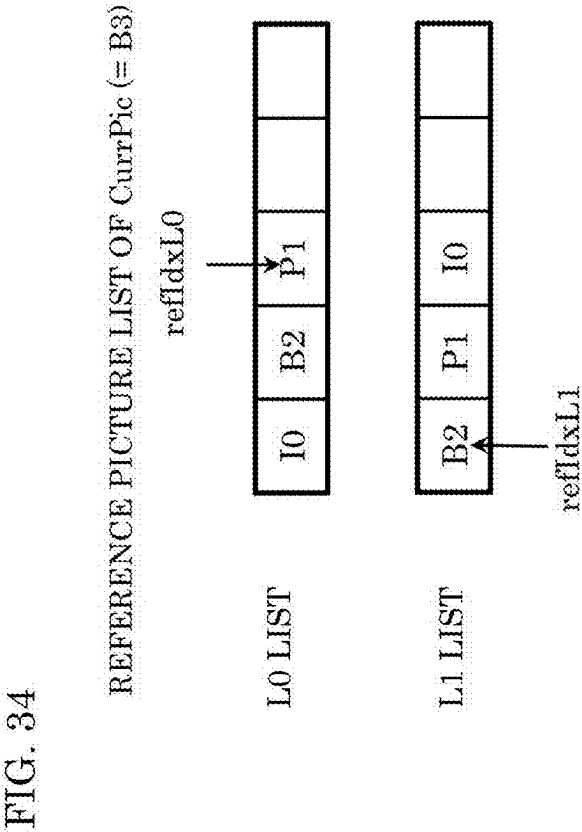


FIG. 35

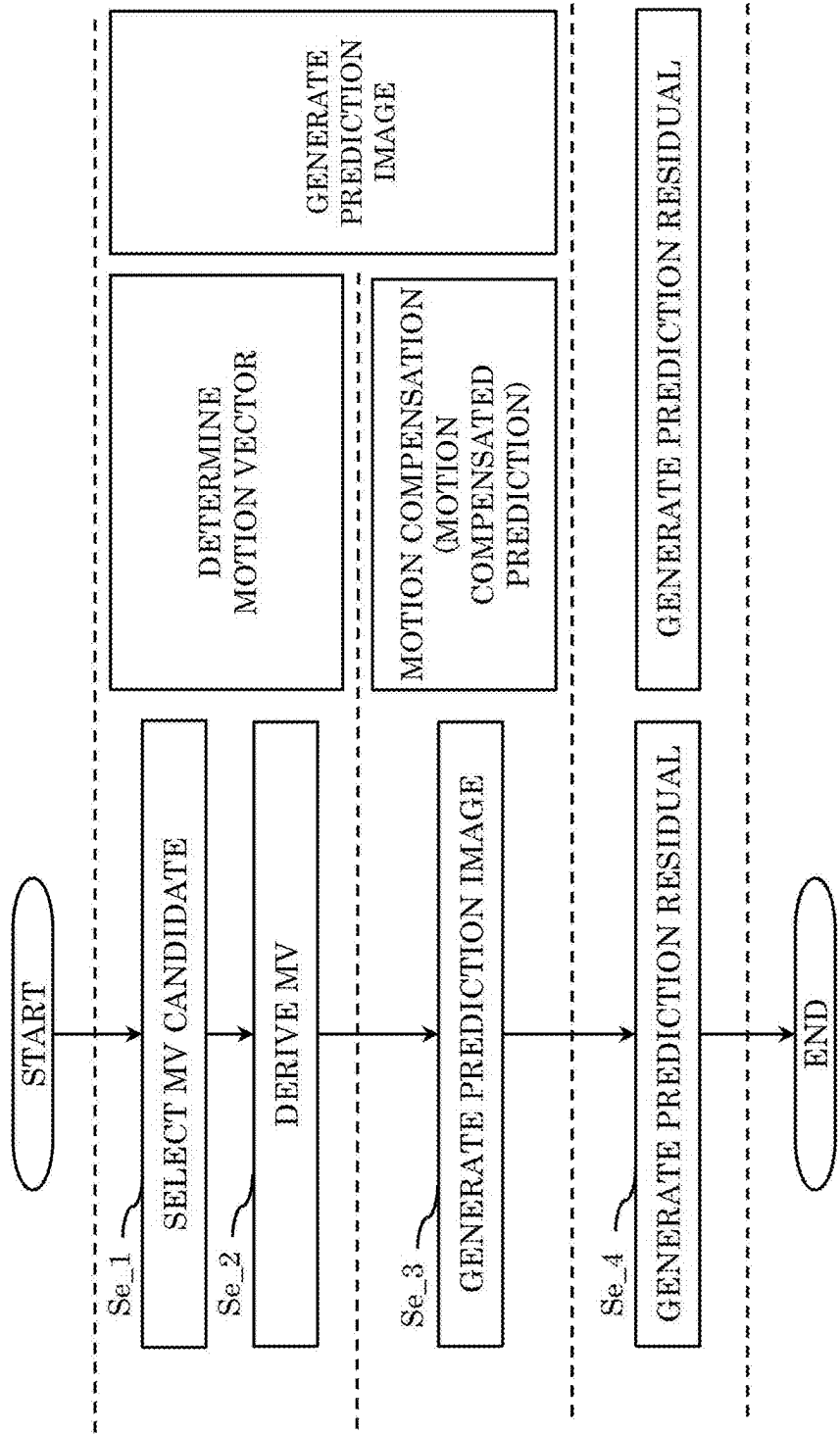


FIG. 36

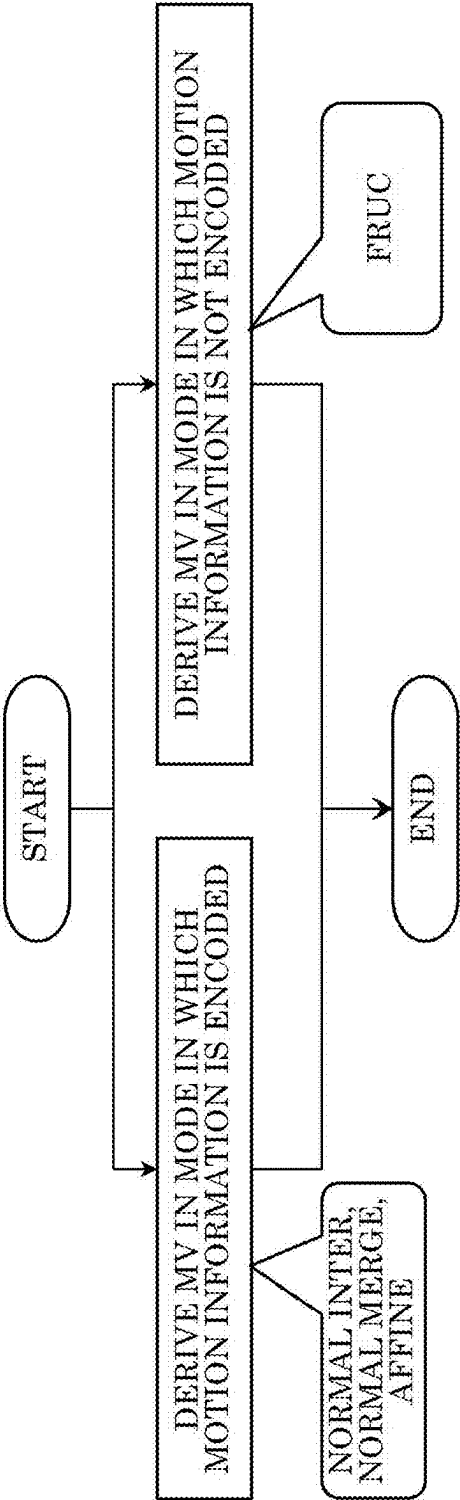


FIG. 37

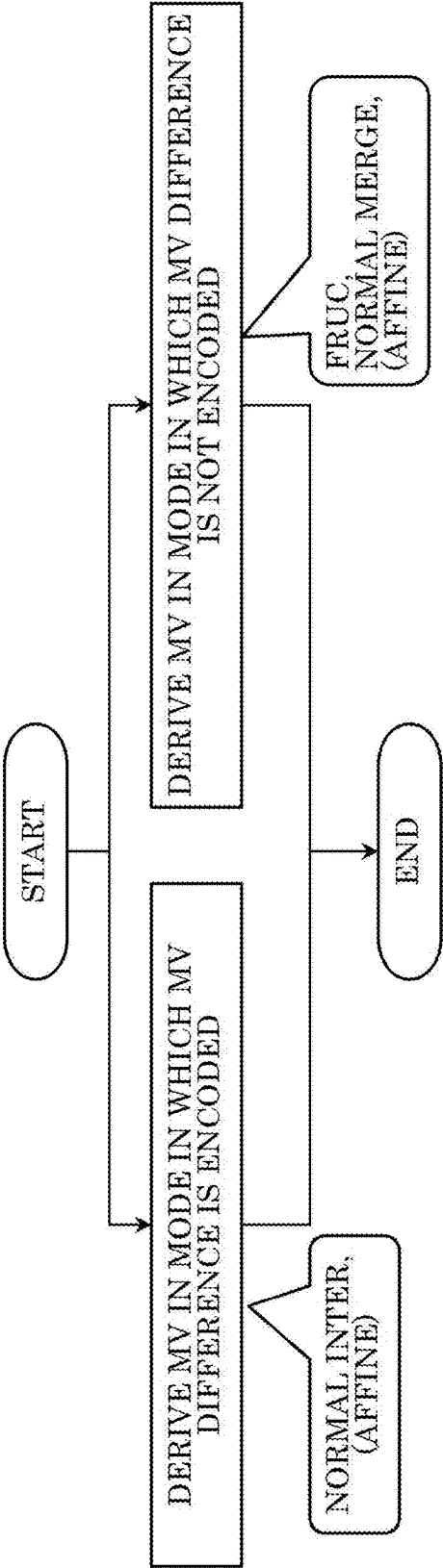


FIG. 38A

MV DERIVATION MODE		MOTION INFORMATION	MV DIFFERENCE
MERGE MODE	INTER MODE	○	○
	OTHER THAN MMVD MODE	○	×
	MMVD MODE	○	△
FRUC MODE		×	×

FIG. 38B

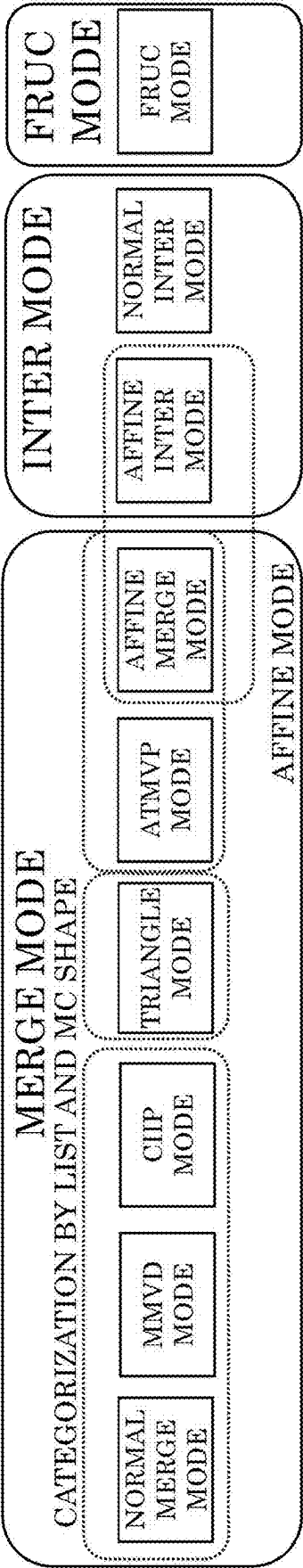


FIG. 39

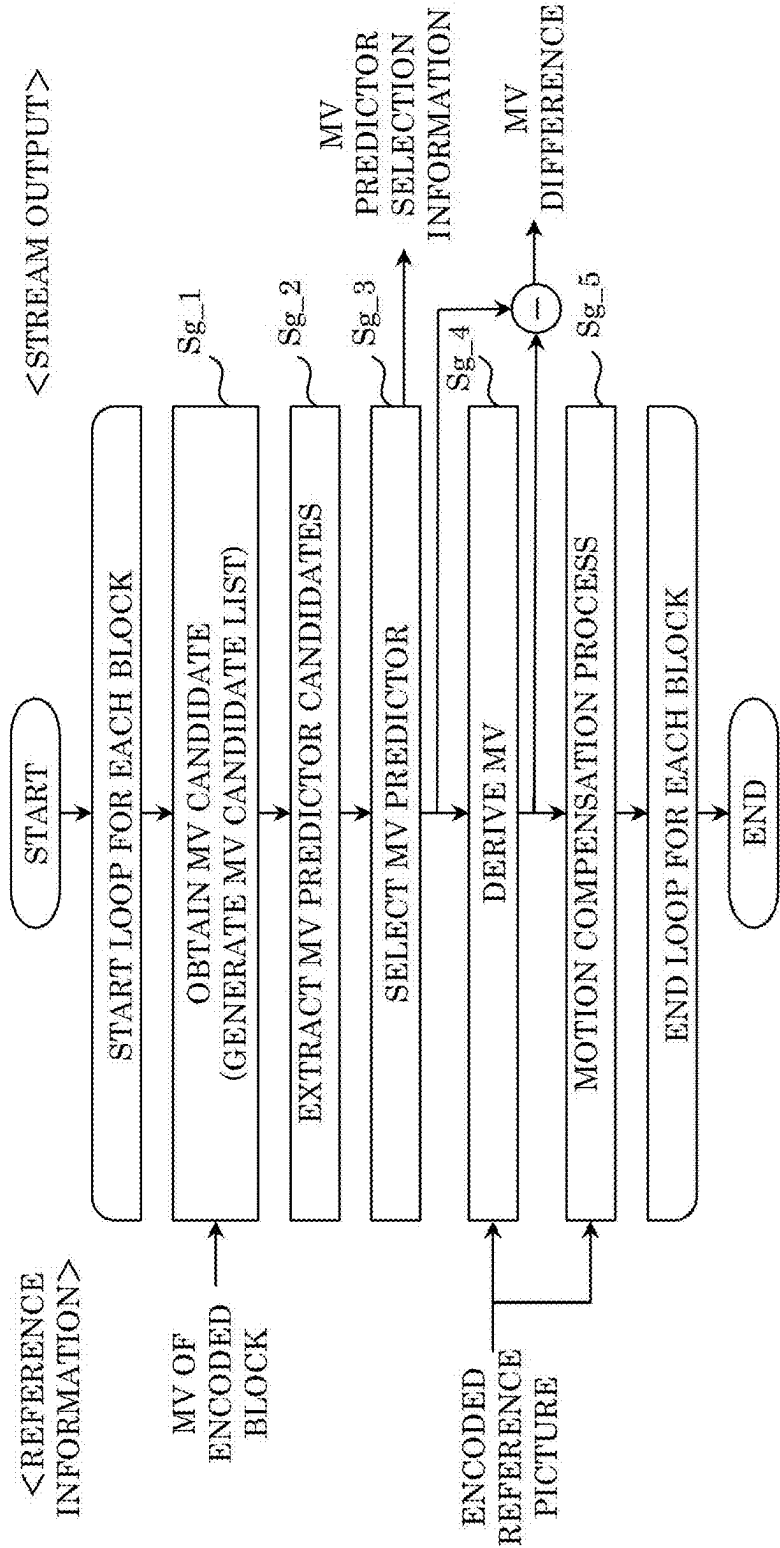


FIG. 40

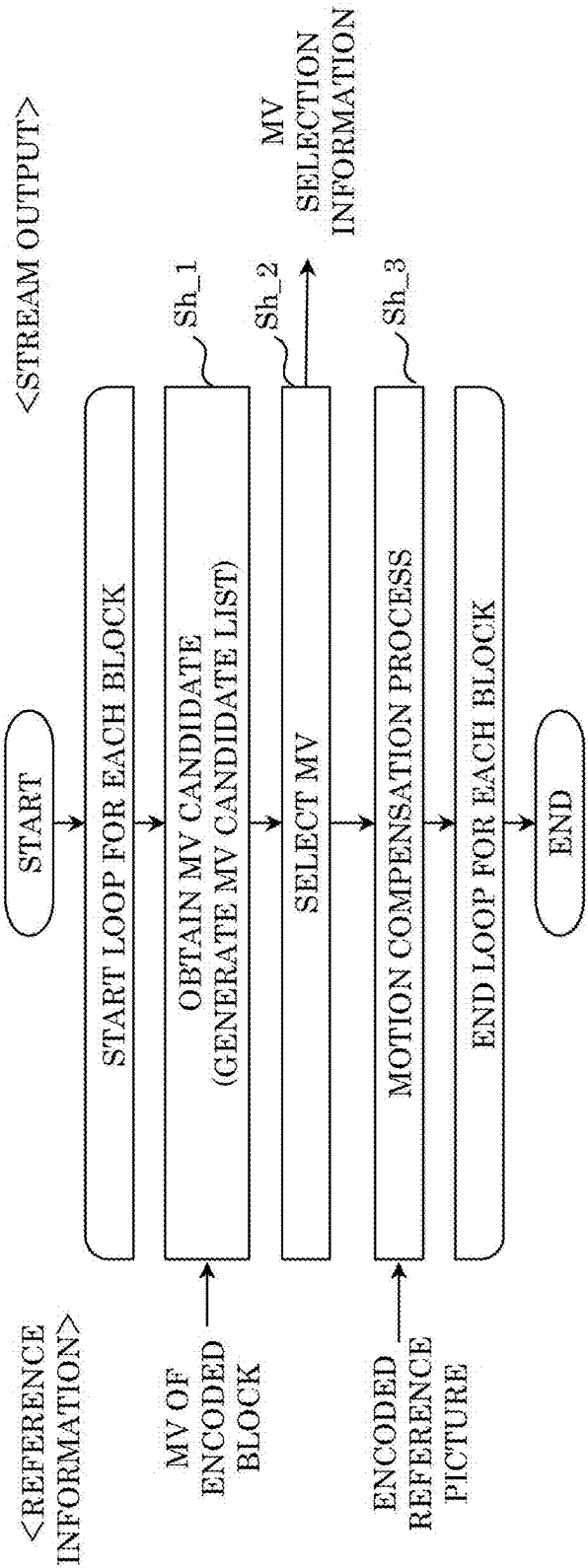


FIG. 41

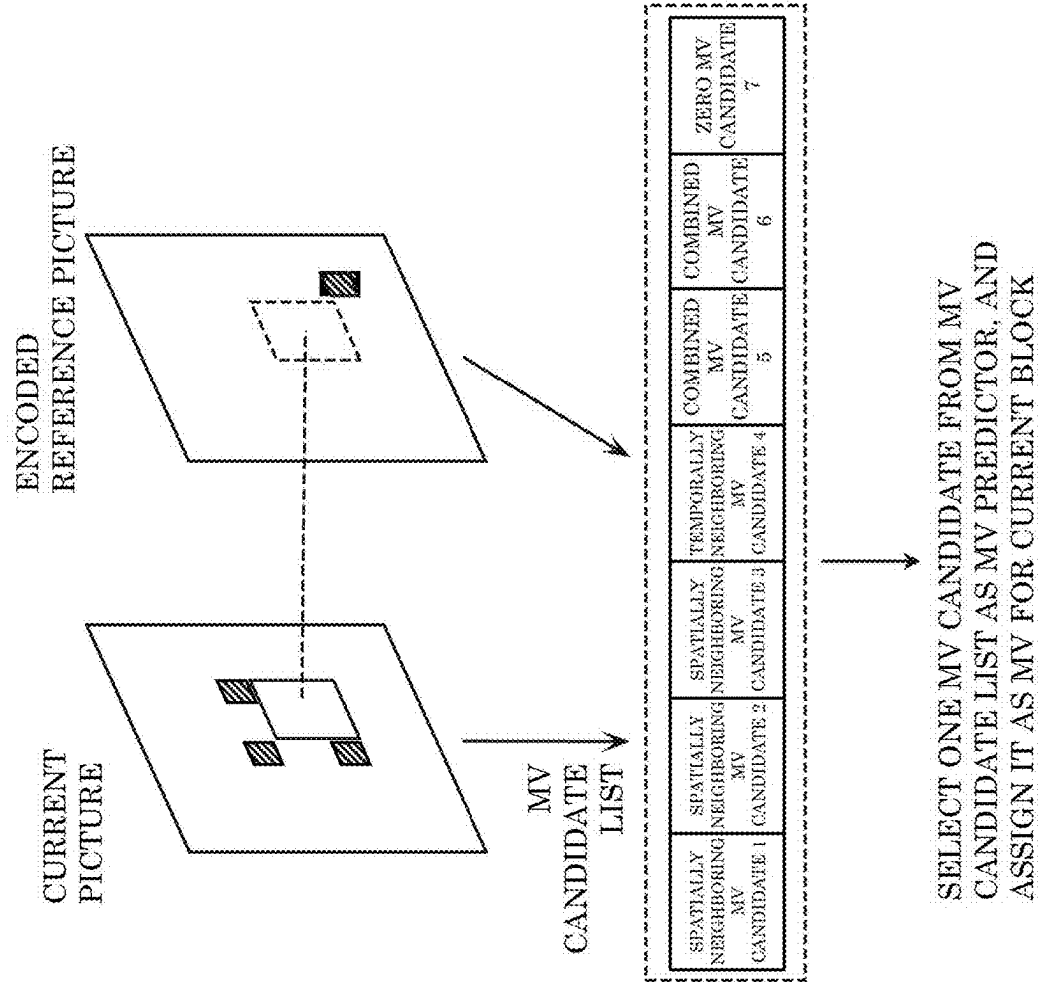


FIG.42

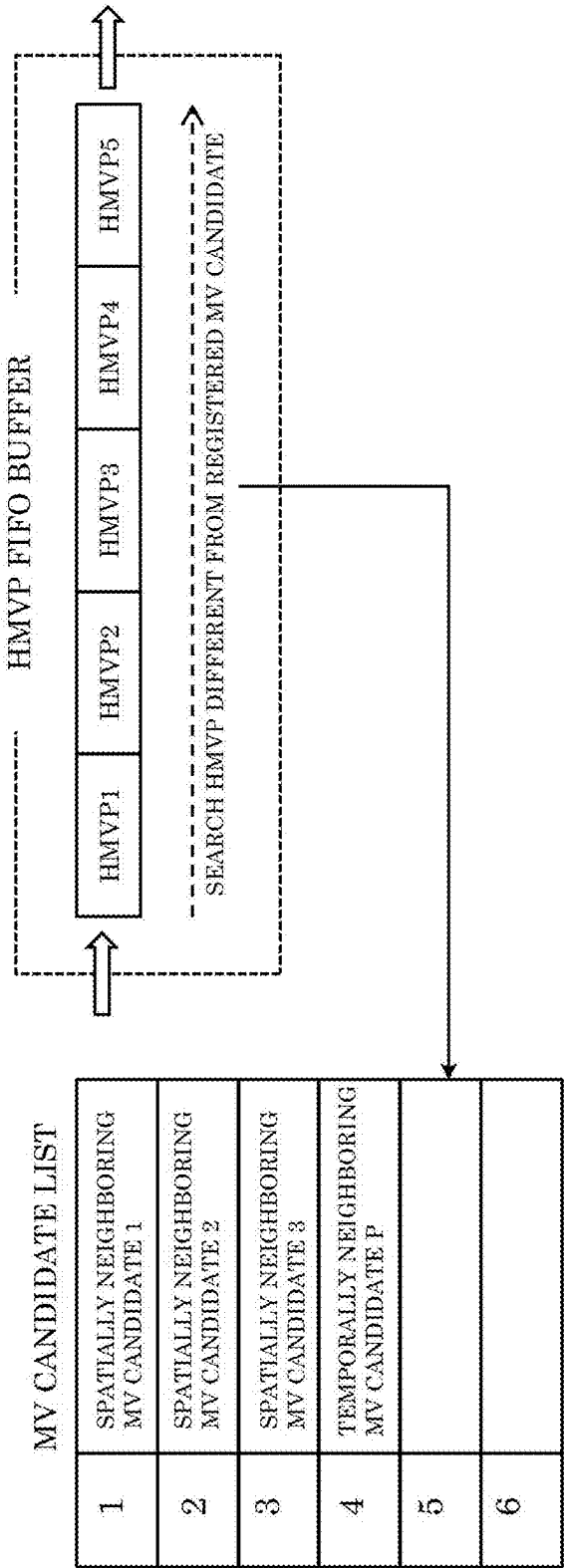


FIG. 43

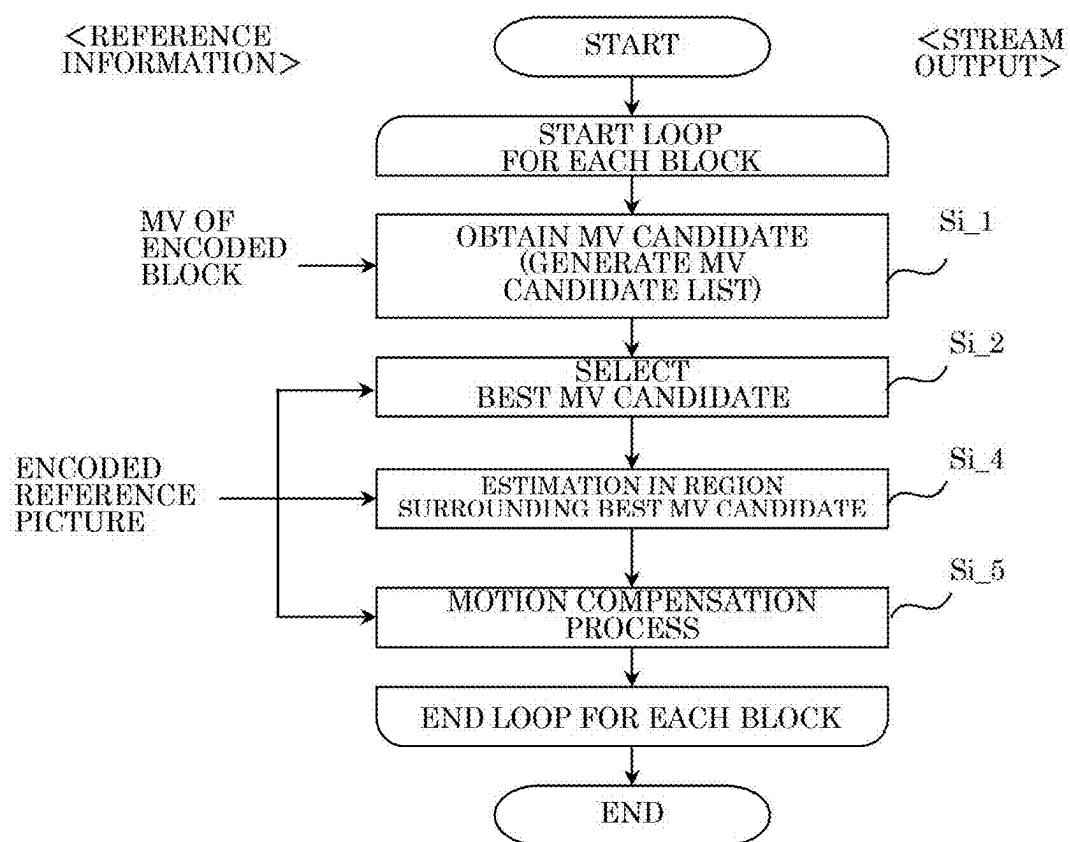


FIG. 44

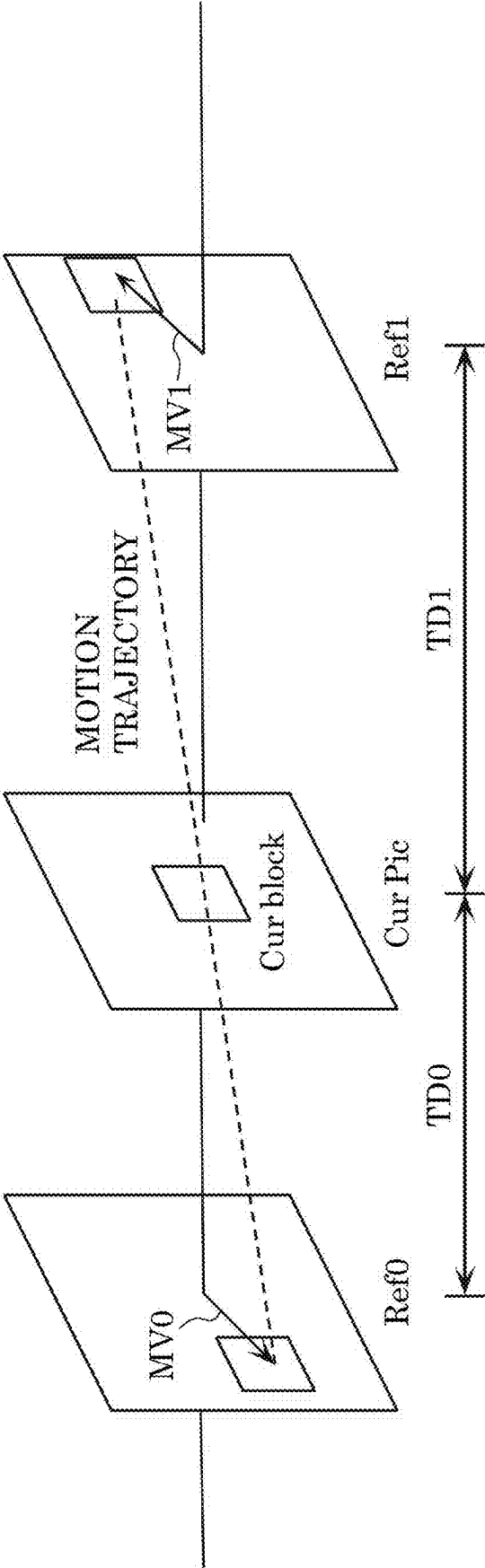


FIG. 45

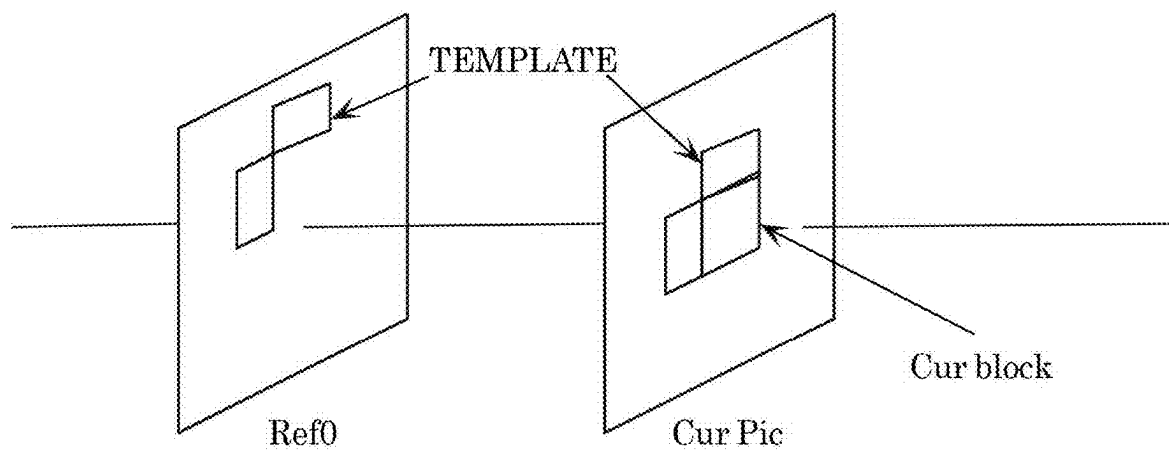


FIG. 46A

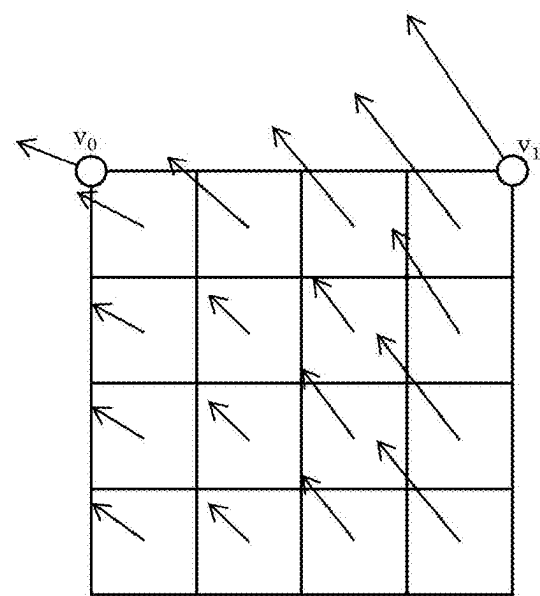


FIG. 46B

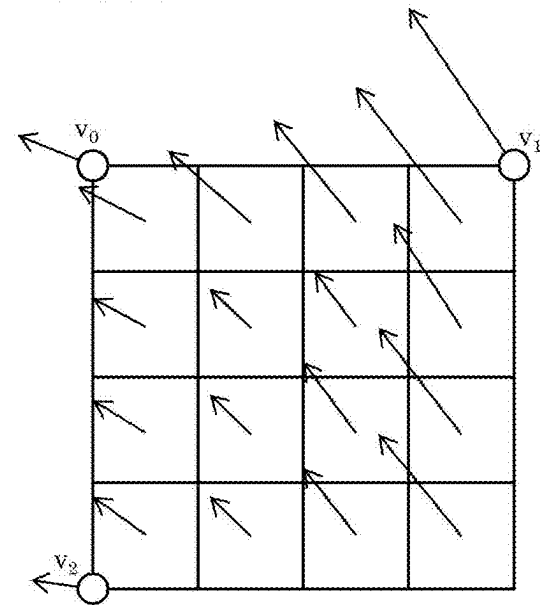


FIG. 47A

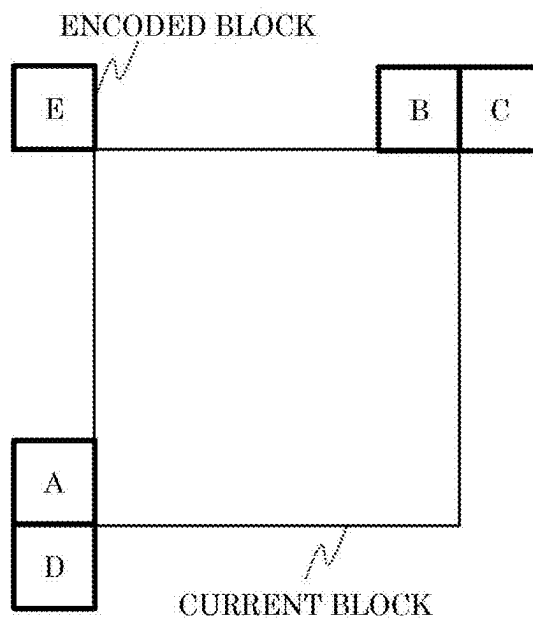


FIG. 47B

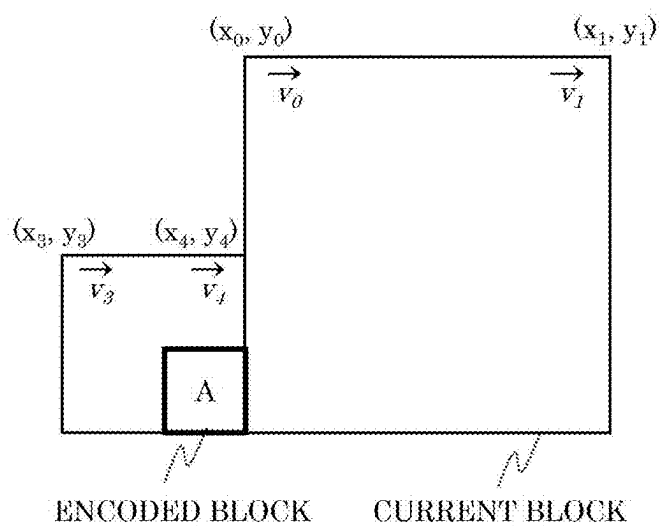


FIG. 47C

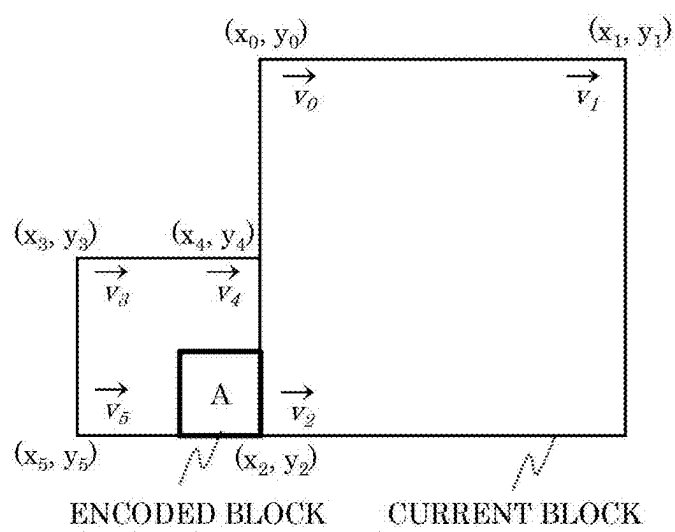


FIG. 48A

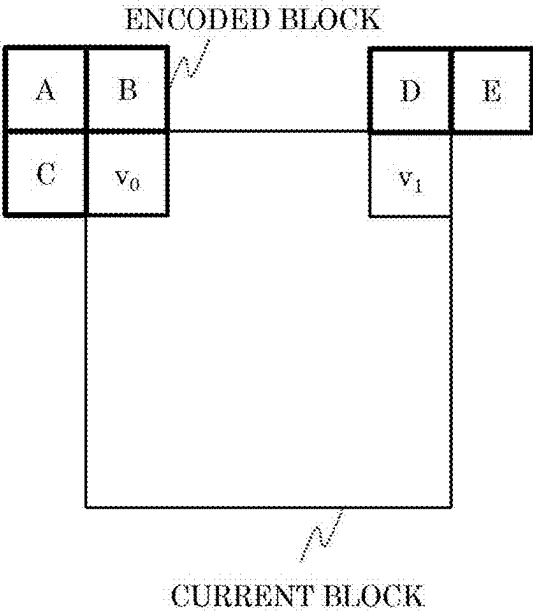


FIG. 48B

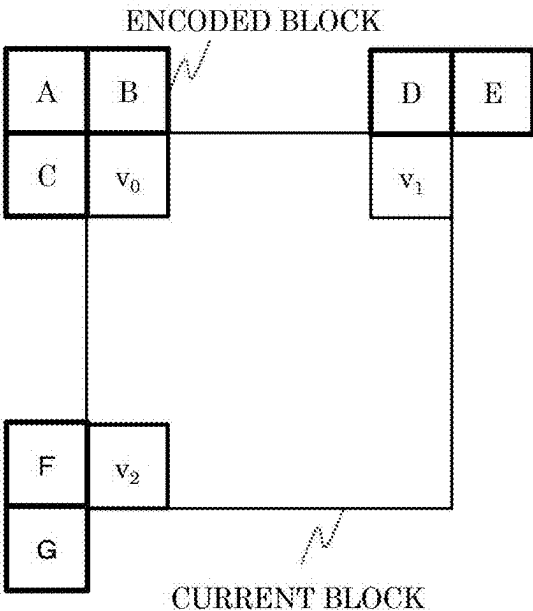


FIG. 49A

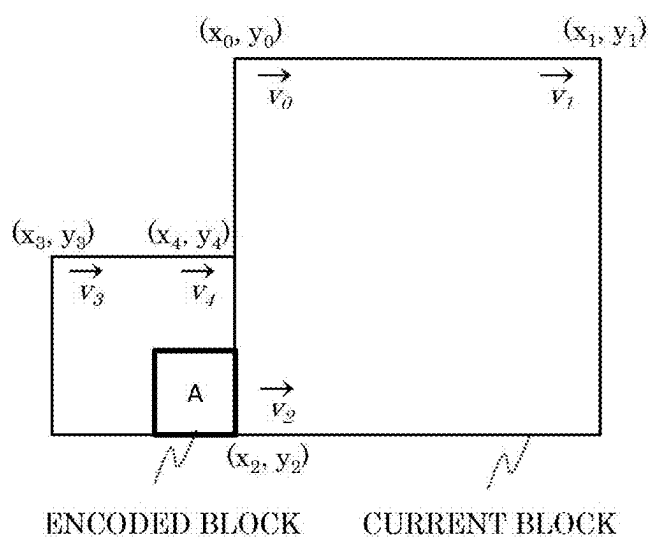


FIG. 49B

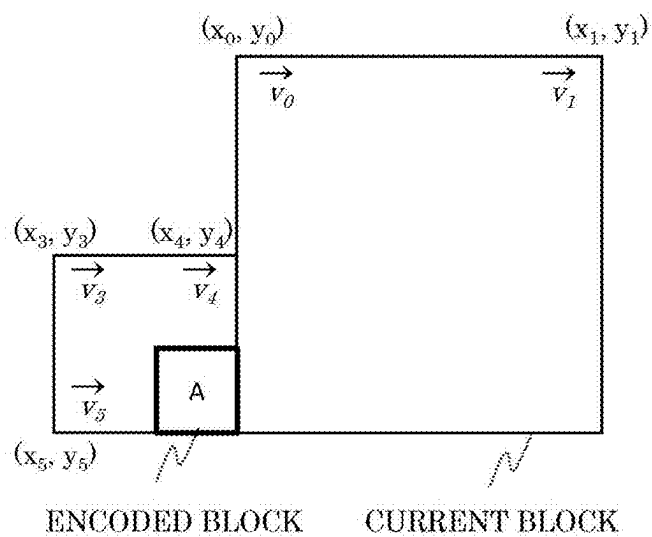


FIG. 50

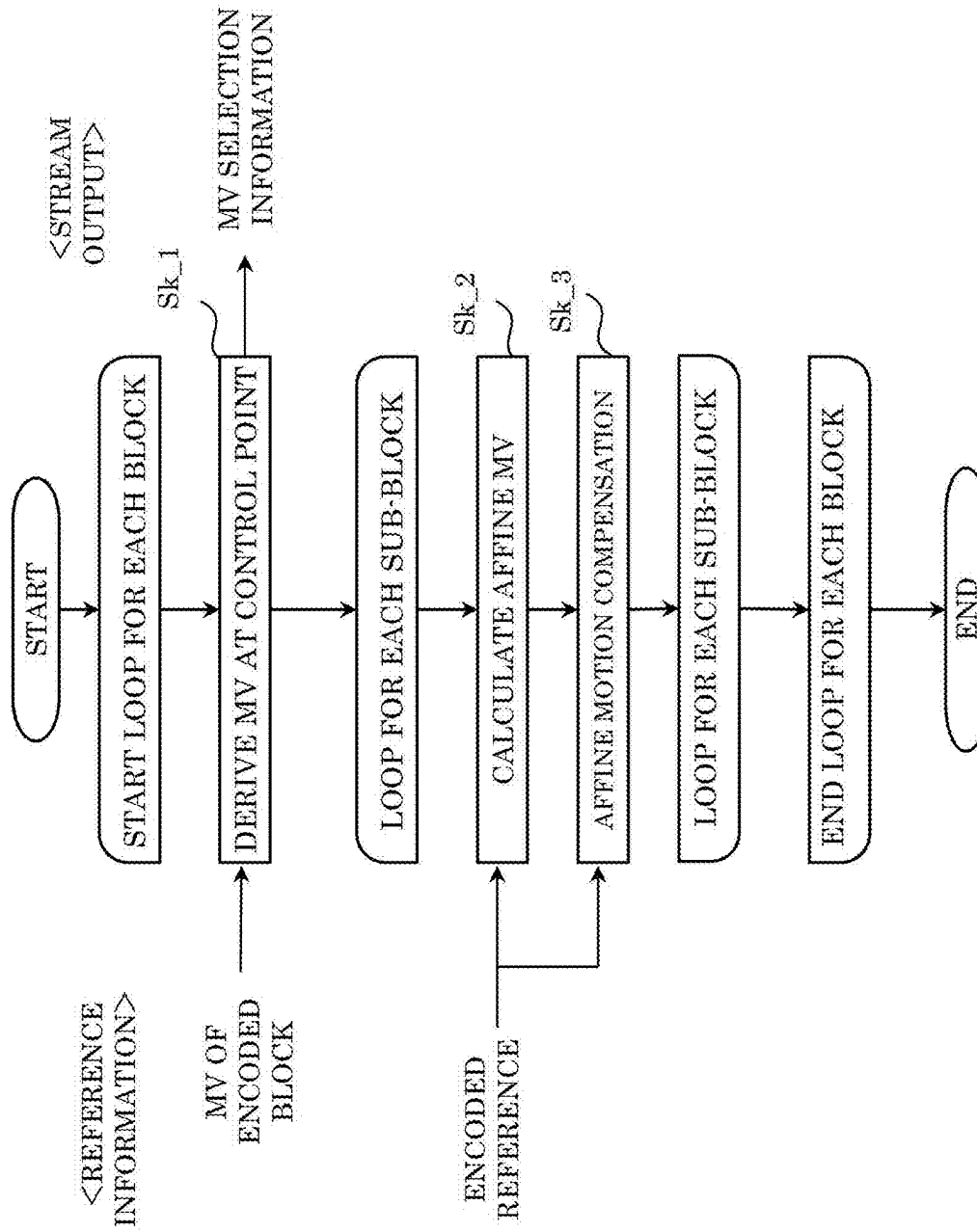


FIG. 51

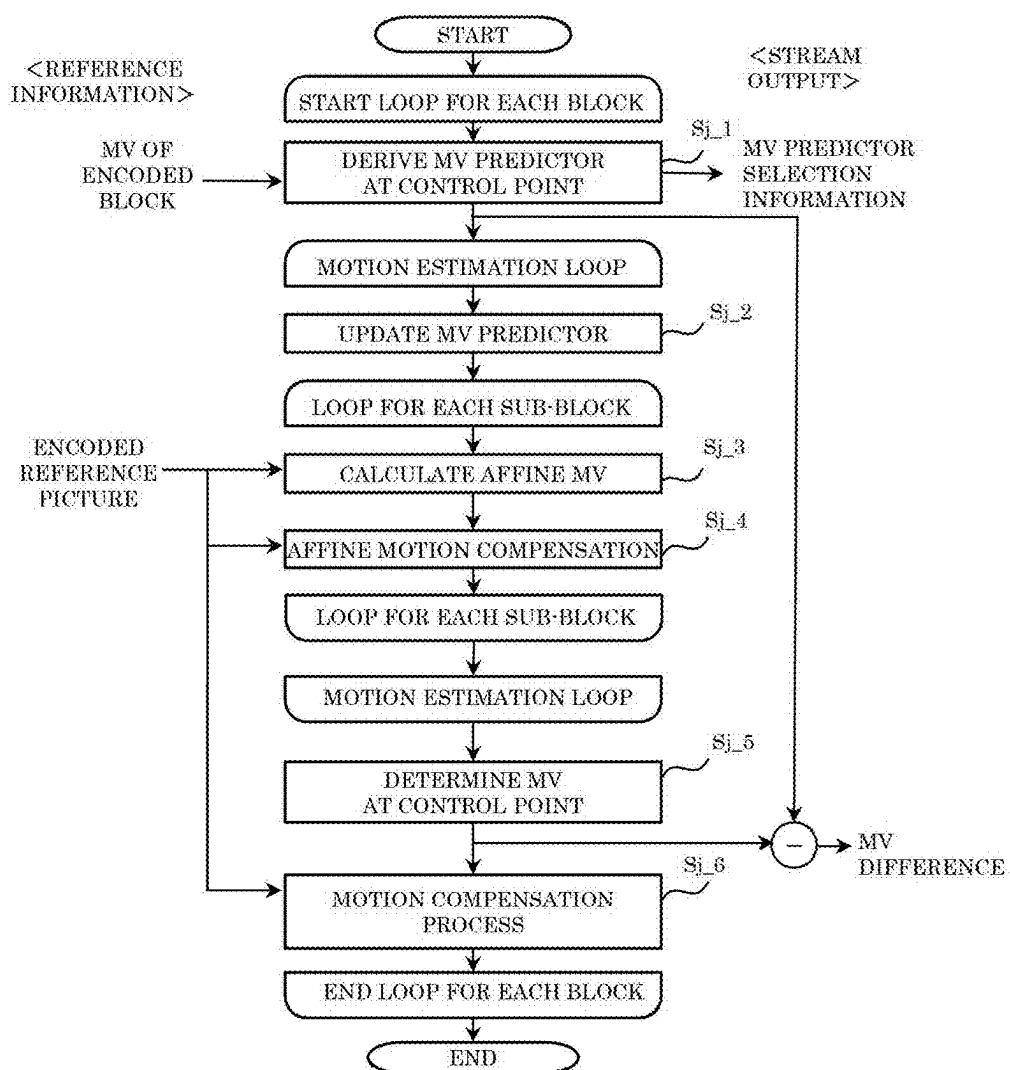


FIG. 52A

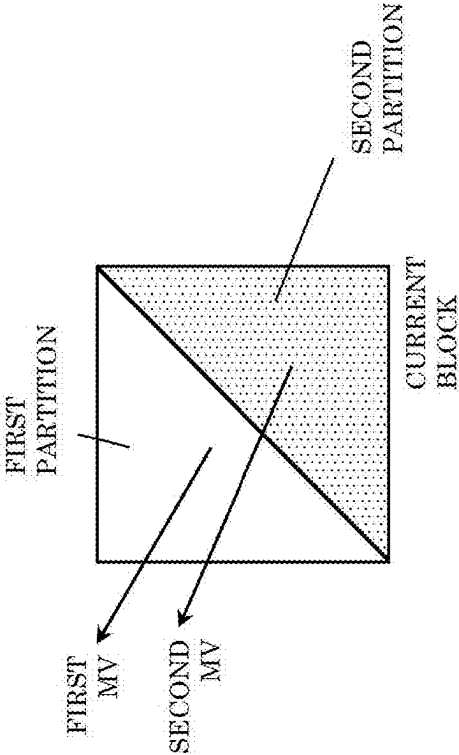


FIG. 52B

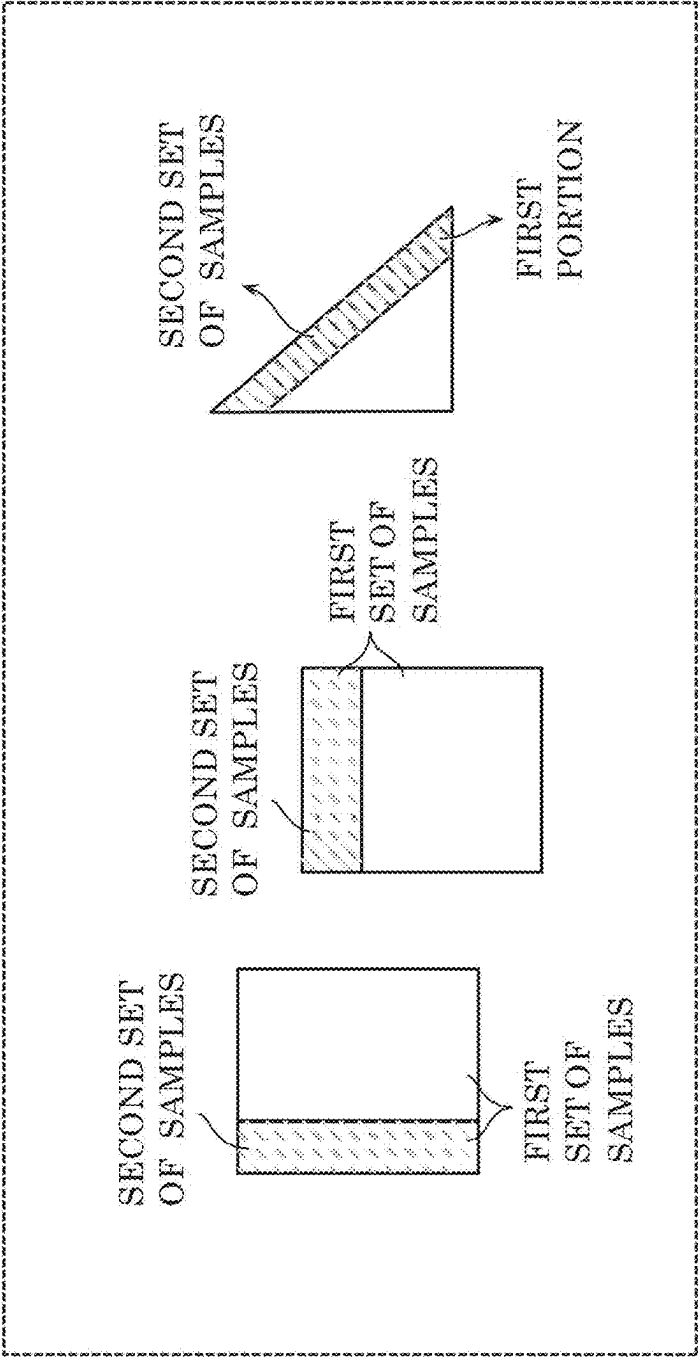


FIG. 52C

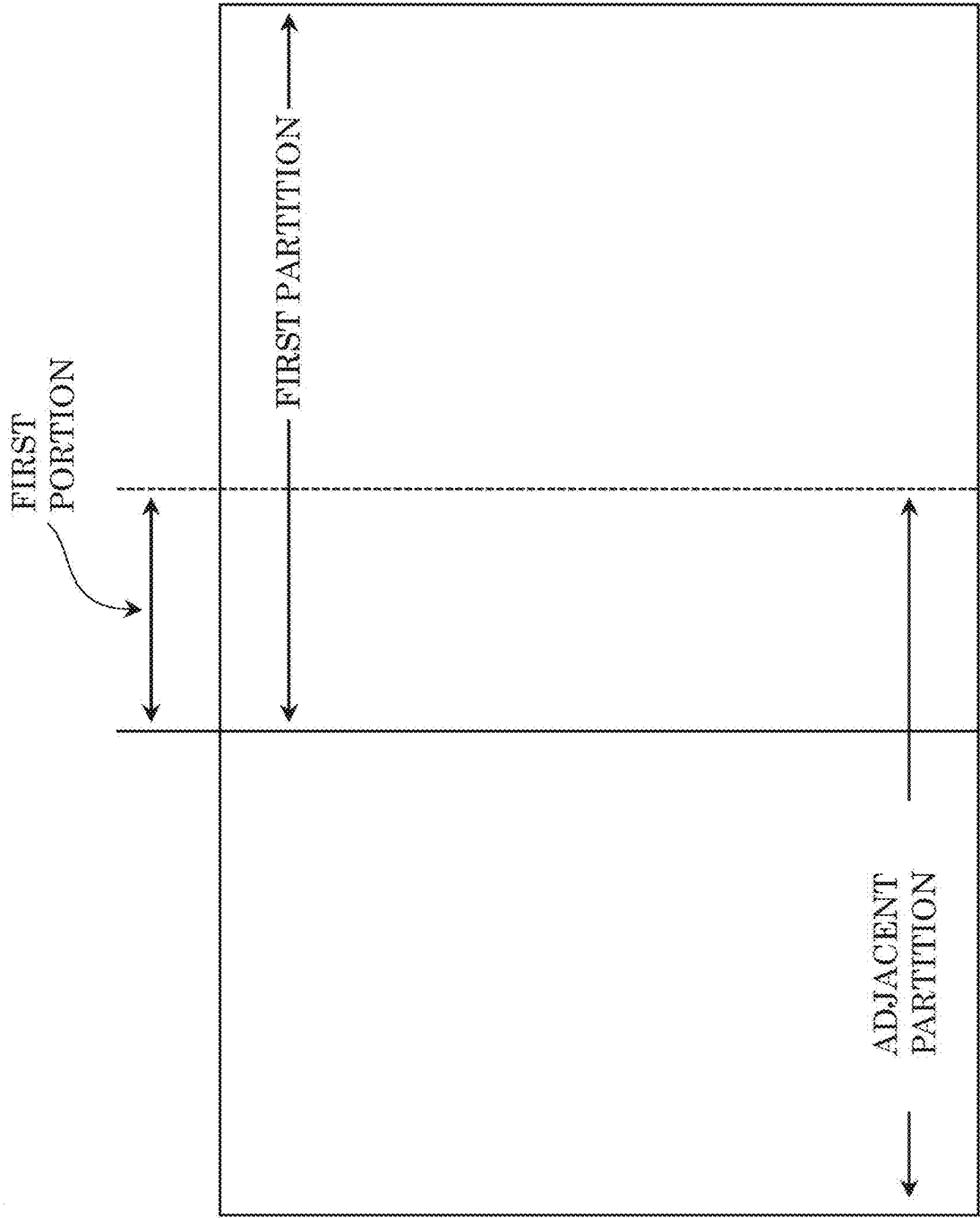


FIG. 53

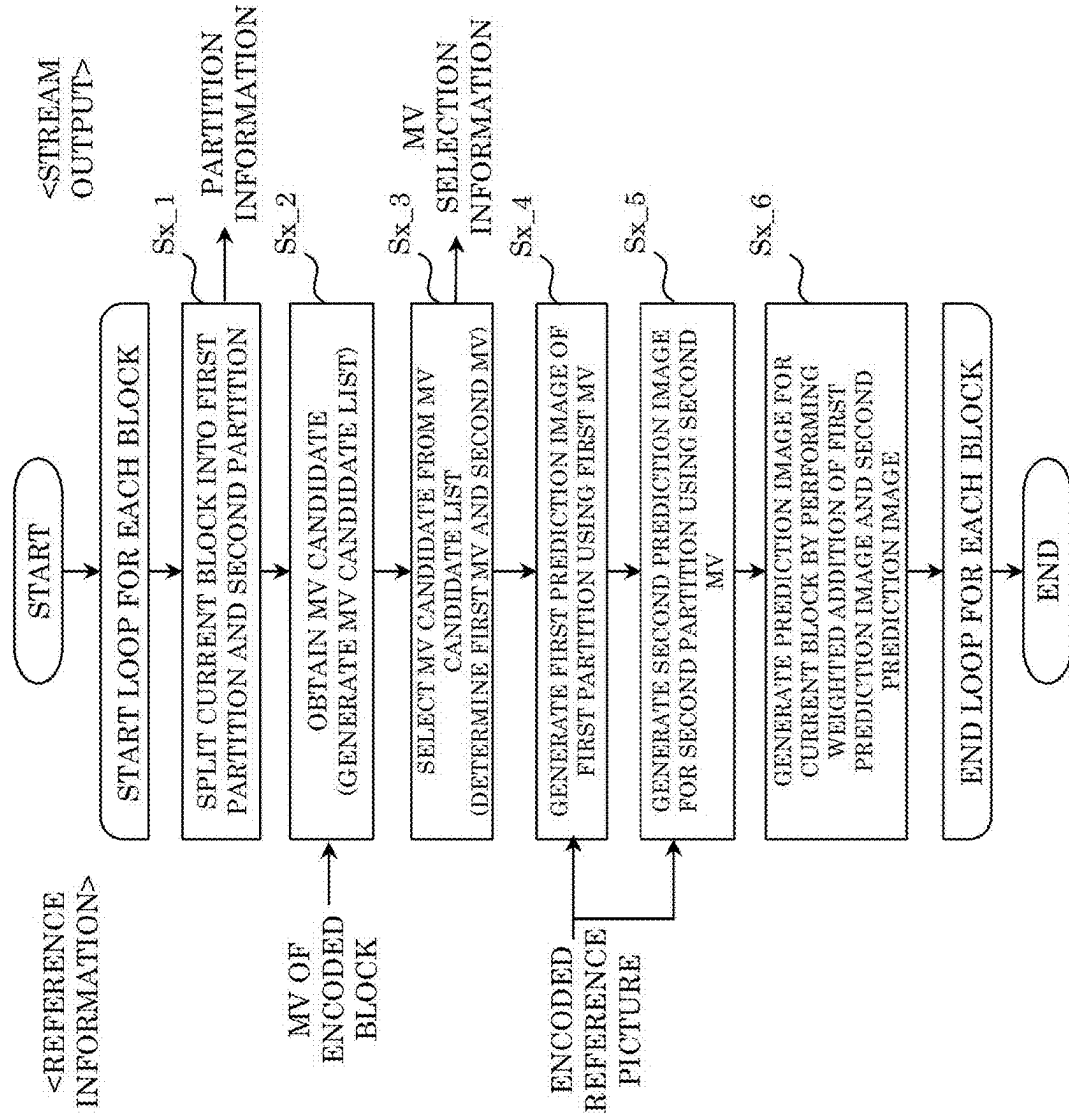


FIG. 54

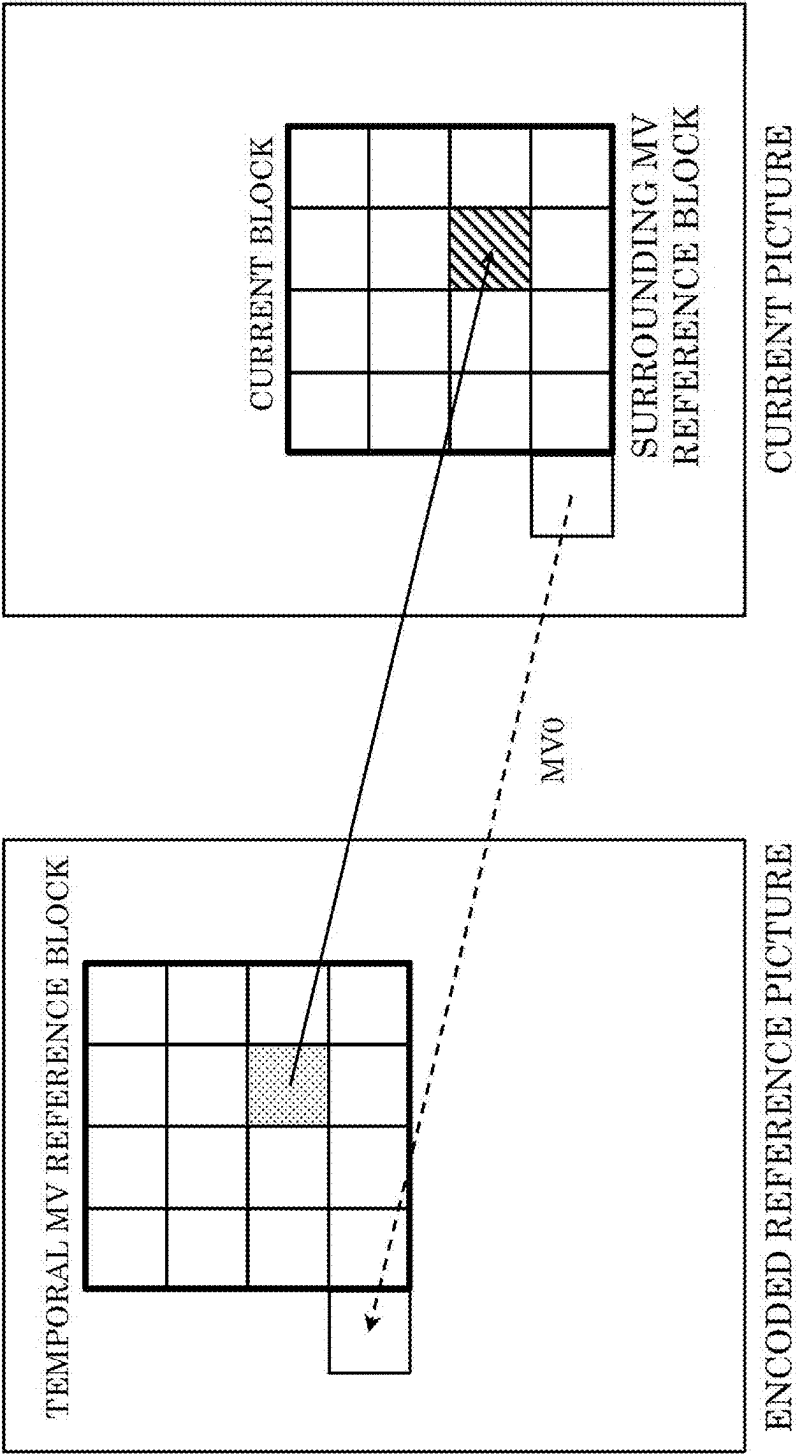


FIG. 55

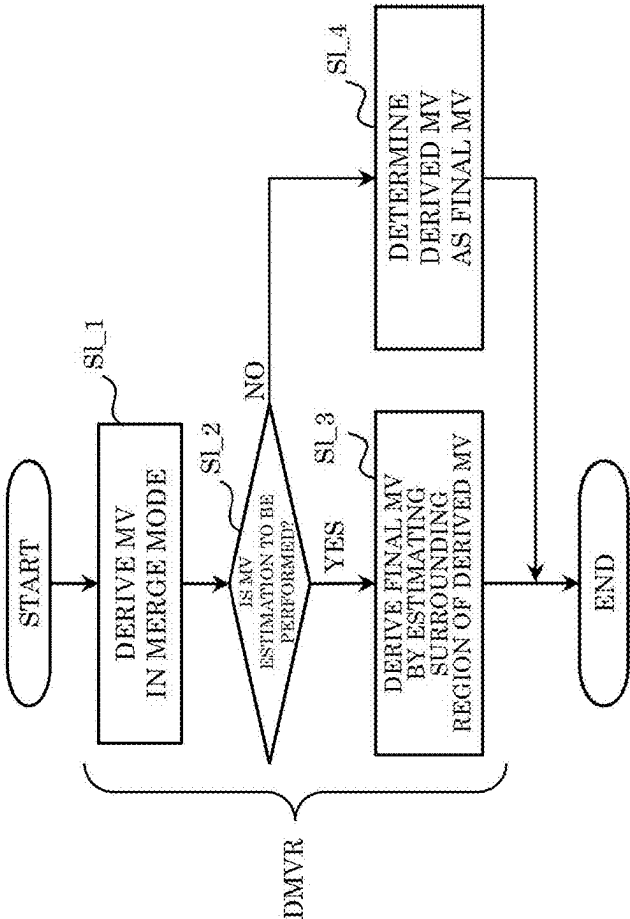


FIG. 56

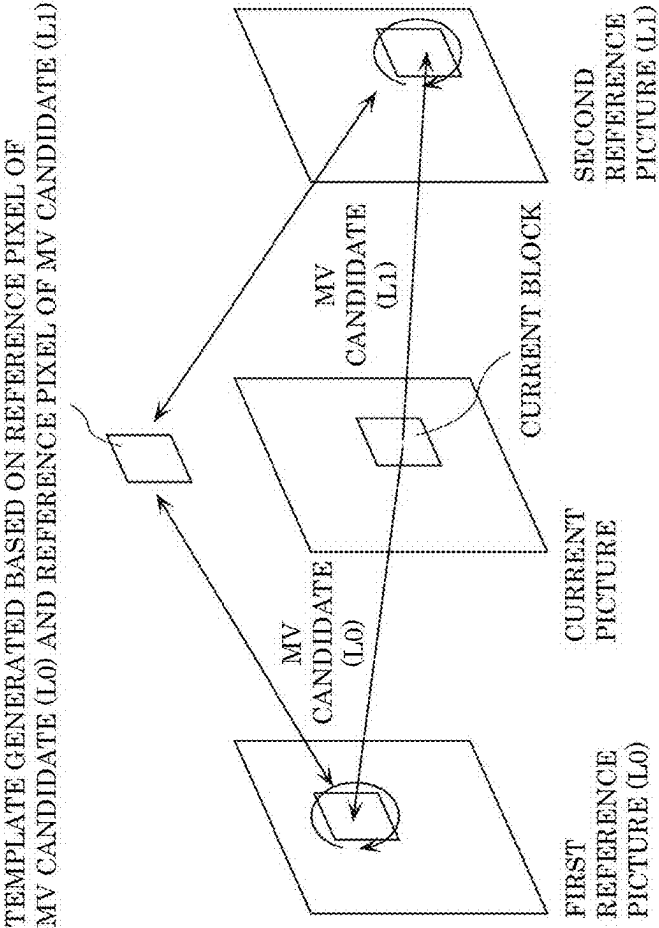


FIG. 57

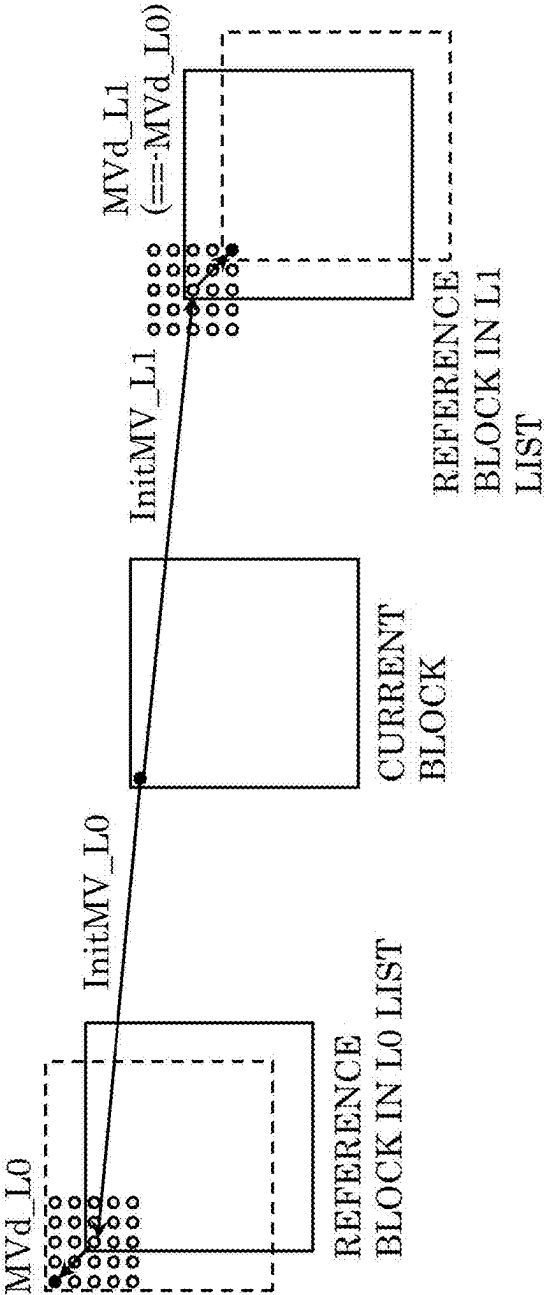


FIG. 58A

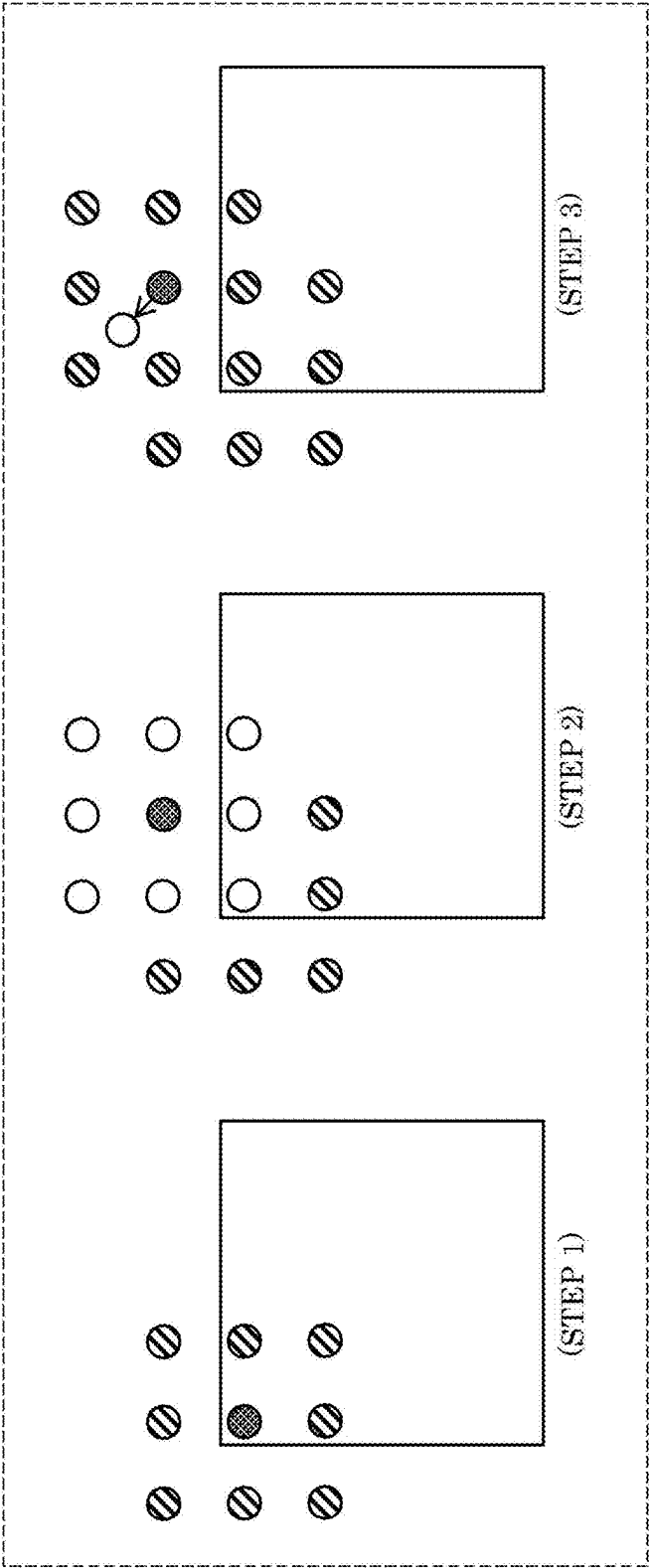


FIG. 58B

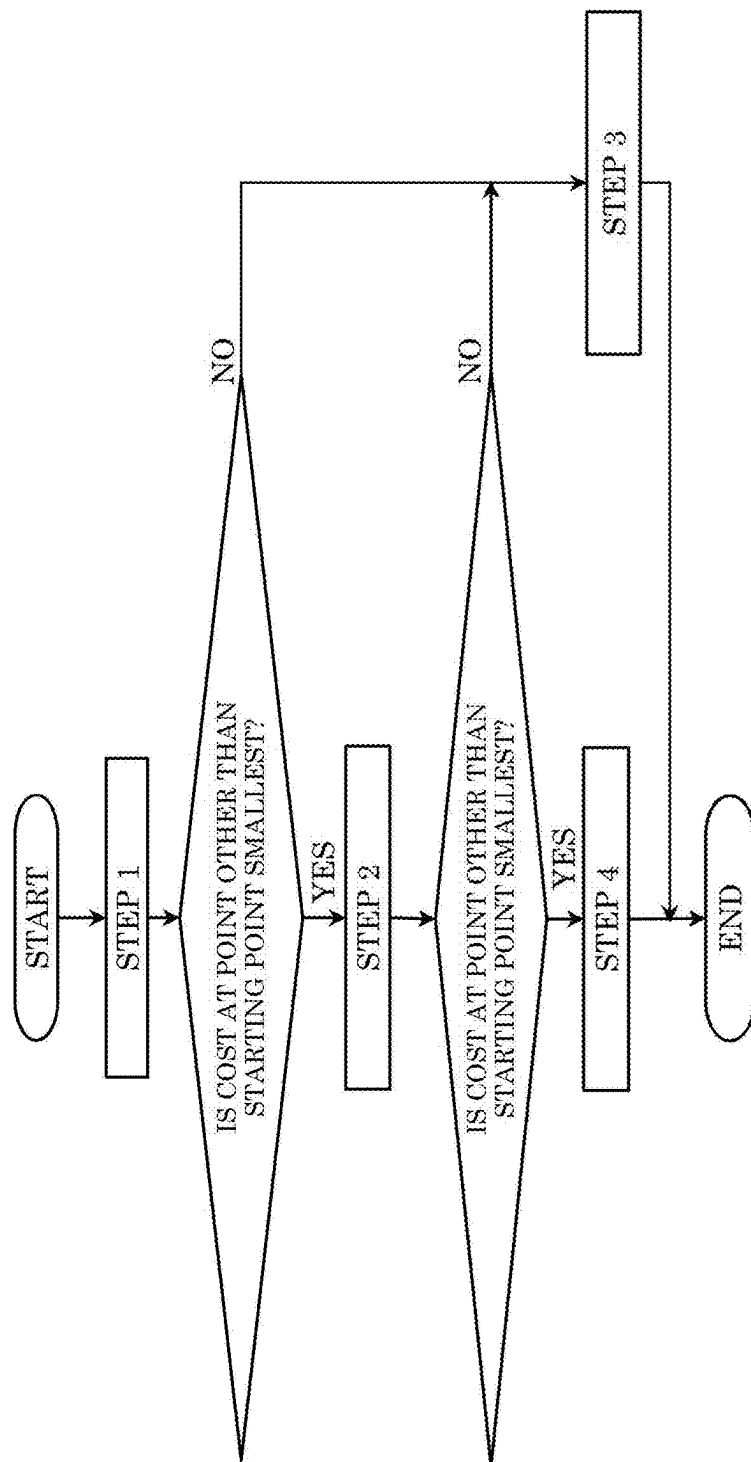


FIG. 59

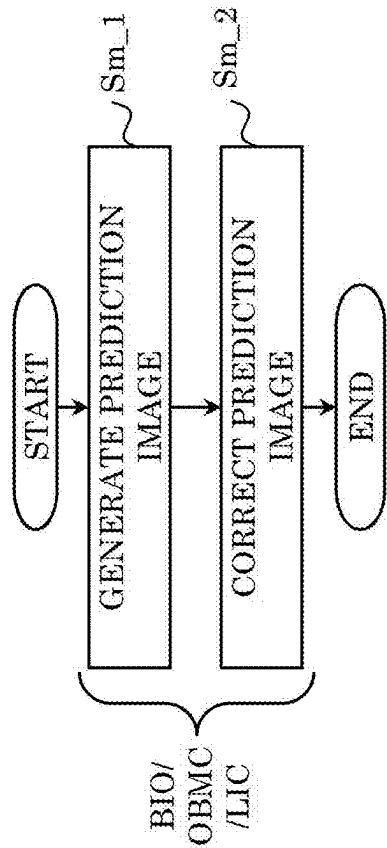


FIG. 60

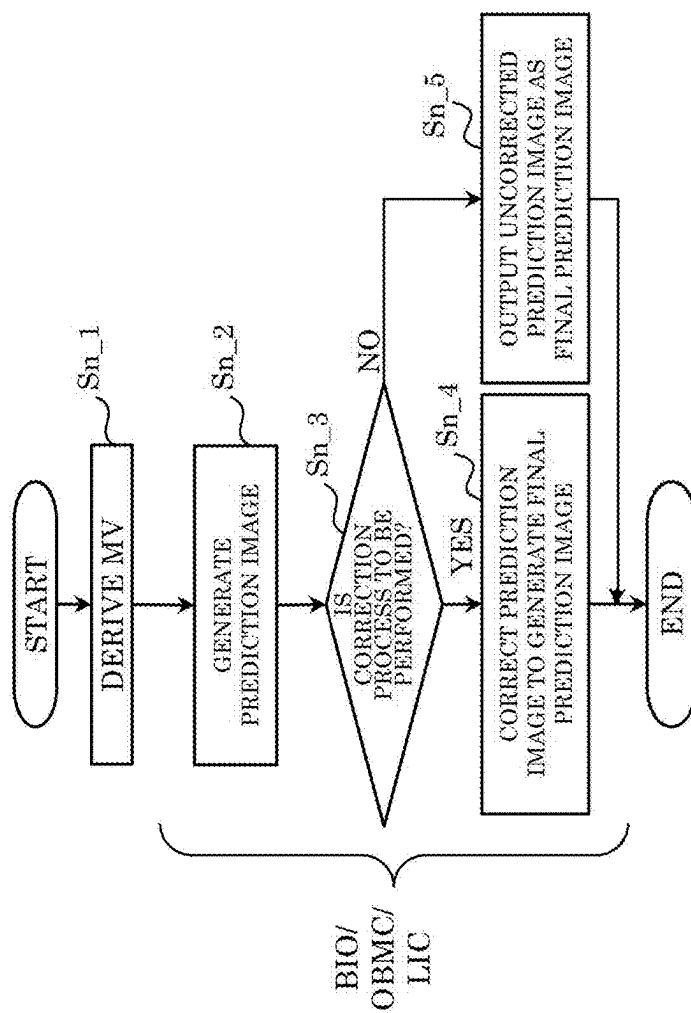


FIG. 61

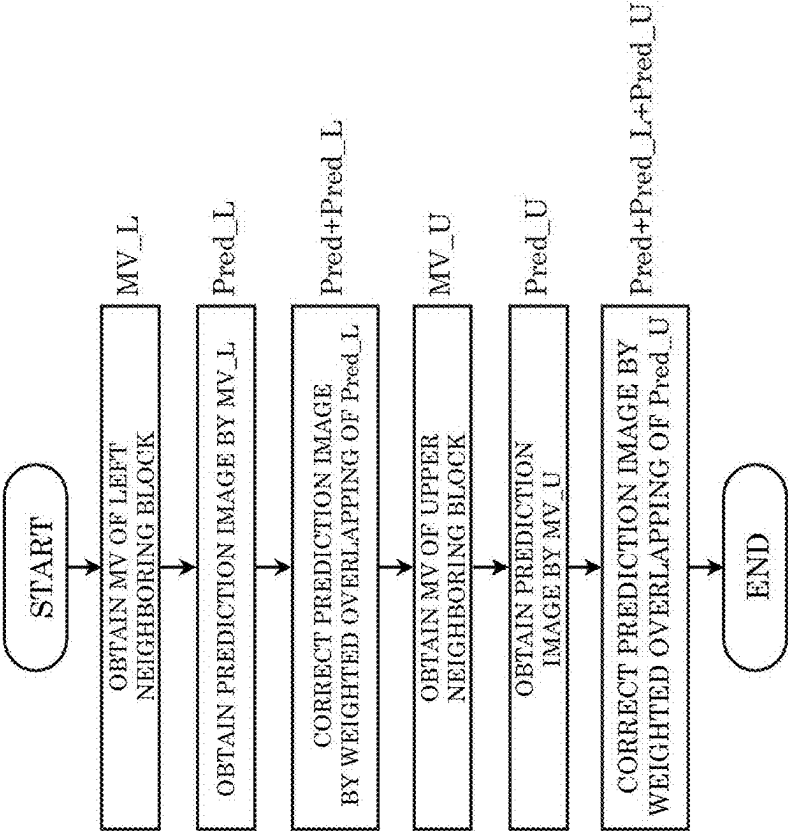


FIG. 62

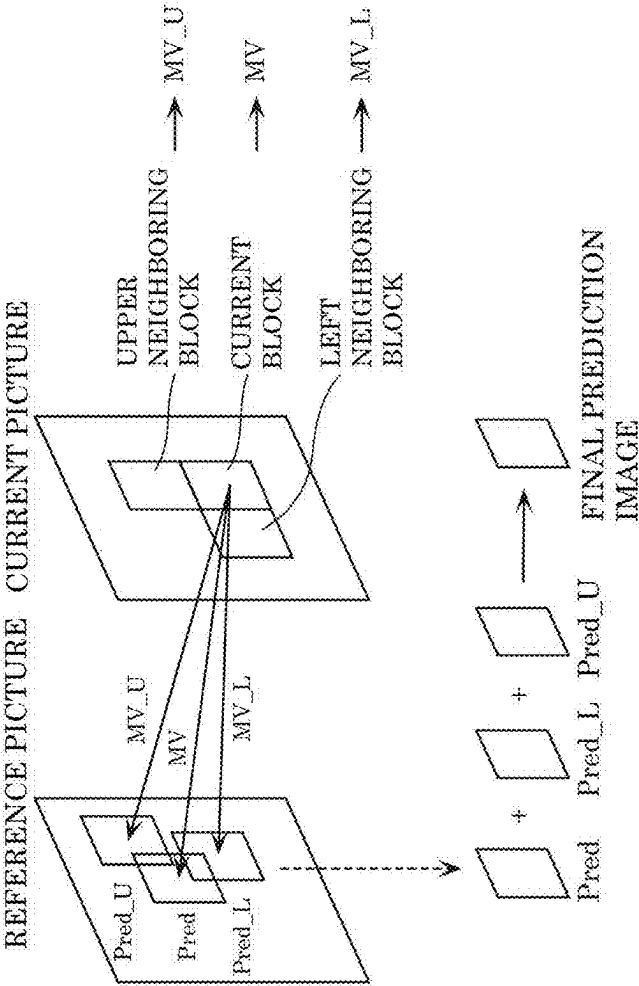


FIG. 63

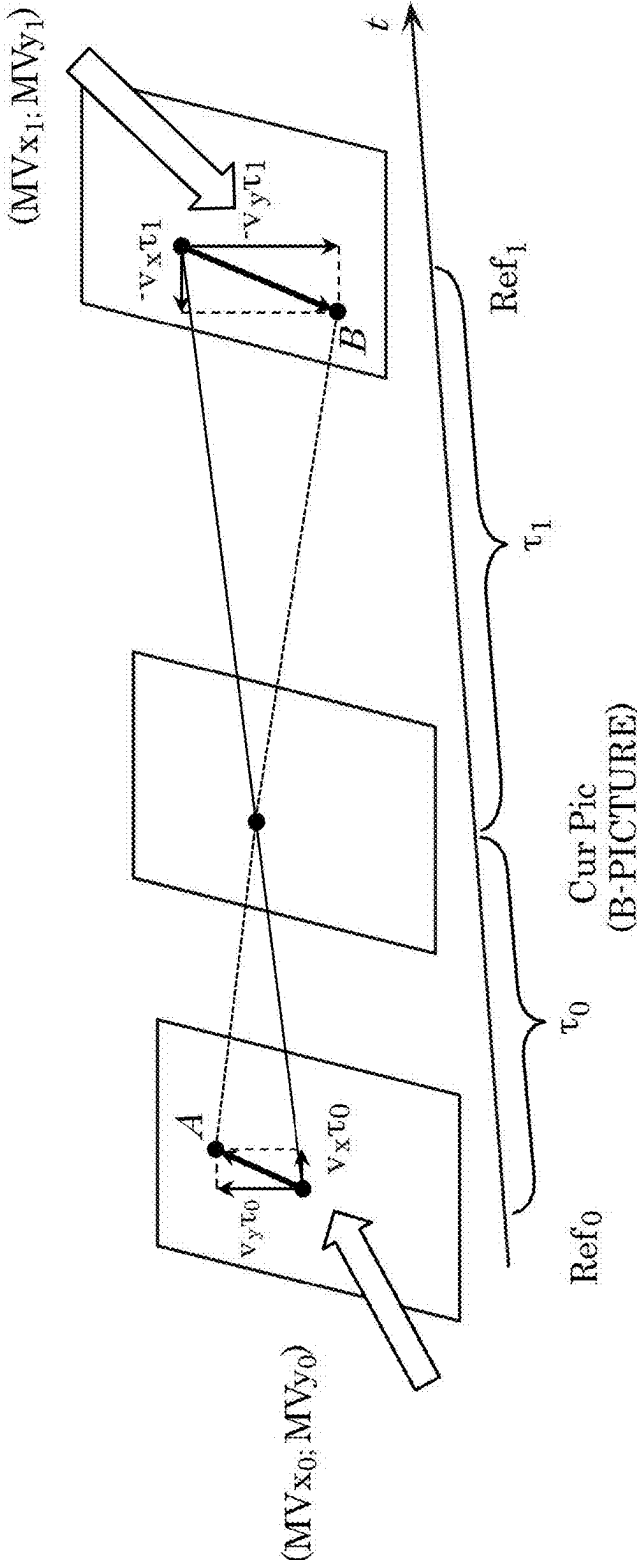


FIG. 64

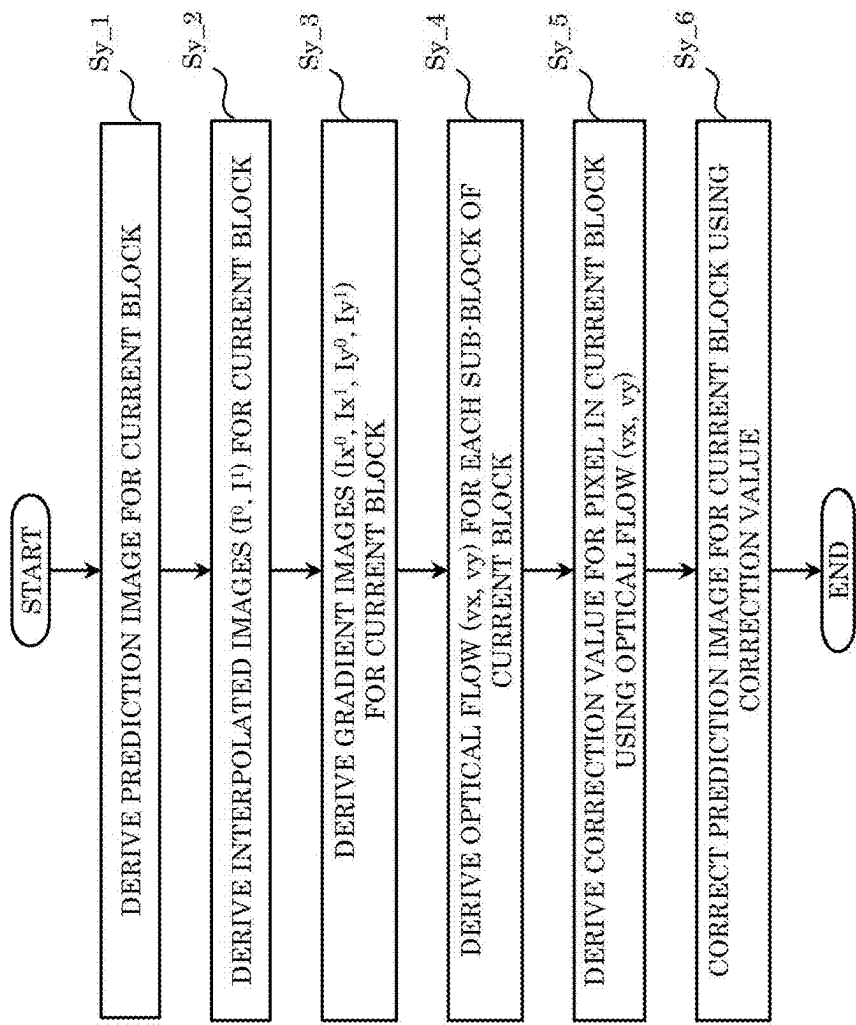


FIG. 65

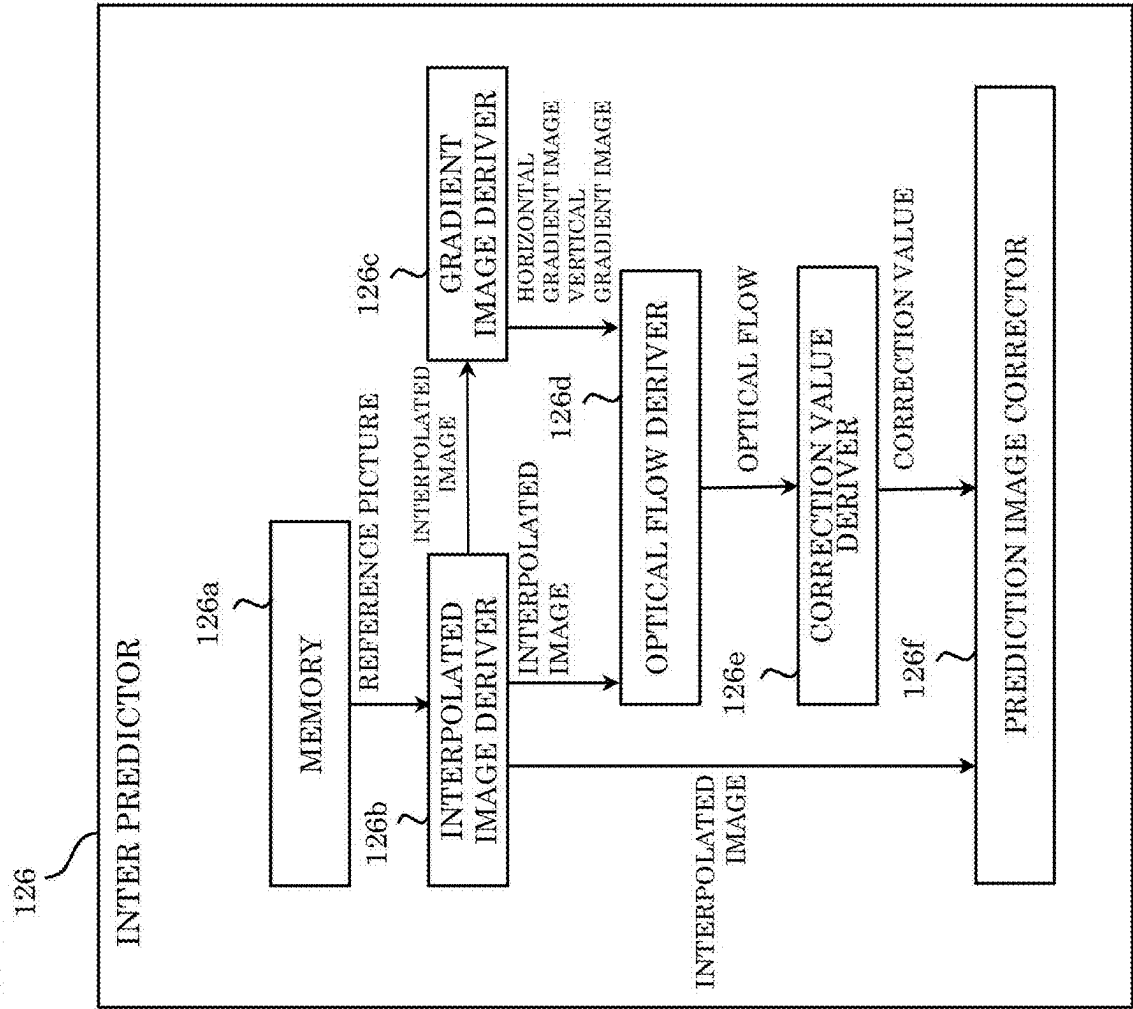


FIG. 66A

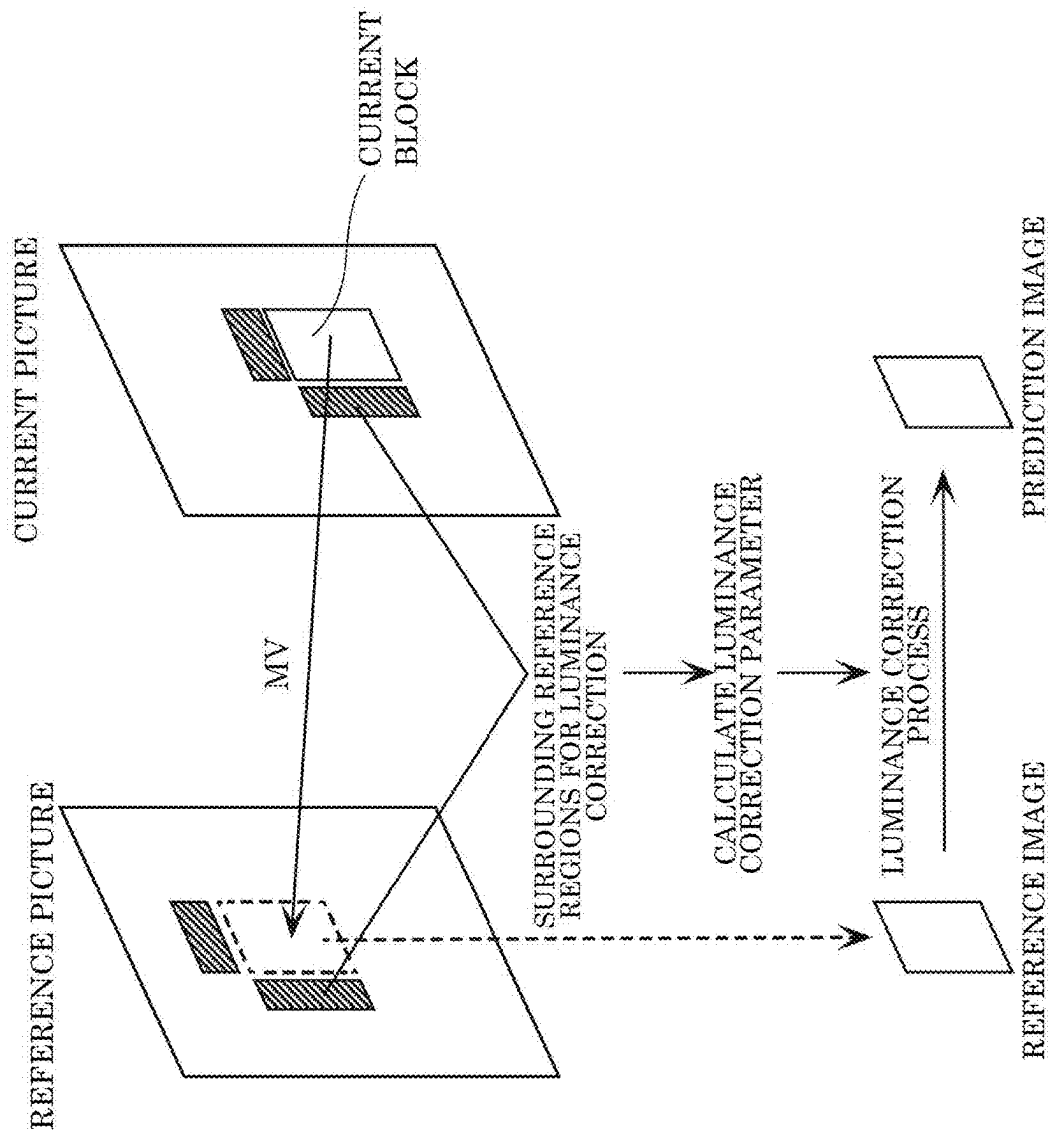


FIG. 66B

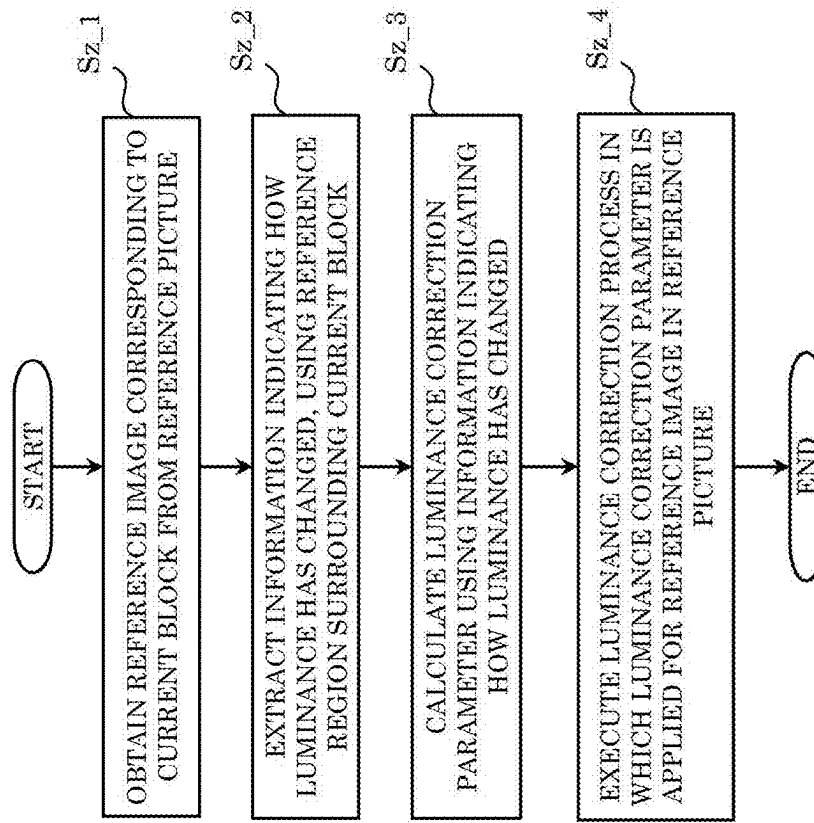


FIG. 67

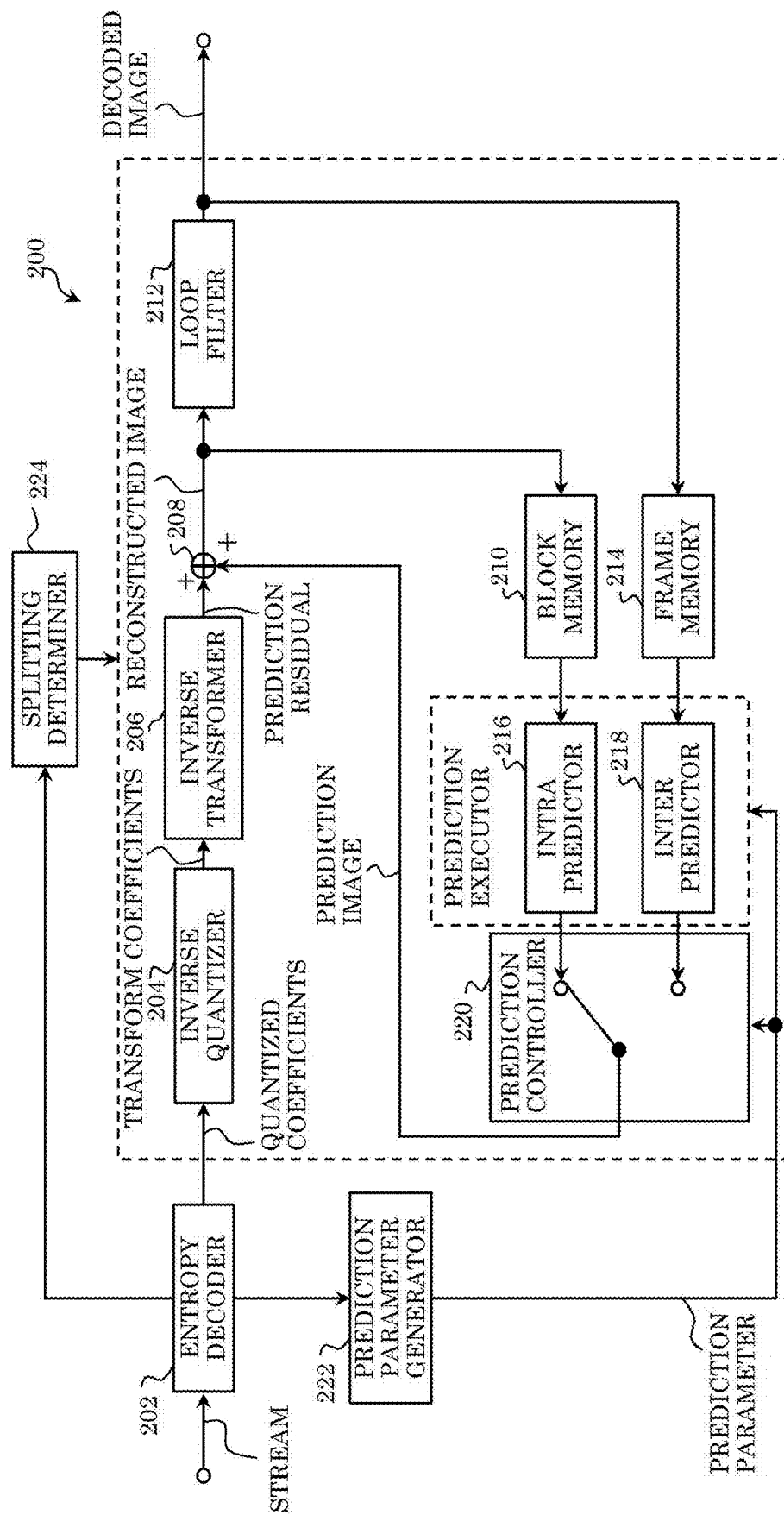


FIG.68

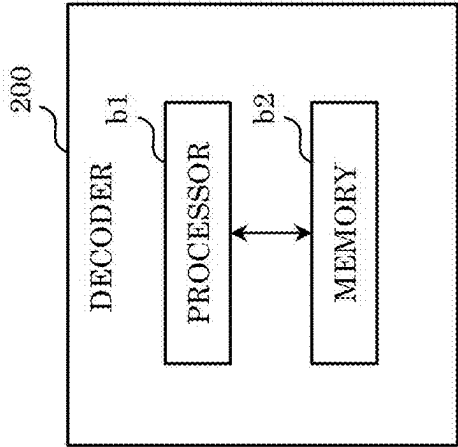


FIG. 69

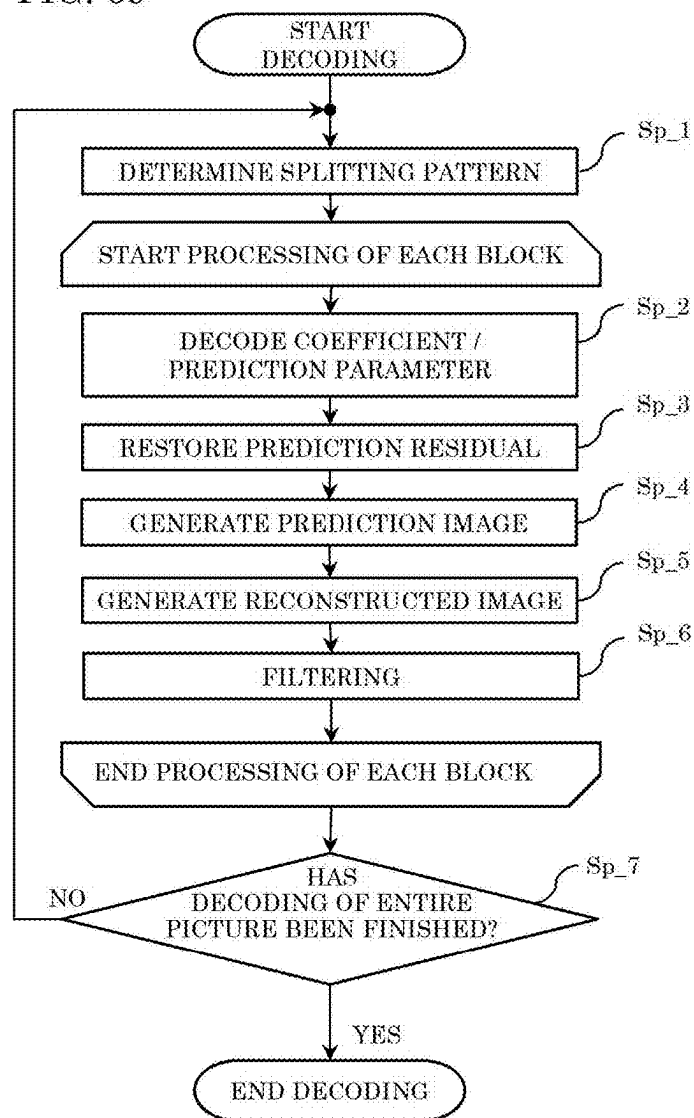


FIG. 70

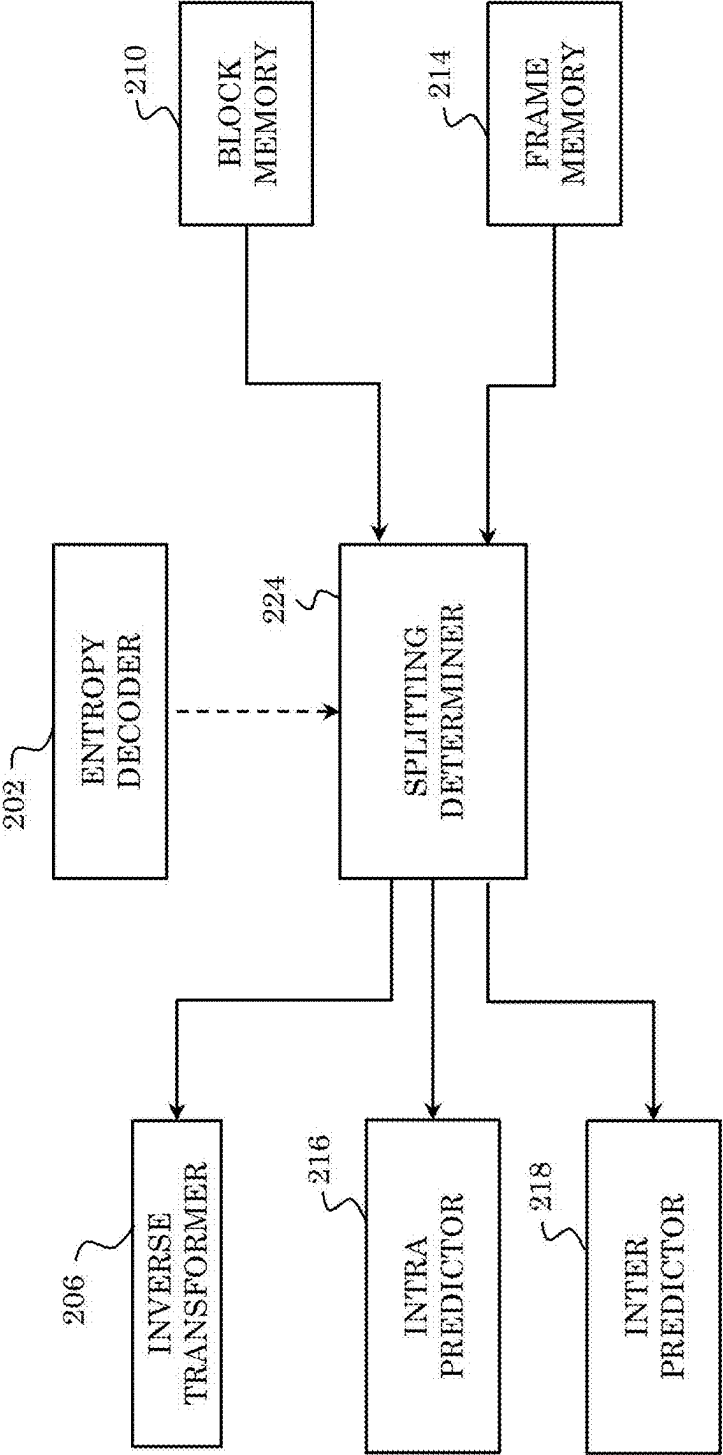


FIG. 71

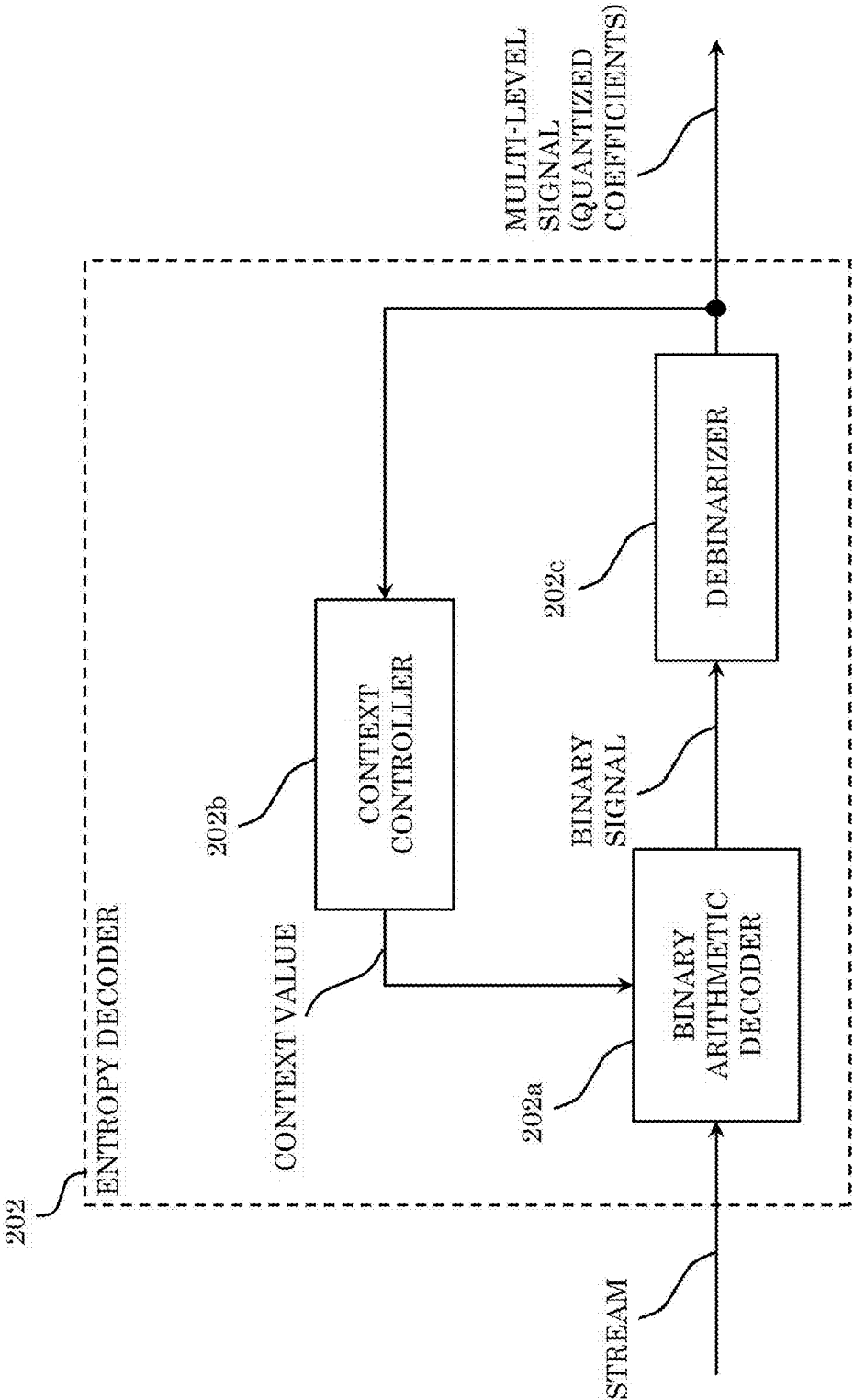


FIG. 72

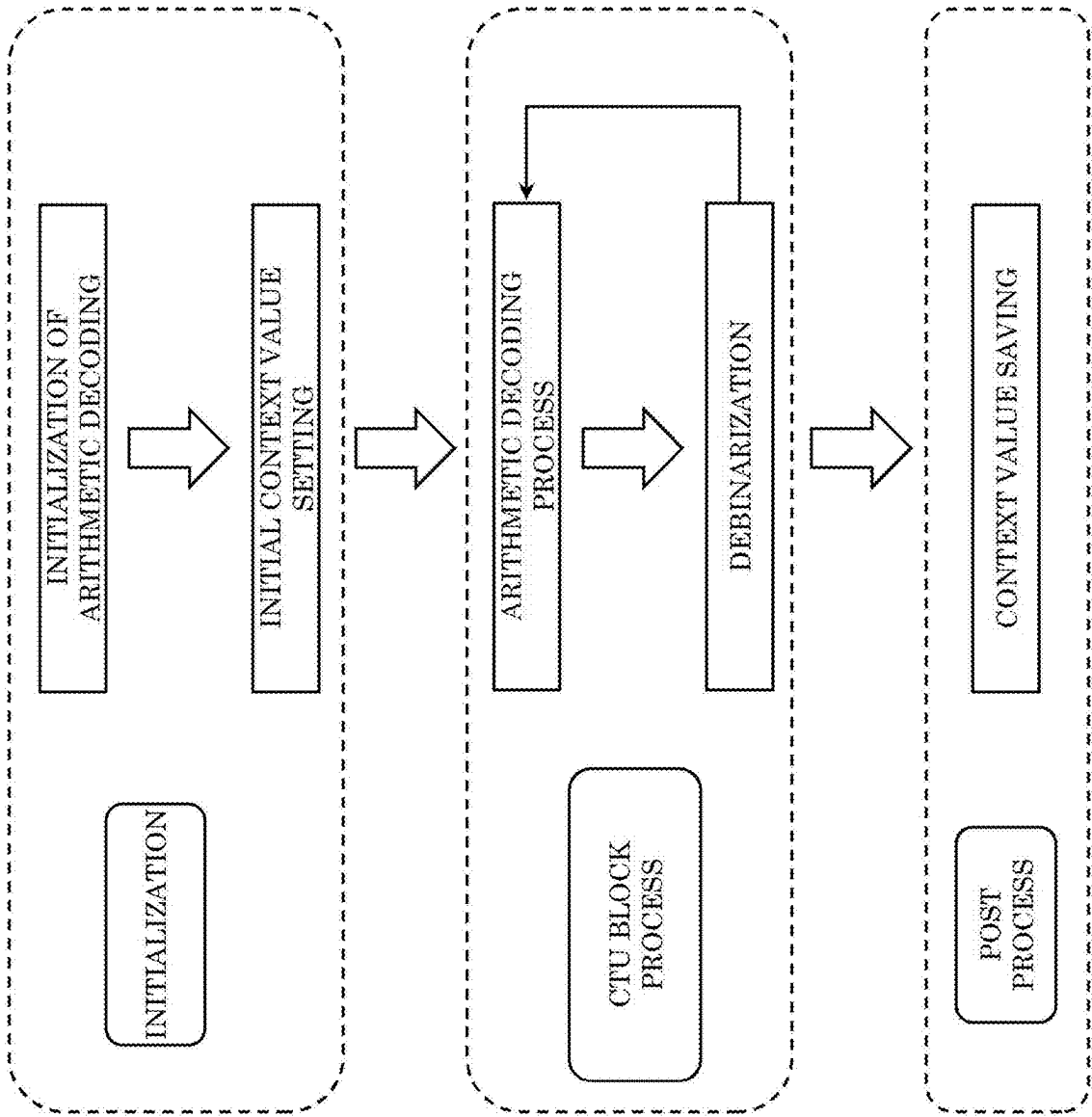


FIG. 73

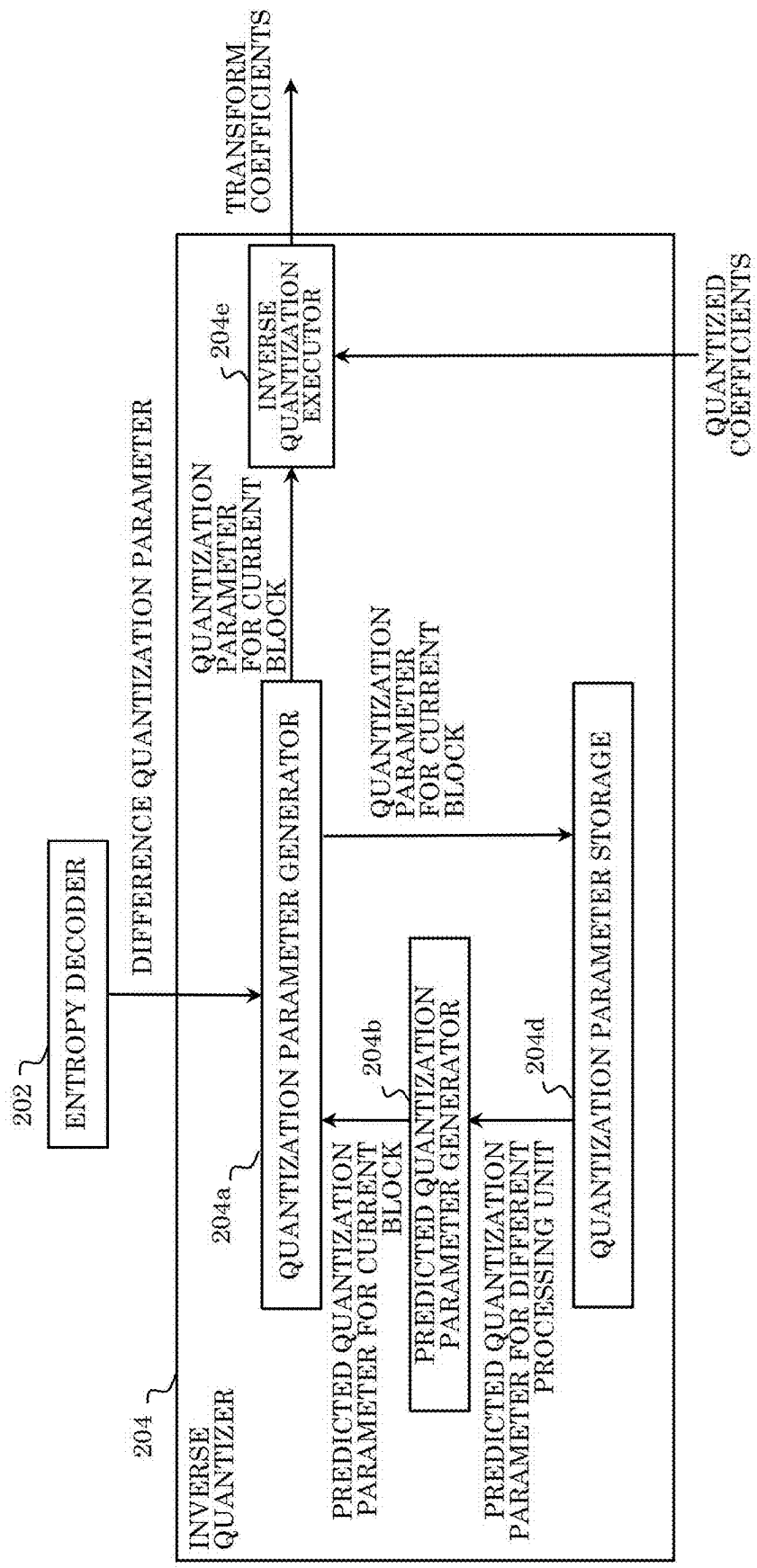


FIG. 74

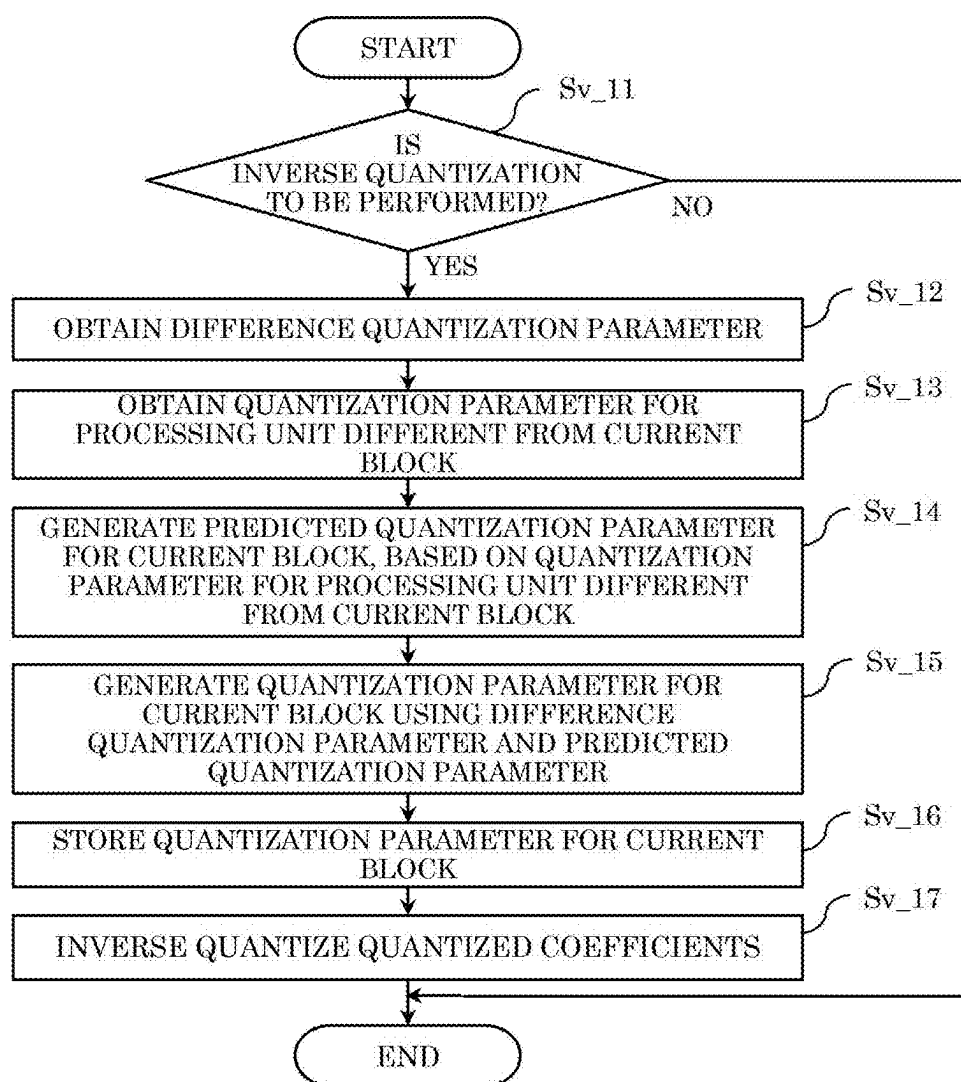


FIG. 75

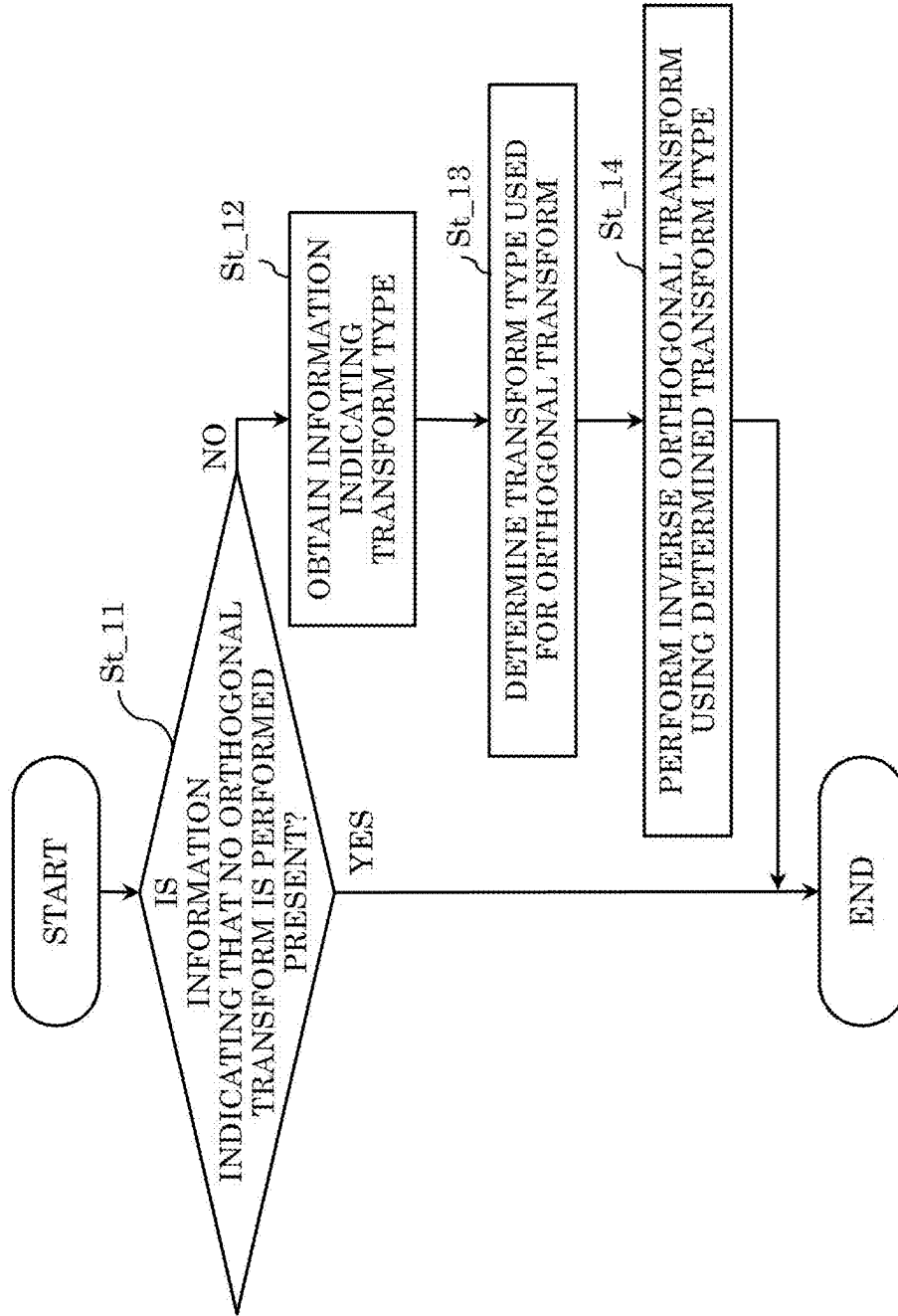


FIG. 76

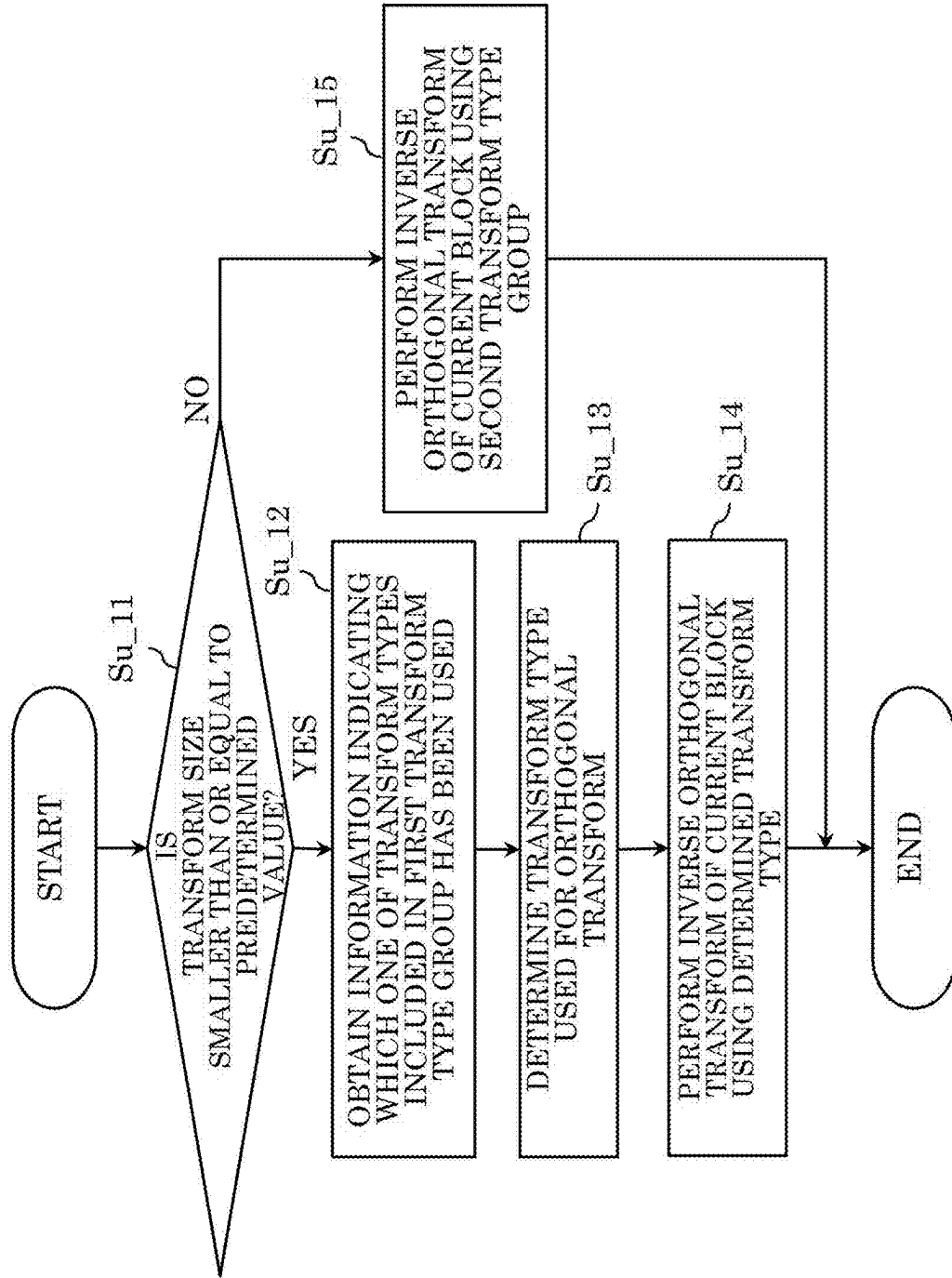


FIG. 77

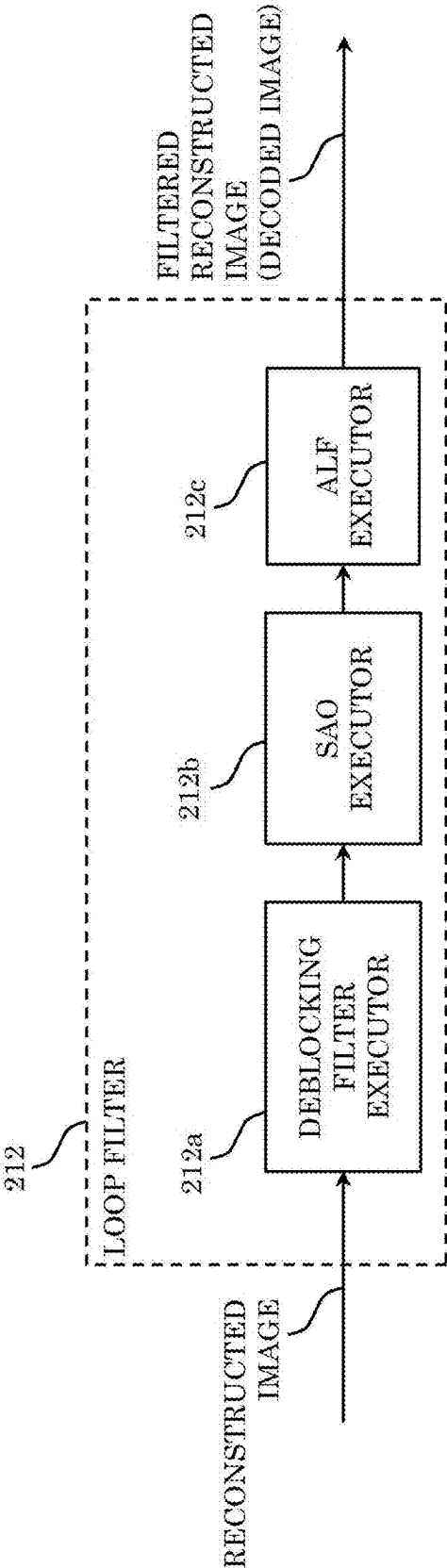


FIG. 78

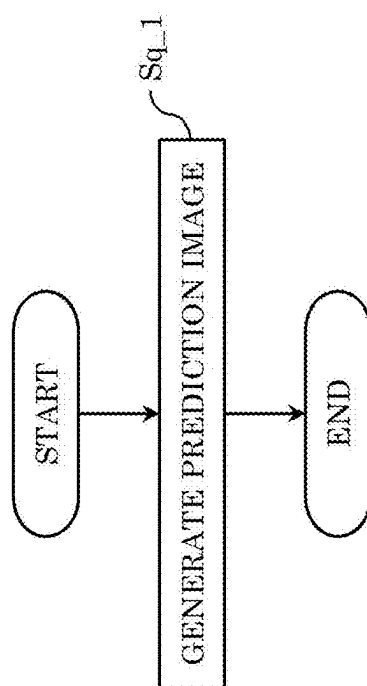


FIG. 79

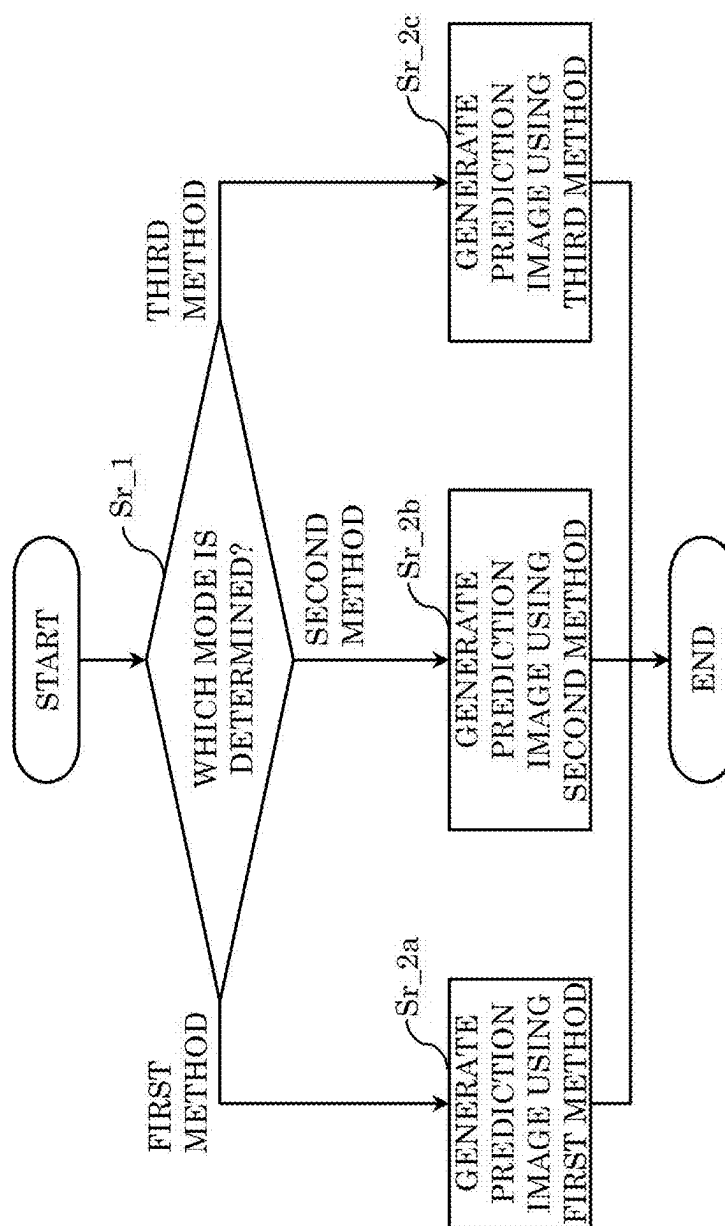


FIG. 80A

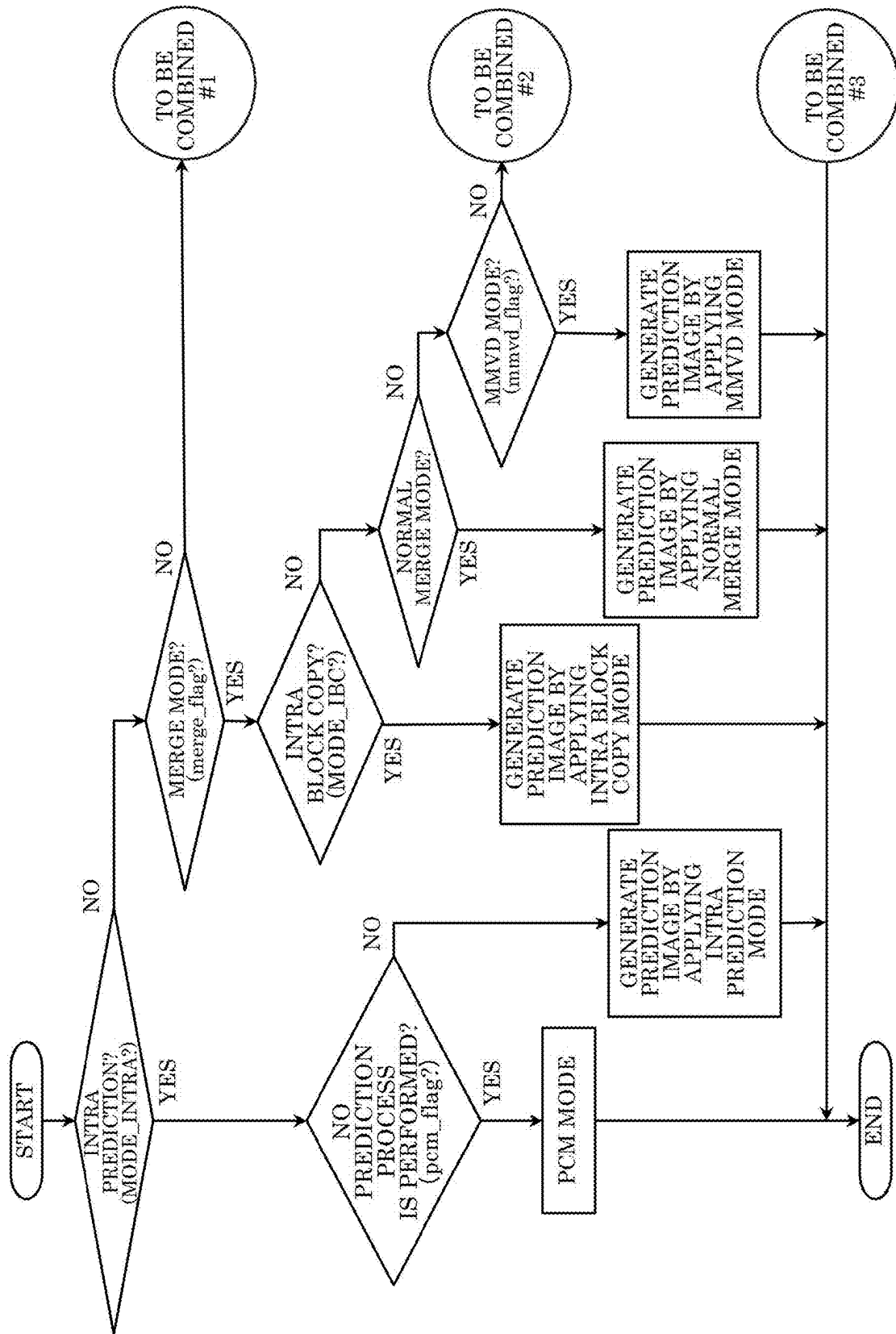


FIG. 80B

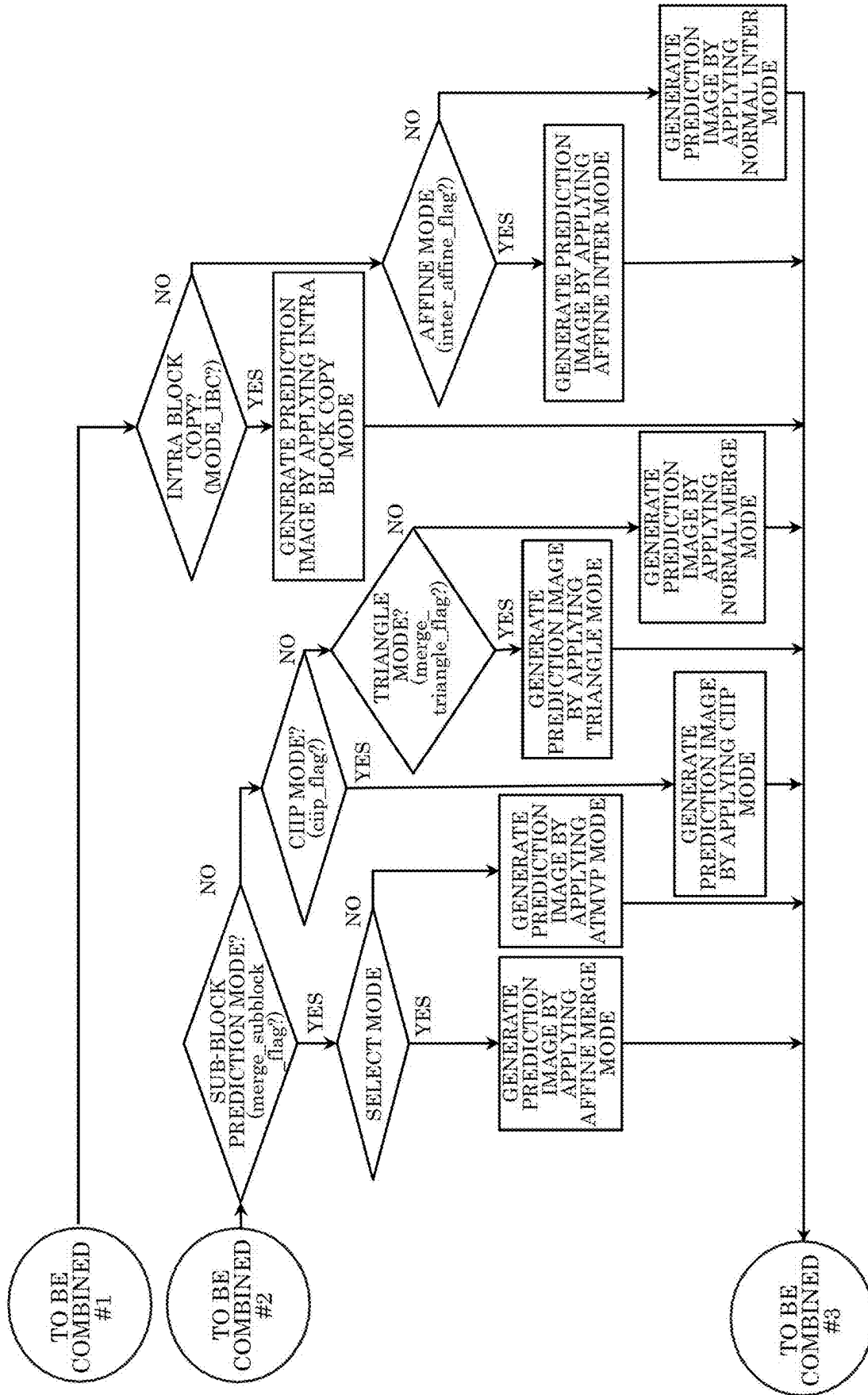


FIG. 81

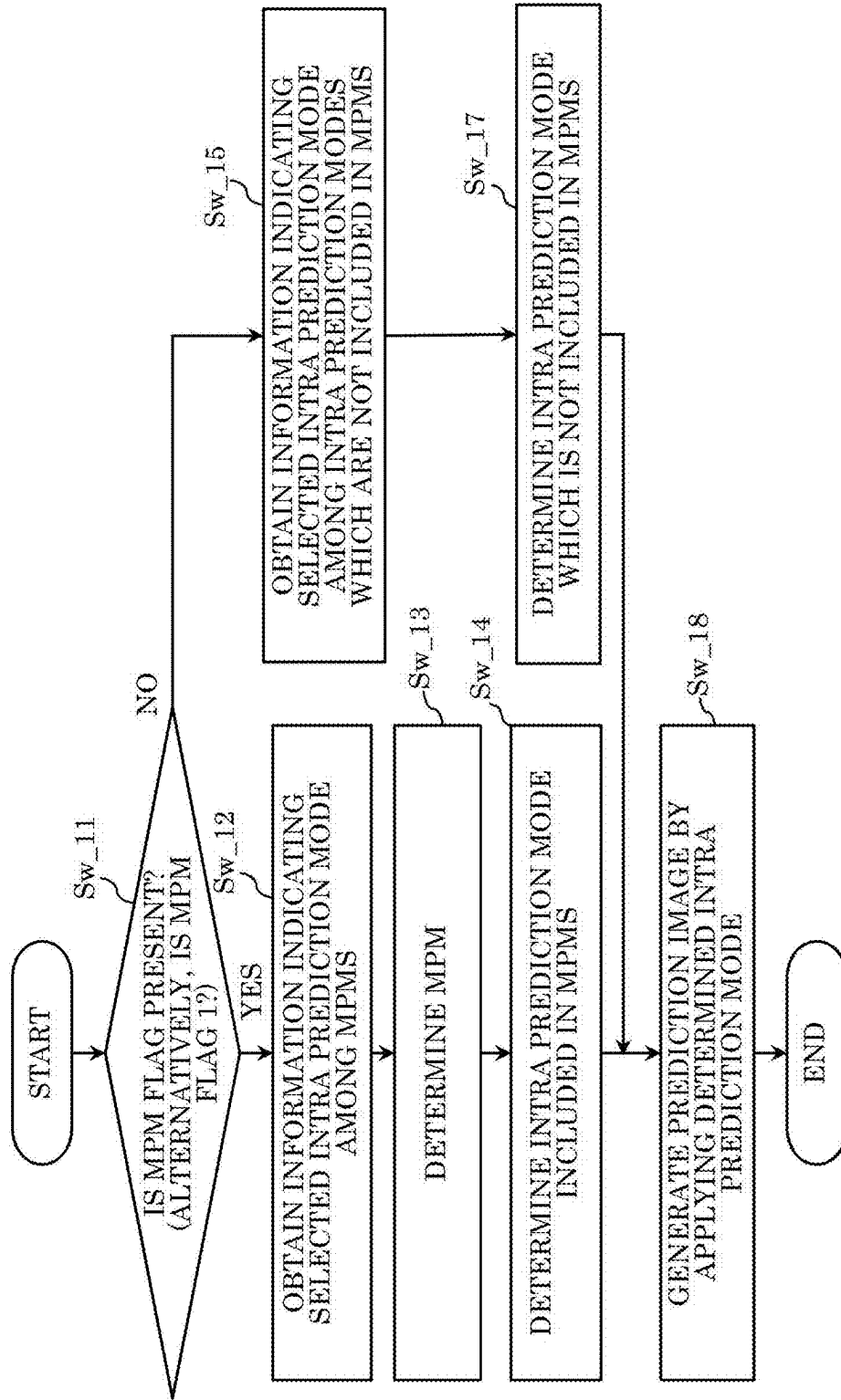


FIG. 82

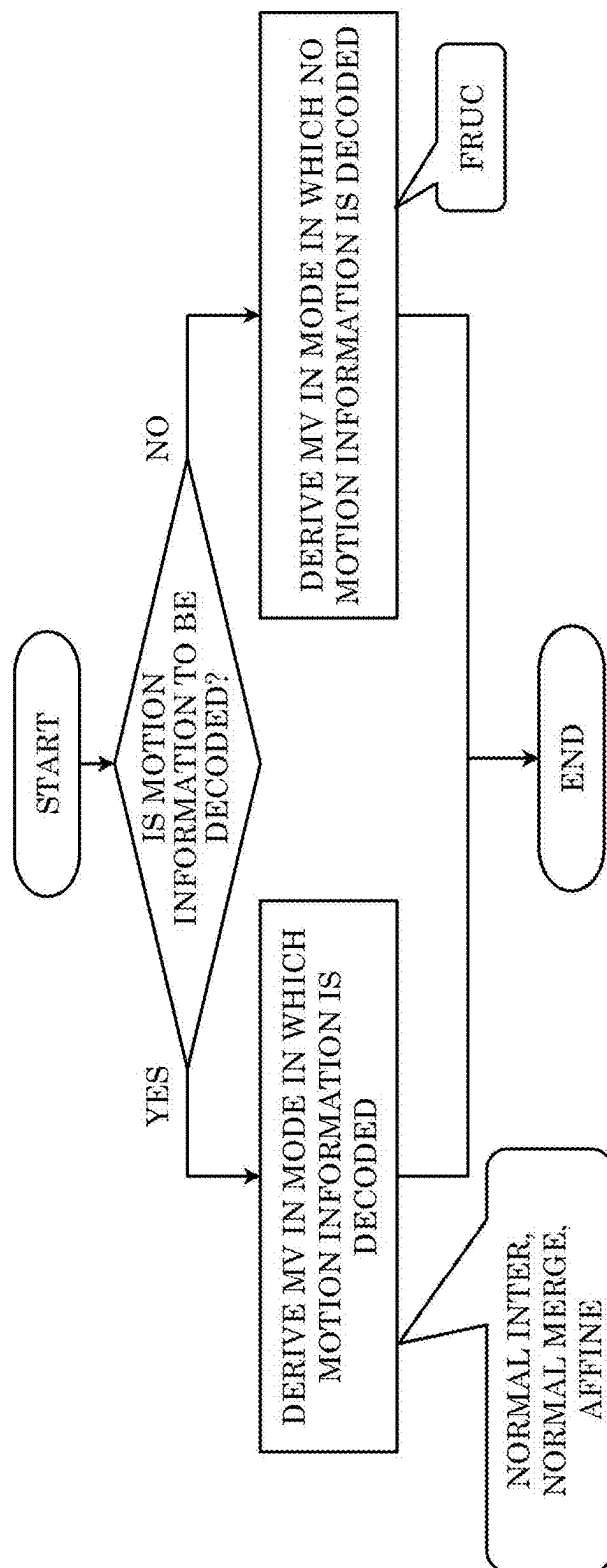


FIG. 83

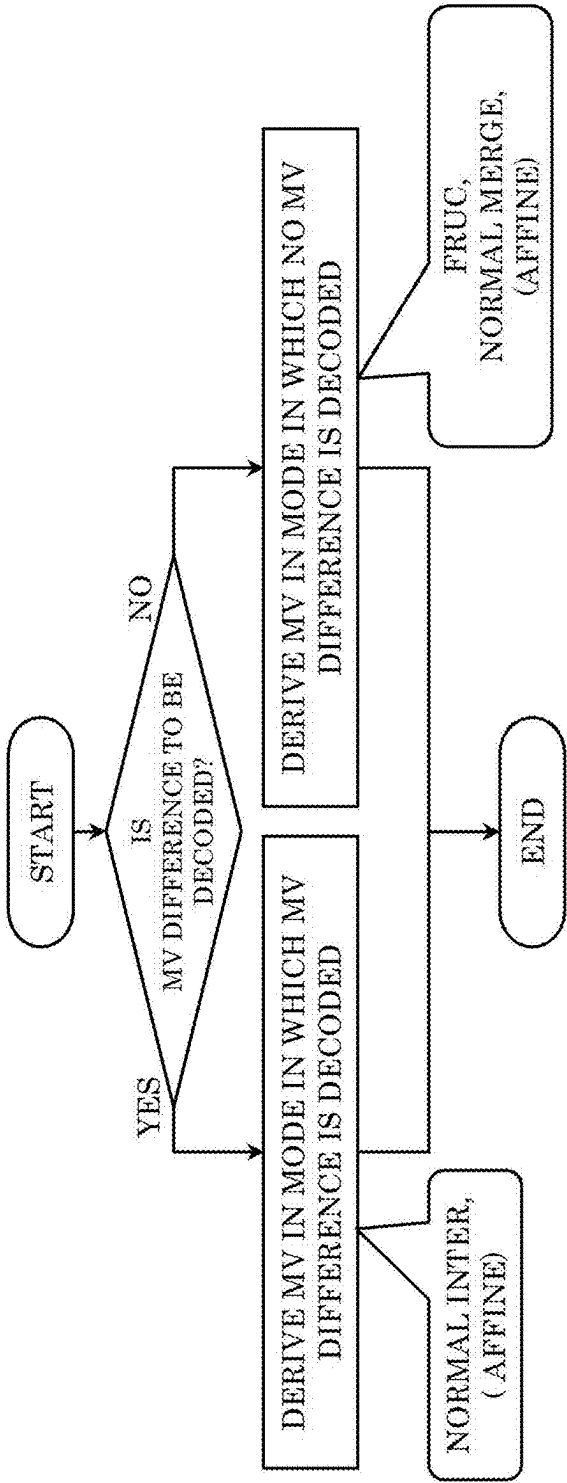


FIG. 84

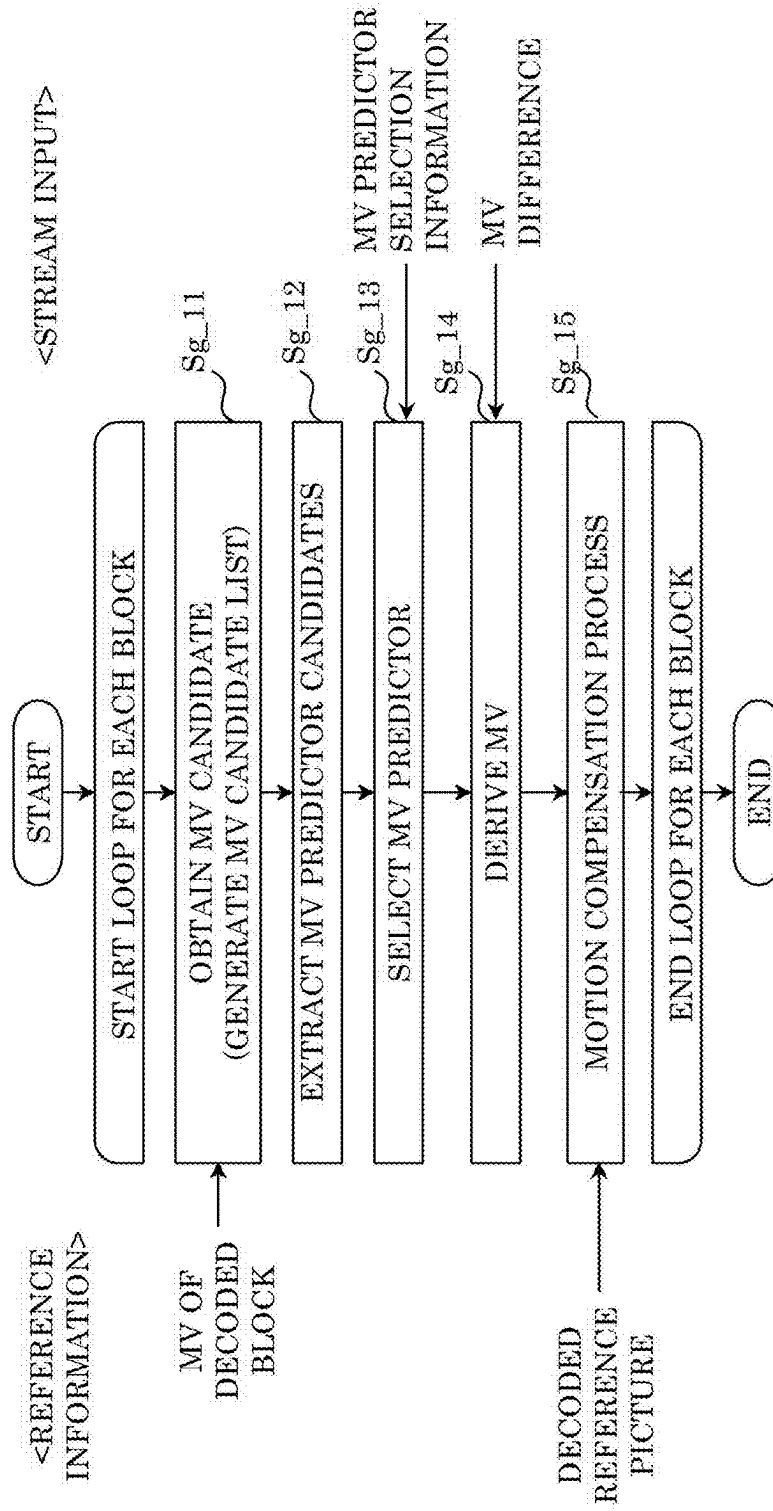


FIG. 85

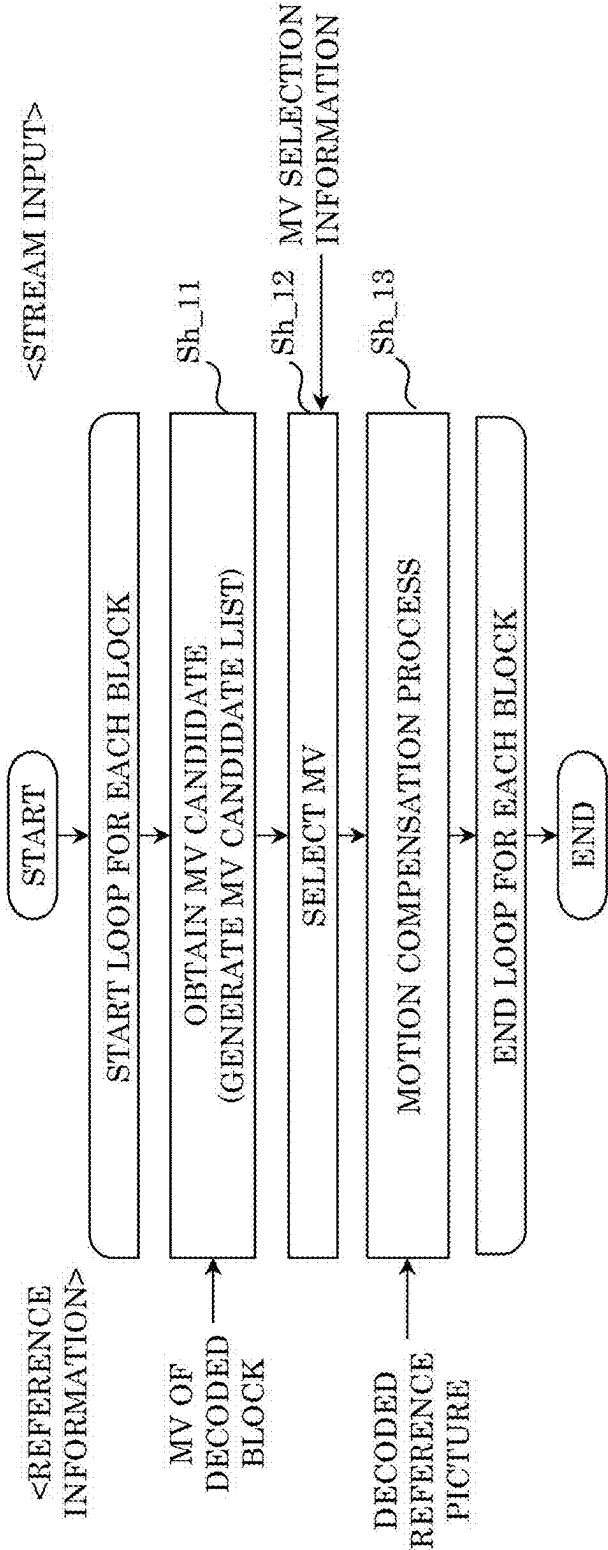


FIG. 86

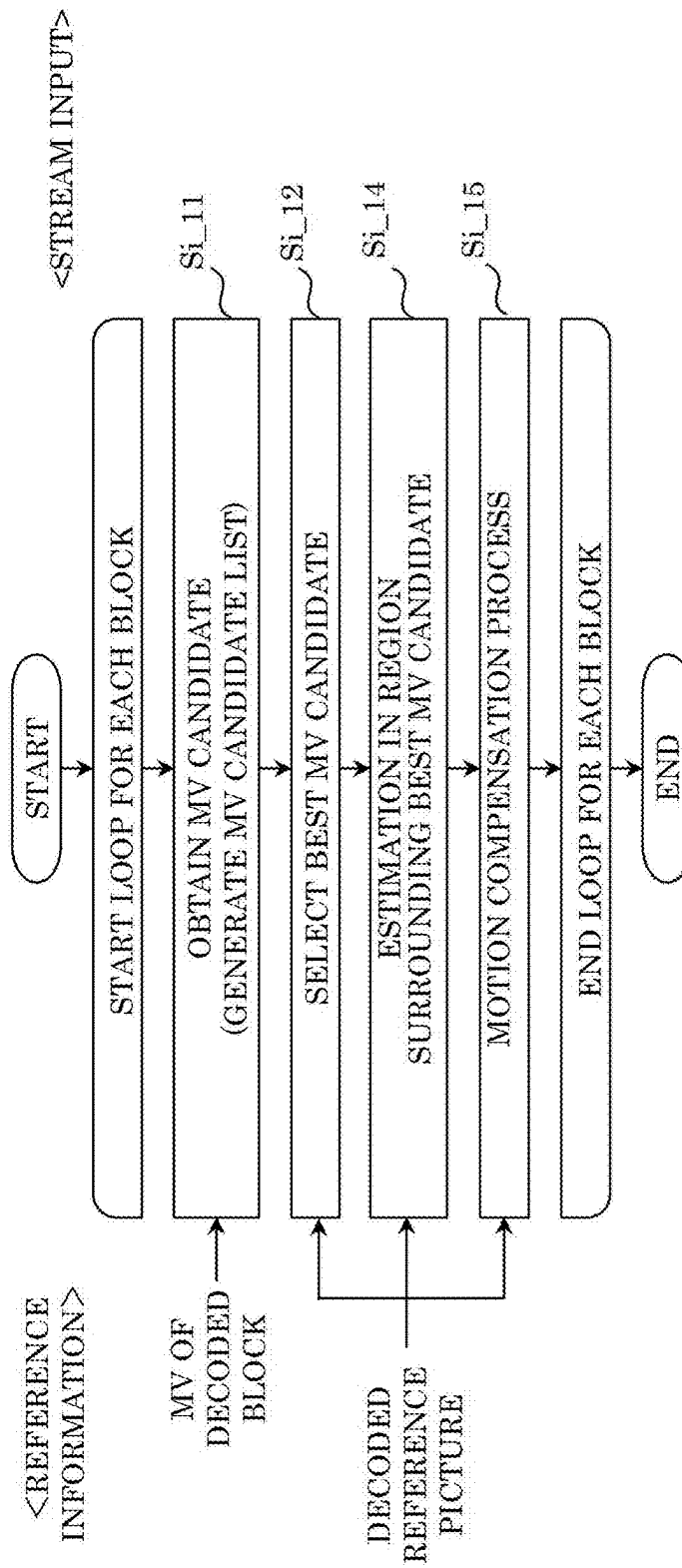


FIG. 87

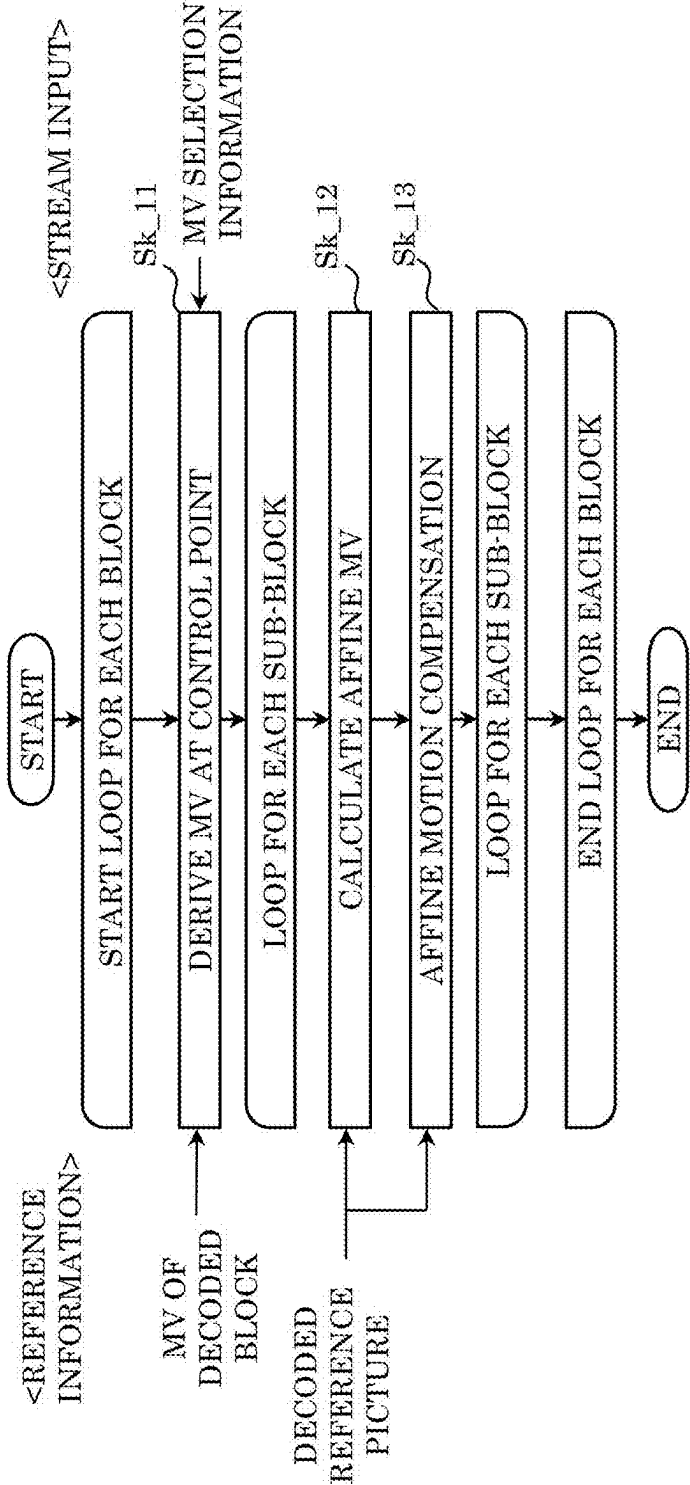


FIG. 88

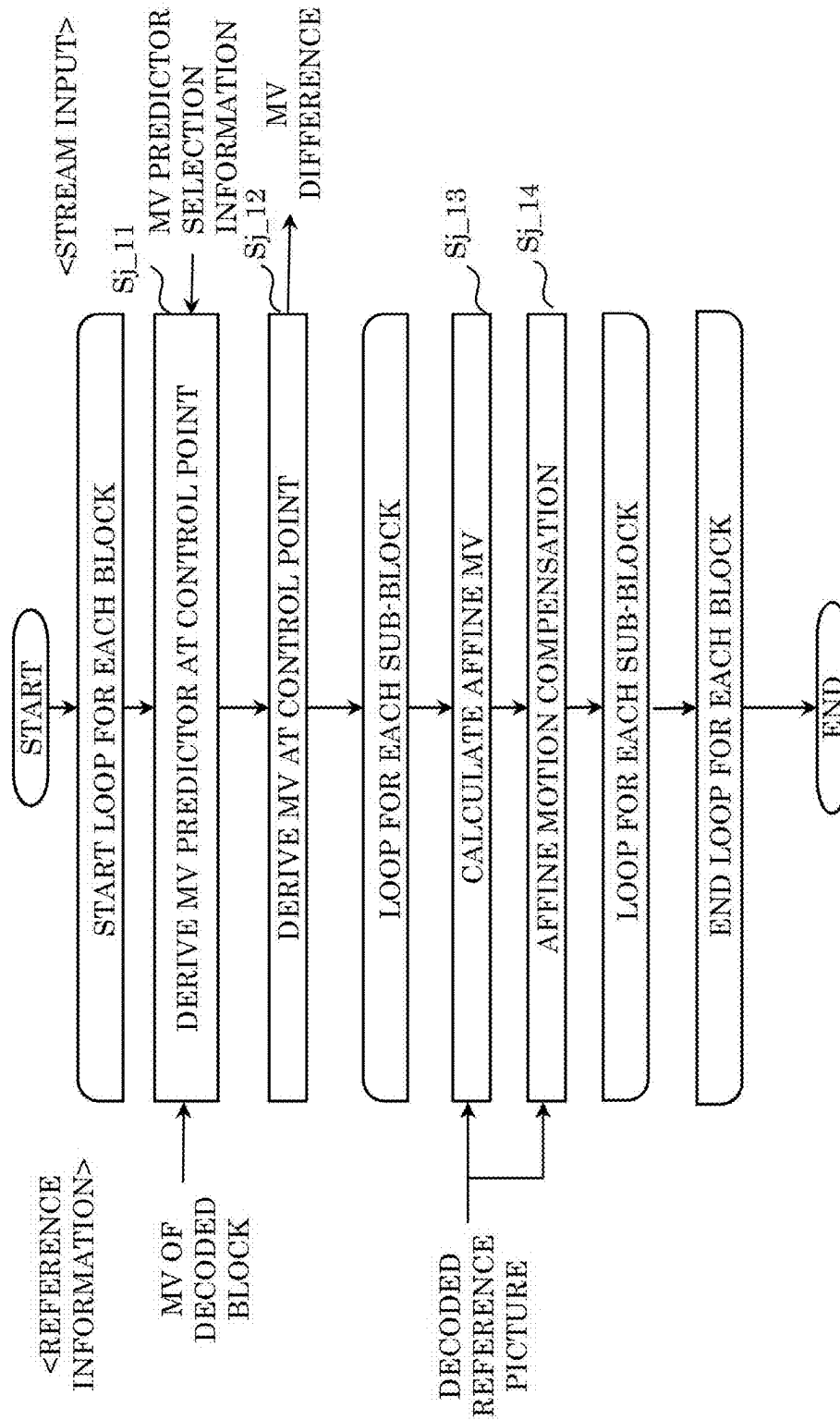


FIG. 89

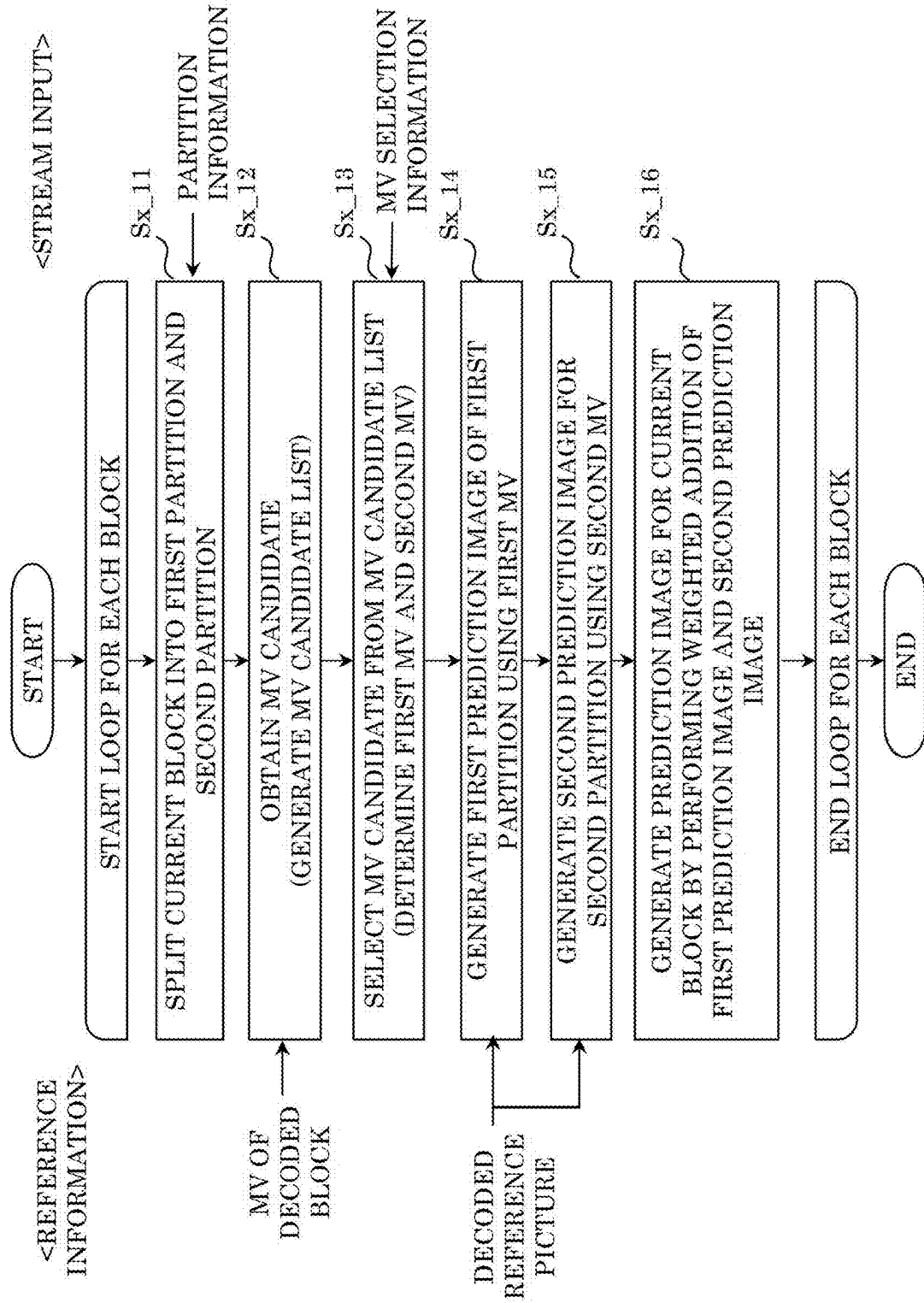


FIG. 90

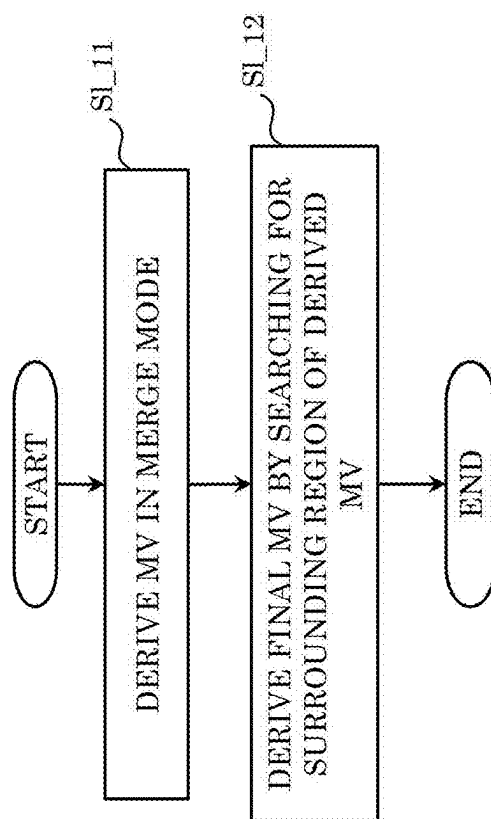


FIG. 91

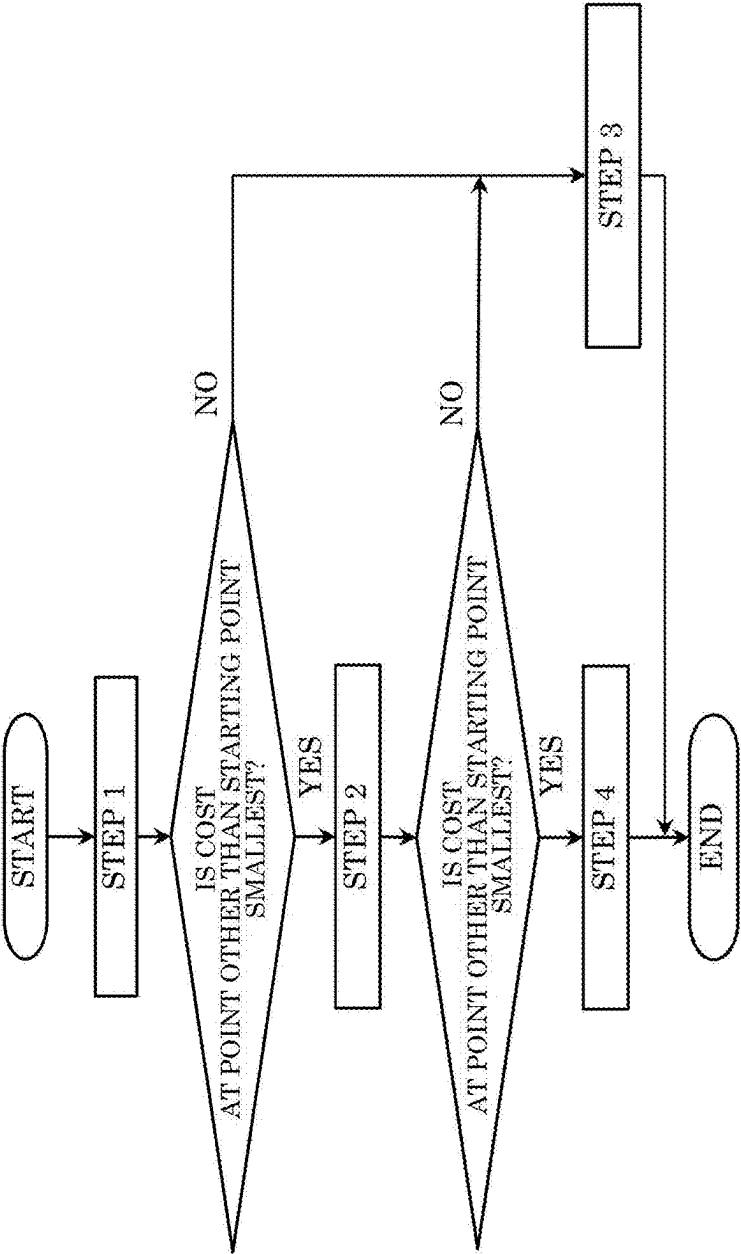


FIG. 92

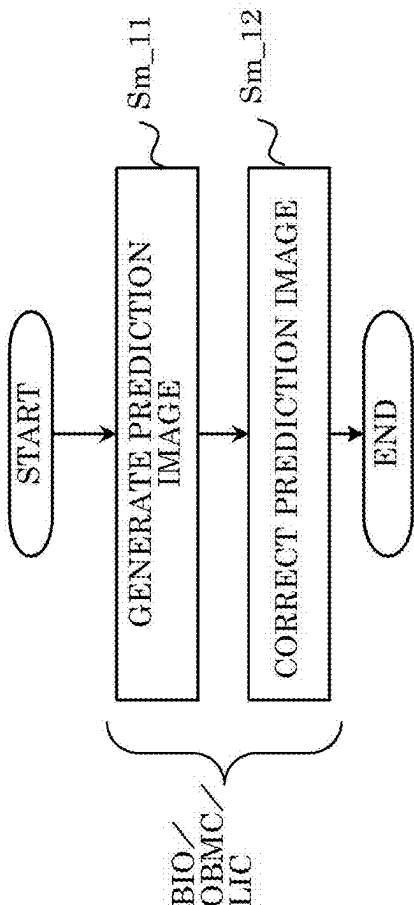


FIG. 93

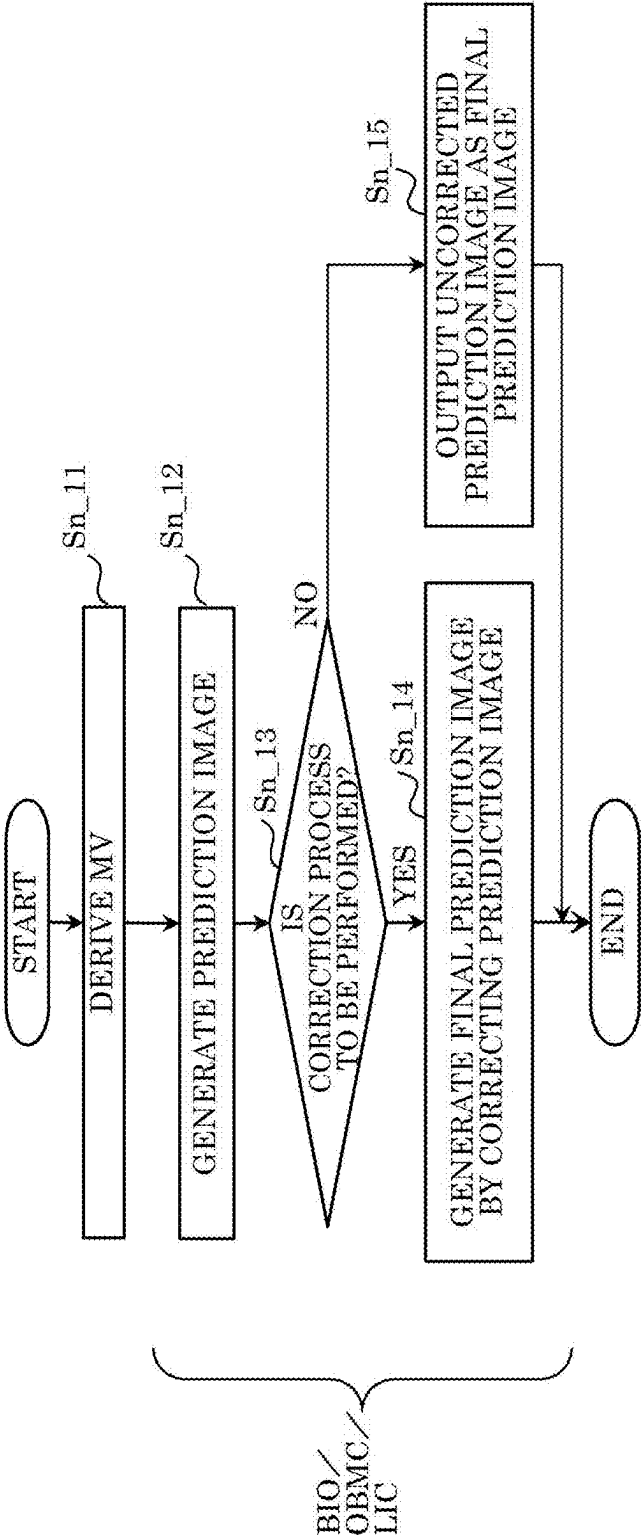


FIG. 94

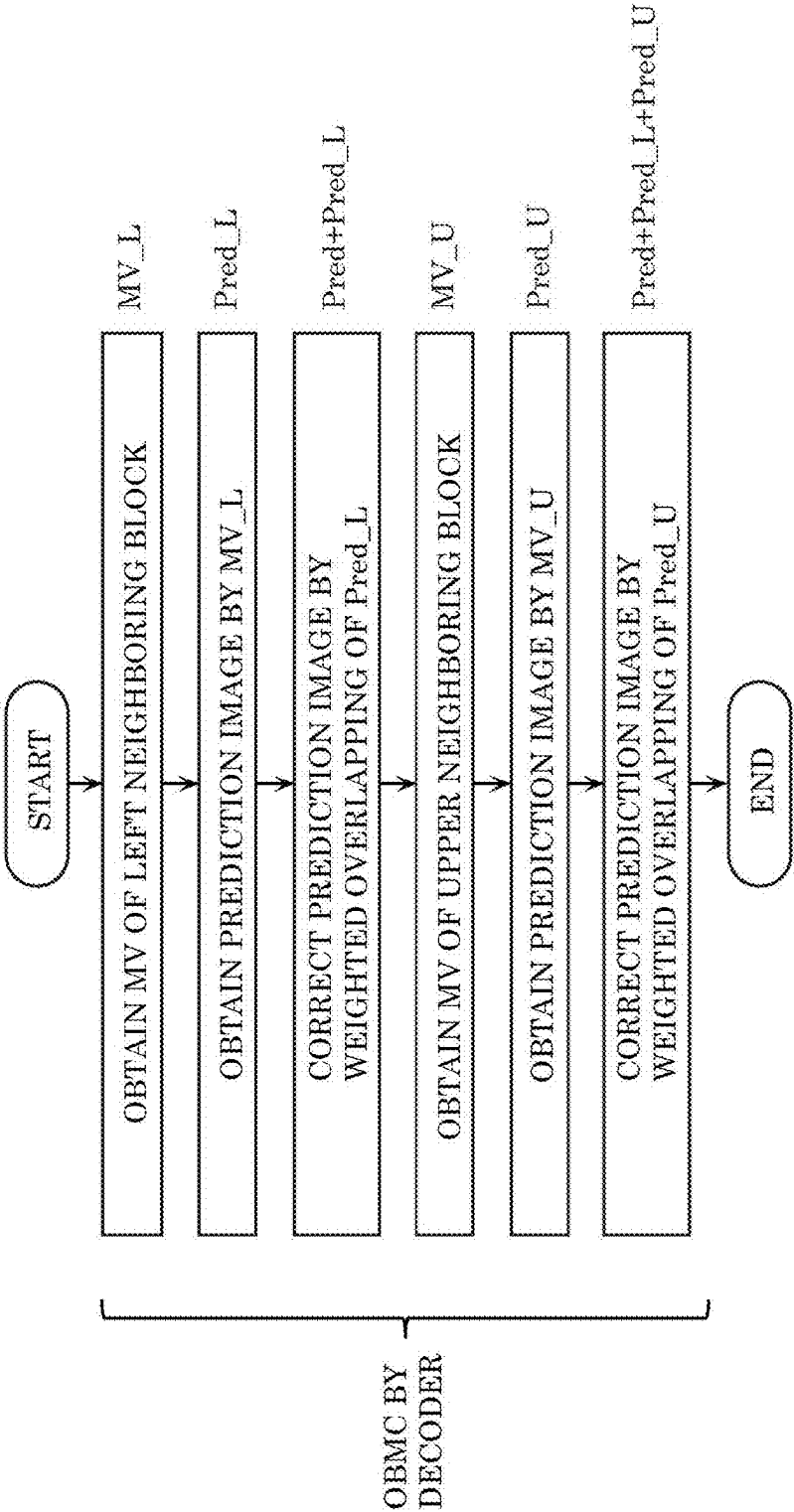


FIG. 95

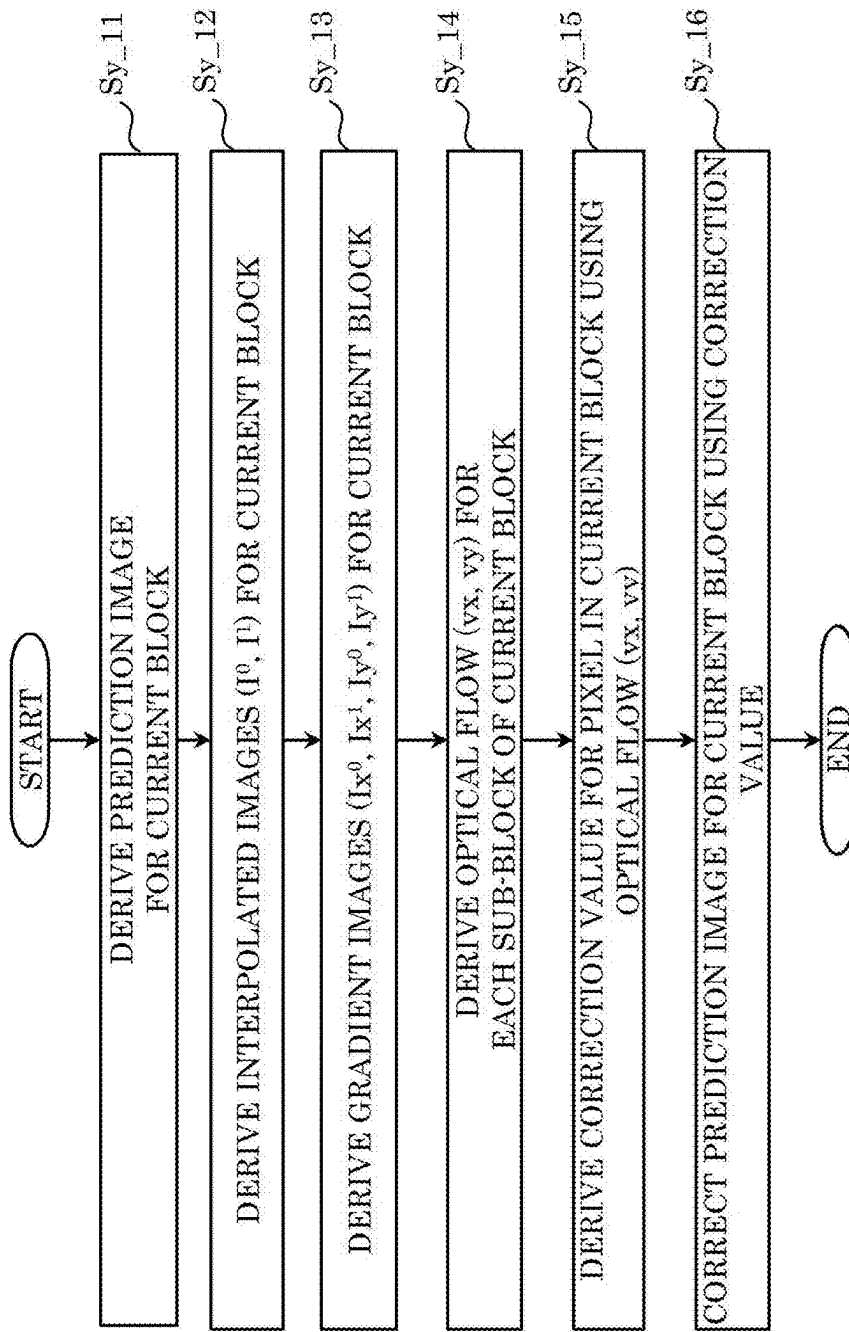


FIG. 96

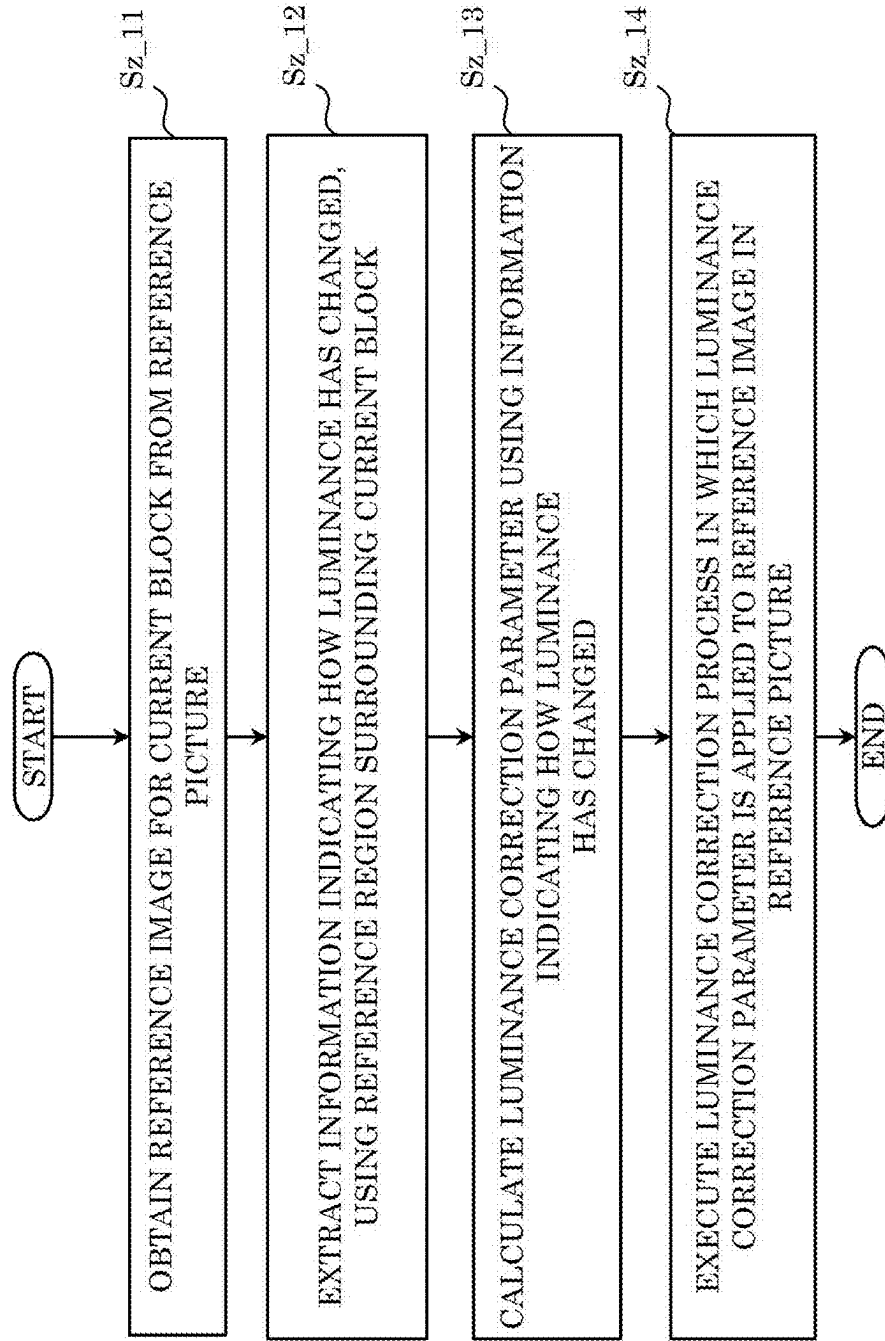


FIG. 97

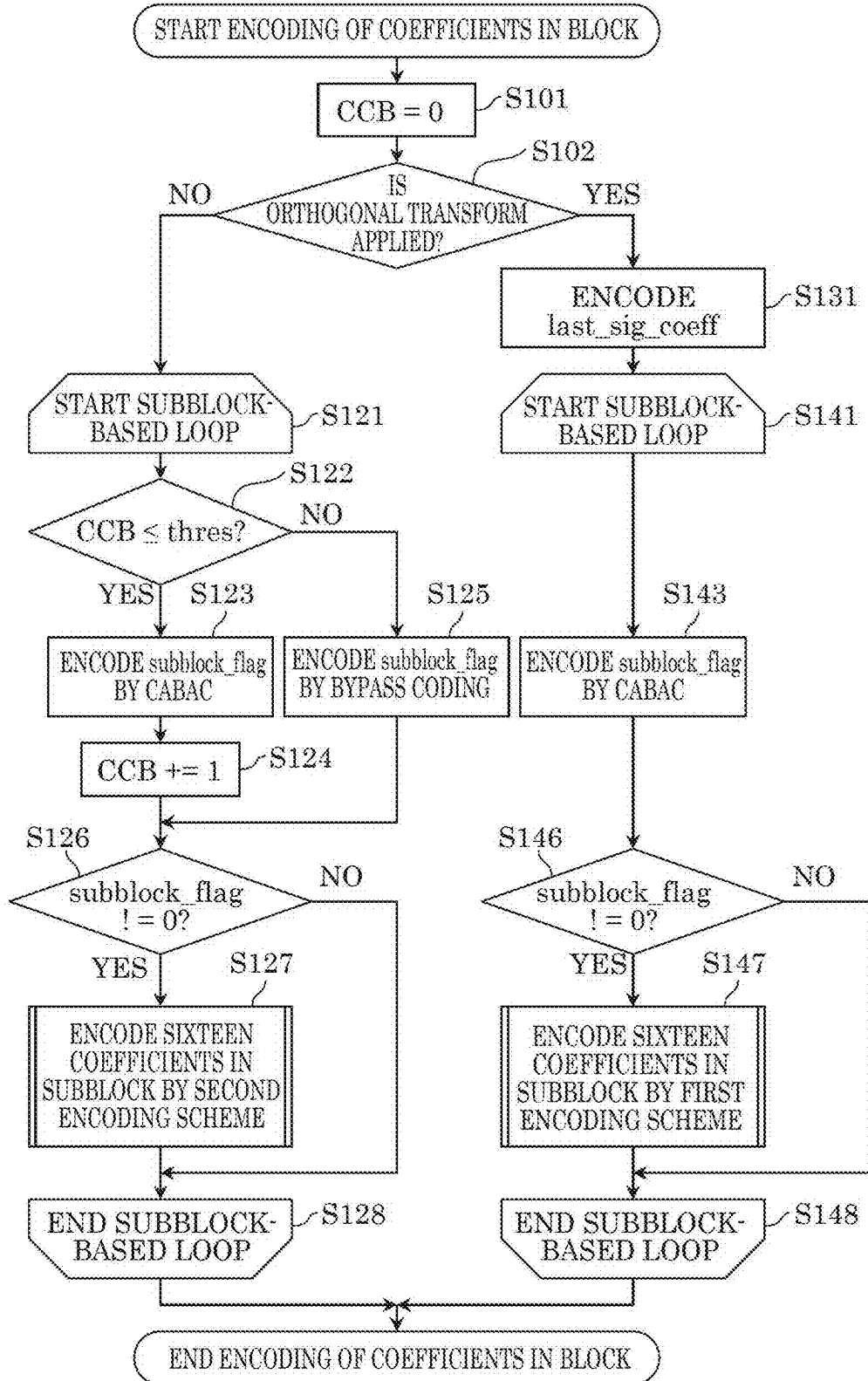


FIG. 98

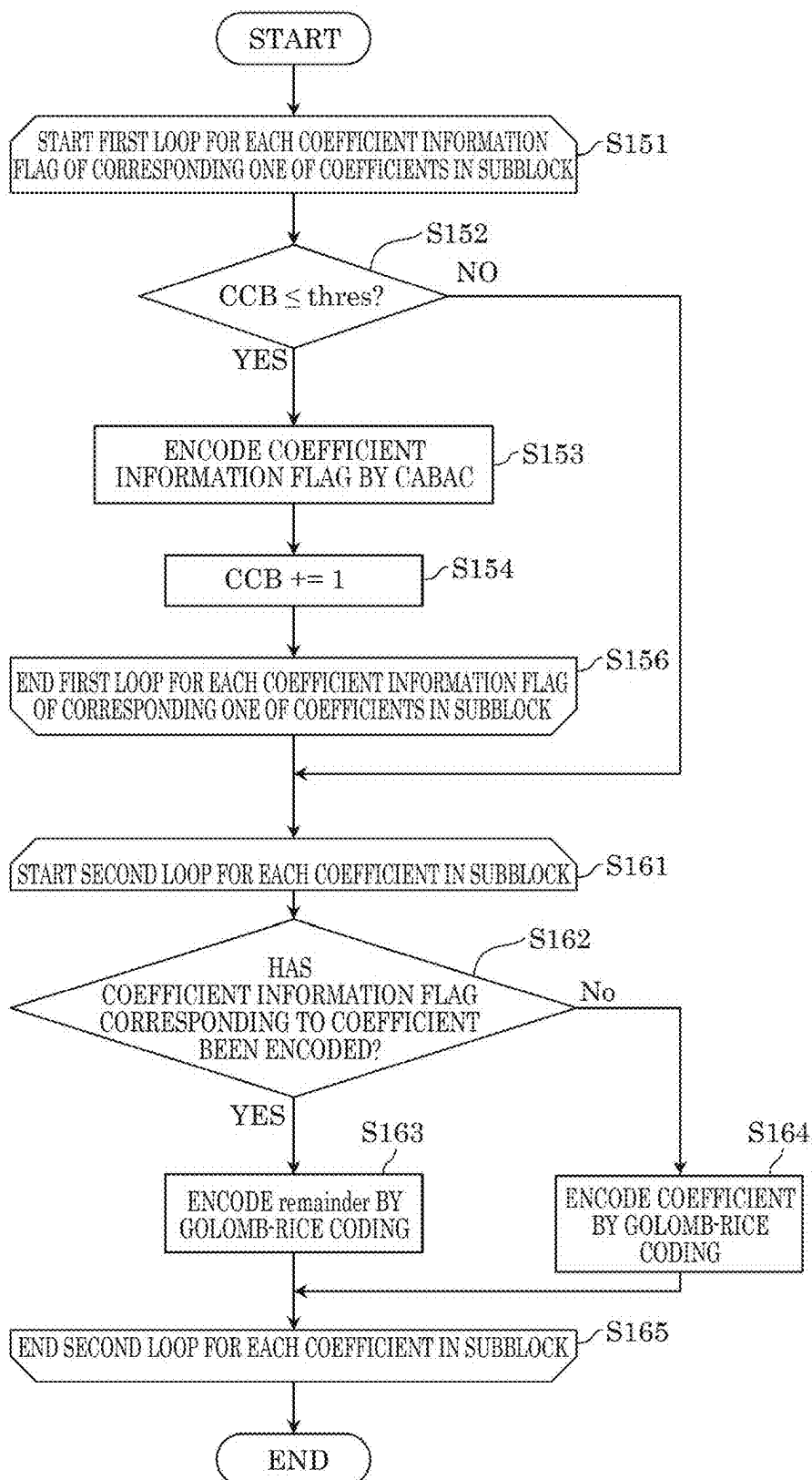


FIG. 99

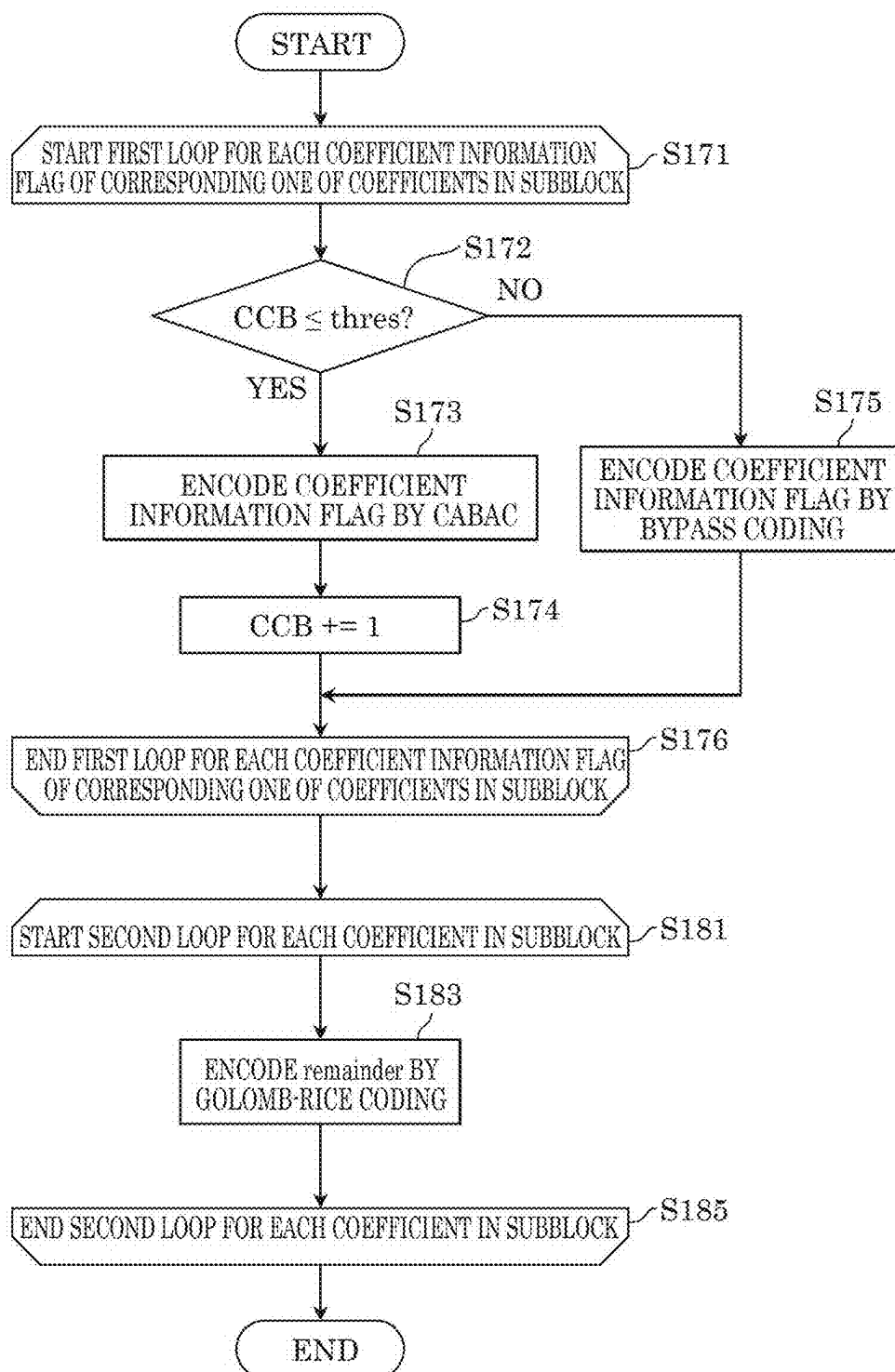


FIG. 100

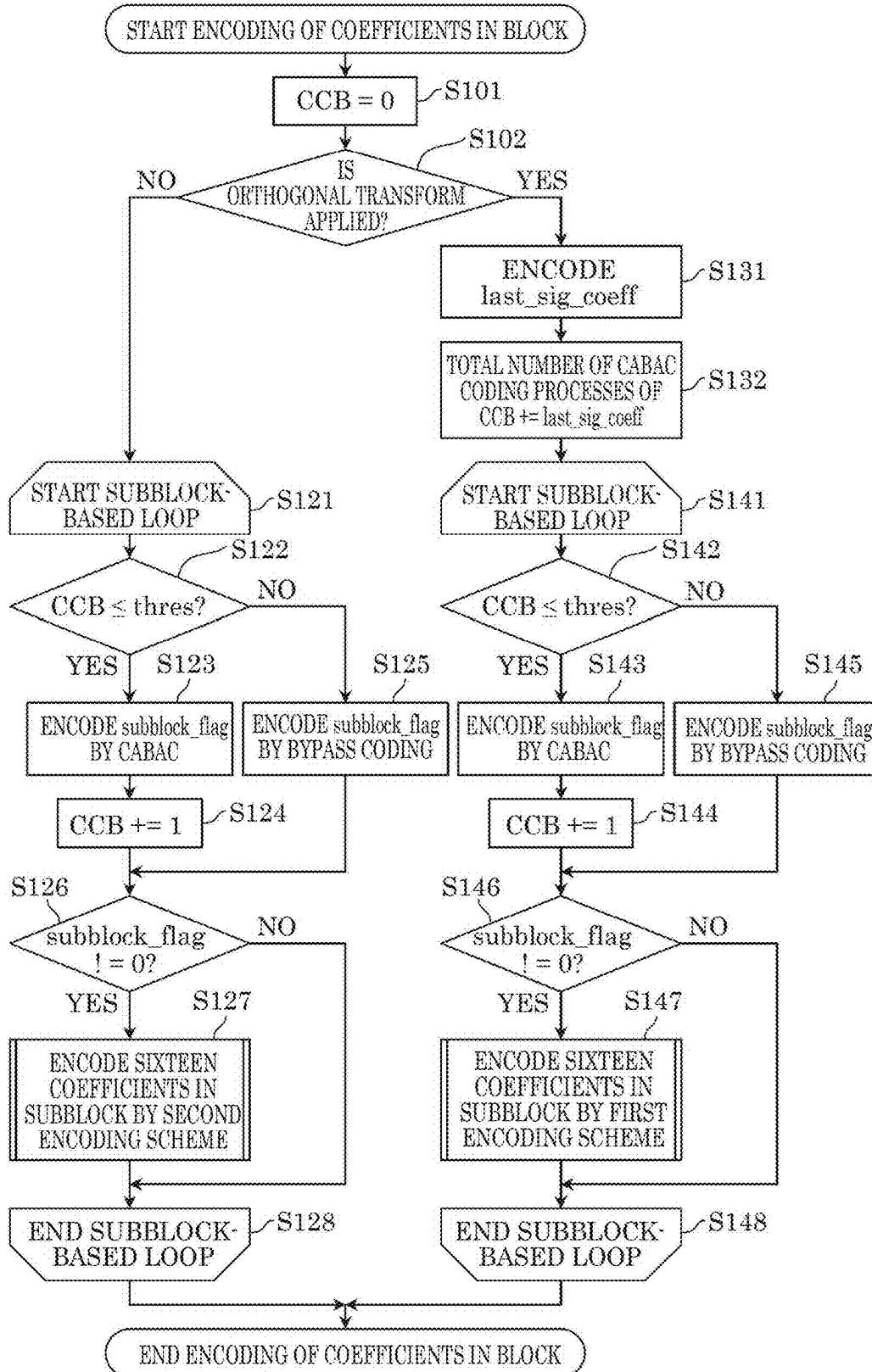


FIG. 101

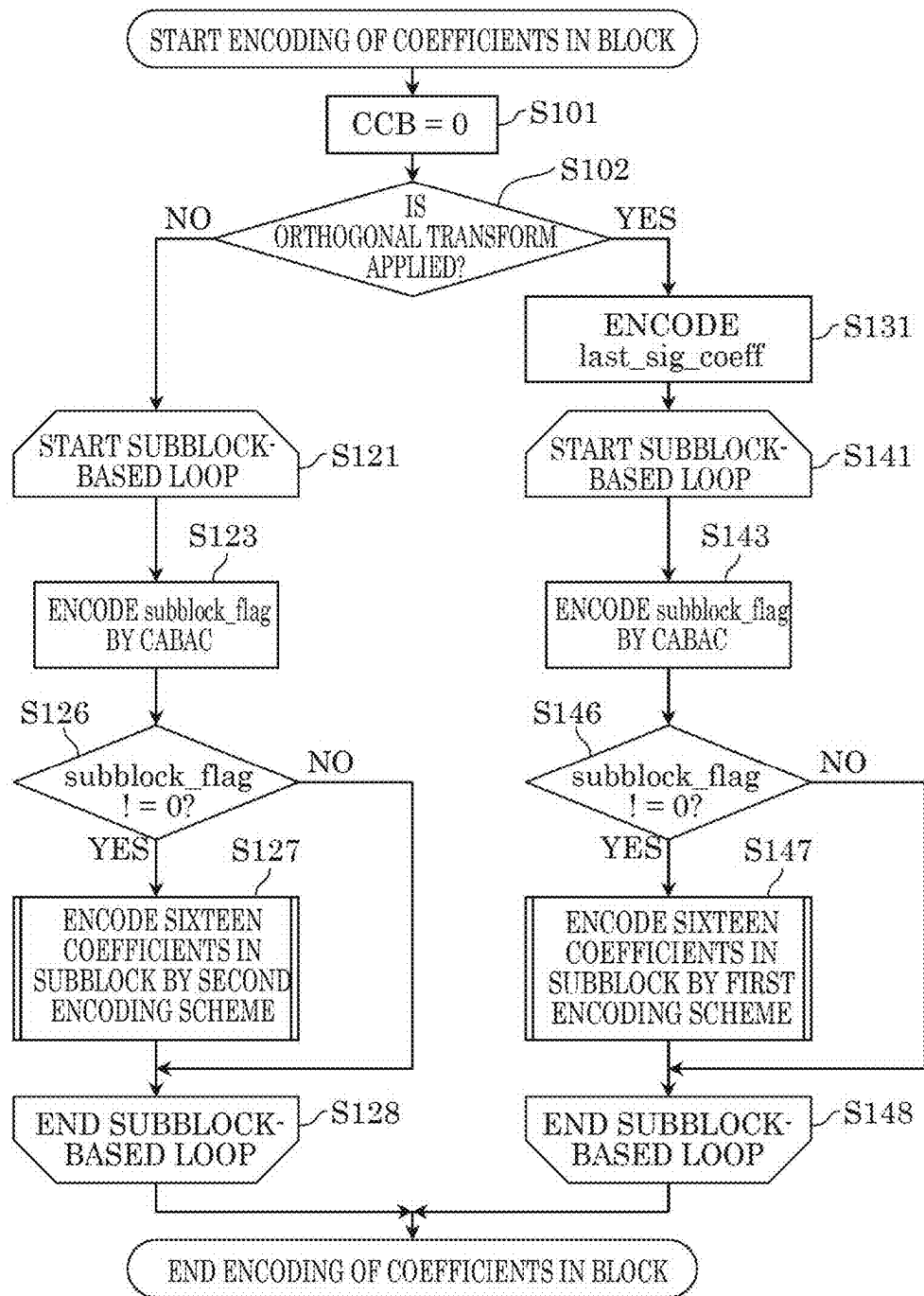


FIG. 102

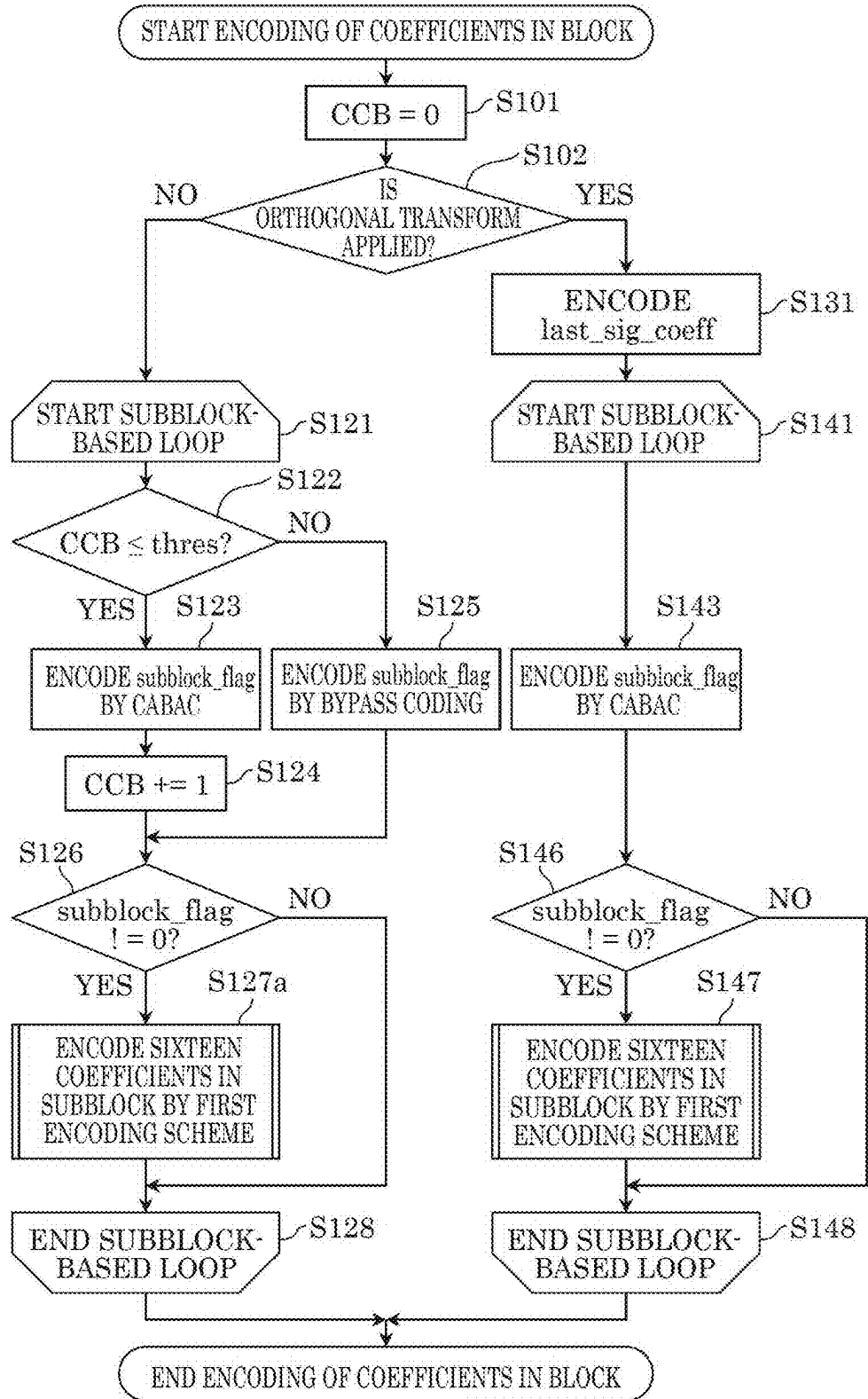


FIG. 103

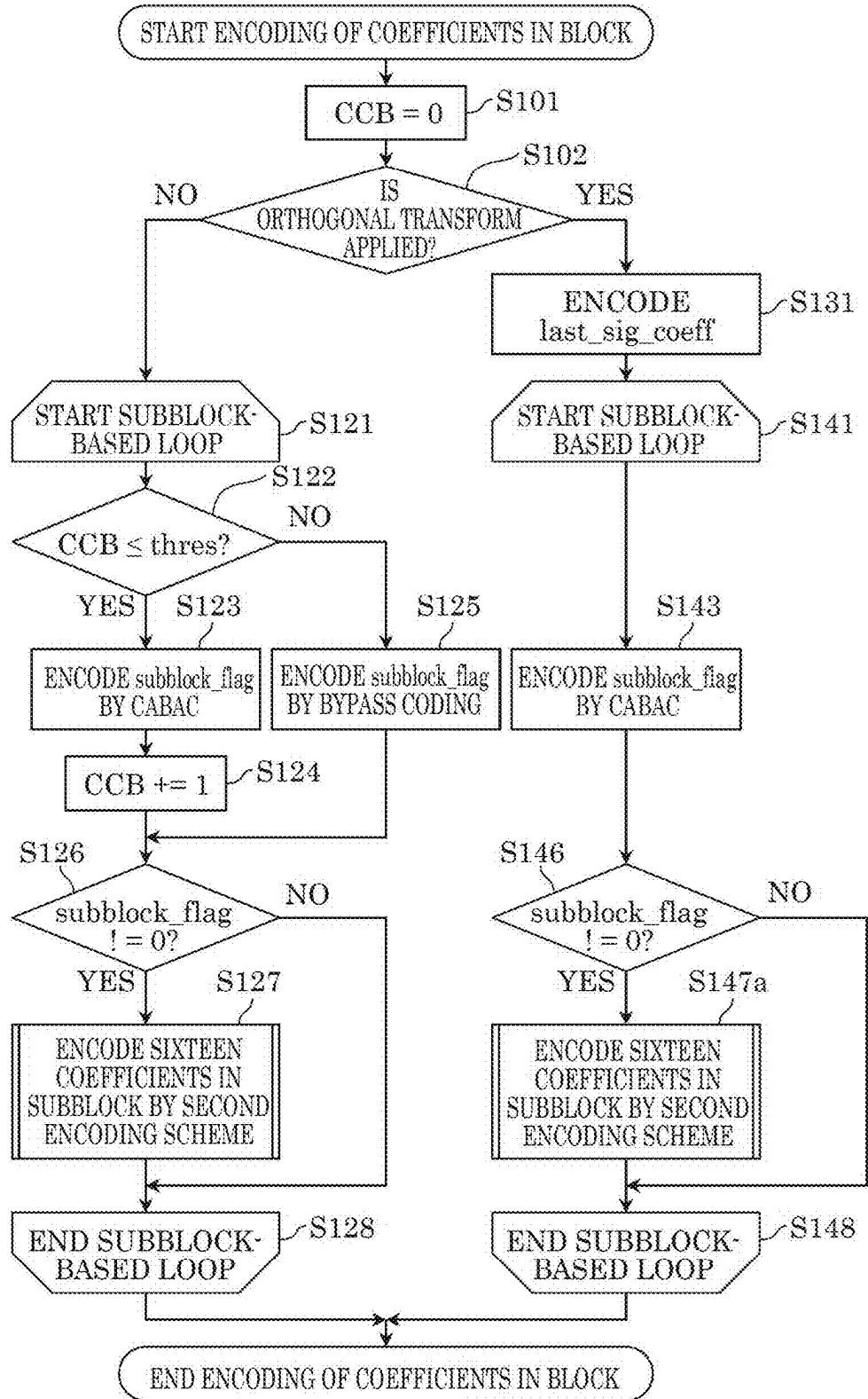


FIG. 104

SYNTAX	EXPLANATION OF SYNTAX
for (n = maxSubPosOfscan; n $\geq$ minSubPos : n --) {	
sig_flag[n], CCB += 1	ENCODING BY CABAC WHEN VALUE OF CCB REMAINS
if (sig_flag[n]) {	
abs_gt1_flag[n], CCB += 1	
if (abs_gt1_flag[n])	
parity_flag[n], CCB += 1	
abs_gt3_flag[n], CCB += 1	
}	
UseCCBPosOfscan = n	
}	
for (n = maxSubPosOfscan; n $\geq$ UseCCBPosOfscan : n --)	
abs_remainder[n]	ENCODE REMAINDER VALUE BY GOLOMB-RICE CODING
for (n = UseCCBPosOfscan - 1; n $\geq$ minSubPos : n --)	
dec_abs_level[n]	ENCODE COEFFICIENT BY GOLOMB-RICE CODING
for (n = maxSubPosOfscan; n $\geq$ minSubpos : n --) {	
if (sig_flag[n])	
sign_flag[n]	BYPASS CODING
}	

FIG. 105

SYNTAX	EXPLANATION OF SYNTAX
for (n = maxSubPosOfscan; n $\geq$ minSubPos; n--) {	
sig_flag[n], CCB += 1	ENCODING BY CABAC WHEN VALUE OF CCB REMAINS, OTHERWISE BYPASS CODING
if (sig_flag[n]) {	
sign_flag[n], CCB += 1	
abs_gt1_flag[n], CCB += 1	
if (abs_gt1_flag[n])	
parity_flag[n], CCB += 1	
}	
}	
for (n = maxSubPosOfscan; n $\geq$ minSubPos; n--) {	
if (abs_gt1_flag[n])	
abs_gt3_flag[n], CCB += 1	ENCODING BY CABAC WHEN VALUE OF CCB REMAINS, OTHERWISE BYPASS CODING
}	
for (n = maxSubPosOfscan; n $\geq$ minSubPos; n--) {	
if (abs_gt3_flag[n])	
abs_gt5_flag[n], CCB += 1	ENCODING BY CABAC WHEN VALUE OF CCB REMAINS, OTHERWISE BYPASS CODING
}	
for (n = maxSubPosOfscan; n $\geq$ minSubPos; n--) {	
if (abs_gt5_flag[n])	
abs_gt7_flag[n], CCB += 1	ENCODING BY CABAC WHEN VALUE OF CCB REMAINS, OTHERWISE BYPASS CODING
}	
for (n = maxSubPosOfscan; n $\geq$ minSubPos; n--) {	
if (abs_gt7_flag[n])	
abs_gt9_flag[n], CCB += 1	ENCODING BY CABAC WHEN VALUE OF CCB REMAINS, OTHERWISE BYPASS CODING
}	
for (n = maxSubPosOfscan; n $\geq$ minSubPos; n--)	
abs_remainder[n]	ENCODE REMAINDER VALUE BY GOLOMB-RICE CODING

FIG. 106

SYNTAX	EXPLANATION OF SYNTAX
for (n = maxSubPosOfscan; n $\geq$ minSubPos; n--) {	ENCODE EIGHT FLAGS BY CABAC WHEN VALUE OF EIGHT OR MORE OF CCB REMAINS
sig_flag[n], CCB += 1	OTHERWISE, ENCODE EIGHT FLAGS BY BYPASS CODING
if (sig_flag[n]) {	
sign_flag[n], CCB += 1	
abs_gt1_flag[n], CCB += 1	
if (abs_gt1_flag[n]) {	
parity_flag[n], CCB += 1	
abs_gt3_flag[n], CCB += 1	
if (abs_gt3_flag[n]) {	
abs_gt5_flag[n], CCB += 1	
if (abs_gt5_flag[n]) {	
abs_gt7_flag[n], CCB += 1	
if (abs_gt7_flag[n])	
abs_gt9_flag[n], CCB += 1	
}	
}	
}	
}	
for (n = maxSubPosOfscan; n $\geq$ minSubPos; n--)	
abs_remainder[n]	ENCODE REMAINDER VALUE BY GOLOMB-RICE CODING

FIG. 107

SYNTAX	EXPLANATION OF SYNTAX
for (n = maxSubPosOfscan; n $\geq$ minSubPos; n++){	ENCODE SEVEN FLAGS BY CABAC WHEN VALUE OF SEVEN OR MORE OF CCB REMAINS
sig_flag[n], CCB += 1	OTHERWISE, ENCODE SEVEN FLAGS BY BYPASS CODING
if (sig_flag[n]){	
abs_gt1_flag[n], CCB += 1	
if (abs_gt1_flag[n]){	
parity_flag[n], CCB += 1	
abs_gt3_flag[n], CCB += 1	
if (abs_gt3_flag[n]){	
abs_gt5_flag[n], CCB += 1	
if (abs_gt5_flag[n]){	
abs_gt7_flag[n], CCB += 1	
if (abs_gt7_flag[n]){	
abs_gt9_flag[n], CCB += 1	
}	
}	
}	
}	
}	
for (n = maxSubPosOfscan; n $\geq$ minSubPos; n++)	
abs_remainder[n]	ENCODE REMAINDER VALUE BY GOLOMB-RICE CODING
for (n = maxSubPosOfscan; n $\geq$ minSubPos; n++){	
if (sig_flag[n])	
sign_flag[n], CCB += 1	ENCODING BY CABAC WHEN VALUE OF CCB REMAINS OTHERWISE BYPASS CODING
}	

FIG. 108

COEFFICIENT INFORMATION FLAGS	CONTEXT NUMBERS FOR USE IN ENCODING OF COEFFICIENT INFORMATION FLAGS
sig_flag	sig_flag (L) + sig_flag (U) (THREE CONTEXT NUMBERS)
parity_flag	ONE CONTEXT NUMBER
gt1_flag	ONE CONTEXT NUMBER
gt3_flag	ONE CONTEXT NUMBER
gt5_flag	ONE CONTEXT NUMBER
gt7_flag	ONE CONTEXT NUMBER
gt9_flag	ONE CONTEXT NUMBER

FIG. 109

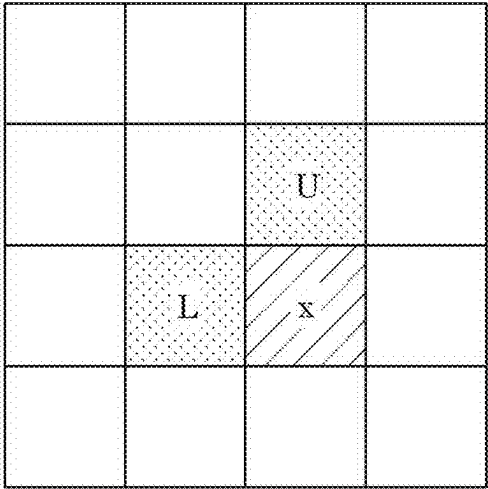


FIG. 110

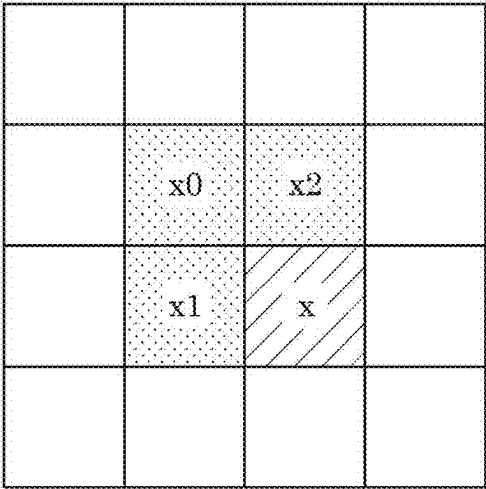


FIG. 111A

x0	x2		
x1	x		

FIG. 111B

x0	x2		
x1	x		

FIG. 112A

x0	x2		
x1	x		

FIG. 112B

	x0	x2	
	x1	x	

FIG. 113

	RICE PARAMETER = 0		RICE PARAMETER = 1		RICE PARAMETER = 2	
	Prefix bins	Suffix bins	Prefix bins	Suffix bins	Prefix bins	Suffix bins
0	0		0	0	0	00
1	10		0	1	0	01
2	110		10	0	0	10
3	1110		10	1	0	11
4	11110		110	0	10	00
5	111110		110	1	10	01
6	1111110		1110	0	10	10
7	11111110		1110	1	10	11
...						

FIG. 114

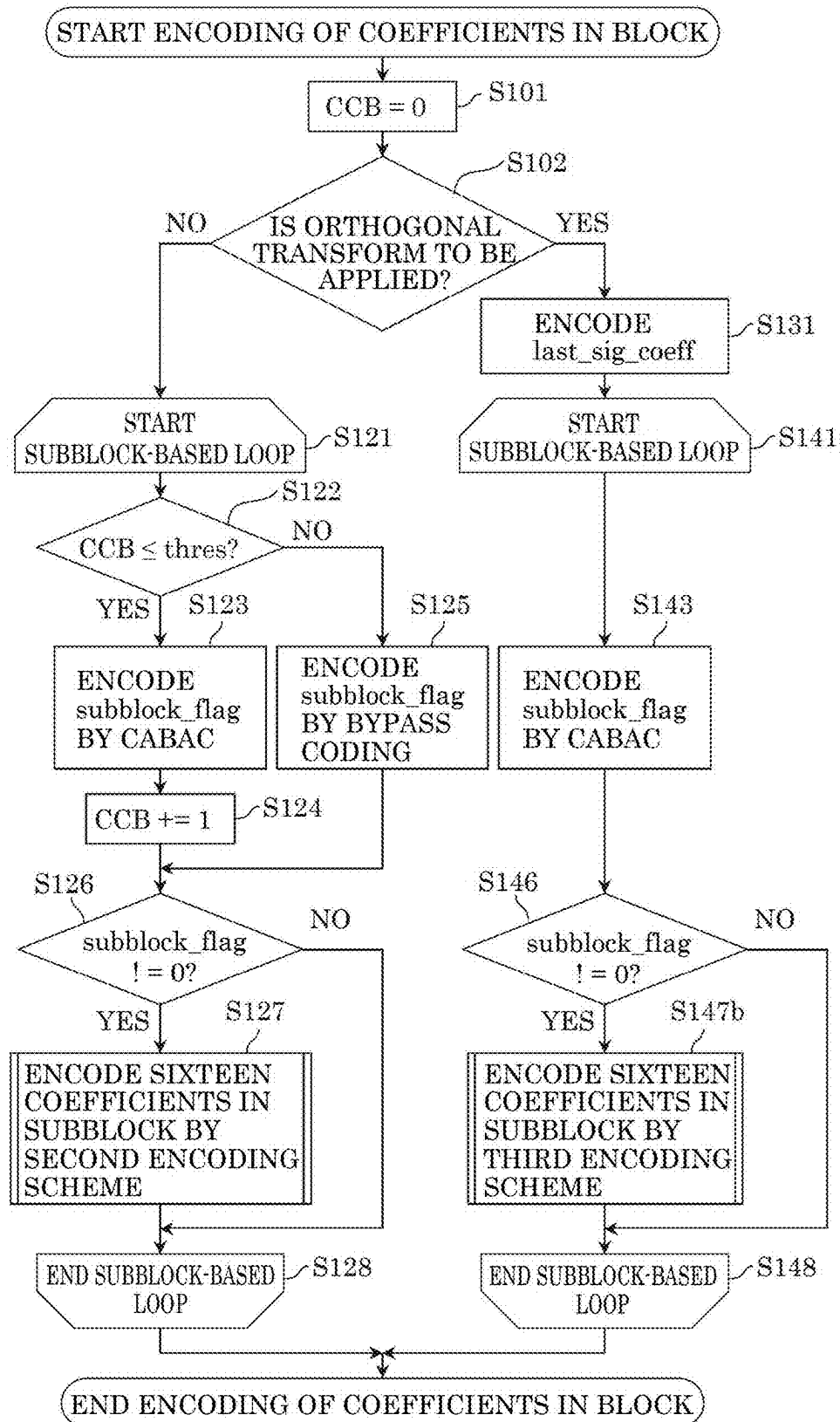


FIG. 115

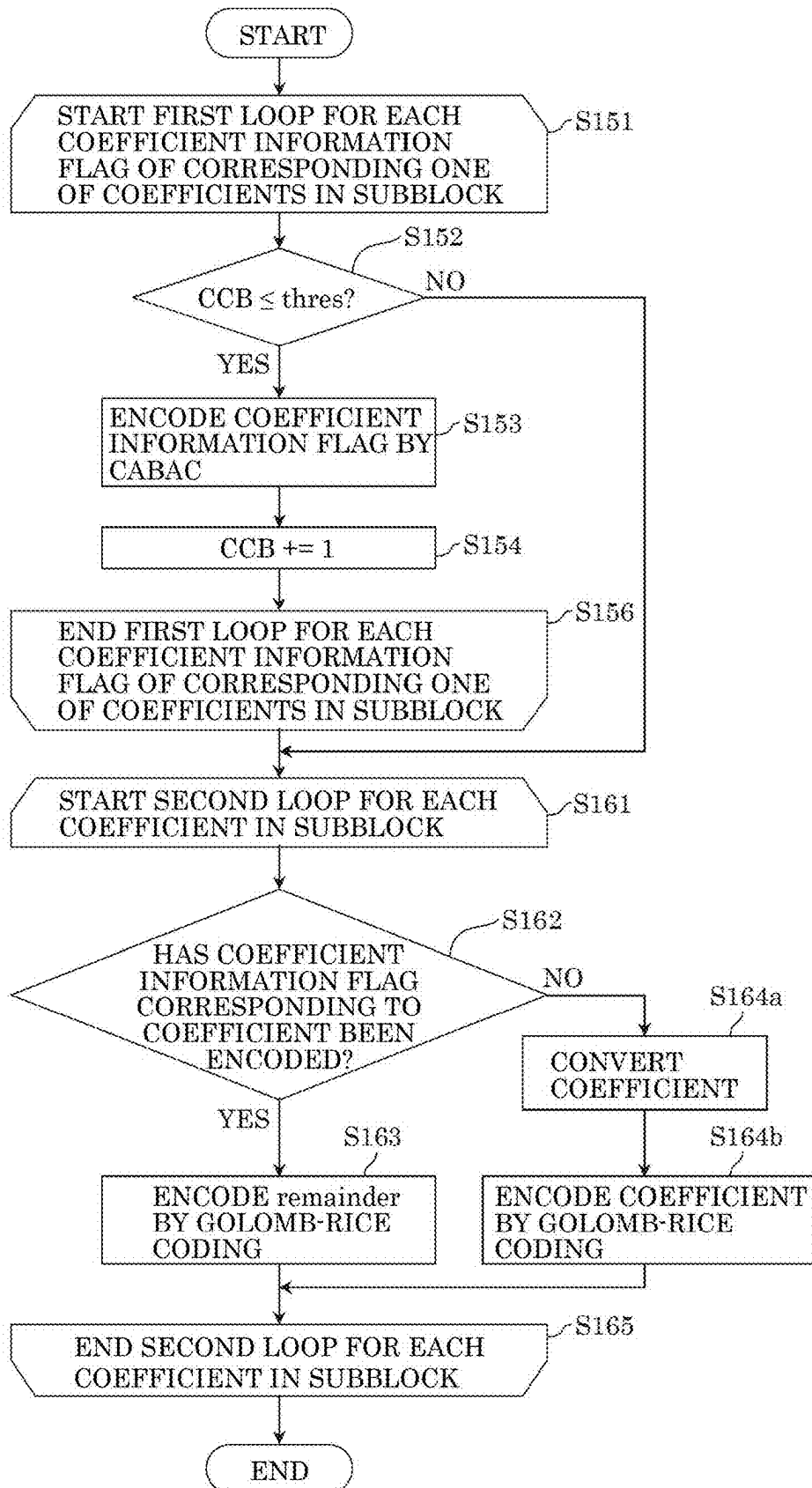


FIG. 116

	a	a3	a0
	a4	a1	
	a2		

FIG. 117

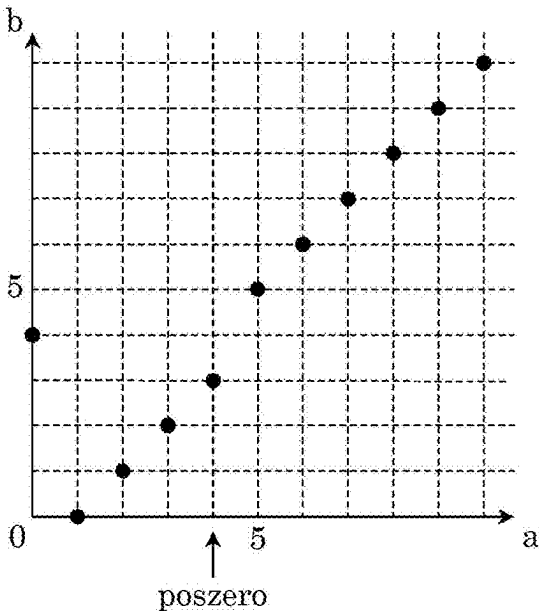


FIG. 118

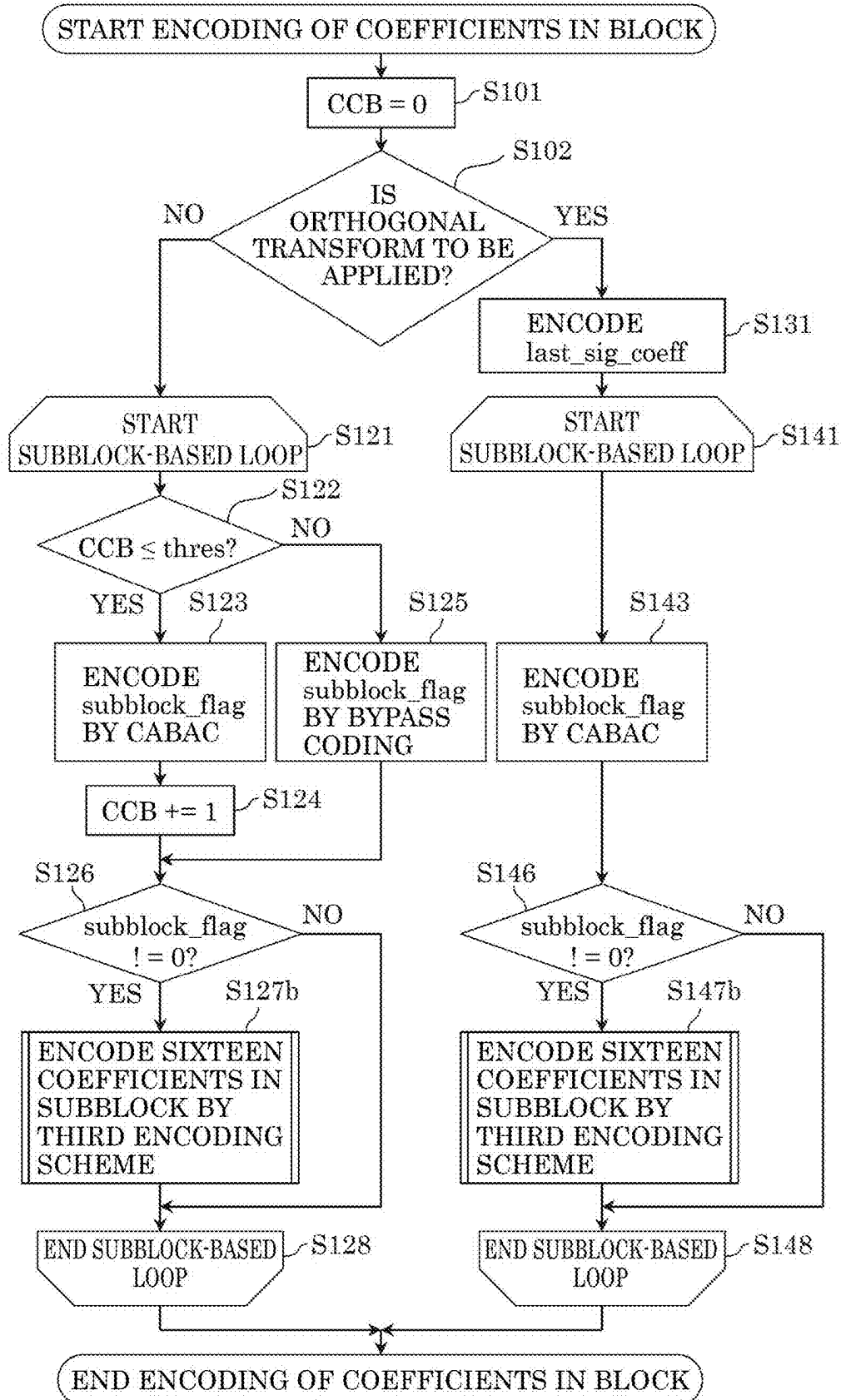


FIG. 119

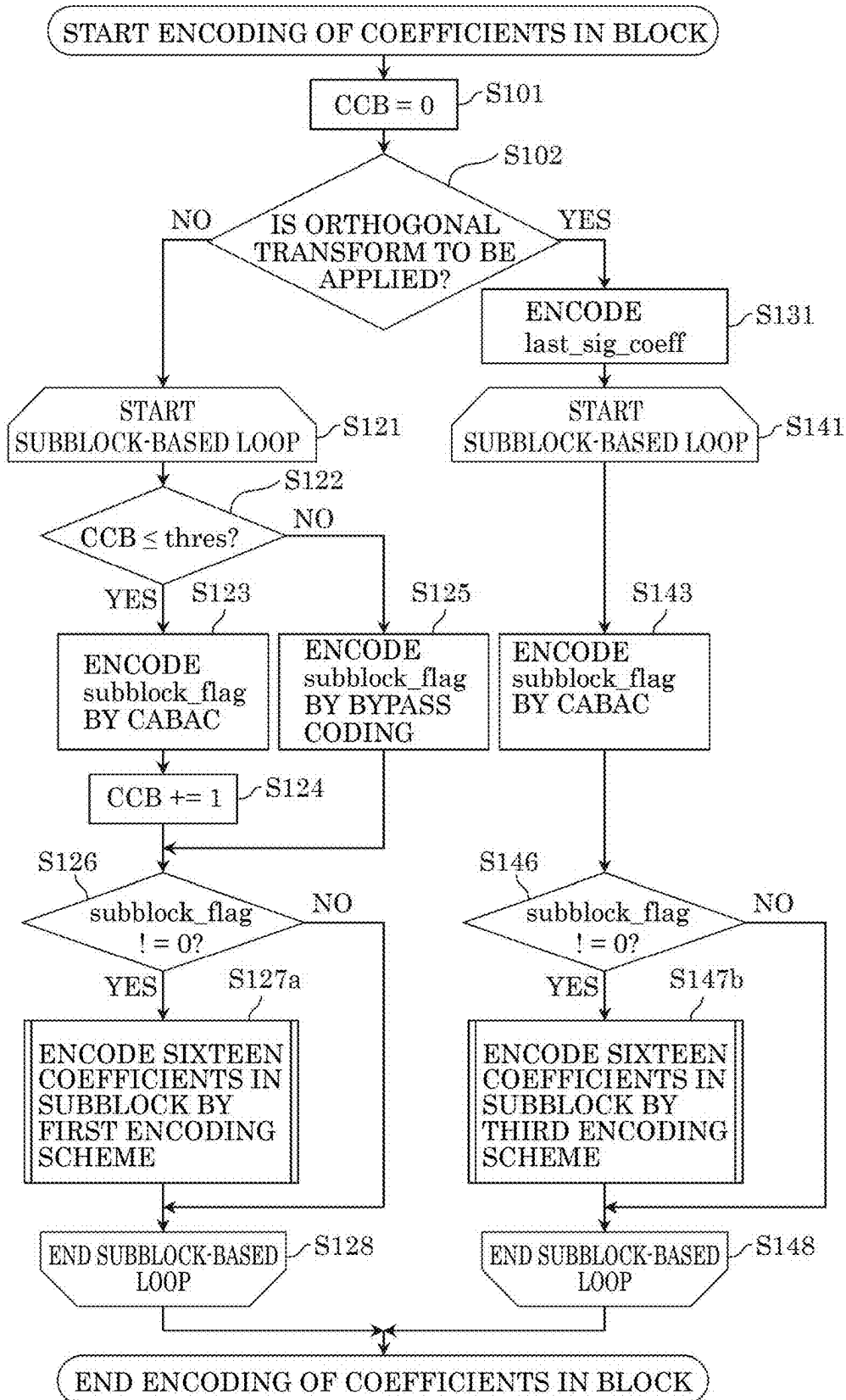


FIG.120

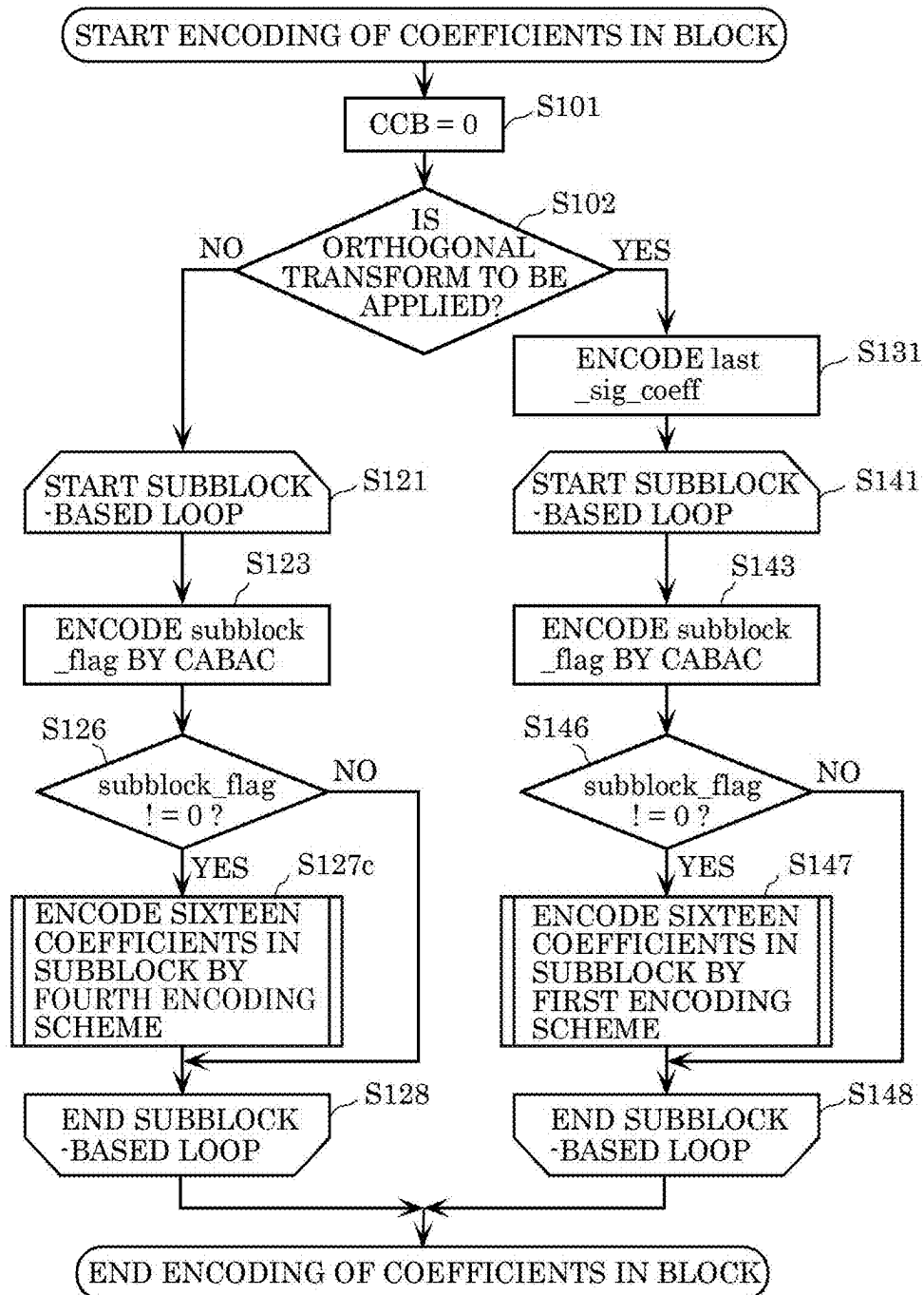


FIG.121

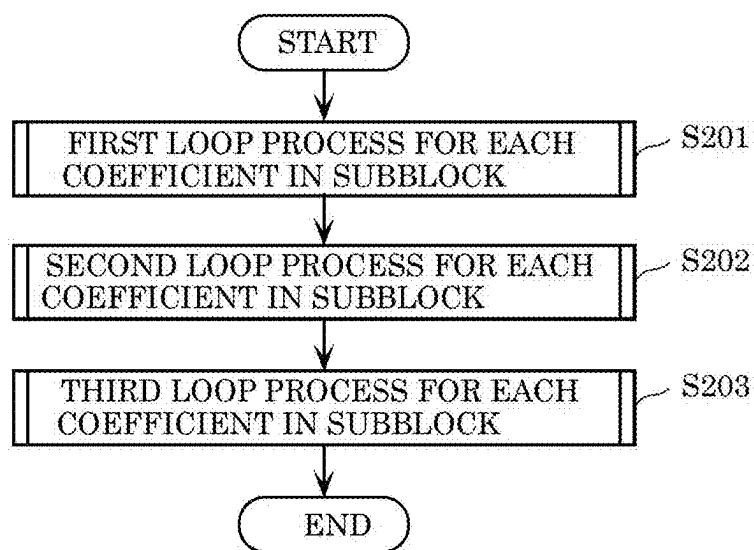


FIG.122

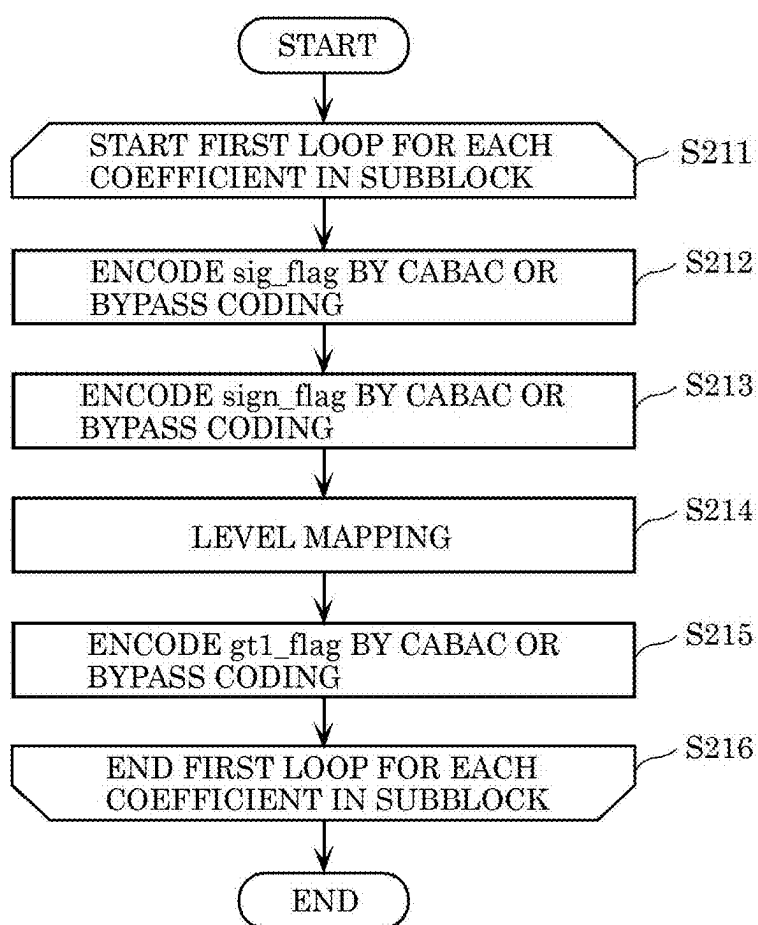


FIG. 123

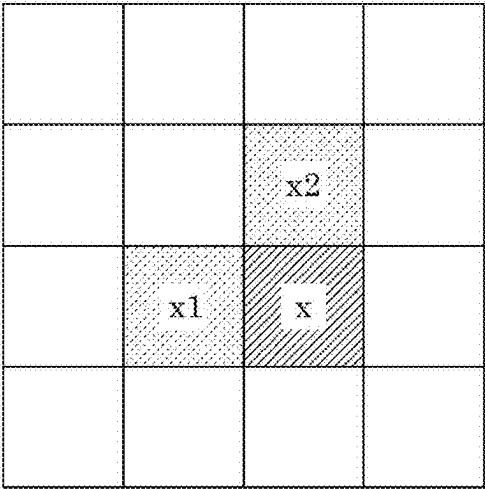


FIG. 124

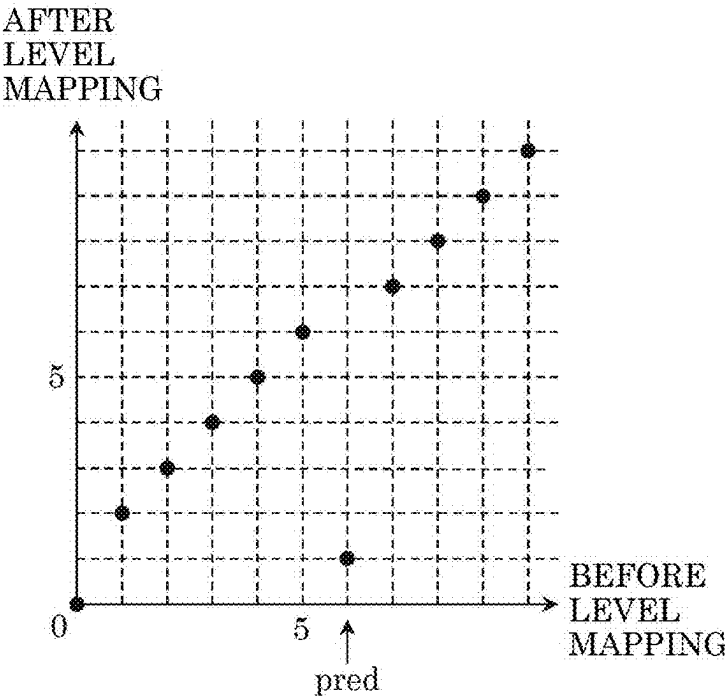


FIG.125

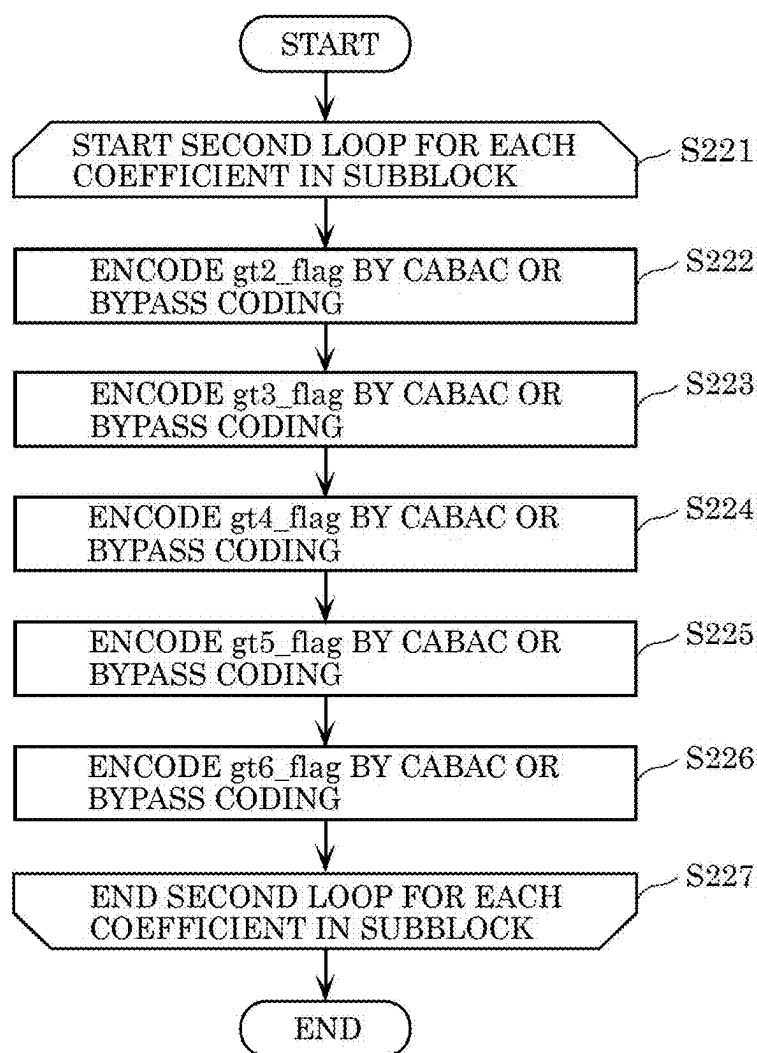


FIG.126

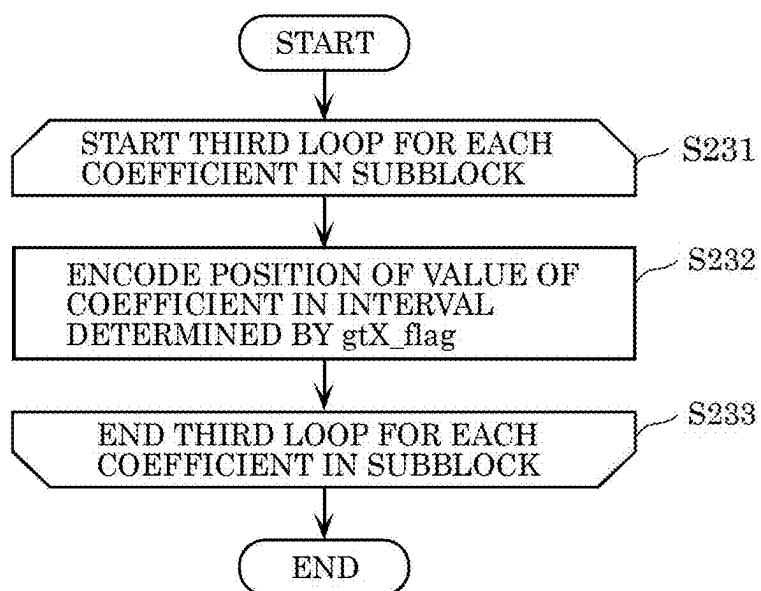


FIG.127

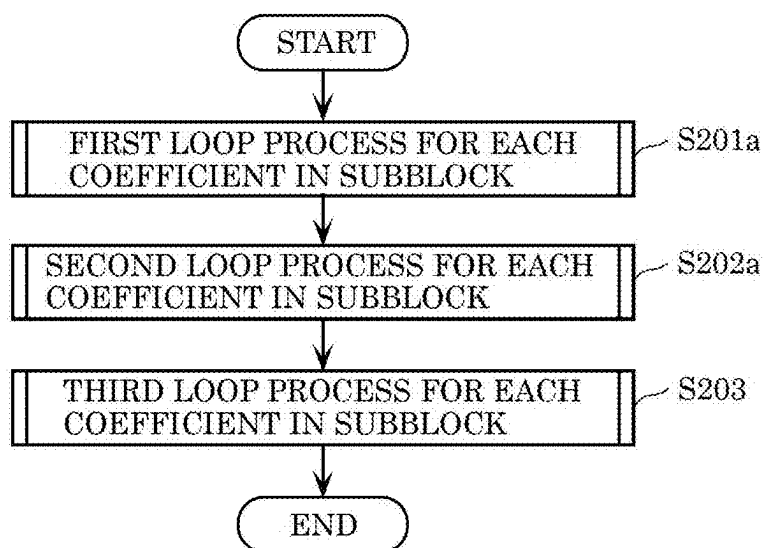


FIG.128

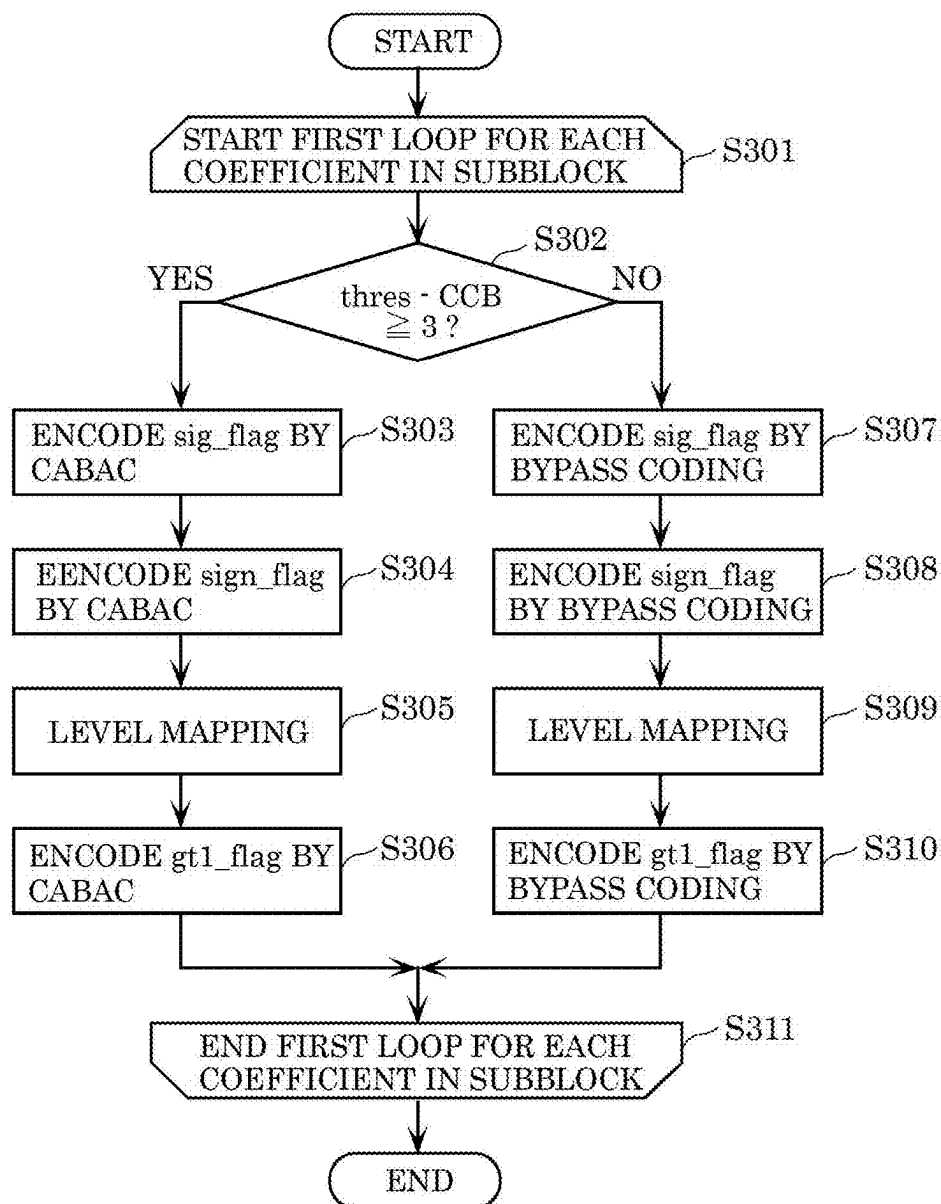


FIG.129

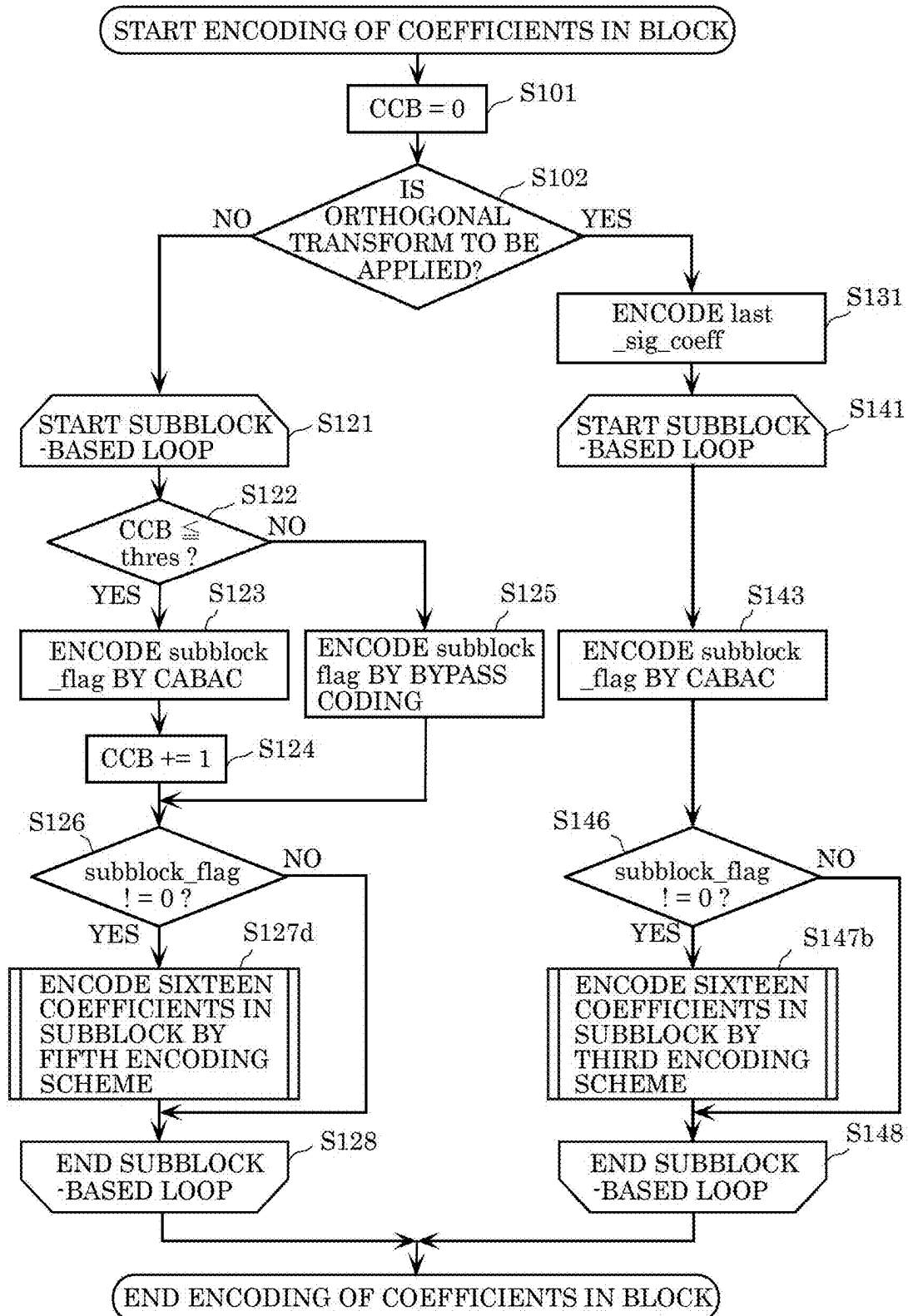


FIG. 130

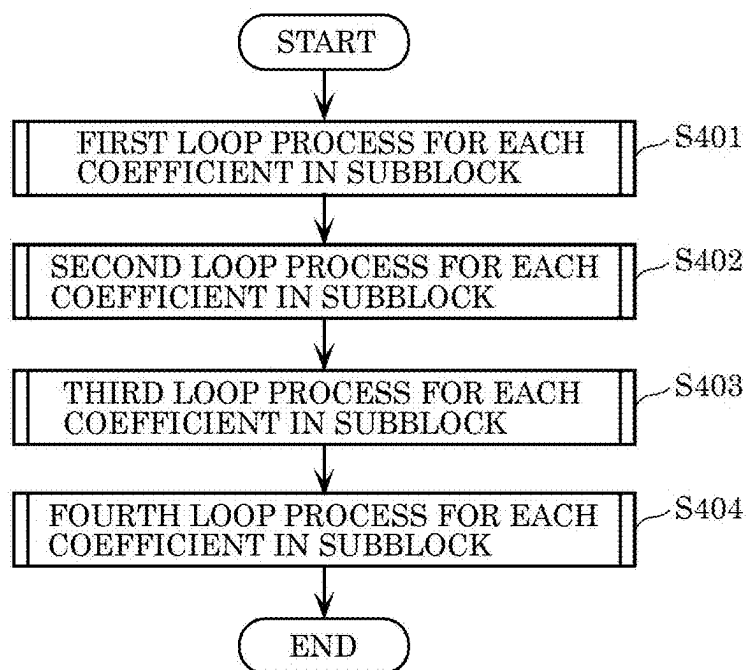


FIG.131

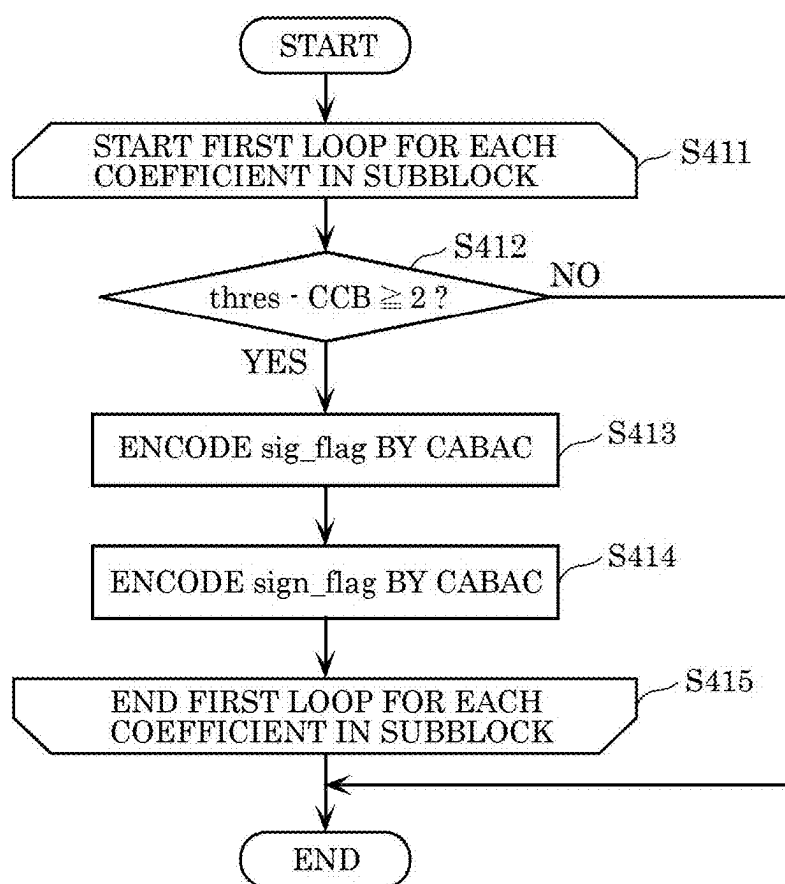


FIG.132

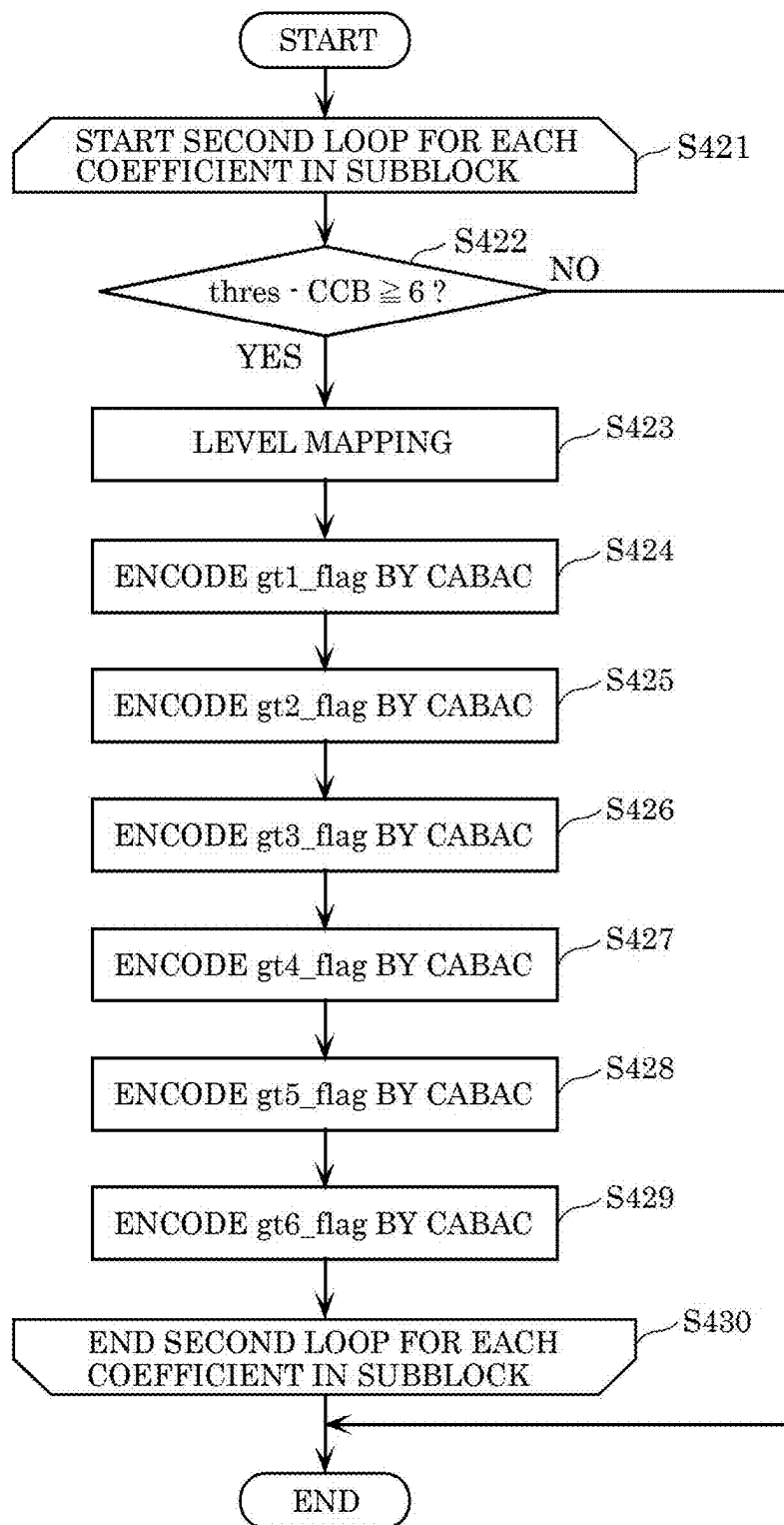


FIG.133

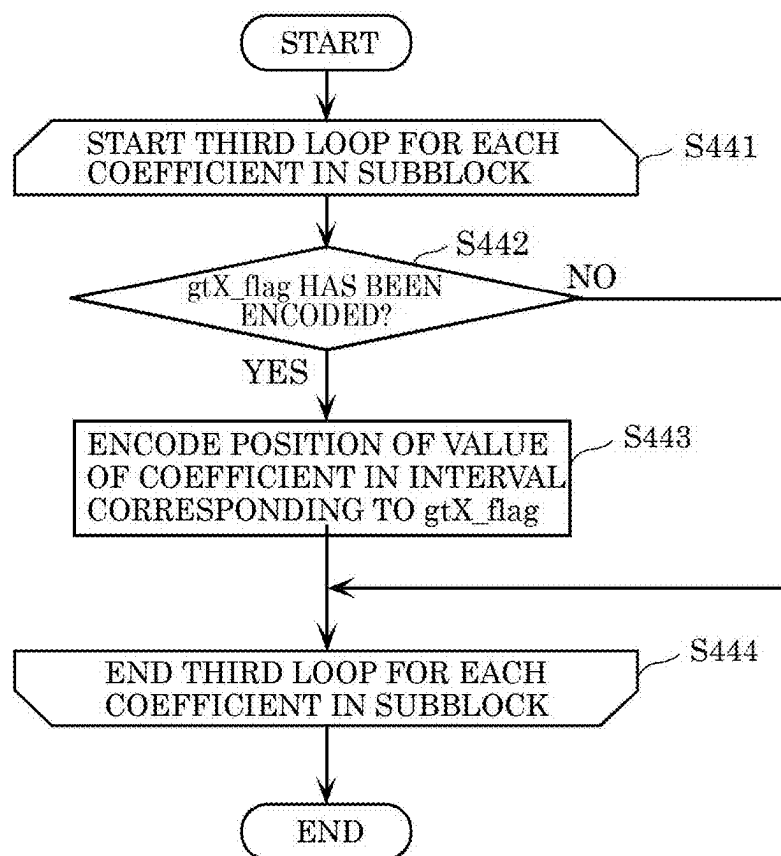


FIG.134

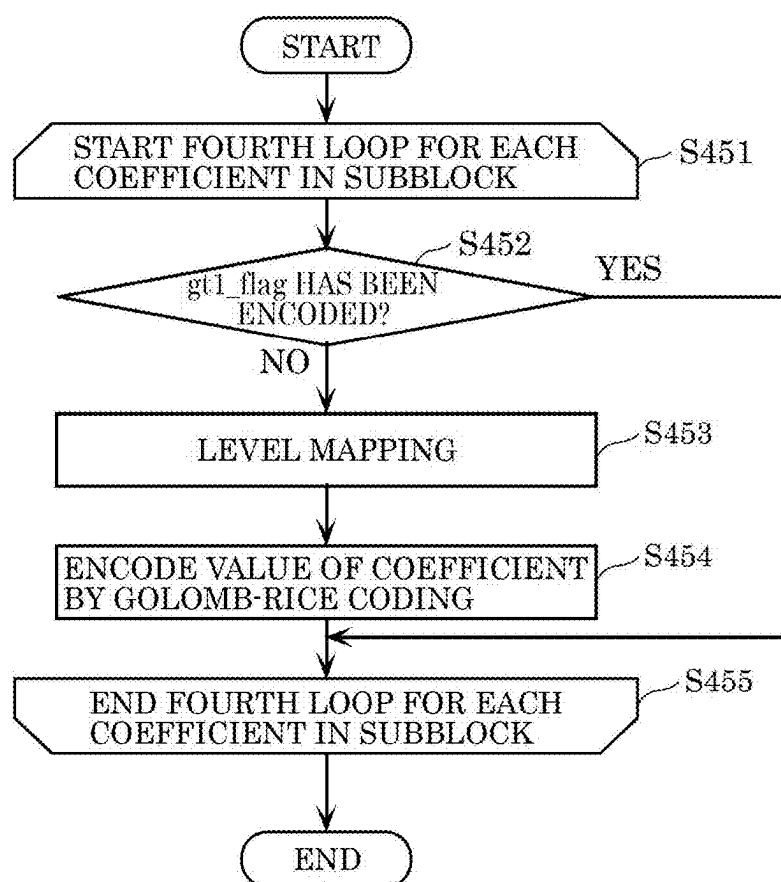


FIG.135

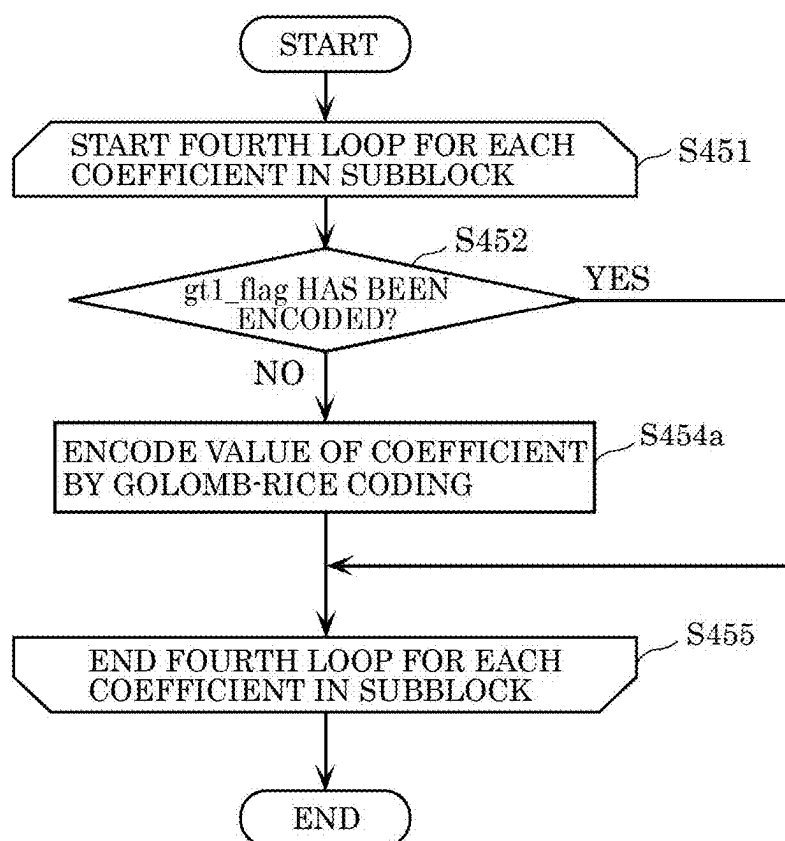


FIG. 136

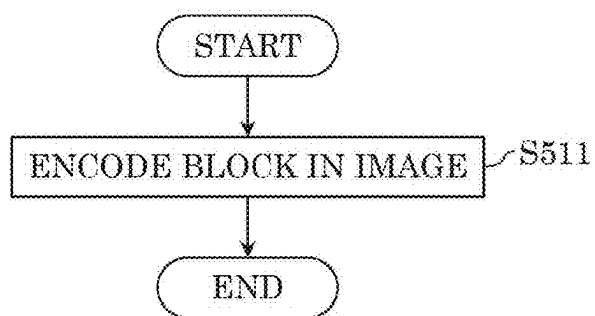


FIG. 137

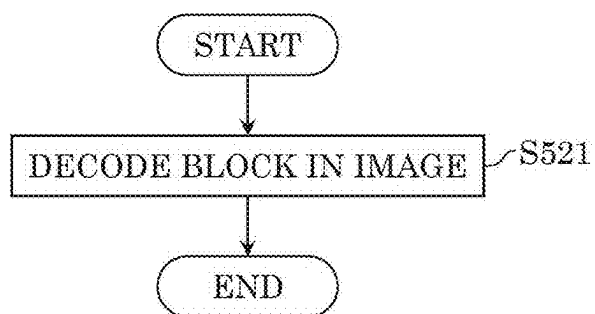


FIG. 138

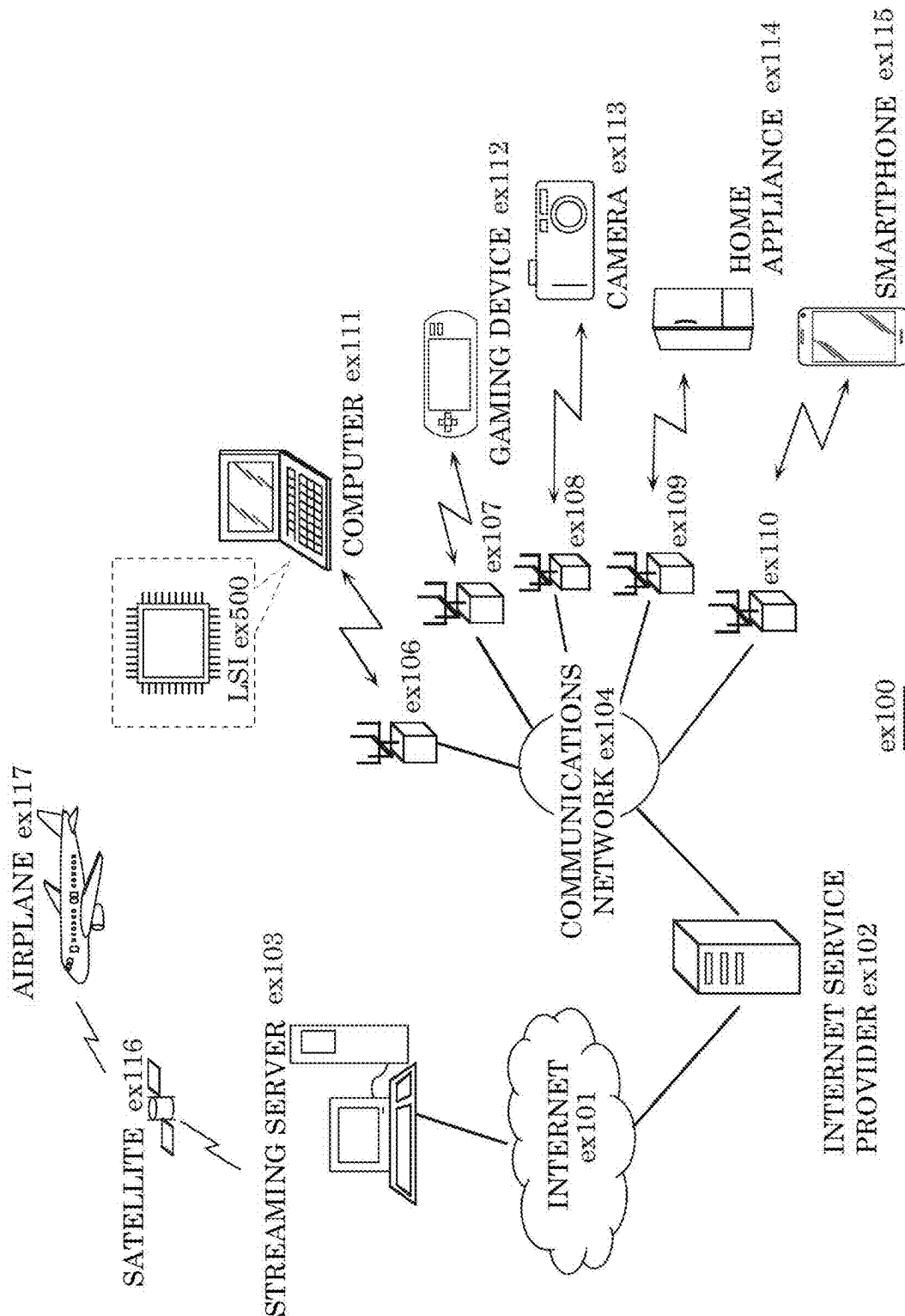


FIG. 139

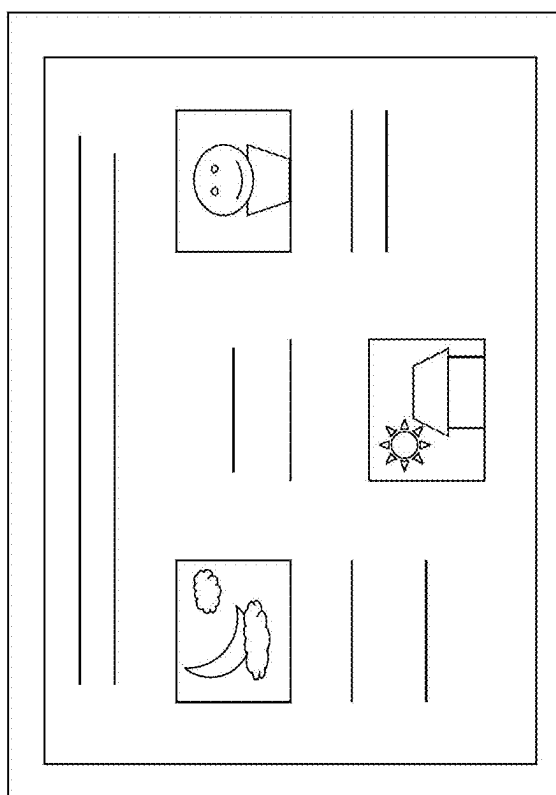


FIG. 140

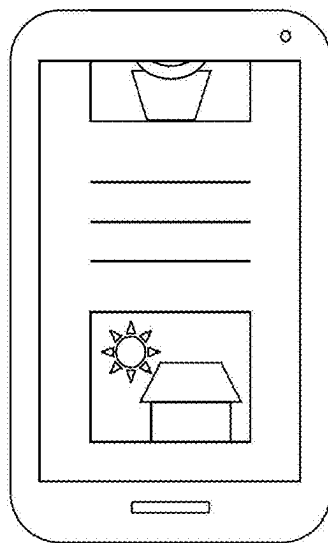


FIG. 141

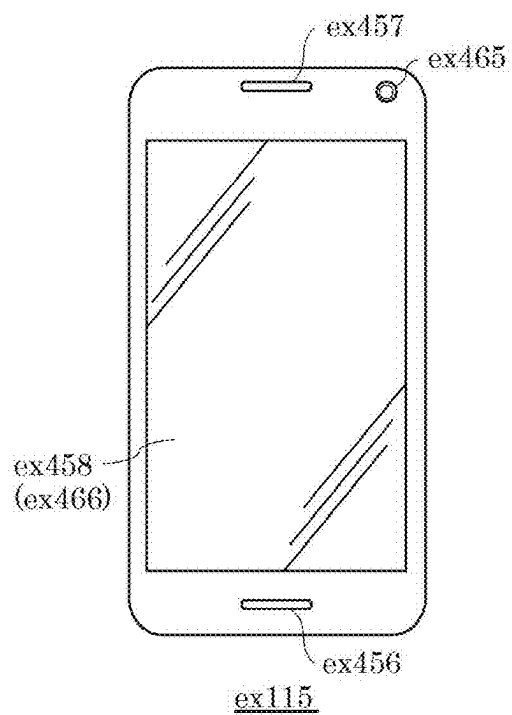
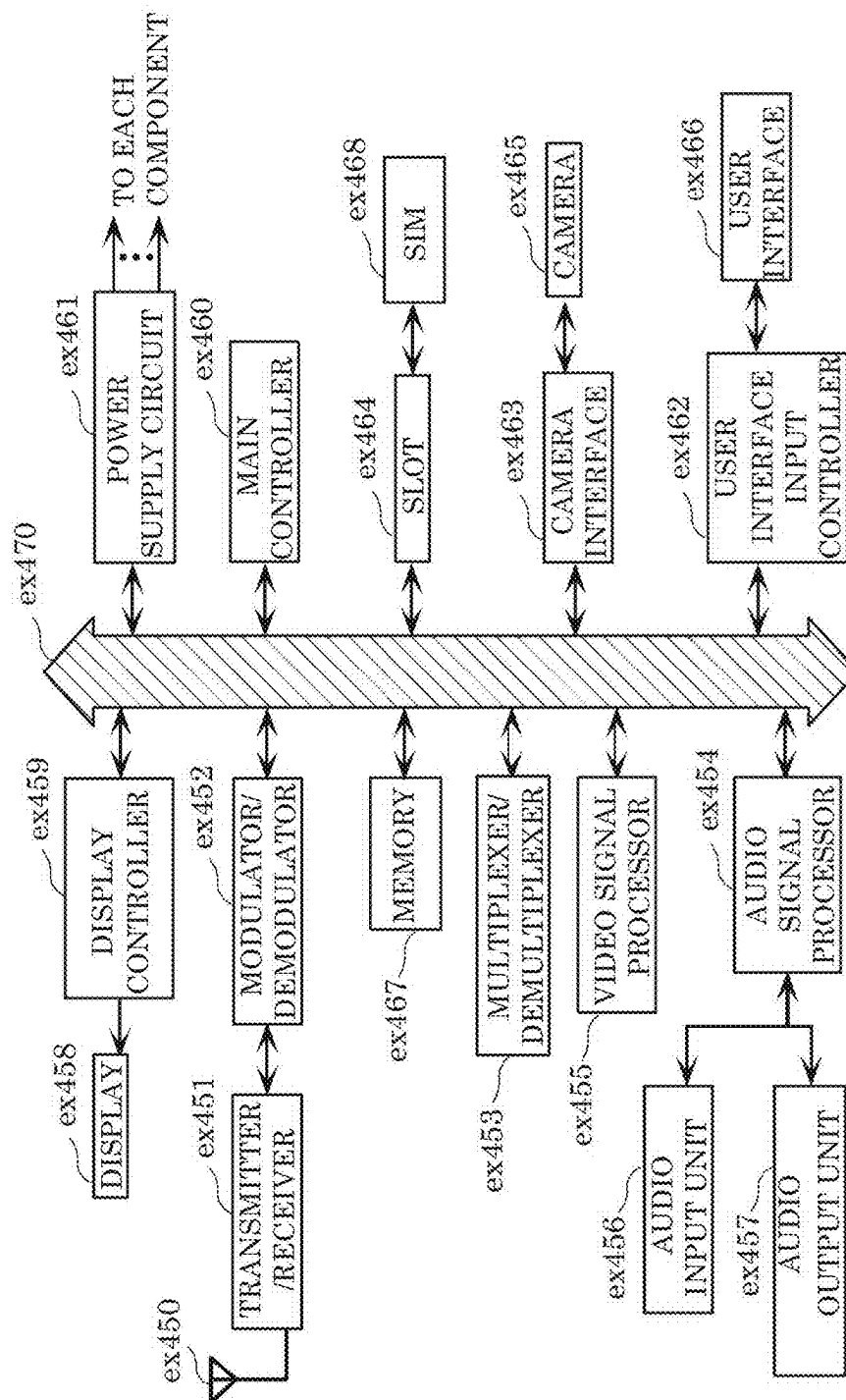


FIG. 142



ENCODER, DECODER, ENCODING METHOD, AND DECODING METHOD

CROSS REFERENCE TO RELATED APPLICATIONS

[0001] This application is a continuation of U.S. application Ser. No. 18/441,617, filed Feb. 14, 2024, which is a continuation of U.S. application Ser. No. 17/691,623, filed Mar. 10, 2022, now U.S. Pat. No. 12,069,266, which is a U.S. continuation application of PCT International Patent Application Number PCT/JP2020/026138 filed on Jul. 3, 2020, claiming the benefit of priority of U.S. Provisional Patent Application No. 62/905,760 filed on Sep. 25, 2019, the entire contents of which are hereby incorporated by reference.

BACKGROUND

1. Technical Field

[0002] The present disclosure relates to video coding, and particularly to systems, constituent elements, and methods in video encoding and decoding.

2. Description of the Related Art

[0003] With advancement in video coding technology, from H.261 and MPEG-1 to H.264/AVC (Advanced Video Coding), MPEG-LA, H.265/HEVC (High Efficiency Video Coding) and H.266/VVC (Versatile Video Codec), there remains a constant need to provide improvements and optimizations to the video coding technology to process an ever-increasing amount of digital video data in various applications. The present disclosure relates to further advancements, improvements and optimizations in video coding.

[0004] Note that H.265 (ISO/IEC 23008-2 HEVC)/HEVC (High Efficiency Video Coding) relates to one example of a conventional standard regarding the above-described video coding technology.

SUMMARY

[0005] For example, an encoder according to an aspect of the present disclosure includes circuitry and memory coupled to the circuitry. In residual coding of a current block to which an orthogonal transform is not applied, when coefficient information flags relating to a coefficient in the current block are encoded, the circuitry: performs a level mapping process to transform the coefficient to a second coefficient by using a predicted value, in which the predicted value is determined based on neighboring coefficients of the coefficient within the current block; encodes second coefficient information flags by Context-based Adaptive Binary Arithmetic Coding (CABAC), each of the second coefficient information flags relating to the second coefficient; and encodes a remainder value of the coefficient with Golomb-Rice code, and when the coefficient information flags are not encoded, in operation, the circuitry: skips the level mapping process; and encodes a value of the coefficient with the Golomb-Rice code.

[0006] Each of embodiments, or each of part of constituent elements and methods in the present disclosure enables, for example, at least one of the following: improvement in coding efficiency, enhancement in image quality, reduction in processing amount of encoding/decoding, reduction in

circuit scale, improvement in processing speed of encoding/decoding, etc. Alternatively, each of embodiments, or each of part of constituent elements and methods in the present disclosure enables, in encoding and decoding, appropriate selection of an element or an operation. The element is, for example, a filter, a block, a size, a motion vector, a reference picture, or a reference block. It is to be noted that the present disclosure includes disclosure regarding configurations and methods which may provide advantages other than the above-described ones. Examples of such configurations and methods include a configuration or method for improving coding efficiency while reducing increase in processing amount.

[0007] Additional benefits and advantages according to an aspect of the present disclosure will become apparent from the specification and drawings. The benefits and/or advantages may be individually obtained by the various embodiments and features of the specification and drawings, and not all of which need to be provided in order to obtain one or more of such benefits and/or advantages.

[0008] It is to be noted that these general or specific aspects may be implemented using a system, an integrated circuit, a computer program, or a computer readable medium (recording medium) such as a CD-ROM, or any combination of systems, methods, integrated circuits, computer programs, and media.

BRIEF DESCRIPTION OF DRAWINGS

[0009] These and other objects, advantages and features of the disclosure will become apparent from the following description thereof taken in conjunction with the accompanying drawings that illustrate a specific embodiment of the present disclosure.

[0010] FIG. 1 is a schematic diagram illustrating one example of a configuration of a transmission system according to an embodiment;

[0011] FIG. 2 is a diagram illustrating one example of a hierarchical structure of data in a stream;

[0012] FIG. 3 is a diagram illustrating one example of a slice configuration;

[0013] FIG. 4 is a diagram illustrating one example of a tile configuration;

[0014] FIG. 5 is a diagram illustrating one example of an encoding structure in scalable encoding;

[0015] FIG. 6 is a diagram illustrating one example of an encoding structure in scalable encoding;

[0016] FIG. 7 is a block diagram illustrating one example of a configuration of an encoder according to an embodiment;

[0017] FIG. 8 is a block diagram illustrating a mounting example of the encoder;

[0018] FIG. 9 is a flow chart illustrating one example of an overall encoding process performed by the encoder;

[0019] FIG. 10 is a diagram illustrating one example of block splitting;

[0020] FIG. 11 is a diagram illustrating one example of a configuration of a splitter;

[0021] FIG. 12 is a diagram illustrating examples of splitting patterns;

[0022] FIG. 13A is a diagram illustrating one example of a syntax tree of a splitting pattern;

[0023] FIG. 13B is a diagram illustrating another example of a syntax tree of a splitting pattern;

[0024] FIG. 14 is a chart illustrating transform basis functions for each transform type;

[0025] FIG. 15 is a diagram illustrating examples of SVT;

[0026] FIG. 16 is a flow chart illustrating one example of a process performed by a transformer;

[0027] FIG. 17 is a flow chart illustrating another example of a process performed by the transformer;

[0028] FIG. 18 is a block diagram illustrating one example of a configuration of a quantizer;

[0029] FIG. 19 is a flow chart illustrating one example of quantization performed by the quantizer;

[0030] FIG. 20 is a block diagram illustrating one example of a configuration of an entropy encoder;

[0031] FIG. 21 is a diagram illustrating a flow of CABAC in the entropy encoder;

[0032] FIG. 22 is a block diagram illustrating one example of a configuration of a loop filter;

[0033] FIG. 23A is a diagram illustrating one example of a filter shape used in an adaptive loop filter (ALF);

[0034] FIG. 23B is a diagram illustrating another example of a filter shape used in an ALF;

[0035] FIG. 23C is a diagram illustrating another example of a filter shape used in an ALF;

[0036] FIG. 23D is a diagram illustrating an example where Y samples (first component) are used for a cross component ALF (CCALF) for Cb and a CCALF for Cr (components different from the first component);

[0037] FIG. 23E is a diagram illustrating a diamond shaped filter;

[0038] FIG. 23F is a diagram illustrating an example for a joint chroma CCALF (JC-CCALF);

[0039] FIG. 23G is a diagram illustrating an example for JC-CCALF weight index candidates;

[0040] FIG. 24 is a block diagram illustrating one example of a specific configuration of a loop filter which functions as a DBF;

[0041] FIG. 25 is a diagram illustrating an example of a deblocking filter having a symmetrical filtering characteristic with respect to a block boundary;

[0042] FIG. 26 is a diagram for illustrating a block boundary on which a deblocking filter process is performed;

[0043] FIG. 27 is a diagram illustrating examples of Bs values;

[0044] FIG. 28 is a flow chart illustrating one example of a process performed by a predictor of the encoder;

[0045] FIG. 29 is a flow chart illustrating another example of a process performed by the predictor of the encoder;

[0046] FIG. 30 is a flow chart illustrating another example of a process performed by the predictor of the encoder;

[0047] FIG. 31 is a diagram illustrating one example of sixty-seven intra prediction modes used in intra prediction;

[0048] FIG. 32 is a flow chart illustrating one example of a process performed by an intra predictor;

[0049] FIG. 33 is a diagram illustrating examples of reference pictures;

[0050] FIG. 34 is a diagram illustrating examples of reference picture lists;

[0051] FIG. 35 is a flow chart illustrating a basic processing flow of inter prediction;

[0052] FIG. 36 is a flow chart illustrating one example of MV derivation;

[0053] FIG. 37 is a flow chart illustrating another example of MV derivation;

[0054] FIG. 38A is a diagram illustrating one example of categorization of modes for MV derivation;

[0055] FIG. 38B is a diagram illustrating one example of categorization of modes for MV derivation;

[0056] FIG. 39 is a flow chart illustrating an example of inter prediction by normal inter mode;

[0057] FIG. 40 is a flow chart illustrating an example of inter prediction by normal merge mode;

[0058] FIG. 41 is a diagram for illustrating one example of an MV derivation process by normal merge mode;

[0059] FIG. 42 is a diagram for illustrating one example of an MV derivation process by a history-based motion vector prediction/predictor (HMVP) mode;

[0060] FIG. 43 is a flow chart illustrating one example of frame rate up conversion (FRUC);

[0061] FIG. 44 is a diagram for illustrating one example of pattern matching (bilateral matching) between two blocks located along a motion trajectory;

[0062] FIG. 45 is a diagram for illustrating one example of pattern matching (template matching) between a template in a current picture and a block in a reference picture;

[0063] FIG. 46A is a diagram for illustrating one example of MV derivation in units of a sub-block in affine mode in which two control points are used;

[0064] FIG. 46B is a diagram for illustrating one example of MV derivation in units of a sub-block in affine mode in which three control points are used;

[0065] FIG. 47A is a conceptual diagram for illustrating one example of MV derivation at control points in an affine mode;

[0066] FIG. 47B is a conceptual diagram for illustrating one example of MV derivation at control points in an affine mode;

[0067] FIG. 47C is a conceptual diagram for illustrating one example of MV derivation at control points in an affine mode;

[0068] FIG. 48A is a diagram for illustrating an affine mode in which two control points are used;

[0069] FIG. 48B is a diagram for illustrating an affine mode in which three control points are used;

[0070] FIG. 49A is a conceptual diagram for illustrating one example of a method for MV derivation at control points when the number of control points for an encoded block and the number of control points for a current block are different from each other;

[0071] FIG. 49B is a conceptual diagram for illustrating another example of a method for MV derivation at control points when the number of control points for an encoded block and the number of control points for a current block are different from each other;

[0072] FIG. 50 is a flow chart illustrating one example of a process in affine merge mode;

[0073] FIG. 51 is a flow chart illustrating one example of a process in affine inter mode;

[0074] FIG. 52A is a diagram for illustrating generation of two triangular prediction images;

[0075] FIG. 52B is a conceptual diagram illustrating examples of a first portion of a first partition and first and second sets of samples;

[0076] FIG. 52C is a conceptual diagram illustrating a first portion of a first partition;

[0077] FIG. 53 is a flow chart illustrating one example of a triangle mode;

[0078] FIG. 54 is a diagram illustrating one example of an advanced temporal motion vector prediction/predictor (ATMVP) mode in which an MV is derived in units of a sub-block;

[0079] FIG. 55 is a diagram illustrating a relationship between a merge mode and dynamic motion vector refreshing (DMVR);

[0080] FIG. 56 is a conceptual diagram for illustrating one example of DMVR;

[0081] FIG. 57 is a conceptual diagram for illustrating another example of DMVR for determining an MV;

[0082] FIG. 58A is a diagram illustrating one example of motion estimation in DMVR;

[0083] FIG. 58B is a flow chart illustrating one example of motion estimation in DMVR;

[0084] FIG. 59 is a flow chart illustrating one example of generation of a prediction image;

[0085] FIG. 60 is a flow chart illustrating another example of generation of a prediction image;

[0086] FIG. 61 is a flow chart illustrating one example of a correction process of a prediction image by overlapped block motion compensation (OBMC);

[0087] FIG. 62 is a conceptual diagram for illustrating one example of a prediction image correction process by OBMC;

[0088] FIG. 63 is a diagram for illustrating a model assuming uniform linear motion;

[0089] FIG. 64 is a flow chart illustrating one example of inter prediction according to BIO; FIG. 65 is a diagram illustrating one example of a configuration of an inter predictor which performs inter prediction according to BIO;

[0090] FIG. 66A is a diagram for illustrating one example of a prediction image generation method using a luminance correction process by local illumination compensation (LIC);

[0091] FIG. 66B is a flow chart illustrating one example of a prediction image generation method using a luminance correction process by LIC;

[0092] FIG. 67 is a block diagram illustrating a configuration of a decoder according to an embodiment;

[0093] FIG. 68 is a block diagram illustrating a mounting example of a decoder;

[0094] FIG. 69 is a flow chart illustrating one example of an overall decoding process performed by the decoder;

[0095] FIG. 70 is a diagram illustrating a relationship between a splitting determiner and other constituent elements;

[0096] FIG. 71 is a block diagram illustrating one example of a configuration of an entropy decoder;

[0097] FIG. 72 is a diagram illustrating a flow of CABAC in the entropy decoder;

[0098] FIG. 73 is a block diagram illustrating one example of a configuration of an inverse quantizer;

[0099] FIG. 74 is a flow chart illustrating one example of inverse quantization performed by the inverse quantizer;

[0100] FIG. 75 is a flow chart illustrating one example of a process performed by an inverse transformer;

[0101] FIG. 76 is a flow chart illustrating another example of a process performed by the inverse transformer;

[0102] FIG. 77 is a block diagram illustrating one example of a configuration of a loop filter;

[0103] FIG. 78 is a flow chart illustrating one example of a process performed by a predictor of the decoder;

[0104] FIG. 79 is a flow chart illustrating another example of a process performed by the predictor of the decoder;

[0105] FIG. 80A is a flow chart illustrating a portion of other example of a process performed by the predictor of the decoder;

[0106] FIG. 80B is a flow chart illustrating the remaining portion of the other example of the process performed by the predictor of the decoder;

[0107] FIG. 81 is a diagram illustrating one example of a process performed by an intra predictor of the decoder;

[0108] FIG. 82 is a flow chart illustrating one example of MV derivation in the decoder;

[0109] FIG. 83 is a flow chart illustrating another example of MV derivation in the decoder;

[0110] FIG. 84 is a flow chart illustrating an example of inter prediction by normal inter mode in the decoder;

[0111] FIG. 85 is a flow chart illustrating an example of inter prediction by normal merge mode in the decoder;

[0112] FIG. 86 is a flow chart illustrating an example of inter prediction by FRUC mode in the decoder;

[0113] FIG. 87 is a flow chart illustrating an example of inter prediction by affine merge mode in the decoder;

[0114] FIG. 88 is a flow chart illustrating an example of inter prediction by affine inter mode in the decoder;

[0115] FIG. 89 is a flow chart illustrating an example of inter prediction by triangle mode in the decoder;

[0116] FIG. 90 is a flow chart illustrating an example of motion estimation by DMVR in the decoder;

[0117] FIG. 91 is a flow chart illustrating one specific example of motion estimation by DMVR in the decoder;

[0118] FIG. 92 is a flow chart illustrating one example of generation of a prediction image in the decoder;

[0119] FIG. 93 is a flow chart illustrating another example of generation of a prediction image in the decoder;

[0120] FIG. 94 is a flow chart illustrating another example of correction of a prediction image by OBMC in the decoder;

[0121] FIG. 95 is a flow chart illustrating another example of correction of a prediction image by BIO in the decoder;

[0122] FIG. 96 is a flow chart illustrating another example of correction of a prediction image by LIC in the decoder;

[0123] FIG. 97 is a flow chart indicating a basic coefficient encoding method according to Aspect 1;

[0124] FIG. 98 is a flow chart indicating a basic first encoding scheme according to Aspect 1;

[0125] FIG. 99 is a flow chart indicating a basic second encoding scheme according to Aspect 1;

[0126] FIG. 100 is a flow chart indicating a coefficient encoding method according to a first example in Aspect 1;

[0127] FIG. 101 is a flow chart indicating a coefficient encoding method according to a second example in Aspect 1;

[0128] FIG. 102 is a flow chart indicating a coefficient encoding method according to a first example in Aspect 2;

[0129] FIG. 103 is a flow chart indicating a coefficient encoding method according to a second example in Aspect 2;

[0130] FIG. 104 is a syntax diagram indicating a basic first encoding scheme according to Aspect 3;

[0131] FIG. 105 is a syntax diagram indicating a basic second encoding scheme according to Aspect 3;

[0132] FIG. 106 is a syntax diagram indicating a second encoding scheme according to a first example in Aspect 3;

[0133] FIG. 107 is a syntax diagram indicating a second encoding scheme according to a second example in Aspect 3;

[0134] FIG. 108 is a relationship diagram indicating basic correspondences between coefficient information flags and context numbers according to Aspect 4;

[0135] FIG. 109 is a conceptual diagram indicating basic neighboring coefficients according to Aspect 4;

[0136] FIG. 110 is a conceptual diagram indicating neighboring coefficients according to a first example in Aspect 4;

[0137] FIG. 111A is a conceptual diagram indicating a current coefficient to be encoded that is present in a position in an edge in a horizontal direction;

[0138] FIG. 111B is a conceptual diagram indicating the current coefficient to be encoded that is present in a position in an edge in a vertical direction;

[0139] FIG. 112A is a conceptual diagram indicating a current coefficient to be encoded that is present neighboring a position in an edge in a horizontal direction;

[0140] FIG. 112B is a conceptual diagram indicating a current coefficient to be encoded that is present neighboring a position in an edge in a vertical direction;

[0141] FIG. 113 is a conceptual diagram indicating correspondences between residual values and Golomb-Rice code according to a third example in Aspect 4;

[0142] FIG. 114 is a flow chart indicating a basic coefficient encoding method according to Aspect 5;

[0143] FIG. 115 is a flow chart indicating a third encoding scheme according to Aspect 5;

[0144] FIG. 116 is a conceptual diagram indicating neighboring coefficients according to Aspect 5;

[0145] FIG. 117 is a conceptual diagram indicating a conversion process according to Aspect 5;

[0146] FIG. 118 is a flow chart indicating a coefficient encoding method according to a first example in Aspect 5;

[0147] FIG. 119 is a flow chart indicating a coefficient encoding method according to a second example in Aspect 5;

[0148] FIG. 120 is a flow chart indicating a basic coefficient encoding method according to Aspect 6;

[0149] FIG. 121 is a flow chart indicating a basic fourth encoding scheme according to Aspect 6;

[0150] FIG. 122 is a flow chart indicating a basic first loop process according to Aspect 6;

[0151] FIG. 123 is a conceptual diagram indicating a plurality of neighboring coefficients according to Aspect 6;

[0152] FIG. 124 is a conceptual diagram indicating level mapping according to Aspect 6;

[0153] FIG. 125 is a flow chart indicating a basic second loop process according to Aspect 6;

[0154] FIG. 126 is a flow chart indicating a basic third loop process according to Aspect 6;

[0155] FIG. 127 is a flow chart indicating a fourth encoding scheme according to an example in Aspect 6;

[0156] FIG. 128 is a flow chart indicating a first loop process according to an example in Aspect 6;

[0157] FIG. 129 is a flow chart indicating a basic coefficient encoding method according to Aspect 7;

[0158] FIG. 130 is a flow chart indicating a basic fifth encoding scheme according to Aspect 7;

[0159] FIG. 131 is a flow chart indicating a basic first loop process according to Aspect 7;

[0160] FIG. 132 is a flow chart indicating a basic second loop process according to Aspect 7;

[0161] FIG. 133 is a flow chart indicating a basic third loop process according to Aspect 7;

[0162] FIG. 134 is a flow chart indicating a basic fourth loop process according to Aspect 7;

[0163] FIG. 135 is a flow chart indicating a fourth loop process according to an example in Aspect 7;

[0164] FIG. 136 is a flowchart indicating an operation by an encoder according to an embodiment;

[0165] FIG. 137 is a flowchart indicating an operation by a decoder according to the embodiment;

[0166] FIG. 138 is a diagram illustrating an overall configuration of a content providing system for implementing a content distribution service;

[0167] FIG. 139 is a diagram illustrating an example of a display screen of a web page;

[0168] FIG. 140 is a diagram illustrating an example of a display screen of a web page;

[0169] FIG. 141 is a diagram illustrating one example of a smartphone; and

[0170] FIG. 142 is a block diagram illustrating an example of a configuration of a smartphone.

DETAILED DESCRIPTION OF THE EMBODIMENT(S)

Introduction

[0171] For example, there is a case where an encoder is capable of transforming a block in an image into data to be easily compressed, by applying orthogonal transform to the block in encoding of the block. There is another case where the encoder is capable of reducing processing delay by not applying orthogonal transform to a block in an image in encoding of the block.

[0172] The properties of the block to which orthogonal transform has been applied and the properties of the block to which orthogonal transform has not been applied are different from each other. The encoding scheme for use in the block to which orthogonal transform has been applied and the encoding scheme for use in the block to which orthogonal transform has not been applied may be different from each other.

[0173] However, in the case where inappropriate encoding scheme has been used for a block to which orthogonal transform has been applied or in the case where inappropriate encoding scheme has been used for a block to which orthogonal transform has not been applied, there is a possibility that, for example, the code amount may increase, or the processing delay may increase. Furthermore, in the case where the encoding scheme that is used for the block to which orthogonal transform has been applied and the encoding scheme that is used for the block to which orthogonal transform has not been applied are significantly different, there is a possibility that the processing becomes complex and the circuit scale increases.

[0174] In view of this, for example, an encoder according to an aspect of the present disclosure includes circuitry and memory coupled to the circuitry. In operation, the circuitry limits a total number of processes of context adaptive coding, and encodes a block in an image. In the encoding of the block, when the total number of processes is within a limited range of the total number of processes, a coefficient information flag is encoded. The coefficient information flag indicates attribution of a coefficient included in the block. In the encoding of the block, in a case where orthogonal

transformation is not applied to the block, when the coefficient information flag is encoded, a conversion process is performed on a value of the coefficient, and a converted value of the coefficient is encoded using the coefficient information flag to be encoded by the context adaptive coding. The conversion process is a process of converting the value of the coefficient using a value determined using neighboring coefficients located around the coefficient in the block. When the coefficient information flag is not encoded, the conversion process is not performed, and the value of the coefficient is encoded by Golomb-Rice coding.

[0175] With this, when the coefficient information flag is encoded by context-adaptive coding, the conversion process is performed, and when the coefficient information flag is not encoded by context-adaptive coding, the conversion process is not performed. This conversion process is performed using neighboring coefficients, and thus the conversion process is influenced by the neighboring coefficients. Accordingly, when using context-adaptive coding which is also influenced by the neighboring coefficients, this conversion process may be effective to reduce the code amount.

[0176] In other words, when the coefficient information flag is encoded by context-adaptive coding, the conversion process that may be effective to reduce the code amount is performed, and when the coefficient information flag is not encoded by context-adaptive coding, the conversion process is not performed. Accordingly, it is possible to reduce the increase in processing delay and reduce the increase in code amount.

[0177] Moreover, for example, the coefficient information flag is a flag indicating whether or not the value of the coefficient is greater than 1.

[0178] With this, it is possible to appropriately determine whether the coefficient information flag indicating whether or not the value of the coefficient is greater than 1 is encoded by context-adaptive coding, according to the limitation of the total number of processes of context-adaptive coding. Accordingly, it is possible to reduce the increase in processing delay and reduce the increase in code amount.

[0179] Moreover, for example, in the encoding of the block, in a case where the orthogonal transformation is applied to the block, when the coefficient information flag is encoded, the conversion process is not performed, and the value of the coefficient is encoded using the coefficient information flag to be encoded by the context adaptive coding, and when the coefficient information flag is not encoded, the conversion process is not performed, and the value of the coefficient is encoded by Golomb-Rice coding.

[0180] With this, the difference decreases between the encoding scheme used for a block to which orthogonal transform is applied and the encoding scheme used for a block to which orthogonal transform is not applied, and thus it is possible to reduce the circuit scale.

[0181] Moreover, for example, the conversion process includes: when the value of the coefficient is equal to the value determined using the neighboring coefficients, converting the value of the coefficient to 1; and when the value of the coefficient is greater than 0 and less than the value determined using the neighboring coefficients, converting the value of the coefficient by adding 1 to the value of the coefficient.

[0182] With this, the value of the coefficient is appropriately converted according to the neighboring coefficients, and thus it is possible to reduce the increase in code amount.

[0183] Moreover, for example, a decoder according to an aspect of the present disclosure includes circuitry and memory coupled to the circuitry. In operation, the circuitry limits a total number of processes of context adaptive decoding, and decodes a block in an image. In the decoding of the block, when the total number of processes is within a limited range of the total number of processes, a coefficient information flag is decoded. The coefficient information flag indicates attribution of a coefficient included in the block. In the decoding of the block, in a case where inverse orthogonal transformation is not applied to the block, when the coefficient information flag is decoded, a value of the coefficient is decoded using the coefficient information flag to be decoded by the context adaptive decoding, and a conversion process is performed on the value of the coefficient to derive a converted value of the coefficient. The conversion process is a process of converting the value of the coefficient using a value determined using neighboring coefficients located around the coefficient in the block. When the coefficient information flag is not decoded, the value of the coefficient is decoded by Golomb-Rice decoding, to derive the value of the coefficient without performing the conversion process.

[0184] With this, when the coefficient information flag is decoded by context-adaptive decoding, the conversion process is performed, and when the coefficient information flag is not decoded by context-adaptive decoding, the conversion process is not performed. This conversion process is performed using neighboring coefficients, and thus the conversion process is influenced by the neighboring coefficients. Accordingly, when using context-adaptive decoding which is also influenced by the neighboring coefficients, this conversion process may be effective to reduce the code amount.

[0185] In other words, when the coefficient information flag is decoded by context-adaptive decoding, the conversion process that may be effective to reduce the code amount is performed, and when the coefficient information flag is not decoded by context-adaptive decoding, the conversion process is not performed. Accordingly, it is possible to reduce the increase in processing delay and reduce the increase in code amount.

[0186] Moreover, for example, the coefficient information flag is a flag indicating whether or not the value of the coefficient is greater than 1.

[0187] With this, it is possible to appropriately determine whether the coefficient information flag indicating whether or not the value of the coefficient is greater than 1 is decoded by context-adaptive decoding, according to the limitation of the total number of processes of context-adaptive decoding. Accordingly, it is possible to reduce the increase in processing delay and reduce the increase in code amount.

[0188] Moreover, for example, in the decoding of the block, in a case where the inverse orthogonal transformation is applied to the block, when the coefficient information flag is decoded, the value of the coefficient is decoded using the coefficient information flag to be decoded by the context adaptive decoding, to derive the value of the coefficient without performing the conversion process, and when the coefficient information flag is not decoded, the value of the coefficient is decoded by Golomb-Rice decoding, to derive the value of the coefficient without performing the conversion process.

[0189] With this, the difference decreases between the decoding scheme used for a block to which inverse orthogo-

nal transform is applied and the decoding scheme used for a block to which inverse orthogonal transform is not applied, and thus it is possible to reduce the circuit scale.

[0190] Moreover, for example, the conversion process includes: when the value of the coefficient is equal to 1, converting the value of the coefficient to the value determined using the neighboring coefficients; and when the value of the coefficient is greater than 0 and less than or equal to the value determined using the neighboring coefficients, converting the value of the coefficient by subtracting 1 from the value of the coefficient.

[0191] With this, the value of the coefficient is appropriately converted according to the neighboring coefficients, and thus it is possible to reduce the increase in code amount.

[0192] Moreover, for example, an encoding method includes limiting a total number of processes of context adaptive coding, and encoding a block in an image. In the encoding of the block, when the total number of processes is within a limited range of the total number of processes, a coefficient information flag is encoded. The coefficient information flag indicates attribution of a coefficient included in the block. In the encoding of the block, in a case where orthogonal transformation is not applied to the block, when the coefficient information flag is encoded, a conversion process is performed on a value of the coefficient, and a converted value of the coefficient is encoded using the coefficient information flag to be encoded by the context adaptive coding. The conversion process is a process of converting the value of the coefficient using a value determined using neighboring coefficients located around the coefficient in the block. When the coefficient information flag is not encoded, the conversion process is not performed, and the value of the coefficient is encoded by Golomb-Rice coding.

[0193] With this, when the coefficient information flag is encoded by context-adaptive coding, the conversion process is performed, and when the coefficient information flag is not encoded by context-adaptive coding, the conversion process is not performed. This conversion process is performed using neighboring coefficients, and thus the conversion process is influenced by the neighboring coefficients. Accordingly, when using context-adaptive coding which is also influenced by the neighboring coefficients, this conversion process may be effective to reduce the code amount.

[0194] In other words, when the coefficient information flag is encoded by context-adaptive coding, the conversion process that may be effective to reduce the code amount is performed, and when the coefficient information flag is not encoded by context-adaptive coding, the conversion process is not performed. Accordingly, it is possible to reduce the increase in processing delay and reduce the increase in code amount.

[0195] Moreover, for example, a decoding method includes limiting a total number of processes of context adaptive decoding, and decoding a block in an image. In the decoding of the block, when the total number of processes is within a limited range of the total number of processes, a coefficient information flag is decoded. The coefficient information flag indicates attribution of a coefficient included in the block. In the decoding of the block, in a case where inverse orthogonal transformation is not applied to the block, when the coefficient information flag is decoded, a value of the coefficient is decoded using the coefficient information flag to be decoded by the context adaptive

decoding, and a conversion process is performed on the value of the coefficient to derive a converted value of the coefficient. The conversion process is a process of converting the value of the coefficient using a value determined using neighboring coefficients located around the coefficient in the block. When the coefficient information flag is not decoded, the value of the coefficient is decoded by Golomb-Rice decoding, to derive the value of the coefficient without performing the conversion process.

[0196] With this, when the coefficient information flag is decoded by context-adaptive decoding, the conversion process is performed, and when the coefficient information flag is not decoded by context-adaptive decoding, the conversion process is not performed. This conversion process is performed using neighboring coefficients, and thus the conversion process is influenced by the neighboring coefficients. Accordingly, when using context-adaptive decoding which is also influenced by the neighboring coefficients, this conversion process may be effective to reduce the code amount.

[0197] In other words, when the coefficient information flag is decoded by context-adaptive decoding, the conversion process that may be effective to reduce the code amount is performed, and when the coefficient information flag is not decoded by context-adaptive decoding, the conversion process is not performed. Accordingly, it is possible to reduce the increase in processing delay and reduce the increase in code amount.

[0198] Moreover, for example, an encoder according to an aspect of the present disclosure includes a splitter, an intra predictor, an inter predictor, a prediction controller, a transformer, a quantizer, an entropy encoder, and a loop filter.

[0199] The splitter splits a current picture to be encoded included in the video into a plurality of blocks. The intra predictor performs intra prediction for generating a prediction image of a current block to be encoded in the current picture, using a reference image in the current picture. The inter predictor performs inter prediction for generating a prediction image of a current block to be encoded, using a reference image in a reference picture different from the current picture.

[0200] The prediction controller controls the intra prediction which is performed by the intra predictor and the inter prediction which is performed by the inter predictor. The transformer transforms prediction residual signals between the prediction image generated by either the intra predictor or the inter predictor and the image of the current block, to generate transform coefficient signals of the current block. The quantizer quantizes the transform coefficient signals. The entropy encoder encodes the quantized transform coefficient signals. The loop filter applies a filter to the current block.

[0201] Moreover, for example, in operation, the entropy encoder limits a total number of processes of context adaptive coding, and encodes a block in an image. In the encoding of the block, when the total number of processes is within a limited range of the total number of processes, a coefficient information flag is encoded. The coefficient information flag indicates attribution of a coefficient included in the block. In the encoding of the block, in a case where orthogonal transformation is not applied to the block, when the coefficient information flag is encoded, a conversion process is performed on a value of the coefficient, and a converted value of the coefficient is encoded using the coefficient information flag to be encoded by the context

adaptive coding. The conversion process is a process of converting the value of the coefficient using a value determined using neighboring coefficients located around the coefficient in the block. When the coefficient information flag is not encoded, the conversion process is not performed, and the value of the coefficient is encoded by Golomb-Rice coding.

[0202] Furthermore, for example, the decoder according to an aspect of the present disclosure is a decoder which decodes a video using a prediction image, and includes an entropy decoder, an inverse quantizer, an inverse transformer, an intra predictor, an inter predictor, a prediction controller, an adder (a reconstructor), and a loop filter.

[0203] The entropy decoder decodes quantized transform coefficient signals of a current block to be decoded in a current picture to be decoded included in the video. The inverse quantizer inverse-quantizes the quantized transform coefficient signals. The inverse transformer inverse-transforms the transform coefficient signals to obtain prediction residual signals of the current block.

[0204] The intra predictor performs intra prediction for generating a prediction image of a current block to be decoded, using a reference image in the current picture. The inter predictor performs inter prediction for generating a prediction image of a current block to be decoded, using a reference image in a reference picture different from the current picture. The prediction controller controls the intra prediction which is performed by the intra predictor and the inter prediction which is performed by the inter predictor.

[0205] The adder adds the prediction image generated by either the intra predictor or the inter predictor and the prediction residual signals, to reconstruct an image of the current block. The loop filter applies a filter to the current block.

[0206] Moreover, for example, in operation, the entropy decoder limits a total number of processes of context adaptive decoding, and decodes a block in an image. In the decoding of the block, when the total number of processes is within a limited range of the total number of processes, a coefficient information flag is decoded. The coefficient information flag indicates attribution of a coefficient included in the block. In the decoding of the block, in a case where inverse orthogonal transformation is not applied to the block, when the coefficient information flag is decoded, a value of the coefficient is decoded using the coefficient information flag to be decoded by the context adaptive decoding, and a conversion process is performed on the value of the coefficient to derive a converted value of the coefficient. The conversion process is a process of converting the value of the coefficient using a value determined using neighboring coefficients located around the coefficient in the block. When the coefficient information flag is not decoded, the value of the coefficient is decoded by Golomb-Rice decoding, to derive the value of the coefficient without performing the conversion process.

[0207] Furthermore, these general or specific aspects may be implemented using a system, an apparatus, a method, an integrated circuit, a computer program, or a non-transitory computer-readable recording medium such as a CD-ROM, or any combination of systems, apparatuses, methods, integrated circuits, computer programs, or computer-readable recording media.

Definitions of Terms

[0208] The respective terms may be defined as indicated below as examples.

(1) Image

[0209] An image is a data unit configured with a set of pixels, is a picture or includes blocks smaller than a picture. Images include a still image in addition to a video.

(2) Picture

[0210] A picture is an image processing unit configured with a set of pixels, and is also referred to as a frame or a field.

(3) Block

[0211] A block is a processing unit which is a set of a particular number of pixels. The block is also referred to as indicated in the following examples. The shapes of blocks are not limited. Examples include a rectangle shape of M×N pixels and a square shape of M×M pixels for the first place, and also include a triangular shape, a circular shape, and other shapes.

Examples of Blocks

[0212] slice/tile/brick

[0213] CTU/super block/basic splitting unit

[0214] VPDU/processing splitting unit for hardware

[0215] CU/processing block unit/prediction block unit (PU)/orthogonal transform block unit (TU)/unit

[0216] sub-block

(4) Pixel/Sample

[0217] A pixel or sample is a smallest point of an image. Pixels or samples include not only a pixel at an integer position but also a pixel at a sub-pixel position generated based on a pixel at an integer position.

(5) Pixel Value/Sample Value

[0218] A pixel value or sample value is an eigen value of a pixel. Pixel or sample values naturally include a luma value, a chroma value, an RGB gradation level and also covers a depth value, or a binary value of 0 or 1.

(6) Flag

[0219] A flag indicates one or more bits, and may be, for example, a parameter or index represented by two or more bits. Alternatively, the flag may indicate not only a binary value represented by a binary number but also a multiple value represented by a number other than the binary number.

(7) Signal

[0220] A signal is the one symbolized or encoded to convey information. Signals include a discrete digital signal and an analog signal which takes a continuous value.

(8) Stream/Bitstream

[0221] A stream or bitstream is a digital data string or a digital data flow. A stream or bitstream may be one stream or may be configured with a plurality of streams having a plurality of hierarchical layers. A stream or bitstream may be

transmitted in serial communication using a single transmission path, or may be transmitted in packet communication using a plurality of transmission paths.

(9) Difference

[0222] In the case of scalar quantity, it is only necessary that a simple difference ($x-y$) and a difference calculation be included. Differences include an absolute value of a difference ($|x-y|$), a squared difference (x^2-y^2), a square root of a difference ($\sqrt{x-y}$), a weighted difference ($ax-by$: a and b are constants), an offset difference ($x-y+a$: a is an offset).

(10) Sum

[0223] In the case of scalar quantity, it is only necessary that a simple sum ($x+y$) and a sum calculation be included. Sums include an absolute value of a sum ($|x+y|$), a squared sum (x^2+y^2), a square root of a sum ($\sqrt{x+y}$), a weighted difference ($ax+by$: a and b are constants), an offset sum ($x+y+a$: a is an offset).

(11) Based on

[0224] A phrase “based on something” means that a thing other than the something may be considered. In addition, “based on” may be used in a case in which a direct result is obtained or a case in which a result is obtained through an intermediate result.

(12) Used, Using

[0225] A phrase “something used” or “using something” means that a thing other than the something may be considered. In addition, “used” or “using” may be used in a case in which a direct result is obtained or a case in which a result is obtained through an intermediate result.

(13) Prohibit, Forbid

[0226] The term “prohibit” or “forbid” can be rephrased as “does not permit” or “does not allow”. In addition, “being not prohibited/forbidden” or “being permitted/allowed” does not always mean “obligation”.

(14) Limit, Restriction/Restrict/Restricted

[0227] The term “limit” or “restriction/restrict/restricted” can be rephrased as “does not permit/allow” or “being not permitted/allowed”. In addition, “being not prohibited/forbidden” or “being permitted/allowed” does not always mean “obligation”. Furthermore, it is only necessary that part of something be prohibited/forbidden quantitatively or qualitatively, and something may be fully prohibited/forbidden.

(15) Chroma

[0228] An adjective, represented by the symbols C_b and C_r , specifying that a sample array or single sample is representing one of the two color difference signals related to the primary colors. The term chroma may be used instead of the term chrominance.

(16) Luma

[0229] An adjective, represented by the symbol or subscript Y or L , specifying that a sample array or single sample

is representing the monochrome signal related to the primary colors. The term luma may be used instead of the term luminance.

Notes Related to the Descriptions

[0230] In the drawings, same reference numbers indicate same or similar components. The sizes and relative locations of components are not necessarily drawn by the same scale.

[0231] Hereinafter, embodiments will be described with reference to the drawings. Note that the embodiments described below each show a general or specific example. The numerical values, shapes, materials, components, the arrangement and connection of the components, steps, the relation and order of the steps, etc., indicated in the following embodiments are mere examples, and are not intended to limit the scope of the claims.

[0232] Embodiments of an encoder and a decoder will be described below. The embodiments are examples of an encoder and a decoder to which the processes and/or configurations presented in the description of aspects of the present disclosure are applicable. The processes and/or configurations can also be implemented in an encoder and a decoder different from those according to the embodiments. For example, regarding the processes and/or configurations as applied to the embodiments, any of the following may be implemented:

[0233] (1) Any of the components of the encoder or the decoder according to the embodiments presented in the description of aspects of the present disclosure may be substituted or combined with another component presented anywhere in the description of aspects of the present disclosure.

[0234] (2) In the encoder or the decoder according to the embodiments, discretionary changes may be made to functions or processes performed by one or more components of the encoder or the decoder, such as addition, substitution, removal, etc., of the functions or processes. For example, any function or process may be substituted or combined with another function or process presented anywhere in the description of aspects of the present disclosure.

[0235] (3) In methods implemented by the encoder or the decoder according to the embodiments, discretionary changes may be made such as addition, substitution, and removal of one or more of the processes included in the method. For example, any process in the method may be substituted or combined with another process presented anywhere in the description of aspects of the present disclosure.

[0236] (4) One or more components included in the encoder or the decoder according to embodiments may be combined with a component presented anywhere in the description of aspects of the present disclosure, may be combined with a component including one or more functions presented anywhere in the description of aspects of the present disclosure, and may be combined with a component that implements one or more processes implemented by a component presented in the description of aspects of the present disclosure.

[0237] (5) A component including one or more functions of the encoder or the decoder according to the embodiments, or a component that implements one or more processes of the encoder or the decoder according to the embodiments, may be combined or substituted

with a component presented anywhere in the description of aspects of the present disclosure, with a component including one or more functions presented anywhere in the description of aspects of the present disclosure, or with a component that implements one or more processes presented anywhere in the description of aspects of the present disclosure.

[0238] (6) In methods implemented by the encoder or the decoder according to the embodiments, any of the processes included in the method may be substituted or combined with a process presented anywhere in the description of aspects of the present disclosure or with any corresponding or equivalent process.

[0239] (7) One or more processes included in methods implemented by the encoder or the decoder according to the embodiments may be combined with a process presented anywhere in the description of aspects of the present disclosure.

[0240] (8) The implementation of the processes and/or configurations presented in the description of aspects of the present disclosure is not limited to the encoder or the decoder according to the embodiments. For example, the processes and/or configurations may be implemented in a device used for a purpose different from the moving picture encoder or the moving picture decoder disclosed in the embodiments.

[System Configuration]

[0241] FIG. 1 is a schematic diagram illustrating one example of a configuration of a transmission system according to an embodiment.

[0242] Transmission system Trs is a system which transmits a stream generated by encoding an image and decodes the transmitted stream. Transmission system Trs like this includes, for example, encoder 100, network Nw, and decoder 200 as illustrated in FIG. 1.

[0243] An image is input to encoder 100. Encoder 100 generates a stream by encoding the input image, and outputs the stream to network Nw. The stream includes, for example, the encoded image and control information for decoding the encoded image. The image is compressed by the encoding.

[0244] It is to be noted that a previous image before being encoded and being input to encoder 100 is also referred to as the original image, the original signal, or the original sample. The image may be a video or a still image. The image is a generic concept of a sequence, a picture, and a block, and thus is not limited to a spatial region having a particular size and to a temporal region having a particular size unless otherwise specified. The image is an array of pixels or pixel values, and the signal representing the image or pixel values are also referred to as samples. The stream may be referred to as a bitstream, an encoded bitstream, a compressed bitstream, or an encoded signal. Furthermore, the encoder may be referred to as an image encoder or a video encoder. The encoding method performed by encoder 100 may be referred to as an encoding method, an image encoding method, or a video encoding method.

[0245] Network Nw transmits the stream generated by encoder 100 to decoder 200. Network Nw may be the Internet, the Wide Area Network (WAN), the Local Area Network (LAN), or any combination of these networks. Network Nw is not always limited to a bi-directional communication network, and may be a uni-directional communication network which transmits broadcast waves of digital

terrestrial broadcasting, satellite broadcasting, or the like. Alternatively, network Nw may be replaced by a medium such as a Digital Versatile Disc (DVD) and a Blu-Ray Disc (BD)®, etc. on which a stream is recorded.

[0246] Decoder 200 generates, for example, a decoded image which is an uncompressed image by decoding a stream transmitted by network Nw. For example, the decoder decodes a stream according to a decoding method corresponding to an encoding method by encoder 100.

[0247] It is to be noted that the decoder may also be referred to as an image decoder or a video decoder, and that the decoding method performed by decoder 200 may also be referred to as a decoding method, an image decoding method, or a video decoding method.

[Data Structure]

[0248] FIG. 2 is a diagram illustrating one example of a hierarchical structure of data in a stream. A stream includes, for example, a video sequence. As illustrated in (a) of FIG. 2, the video sequence includes a video parameter set (VPS), a sequence parameter set (SPS), a picture parameter set (PPS), supplemental enhancement information (SEI), and a plurality of pictures.

[0249] In a video having a plurality of layers, a VPS includes: a coding parameter which is common between some of the plurality of layers; and a coding parameter related to some of the plurality of layers included in the video or an individual layer.

[0250] An SPS includes a parameter which is used for a sequence, that is, a coding parameter which decoder 200 refers to in order to decode the sequence. For example, the coding parameter may indicate the width or height of a picture. It is to be noted that a plurality of SPSs may be present.

[0251] A PPS includes a parameter which is used for a picture, that is, a coding parameter which decoder 200 refers to in order to decode each of the pictures in the sequence. For example, the coding parameter may include a reference value for the quantization width which is used to decode a picture and a flag indicating application of weighted prediction. It is to be noted that a plurality of PPSs may be present. Each of the SPS and the PPS may be simply referred to as a parameter set.

[0252] As illustrated in (b) of FIG. 2, a picture may include a picture header and at least one slice. A picture header includes a coding parameter which decoder 200 refers to in order to decode the at least one slice.

[0253] As illustrated in (c) of FIG. 2, a slice includes a slice header and at least one brick. A slice header includes a coding parameter which decoder 200 refers to in order to decode the at least one brick.

[0254] As illustrated in (d) of FIG. 2, a brick includes at least one coding tree unit (CTU).

[0255] It is to be noted that a picture may not include any slice and may include a tile group instead of a slice. In this case, the tile group includes at least one tile. In addition, a brick may include a slice.

[0256] A CTU is also referred to as a super block or a basis splitting unit. As illustrated in (e) of FIG. 2, a CTU like this includes a CTU header and at least one coding unit (CU). A CTU header includes a coding parameter which decoder 200 refers to in order to decode the at least one CU.

[0257] A CU may be split into a plurality of smaller CUs. As illustrated in (f) of FIG. 2, a CU includes a CU header,

prediction information, and residual coefficient information. Prediction information is information for predicting the CU, and the residual coefficient information is information indicating a prediction residual to be described later. Although a CU is basically the same as a prediction unit (PU) and a transform unit (TU), it is to be noted that, for example, an SBT to be described later may include a plurality of TUs smaller than the CU. In addition, the CU may be processed for each virtual pipeline decoding unit (VPDU) included in the CU. The VPDU is, for example, a fixed unit which can be processed at one stage when pipeline processing is performed in hardware.

[0258] It is to be noted that a stream may not include part of the hierarchical layers illustrated in FIG. 2. The order of the hierarchical layers may be exchanged, or any of the hierarchical layers may be replaced by another hierarchical layer. Here, a picture which is a target for a process which is about to be performed by a device such as encoder **100** or decoder **200** is referred to as a current picture. A current picture means a current picture to be encoded when the process is an encoding process, and a current picture means a current picture to be decoded when the process is a decoding process. Likewise, for example, a CU or a block of CUs which is a target for a process which is about to be performed by a device such as encoder **100** or decoder **200** is referred to as a current block. A current block means a current block to be encoded when the process is an encoding process, and a current block means a current block to be decoded when the process is a decoding process.

[Picture Structure: Slice/Tile]

[0259] A picture may be configured with one or more slice units or tile units in order to decode the picture in parallel.

[0260] Slices are basic encoding units included in a picture. A picture may include, for example, one or more slices. In addition, a slice includes one or more successive coding tree units (CTUs).

[0261] FIG. 3 is a diagram illustrating one example of a slice configuration. For example, a picture includes 11×8 CTUs, and is split into four slices (slices 1 to 4). Slice 1 includes sixteen CTUs, slice 2 includes twenty-one CTUs, slice 3 includes twenty-nine CTUs, and slice 4 includes twenty-two CTUs. Here, each CTU in the picture belongs to one of the slices. The shape of each slice is a shape obtained by splitting the picture horizontally. A boundary of each slice does not need to coincide with an image end, and may coincide with any of the boundaries between CTUs in the image. The processing order of the CTUs in a slice (an encoding order or a decoding order) is, for example, a raster-scan order. A slice includes a slice header and encoded data. Features of the slice may be written in the slice header. The features include a CTU address of a top CTU in the slice, a slice type, etc.

[0262] A tile is a unit of a rectangular region included in a picture. Each of tiles may be assigned with a number referred to as TileId in raster-scan order.

[0263] FIG. 4 is a diagram illustrating one example of a tile configuration. For example, a picture includes 11×8 CTUs, and is split into four tiles of rectangular regions (tiles 1 to 4). When tiles are used, the processing order of CTUs is changed from the processing order in the case where no tile is used. When no tile is used, a plurality of CTUs in a picture are processed in raster-scan order. When a plurality of tiles are used, at least one CTU in each of the plurality of

tiles is processed in raster-scan order. For example, as illustrated in FIG. 4, the processing order of the CTUs included in tile 1 is the order which starts from the left-end of the first column of tile 1 toward the right-end of the first column of tile 1 and then starts from the left-end of the second column of tile 1 toward the right-end of the second column of tile 1.

[0264] It is to be noted that one tile may include one or more slices, and one slice may include one or more tiles.

[0265] It is to be noted that a picture may be configured with one or more tile sets. A tile set may include one or more tile groups, or one or more tiles. A picture may be configured with only one of a tile set, a tile group, and a tile. For example, an order for scanning a plurality of tiles for each tile set in raster scan order is assumed to be a basic encoding order of tiles. A set of one or more tiles which are continuous in the basic encoding order in each tile set is assumed to be a tile group. Such a picture may be configured by splitter **102** (see FIG. 7) to be described later.

[Scalable Encoding]

[0266] FIGS. 5 and 6 are diagrams illustrating examples of scalable stream structures.

[0267] As illustrated in FIG. 5, encoder **100** may generate a temporally/spatially scalable stream by dividing each of a plurality of pictures into any of a plurality of layers and encoding the picture in the layer. For example, encoder **100** encodes the picture for each layer, thereby achieving scalability where an enhancement layer is present above a base layer. Such encoding of each picture is also referred to as scalable encoding. In this way, decoder **200** is capable of switching image quality of an image which is displayed by decoding the stream. In other words, decoder **200** determines up to which layer to decode based on internal factors such as the processing ability of decoder **200** and external factors such as a state of a communication bandwidth. As a result, decoder **200** is capable of decoding a content while freely switching between low resolution and high resolution. For example, the user of the stream watches a video of the stream halfway using a smartphone on the way to home, and continues watching the video at home on a device such as a TV connected to the Internet. It is to be noted that each of the smartphone and the device described above includes decoder **200** having the same or different performances. In this case, when the device decodes layers up to the higher layer in the stream, the user can watch the video at high quality at home. In this way, encoder **100** does not need to generate a plurality of streams having different image qualities of the same content, and thus the processing load can be reduced.

[0268] Furthermore, the enhancement layer may include meta information based on statistical information on the image. Decoder **200** may generate a video whose image quality has been enhanced by performing super-resolution imaging on a picture in the base layer based on the metadata. Super-resolution imaging may be any of improvement in the Signal-to-Noise (SN) ratio in the same resolution and increase in resolution. Metadata may include information for identifying a linear or a non-linear filter coefficient, as used in a super-resolution process, or information identifying a parameter value in a filter process, machine learning, or a least squares method used in super-resolution processing.

[0269] Alternatively, a configuration may be provided in which a picture is divided into, for example, tiles in accor-

dance with, for example, the meaning of an object in the picture. In this case, decoder **200** may decode only a partial region in a picture by selecting a tile to be decoded. In addition, an attribute of the object (person, car, ball, etc.) and a position of the object in the picture (coordinates in identical images) may be stored as metadata. In this case, decoder **200** is capable of identifying the position of a desired object based on the metadata, and determining the tile including the object. For example, as illustrated in FIG. 6, the metadata may be stored using a data storage structure different from image data, such as SEI in HEVC. This metadata indicates, for example, the position, size, or color of a main object.

[0270] Metadata may be stored in units of a plurality of pictures, such as a stream, a sequence, or a random access unit. In this way, decoder **200** is capable of obtaining, for example, the time at which a specific person appears in the video, and by fitting the time information with picture unit information, is capable of identifying a picture in which the object is present and determining the position of the object in the picture.

[Encoder]

[0271] Next, encoder **100** according to this embodiment is described. FIG. 7 is a block diagram illustrating one example of a configuration of encoder **100** according to this embodiment. Encoder **100** encodes an image in units of a block.

[0272] As illustrated in FIG. 7, encoder **100** is an apparatus which encodes an image in units of a block, and includes splitter **102**, subtractor **104**, transformer **106**, quantizer **108**, entropy encoder **110**, inverse quantizer **112**, inverse transformer **114**, adder **116**, block memory **118**, loop filter **120**, frame memory **122**, intra predictor **124**, inter predictor **126**, prediction controller **128**, and prediction parameter generator **130**. It is to be noted that intra predictor **124** and inter predictor **126** are configured as part of a prediction executor.

[Mounting Example of Encoder]

[0273] FIG. 8 is a block diagram illustrating a mounting example of encoder **100**. Encoder **100** includes processor **a1** and memory **a2**. For example, the plurality of constituent elements of encoder **100** illustrated in FIG. 7 are mounted on processor **a1** and memory **a2** illustrated in FIG. 8.

[0274] Processor **a1** is circuitry which performs information processing and is accessible to memory **a2**. For example, processor **a1** is dedicated or general electronic circuitry which encodes an image. Processor **a1** may be a processor such as a CPU. In addition, processor **a1** may be an aggregate of a plurality of electronic circuits. In addition, for example, processor **a1** may take the roles of two or more constituent elements other than a constituent element for storing information out of the plurality of constituent elements of encoder **100** illustrated in FIG. 7, etc.

[0275] Memory **a2** is dedicated or general memory for storing information that is used by processor **a1** to encode the image. Memory **a2** may be electronic circuitry, and may be connected to processor **a1**. In addition, memory **a2** may be included in processor **a1**. In addition, memory **a2** may be an aggregate of a plurality of electronic circuits. In addition, memory **a2** may be a magnetic disc, an optical disc, or the

like, or may be represented as storage, a medium, or the like. In addition, memory **a2** may be non-volatile memory, or volatile memory.

[0276] For example, memory **a2** may store an image to be encoded or a stream corresponding to an encoded image. In addition, memory **a2** may store a program for causing processor **a1** to encode an image.

[0277] In addition, for example, memory **a2** may take the roles of two or more constituent elements for storing information out of the plurality of constituent elements of encoder **100** illustrated in FIG. 7. More specifically, memory **a2** may take the roles of block memory **118** and frame memory **122** illustrated in FIG. 7. More specifically, memory **a2** may store a reconstructed image (specifically, a reconstructed block, a reconstructed picture, or the like).

[0278] It is to be noted that, in encoder **100**, not all of the plurality of constituent elements indicated in FIG. 7, etc. may be implemented, and not all the processes described above may be performed. Part of the constituent elements indicated in FIG. 7 may be included in another device, or part of the processes described above may be performed by another device.

[0279] Hereinafter, an overall flow of processes performed by encoder **100** is described, and then each of constituent elements included in encoder **100** is described.

[Overall Flow of Encoding Process]

[0280] FIG. 9 is a flow chart illustrating one example of an overall encoding process performed by encoder **100**.

[0281] First, splitter **102** of encoder **100** splits each of pictures included in an original image into a plurality of blocks having a fixed size (128×128 pixels) (Step Sa_1). Splitter **102** then selects a splitting pattern for the fixed-size block (Step Sa_2). In other words, splitter **102** further splits the fixed-size block into a plurality of blocks which form the selected splitting pattern. Encoder **100** performs, for each of the plurality of blocks, Steps Sa_3 to Sa_9 for the block.

[0282] Prediction controller **128** and a prediction executor which is configured with intra predictor **124** and inter predictor **126** generate a prediction image of a current block (Step Sa_3). It is to be noted that the prediction image is also referred to as a prediction signal, a prediction block, or prediction samples.

[0283] Next, subtractor **104** generates the difference between a current block and a prediction image as a prediction residual (Step Sa_4). It is to be noted that the prediction residual is also referred to as a prediction error.

[0284] Next, transformer **106** transforms the prediction image and quantizer **108** quantizes the result, to generate a plurality of quantized coefficients (Step Sa_5).

[0285] Next, entropy encoder **110** encodes (specifically, entropy encodes) the plurality of quantized coefficients and a prediction parameter related to generation of a prediction image to generate a stream (Step Sa_6).

[0286] Next, inverse quantizer **112** performs inverse quantization of the plurality of quantized coefficients and inverse transformer **114** performs inverse transform of the result, to restore a prediction residual (Step Sa_7).

[0287] Next, adder **116** adds the prediction image to the restored prediction residual to reconstruct the current block (Step Sa_8). In this way, the reconstructed image is generated. It is to be noted that the reconstructed image is also referred to as a reconstructed block, and, in particular, that

a reconstructed image generated by encoder **100** is also referred to as a local decoded block or a local decoded image.

[0288] When the reconstructed image is generated, loop filter **120** performs filtering of the reconstructed image as necessary (Step Sa_9).

[0289] Encoder **100** then determines whether encoding of the entire picture has been finished (Step Sa_10). When determining that the encoding has not yet been finished (No in Step Sa_10), processes from Step Sa_2 are executed repeatedly.

[0290] Although encoder **100** selects one splitting pattern for a fixed-size block, and encodes each block according to the splitting pattern in the above-described example, it is to be noted that each block may be encoded according to a corresponding one of a plurality of splitting patterns. In this case, encoder **100** may evaluate a cost for each of the plurality of splitting patterns, and, for example, may select the stream obtained by encoding according to the splitting pattern which yields the smallest cost as a stream which is output finally.

[0291] Alternatively, the processes in Steps Sa_1 to Sa_10 may be performed sequentially by encoder **100**, or two or more of the processes may be performed in parallel or may be reordered.

[0292] The encoding process by encoder **100** is hybrid encoding using prediction encoding and transform encoding. In addition, prediction encoding is performed by an encoding loop configured with subtractor **104**, transformer **106**, quantizer **108**, inverse quantizer **112**, inverse transformer **114**, adder **116**, loop filter **120**, block memory **118**, frame memory **122**, intra predictor **124**, inter predictor **126**, and prediction controller **128**. In other words, the prediction executor configured with intra predictor **124** and inter predictor **126** is part of the encoding loop.

[Splitter]

[0293] Splitter **102** splits each of pictures included in the original image into a plurality of blocks, and outputs each block to subtractor **104**. For example, splitter **102** first splits a picture into blocks of a fixed size (for example, 128×128 pixels). The fixed-size block is also referred to as a coding tree unit (CTU). Splitter **102** then splits each fixed-size block into blocks of variable sizes (for example, 64×64 pixels or smaller), based on recursive quadtree and/or binary tree block splitting. In other words, splitter **102** selects a splitting pattern. The variable-size block is also referred to as a coding unit (CU), a prediction unit (PU), or a transform unit (TU). It is to be noted that, in various kinds of mounting examples, there is no need to differentiate between CU, PU, and TU; all or some of the blocks in a picture may be processed in units of a CU, a PU, or a TU.

[0294] FIG. 10 is a diagram illustrating one example of block splitting according to this embodiment. In FIG. 10, the solid lines represent block boundaries of blocks split by quadtree block splitting, and the dashed lines represent block boundaries of blocks split by binary tree block splitting.

[0295] Here, block **10** is a square block having 128×128 pixels. This block **10** is first split into four square 64×64 pixel blocks (quadtree block splitting).

[0296] The upper-left 64×64 pixel block is further vertically split into two rectangle 32×64 pixel blocks, and the left 32×64 pixel block is further vertically split into two rect-

angle 16×64 pixel blocks (binary tree block splitting). As a result, the upper-left square 64×64 pixel block is split into two 16×64 pixel blocks **11** and **12** and one 32×64 pixel block **13**.

[0297] The upper-right square 64×64 pixel block is horizontally split into two rectangle 64×32 pixel blocks **14** and **15** (binary tree block splitting).

[0298] The lower-left square 64×64 pixel block is first split into four square 32×32 pixel blocks (quadtree block splitting). The upper-left block and the lower-right block among the four square 32×32 pixel blocks are further split. The upper-left square 32×32 pixel block is vertically split into two rectangle 16×32 pixel blocks, and the right 16×32 pixel block is further horizontally split into two 16×16 pixel blocks (binary tree block splitting). The lower-right 32×32 pixel block is horizontally split into two 32×16 pixel blocks (binary tree block splitting). The upper-right square 32×32 pixel block is horizontally split into two rectangle 32×16 pixel blocks (binary tree block splitting). As a result, the lower-left square 64×64 pixel block is split into rectangle 16×32 pixel block **16**, two square 16×16 pixel blocks **17** and **18**, two square 32×32 pixel blocks **19** and **20**, and two rectangle 32×16 pixel blocks **21** and **22**.

[0299] The lower-right 64×64 pixel block **23** is not split.

[0300] As described above, in FIG. 10, block **10** is split into thirteen variable-size blocks **11** through **23** based on recursive quadtree and binary tree block splitting. Such splitting is also referred to as quad-tree plus binary tree splitting (QTBT).

[0301] It is to be noted that, in FIG. 10, one block is split into four or two blocks (quadtree or binary tree block splitting), but splitting is not limited to these examples. For example, one block may be split into three blocks (ternary block splitting). Splitting including such ternary block splitting is also referred to as multi type tree (MBT) splitting.

[0302] FIG. 11 is a diagram illustrating one example of a configuration of splitter **102**. As illustrated in FIG. 11, splitter **102** may include block splitting determiner **102a**. Block splitting determiner **102a** may perform the following processes as examples.

[0303] For example, block splitting determiner **102a** collects block information from either block memory **118** or frame memory **122**, and determines the above-described splitting pattern based on the block information. Splitter **102** splits the original image according to the splitting pattern, and outputs at least one block obtained by the splitting to subtractor **104**.

[0304] In addition, for example, block splitting determiner **102a** outputs a parameter indicating the above-described splitting pattern to transformer **106**, inverse transformer **114**, intra predictor **124**, inter predictor **126**, and entropy encoder **110**. Transformer **106** may transform a prediction residual based on the parameter. Intra predictor **124** and inter predictor **126** may generate a prediction image based on the parameter. In addition, entropy encoder **110** may entropy encode the parameter.

[0305] The parameter related to the splitting pattern may be written in a stream as indicated below as one example.

[0306] FIG. 12 is a diagram illustrating examples of splitting patterns. Examples of splitting patterns include: splitting into four regions (QT) in which a block is split into two regions both horizontally and vertically; splitting into three regions (HT or VT) in which a block is split in the same direction in a ratio of 1:2:1; splitting into two regions (HB

or VB) in which a block is split in the same direction in a ratio of 1:1; and no splitting (NS).

[0307] It is to be noted that the splitting pattern does not have any block splitting direction in the case of splitting into four regions and no splitting, and that the splitting pattern has splitting direction information in the case of splitting into two regions or three regions.

[0308] FIGS. 13A and 13B are each a diagram illustrating one example of a syntax tree of a splitting pattern. In the example of FIG. 13A, first, information indicating whether to perform splitting (S: Split flag) is present, and information indicating whether to perform splitting into four regions (QT: QT flag) is present next. Information indicating which one of splitting into three regions and two regions is to be performed (TT: TT flag or BT: BT flag) is present next, and lastly, information indicating a division direction (Ver: Vertical flag or Hor: Horizontal flag) is present. It is to be noted that each of at least one block obtained by splitting according to such a splitting pattern may be further split repeatedly in a similar process. In other words, as one example, whether splitting is performed, whether splitting into four regions is performed, which one of the horizontal direction and the vertical direction is the direction in which a splitting method is to be performed, which one of splitting into three regions and splitting into two regions is to be performed may be recursively determined, and the determination results may be encoded in a stream according to the encoding order disclosed by the syntax tree illustrated in FIG. 13A.

[0309] In addition, although information items respectively indicating S, QT, TT, and Ver are arranged in the listed order in the syntax tree illustrated in FIG. 13A, information items respectively indicating S, QT, Ver, and BT may be arranged in the listed order. In other words, in the example of FIG. 13B, first, information indicating whether to perform splitting (S: Split flag) is present, and information indicating whether to perform splitting into four regions (QT: QT flag) is present next. Information indicating the splitting direction (Ver: Vertical flag or Hor: Horizontal flag) is present next, and lastly, information indicating which one of splitting into two regions and splitting into three regions is to be performed (BT: BT flag or TT: TT flag) is present.

[0310] It is to be noted that the splitting patterns described above are examples, and splitting patterns other than the described splitting patterns may be used, or part of the described splitting patterns may be used.

[Subtractor]

[0311] Subtractor 104 subtracts a prediction image (prediction image that is input from prediction controller 128) from the original image in units of a block input from splitter 102 and split by splitter 102. In other words, subtractor 104 calculates prediction residuals of a current block. Subtractor 104 then outputs the calculated prediction residuals to transformer 106.

[0312] The original signal is an input signal which has been input to encoder 100 and represents an image of each picture included in a video (for example, a luma signal and two chroma signals).

[Transformer]

[0313] Transformer 106 transforms prediction residuals in spatial domain into transform coefficients in frequency domain, and outputs the transform coefficients to quantizer

108. More specifically, transformer 106 applies, for example, a predefined discrete cosine transform (DCT) or discrete sine transform (DST) to prediction residuals in spatial domain.

[0314] It is to be noted that transformer 106 may adaptively select a transform type from among a plurality of transform types, and transform prediction residuals into transform coefficients by using a transform basis function corresponding to the selected transform type. This sort of transform is also referred to as explicit multiple core transform (EMT) or adaptive multiple transform (AMT). In addition, a transform basis function is also simply referred to as a basis.

[0315] The transform types include, for example, DCT-II, DCT-V, DCT-VIII, DST-I, and DST-VII. It is to be noted that these transform types may be represented as DCT2, DCT5, DCT8, DST1, and DST7. FIG. 14 is a chart illustrating transform basis functions for each transform type. In FIG. 14, N indicates the number of input pixels. For example, selection of a transform type from among the plurality of transform types may depend on a prediction type (one of intra prediction and inter prediction), and may depend on an intra prediction mode.

[0316] Information indicating whether to apply such EMT or AMT (referred to as, for example, an EMT flag or an AMT flag) and information indicating the selected transform type is normally signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, brick level, or CTU level).

[0317] In addition, transformer 106 may re-transform the transform coefficients (which are transform results). Such re-transform is also referred to as adaptive secondary transform (AST) or non-separable secondary transform (NSST). For example, transformer 106 performs re-transform in units of a sub-block (for example, 4×4 pixel sub-block) included in a transform coefficient block corresponding to an intra prediction residual. Information indicating whether to apply NSST and information related to a transform matrix for use in NSST are normally signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, brick level, or CTU level).

[0318] Transformer 106 may employ a separable transform and a non-separable transform. A separable transform is a method in which a transform is performed a plurality of times by separately performing a transform for each of directions according to the number of dimensions of inputs. A non-separable transform is a method of performing a collective transform in which two or more dimensions in multidimensional inputs are collectively regarded as a single dimension.

[0319] In one example of the non-separable transform, when an input is a 4×4 pixel block, the 4×4 pixel block is regarded as a single array including sixteen elements, and the transform applies a 16×16 transform matrix to the array.

[0320] In another example of the non-separable transform, an input block of 4×4 pixels is regarded as a single array including sixteen elements, and then a transform (hypercube givens transform) in which givens revolution is performed on the array a plurality of times may be performed.

[0321] In the transform in transformer 106, the transform types of transform basis functions to be transformed into the frequency domain according to regions in a CU can be switched. Examples include a spatially varying transform (SVT).

[0322] FIG. 15 is a diagram illustrating one example of SVT.

[0323] In SVT, as illustrated in FIG. 15, CUs are split into two equal regions horizontally or vertically, and only one of the regions is transformed into the frequency domain. A transform type can be set for each region. For example, DST7 and DCT8 are used. For example, among the two regions obtained by splitting a CU vertically into two equal regions, DST7 and DCT8 may be used for the region at position 0. Alternatively, among the two regions, DST7 is used for the region at position 1. Likewise, among the two regions obtained by splitting a CU horizontally into two equal regions, DST7 and DCT8 are used for the region at position 0. Alternatively, among the two regions, DST7 is used for the region at position 1. Although only one of the two regions in a CU is transformed and the other is not transformed in the example illustrated in FIG. 15, each of the two regions may be transformed. In addition, splitting method may include not only splitting into two regions but also splitting into four regions. In addition, the splitting method can be more flexible. For example, information indicating the splitting method may be encoded and may be signaled in the same manner as the CU splitting. It is to be noted that SVT is also referred to as sub-block transform (SBT).

[0324] The AMT and EMT described above may be referred to as MTS (multiple transform selection). When MTS is applied, a transform type that is DST7, DCT8, or the like can be selected, and the information indicating the selected transform type may be encoded as index information for each CU. There is another process referred to as IMTS (implicit MTS) as a process for selecting, based on the shape of a CU, a transform type to be used for orthogonal transform performed without encoding index information. When IMTS is applied, for example, when a CU has a rectangle shape, orthogonal transform of the rectangle shape is performed using DST7 for the short side and DCT2 for the long side. In addition, for example, when a CU has a square shape, orthogonal transform of the rectangle shape is performed using DCT2 when MTS is effective in a sequence and using DST7 when MTS is ineffective in the sequence. DCT2 and DST7 are mere examples. Other transform types may be used, and it is also possible to change the combination of transform types for use to a different combination of transform types. IMTS may be used only for intra prediction blocks, or may be used for both intra prediction blocks and inter prediction block.

[0325] The three processes of MTS, SBT, and IMTS have been described above as selection processes for selectively switching transform types for use in orthogonal transform. However, all of the three selection processes may be made effective, or only part of the selection processes may be selectively made effective. Whether each of the selection processes is made effective can be identified based on flag information or the like in a header such as an SPS. For example, when all of the three selection processes are effective, one of the three selection processes is selected for each CU and orthogonal transform of the CU is performed. It is to be noted that the selection processes for selectively

switching the transform types may be selection processes different from the above three selection processes, or each of the three selection processes may be replaced by another process as long as at least one of the following four functions [1] to [4] can be achieved. Function [1] is a function for performing orthogonal transform of the entire CU and encoding information indicating the transform type used in the transform. Function [2] is a function for performing orthogonal transform of the entire CU and determining the transform type based on a predetermined rule without encoding information indicating the transform type. Function [3] is a function for performing orthogonal transform of a partial region of a CU and encoding information indicating the transform type used in the transform. Function [4] is a function for performing orthogonal transform of a partial region of a CU and determining the transform type based on a predetermined rule without encoding information indicating the transform type used in the transform.

[0326] It is to be noted that whether each of MTS, IMTS, and SBT is applied may be determined for each processing unit. For example, whether each of MTS, IMTS, and SBT is applied may be determined for each sequence, picture, brick, slice, CTU, or CU.

[0327] It is to be noted that a tool for selectively switching transform types in the present disclosure may be rephrased by a method for selectively selecting a basis for use in a transform process, a selection process, or a process for selecting a basis. In addition, the tool for selectively switching transform types may be rephrased by a mode for adaptively selecting a transform type.

[0328] FIG. 16 is a flow chart illustrating one example of a process performed by transformer 106.

[0329] For example, transformer 106 determines whether to perform orthogonal transform (Step St_1). Here, when determining to perform orthogonal transform (Yes in Step St_1), transformer 106 selects a transform type for use in orthogonal transform from a plurality of transform types (Step St_2). Next, transformer 106 performs orthogonal transform by applying the selected transform type to the prediction residual of a current block (Step St_3). Transformer 106 then outputs information indicating the selected transform type to entropy encoder 110, so as to allow entropy encoder 110 to encode the information (Step St_4). On the other hand, when determining not to perform orthogonal transform (No in Step St_1), transformer 106 outputs information indicating that no orthogonal transform is performed, so as to allow entropy encoder 110 to encode the information (Step St_5). It is to be noted that whether to perform orthogonal transform in Step St_1 may be determined based on, for example, the size of a transform block, a prediction mode applied to the CU, etc. Alternatively, orthogonal transform may be performed using a predefined transform type without encoding information indicating the transform type for use in orthogonal transform.

[0330] FIG. 17 is a flow chart illustrating another example of a process performed by transformer 106. It is to be noted that the example illustrated in FIG. 17 is an example of orthogonal transform in the case where transform types for use in orthogonal transform are selectively switched as in the case of the example illustrated in FIG. 16.

[0331] As one example, a first transform type group may include DCT2, DST7, and DCT8. As another example, a second transform type group may include DCT2. The transform types included in the first transform type group and the

transform types included in the second transform type group may partly overlap with each other, or may be totally different from each other.

[0332] More specifically, transformer **106** determines whether a transform size is smaller than or equal to a predetermined value (Step Su_1). Here, when determining that the transform size is smaller than or equal to the predetermined value (Yes in Step Su_1), transformer **106** performs orthogonal transform of the prediction residual of the current block using the transform type included in the first transform type group (Step Su_2). Next, transformer **106** outputs information indicating the transform type to be used among at least one transform type included in the first transform type group to entropy encoder **110**, so as to allow entropy encoder **110** to encode the information (Step Su_3). On the other hand, when determining that the transform size is not smaller than or equal to the predetermined value (No in Step Su_1), transformer **106** performs orthogonal transform of the prediction residual of the current block using the second transform type group (Step Su_4).

[0333] In Step Su_3, the information indicating the transform type for use in orthogonal transform may be information indicating a combination of the transform type to be applied vertically in the current block and the transform type to be applied horizontally in the current block. The first type group may include only one transform type, and the information indicating the transform type for use in orthogonal transform may not be encoded. The second transform type group may include a plurality of transform types, and information indicating the transform type for use in orthogonal transform among the one or more transform types included in the second transform type group may be encoded.

[0334] Alternatively, a transform type may be determined based only on a transform size. It is to be noted that such determinations are not limited to the determination as to whether the transform size is smaller than or equal to the predetermined value, and other processes are also possible as long as the processes are for determining a transform type for use in orthogonal transform based on the transform size.

[Quantizer]

[0335] Quantizer **108** quantizes the transform coefficients output from transformer **106**. More specifically, quantizer **108** scans, in a determined scanning order, the transform coefficients of the current block, and quantizes the scanned transform coefficients based on quantization parameters (QP) corresponding to the transform coefficients. Quantizer **108** then outputs the quantized transform coefficients (hereinafter also referred to as quantized coefficients) of the current block to entropy encoder **110** and inverse quantizer **112**.

[0336] A determined scanning order is an order for quantizing/inverse quantizing transform coefficients. For example, a determined scanning order is defined as ascending order of frequency (from low to high frequency) or descending order of frequency (from high to low frequency).

[0337] A quantization parameter (QP) is a parameter defining a quantization step (quantization width). For example, when the value of the quantization parameter increases, the quantization step also increases. In other words, when the value of the quantization parameter increases, an error in quantized coefficients (quantization error) increases.

[0338] In addition, a quantization matrix may be used for quantization. For example, several kinds of quantization matrices may be used correspondingly to frequency transform sizes such as 4×4 and 8×8, prediction modes such as intra prediction and inter prediction, and pixel components such as luma and chroma pixel components. It is to be noted that quantization means digitalizing values sampled at predetermined intervals correspondingly to predetermined levels. In this technical field, quantization may be represented as other expressions such as rounding and scaling.

[0339] Methods using quantization matrices include a method using a quantization matrix which has been set directly at the encoder **100** side and a method using a quantization matrix which has been set as a default (default matrix). At the encoder **100** side, a quantization matrix suitable for features of an image can be set by directly setting a quantization matrix. This case, however, has a disadvantage of increasing a coding amount for encoding the quantization matrix. It is to be noted that a quantization matrix to be used to quantize the current block may be generated based on a default quantization matrix or an encoded quantization matrix, instead of directly using the default quantization matrix or the encoded quantization matrix.

[0340] There is a method for quantizing a high-frequency coefficient and a low-frequency coefficient in the same manner without using a quantization matrix. It is to be noted that this method is equivalent to a method using a quantization matrix (flat matrix) whose all coefficients have the same value.

[0341] The quantization matrix may be encoded, for example, at the sequence level, picture level, slice level, brick level, or CTU level.

[0342] When using a quantization matrix, quantizer **108** scales, for each transform coefficient, for example a quantization width which can be calculated based on a quantization parameter, etc., using the value of the quantization matrix. The quantization process performed without using any quantization matrix may be a process of quantizing transform coefficients based on a quantization width calculated based on a quantization parameter, etc. It is to be noted that, in the quantization process performed without using any quantization matrix, the quantization width may be multiplied by a predetermined value which is common for all the transform coefficients in a block.

[0343] FIG. 18 is a block diagram illustrating one example of a configuration of quantizer **108**.

[0344] For example, quantizer **108** includes difference quantization parameter generator **108a**, predicted quantization parameter generator **108b**, quantization parameter generator **108c**, quantization parameter storage **108d**, and quantization executor **108e**.

[0345] FIG. 19 is a flow chart illustrating one example of quantization performed by quantizer **108**.

[0346] As one example, quantizer **108** may perform quantization for each CU based on the flow chart illustrated in FIG. 19. More specifically, quantization parameter generator **108c** determines whether to perform quantization (Step Sv_1). Here, when determining to perform quantization (Yes in Step Sv_1), quantization parameter generator **108c** generates a quantization parameter for a current block (Step Sv_2), and stores the quantization parameter into quantization parameter storage **108d** (Step Sv_3).

[0347] Next, quantization executor **108e** quantizes transform coefficients of the current block using the quantization parameter generated in Step Sv_2 (Step Sv_4). Predicted quantization parameter generator **108b** then obtains a quantization parameter for a processing unit different from the current block from quantization parameter storage **108d** (Step Sv_5). Predicted quantization parameter generator **108b** generates a predicted quantization parameter of the current block based on the obtained quantization parameter (Step Sv_6). Difference quantization parameter generator **108a** calculates the difference between the quantization parameter of the current block generated by quantization parameter generator **108c** and the predicted quantization parameter of the current block generated by predicted quantization parameter generator **108b** (Step Sv_7). The difference quantization parameter is generated by calculating the difference. Difference quantization parameter generator **108a** outputs the difference quantization parameter to entropy encoder **110**, so as to allow entropy encoder **110** to encode the difference quantization parameter (Step Sv_8).

[0348] It is to be noted that the difference quantization parameter may be encoded, for example, at the sequence level, picture level, slice level, brick level, or CTU level. In addition, the initial value of the quantization parameter may be encoded at the sequence level, picture level, slice level, brick level, or CTU level. At this time, the quantization parameter may be generated using the initial value of the quantization parameter and the difference quantization parameter.

[0349] It is to be noted that quantizer **108** may include a plurality of quantizers, and may apply dependent quantization in which transform coefficients are quantized using a quantization method selected from a plurality of quantization methods.

[Entropy Encoder]

[0350] FIG. 20 is a block diagram illustrating one example of a configuration of entropy encoder **110**.

[0351] Entropy encoder **110** generates a stream by entropy encoding the quantized coefficients input from quantizer **108** and a prediction parameter input from prediction parameter generator **130**. For example, context-based adaptive binary arithmetic coding (CABAC) is used as the entropy encoding. More specifically, entropy encoder **110** includes binarizer **110a**, context controller **110b**, and binary arithmetic encoder **110c**. Binarizer **110a** performs binarization in which multi-level signals such as quantized coefficients and a prediction parameter are transformed into binary signals. Examples of binarization methods include truncated Rice binarization, exponential Golomb codes, and fixed length binarization. Context controller **110b** derives a context value according to a feature or a surrounding state of a syntax element, that is, an occurrence probability of a binary signal. Examples of methods for deriving a context value include bypass, referring to a syntax element, referring to an upper and left adjacent blocks, referring to hierarchical information, and others. Binary arithmetic encoder **110c** arithmetically encodes the binary signal using the derived context value.

[0352] FIG. 21 is a diagram illustrating a flow of CABAC in entropy encoder **110**.

[0353] First, initialization is performed in CABAC in entropy encoder **110**. In the initialization, initialization in binary arithmetic encoder **110c** and setting of an initial

context value are performed. For example, binarizer **110a** and binary arithmetic encoder **110c** execute binarization and arithmetic encoding of a plurality of quantization coefficients in a CTU sequentially. At this time, context controller **110b** updates the context value each time arithmetic encoding is performed. Context controller **110b** then saves the context value as a post process. The saved context value is used, for example, to initialize the context value for the next CTU.

[Inverse Quantizer]

[0354] Inverse quantizer **112** inverse quantizes quantized coefficients which have been input from quantizer **108**. More specifically, inverse quantizer **112** inverse quantizes, in a determined scanning order, quantized coefficients of the current block. Inverse quantizer **112** then outputs the inverse quantized transform coefficients of the current block to inverse transformer **114**.

[Inverse Transformer]

[0355] Inverse transformer **114** restores prediction errors by inverse transforming the transform coefficients which have been input from inverse quantizer **112**. More specifically, inverse transformer **114** restores the prediction residuals of the current block by performing an inverse transform corresponding to the transform applied to the transform coefficients by transformer **106**. Inverse transformer **114** then outputs the restored prediction residuals to adder **116**.

[0356] It is to be noted that since information is normally lost in quantization, the restored prediction residuals do not match the prediction errors calculated by subtractor **104**. In other words, the restored prediction residuals normally include quantization errors.

[Adder]

[0357] Adder **116** reconstructs the current block by adding the prediction residuals which have been input from inverse transformer **114** and prediction images which have been input from prediction controller **128**. Consequently, a reconstructed image is generated. Adder **116** then outputs the reconstructed image to block memory **118** and loop filter **120**.

[Block Memory]

[0358] Block memory **118** is storage for storing a block which is included in a current picture and is referred to in intra prediction. More specifically, block memory **118** stores a reconstructed image output from adder **116**.

[Frame Memory]

[0359] Frame memory **122** is, for example, storage for storing reference pictures for use in inter prediction, and is also referred to as a frame buffer. More specifically, frame memory **122** stores a reconstructed image filtered by loop filter **120**.

[Loop Filter]

[0360] Loop filter **120** applies a loop filter to a reconstructed image output by adder **116**, and outputs the filtered reconstructed image to frame memory **122**. A loop filter is a filter used in an encoding loop (in-loop filter). Examples of

loop filters include, for example, an adaptive loop filter (ALF), a deblocking filter (DF or DBF), a sample adaptive offset (SAO), etc.

[0361] FIG. 22 is a block diagram illustrating one example of a configuration of loop filter 120.

[0362] For example, as illustrated in FIG. 22, loop filter 120 includes deblocking filter executor 120a, SAO executor 120b, and ALF executor 120c. Deblocking filter executor 120a performs a deblocking filter process of the reconstructed image. SAO executor 120b performs a SAO process of the reconstructed image after being subjected to the deblocking filter process. ALF executor 120c performs an ALF process of the reconstructed image after being subjected to the SAO process. The ALF and deblocking filter processes are described later in detail. The SAO process is a process for enhancing image quality by reducing ringing (a phenomenon in which pixel values are distorted like waves around an edge) and correcting deviation in pixel value. Examples of SAO processes include an edge offset process and a band offset process. It is to be noted that loop filter 120 does not always need to include all the constituent elements disclosed in FIG. 22, and may include only part of the constituent elements. In addition, loop filter 120 may be configured to perform the above processes in a processing order different from the one disclosed in FIG. 22.

[Loop Filter>Adaptive Loop Filter]

[0363] In an ALF, a least square error filter for removing compression artifacts is applied. For example, one filter selected from among a plurality of filters based on the direction and activity of local gradients is applied for each of 2x2 pixel sub-blocks in the current block.

[0364] More specifically, first, each sub-block (for example, each 2x2 pixel sub-block) is categorized into one out of a plurality of classes (for example, fifteen or twenty-five classes). The categorization of the sub-block is based on, for example, gradient directionality and activity. In a specific example, category index C (for example, $C=5D+A$) is calculated based on gradient directionality D (for example, 0 to 2 or 0 to 4) and gradient activity A (for example, 0 to 4). Then, based on category index C, each sub-block is categorized into one out of a plurality of classes.

[0365] For example, gradient directionality D is calculated by comparing gradients of a plurality of directions (for example, the horizontal, vertical, and two diagonal directions). Moreover, for example, gradient activity A is calculated by adding gradients of a plurality of directions and quantizing the result of the addition.

[0366] The filter to be used for each sub-block is determined from among the plurality of filters based on the result of such categorization.

[0367] The filter shape to be used in an ALF is, for example, a circular symmetric filter shape. FIG. 23A through FIG. 23C illustrate examples of filter shapes used in ALFs. FIG. 23A illustrates a 5x5 diamond shape filter, FIG. 23B illustrates a 7x7 diamond shape filter, and FIG. 23C illustrates a 9x9 diamond shape filter. Information indicating the filter shape is normally signaled at the picture level. It is to be noted that the signaling of such information indicating the filter shape does not necessarily need to be performed at the picture level, and may be performed at another level (for example, at the sequence level, slice level, brick level, CTU level, or CU level).

[0368] The ON or OFF of the ALF is determined, for example, at the picture level or CU level. For example, the decision of whether to apply the ALF to luma may be made at the CU level, and the decision of whether to apply ALF to chroma may be made at the picture level. Information indicating ON or OFF of the ALF is normally signaled at the picture level or CU level. It is to be noted that the signaling of information indicating ON or OFF of the ALF does not necessarily need to be performed at the picture level or CU level, and may be performed at another level (for example, at the sequence level, slice level, brick level, or CTU level).

[0369] In addition, as described above, one filter is selected from the plurality of filters, and an ALF process of a sub-block is performed. A coefficient set of coefficients to be used for each of the plurality of filters (for example, up to the fifteenth or twenty-fifth filter) is normally signaled at the picture level. It is to be noted that the coefficient set does not always need to be signaled at the picture level, and may be signaled at another level (for example, the sequence level, slice level, brick level, CTU level, CU level, or sub-block level).

[Loop Filter>Cross Component Adaptive Loop Filter]

[0370] FIG. 23D is a diagram illustrating an example where Y samples (first component) are used for a cross component ALF (CCALF) for Cb and a CCALF for Cr (components different from the first component). FIG. 23E is a diagram illustrating a diamond shaped filter.

[0371] One example of CC-ALF operates by applying a linear, diamond shaped filter (FIGS. 23D, 23E) to a luma channel for each chroma component. The filter coefficients, for example, may be transmitted in the APS, scaled by a factor of 2^{10} , and rounded for fixed point representation. The application of the filters is controlled on a variable block size and signaled by a context-coded flag received for each block of samples. The block size along with a CC-ALF enabling flag is received at the slice-level for each chroma component. Syntax and semantics for CC-ALF are provided in the Appendix. In the contribution, the following block sizes (in chroma samples) were supported: 16x16, 32x32, 64x64, and 128x128.

[Loop Filter>Joint Chroma Cross Component Adaptive Loop Filter]

[0372] FIG. 23F is a diagram illustrating an example for a joint chroma CCALF (JC-CCALF).

[0373] One example of JC-CCALF, where only one CCALF filter will be used to generate one CCALF filtered output as a chroma refinement signal for one color component only, while a properly weighted version of the same chroma refinement signal will be applied to the other color component. In this way, the complexity of existing CCALF is reduced roughly by half.

[0374] The weight value is coded into a sign flag and a weight index. The weight index (denoted as weight_index) is coded into 3 bits, and specifies the magnitude of the JC-CCALF weight JcCcWeight. It cannot be equal to 0. The magnitude of JcCcWeight is determined as follows.

[0375] If weight_index is less than or equal to 4, JcCcWeight is equal to $\text{weight_index} \gg 2$.

[0376] Otherwise, JcCcWeight is equal to $4/(\text{weight_index}-4)$.

[0377] The block-level on/off control of ALF filtering for Cb and Cr are separate. This is the same as in CCALF, and two separate sets of block-level on/off control flags will be coded. Different from CCALF, herein, the Cb, Cr on/off control block sizes are the same, and thus, only one block size variable is coded.

[Loop Filter>Deblocking Filter]

[0378] In a deblocking filter process, loop filter 120 performs a filter process on a block boundary in a reconstructed image so as to reduce distortion which occurs at the block boundary.

[0379] FIG. 24 is a block diagram illustrating one example of a specific configuration of deblocking filter executor 120a.

[0380] For example, deblocking filter executor 120a includes: boundary determiner 1201; filter determiner 1203; filter executor 1205; process determiner 1208; filter characteristic determiner 1207; and switches 1202, 1204, and 1206.

[0381] Boundary determiner 1201 determines whether a pixel to be deblock filtered (that is, a current pixel) is present around a block boundary. Boundary determiner 1201 then outputs the determination result to switch 1202 and process determiner 1208.

[0382] In the case where boundary determiner 1201 has determined that a current pixel is present around a block boundary, switch 1202 outputs an unfiltered image to switch 1204. In the opposite case where boundary determiner 1201 has determined that no current pixel is present around a block boundary, switch 1202 outputs an unfiltered image to switch 1206. It is to be noted that the unfiltered image is an image configured with a current pixel and at least one surrounding pixel located around the current pixel.

[0383] Filter determiner 1203 determines whether to perform deblocking filtering of the current pixel, based on the pixel value of at least one surrounding pixel located around the current pixel. Filter determiner 1203 then outputs the determination result to switch 1204 and process determiner 1208.

[0384] In the case where filter determiner 1203 has determined to perform deblocking filtering of the current pixel, switch 1204 outputs the unfiltered image obtained through switch 1202 to filter executor 1205. In the opposite case where filter determiner 1203 has determined not to perform deblocking filtering of the current pixel, switch 1204 outputs the unfiltered image obtained through switch 1202 to switch 1206.

[0385] When obtaining the unfiltered image through switches 1202 and 1204, filter executor 1205 executes, for the current pixel, deblocking filtering having the filter characteristic determined by filter characteristic determiner 1207. Filter executor 1205 then outputs the filtered pixel to switch 1206.

[0386] Under control by process determiner 1208, switch 1206 selectively outputs a pixel which has not been deblock filtered and a pixel which has been deblock filtered by filter executor 1205.

[0387] Process determiner 1208 controls switch 1206 based on the results of determinations made by boundary determiner 1201 and filter determiner 1203. In other words, process determiner 1208 causes switch 1206 to output the pixel which has been deblock filtered when boundary determiner 1201 has determined that the current pixel is present

around the block boundary and filter determiner 1203 has determined to perform deblocking filtering of the current pixel. In addition, in a case other than the above case, process determiner 1208 causes switch 1206 to output the pixel which has not been deblock filtered. A filtered image is output from switch 1206 by repeating output of a pixel in this way. It is to be noted that the configuration illustrated in FIG. 24 is one example of a configuration in deblocking filter executor 120a. Deblocking filter executor 120a may have another configuration.

[0388] FIG. 25 is a diagram illustrating an example of a deblocking filter having a symmetrical filtering characteristic with respect to a block boundary.

[0389] In a deblocking filter process, one of two deblocking filters having different characteristics, that is, a strong filter and a weak filter is selected using pixel values and quantization parameters, for example. In the case of the strong filter, pixels p0 to p2 and pixels q0 to q2 are present across a block boundary as illustrated in FIG. 25, the pixel values of the respective pixels q0 to q2 are changed to pixel values q'0 to q'2 by performing computations according to the expressions below.

$$q'0 = (p1 + 2 \times p0 + 2 \times q0 + 2 \times q1 + q2 + 4) / 8$$

$$q'1 = (p0 + q0 + q1 + q2 + 2) / 4$$

$$q'2 = (p0 + q0 + q1 + 3 \times q2 + 2 \times q3 + 4) / 8$$

[0390] It is to be noted that, in the above expressions, p0 to p2 and q0 to q2 are the pixel values of respective pixels p0 to p2 and pixels q0 to q2. In addition, q3 is the pixel value of neighboring pixel q3 located at the opposite side of pixel q2 with respect to the block boundary. In addition, in the right side of each of the expressions, coefficients which are multiplied with the respective pixel values of the pixels to be used for deblocking filtering are filter coefficients.

[0391] Furthermore, in the deblocking filtering, clipping may be performed so that the calculated pixel values do not change over a threshold value. In the clipping process, the pixel values calculated according to the above expressions are clipped to a value obtained according to “a pre-computation pixel value $\pm 2 \times$ a threshold value” using the threshold value determined based on a quantization parameter. In this way, it is possible to prevent excessive smoothing.

[0392] FIG. 26 is a diagram for illustrating one example of a block boundary on which a deblocking filter process is performed. FIG. 27 is a diagram illustrating examples of Bs values.

[0393] The block boundary on which the deblocking filter process is performed is, for example, a boundary between CUs, PUs, or TUs having 8x8 pixel blocks as illustrated in FIG. 26. The deblocking filter process is performed, for example, in units of four rows or four columns. First, boundary strength (Bs) values are determined as indicated in FIG. 27 for block P and block Q illustrated in FIG. 26.

[0394] According to the Bs values in FIG. 27, whether to perform deblocking filter processes of block boundaries belonging to the same image using different strengths may be determined. The deblocking filter process for a chroma signal is performed when a Bs value is 2. The deblocking filter process for a luma signal is performed when a Bs value is 1 or more and a determined condition is satisfied. It is to

be noted that conditions for determining Bs values are not limited to those indicated in FIG. 27, and a Bs value may be determined based on another parameter.

[Predictor (Intra Predictor, Inter Predictor, Prediction Controller)]

[0395] FIG. 28 is a flow chart illustrating one example of a process performed by a predictor of encoder 100. It is to be noted that the predictor, as one example, includes all or part of the following constituent elements: intra predictor 124; inter predictor 126; and prediction controller 128. The prediction executor includes, for example, intra predictor 124 and inter predictor 126.

[0396] The predictor generates a prediction image of a current block (Step Sb_1). It is to be noted that the prediction image is, for example, an intra prediction image (intra prediction signal) or an inter prediction image (inter prediction signal). More specifically, the predictor generates the prediction image of the current block using a reconstructed image which has been already obtained for another block through generation of a prediction image, generation of a prediction residual, generation of quantized coefficients, restoring of a prediction residual, and addition of a prediction image.

[0397] The reconstructed image may be, for example, an image in a reference picture or an image of an encoded block (that is, the other block described above) in a current picture which is the picture including the current block. The encoded block in the current picture is, for example, a neighboring block of the current block.

[0398] FIG. 29 is a flow chart illustrating another example of a process performed by the predictor of encoder 100.

[0399] The predictor generates a prediction image using a first method (Step Sc_1a), generates a prediction image using a second method (Step Sc_1b), and generates a prediction image using a third method (Step Sc_1c). The first method, the second method, and the third method may be mutually different methods for generating a prediction image. Each of the first to third methods may be an inter prediction method, an intra prediction method, or another prediction method. The above-described reconstructed image may be used in these prediction methods.

[0400] Next, the predictor evaluates the prediction images generated in Steps Sc_1a, Sc_1b, and Sc_1c (Step Sc_2). For example, the predictor calculates costs C for the prediction images generated in Step Sc_1a, Sc_1b, and Sc_1c, and evaluates the prediction images by comparing the costs C of the prediction images. It is to be noted that cost C is calculated according to an expression of an R-D optimization model, for example, $C=D+\lambda \times R$. In this expression, D indicates compression artifacts of a prediction image, and is represented as, for example, a sum of absolute differences between the pixel value of a current block and the pixel value of a prediction image. In addition, R indicates a bit rate of a stream. In addition, λ indicates, for example, a multiplier according to the method of Lagrange multiplier.

[0401] The predictor then selects one of the prediction images generated in Steps Sc_1a, Sc_1b, and Sc_1c (Step Sc_3). In other words, the predictor selects a method or a mode for obtaining a final prediction image. For example, the predictor selects the prediction image having the smallest cost C, based on costs C calculated for the prediction images. Alternatively, the evaluation in Step Sc_2 and the selection of the prediction image in Step Sc_3 may be made

based on a parameter which is used in an encoding process. Encoder 100 may transform information for identifying the selected prediction image, the method, or the mode into a stream. The information may be, for example, a flag or the like. In this way, decoder 200 is capable of generating a prediction image according to the method or the mode selected by encoder 100, based on the information. It is to be noted that, in the example illustrated in FIG. 29, the predictor selects any of the prediction images after the prediction images are generated using the respective methods. However, the predictor may select a method or a mode based on a parameter for use in the above-described encoding process before generating prediction images, and may generate a prediction image according to the method or mode selected.

[0402] For example, the first method and the second method may be intra prediction and inter prediction, respectively, and the predictor may select a final prediction image for a current block from prediction images generated according to the prediction methods.

[0403] FIG. 30 is a flow chart illustrating another example of a process performed by the predictor of encoder 100.

[0404] First, the predictor generates a prediction image using intra prediction (Step Sd_1a), and generates a prediction image using inter prediction (Step Sd_1b). It is to be noted that the prediction image generated by intra prediction is also referred to as an intra prediction image, and the prediction image generated by inter prediction is also referred to as an inter prediction image.

[0405] Next, the predictor evaluates each of the intra prediction image and the inter prediction image (Step Sd_2). Cost C described above may be used in the evaluation. The predictor may then select the prediction image for which the smallest cost C has been calculated among the intra prediction image and the inter prediction image, as the final prediction image for the current block (Step Sd_3). In other words, the prediction method or the mode for generating the prediction image for the current block is selected.

[Intra Predictor]

[0406] Intra predictor 124 generates a prediction image (that is, intra prediction image) of a current block by performing intra prediction (also referred to as intra frame prediction) of the current block by referring to a block or blocks in the current picture which is or are stored in block memory 118. More specifically, intra predictor 124 generates an intra prediction image by performing intra prediction by referring to pixel values (for example, luma and/or chroma values) of a block or blocks neighboring the current block, and then outputs the intra prediction image to prediction controller 128.

[0407] For example, intra predictor 124 performs intra prediction by using one mode from among a plurality of intra prediction modes which have been predefined. The intra prediction modes normally include one or more non-directional prediction modes and a plurality of directional prediction modes.

[0408] The one or more non-directional prediction modes include, for example, planar prediction mode and DC prediction mode defined in the H.265/HEVC standard.

[0409] The plurality of directional prediction modes include, for example, the thirty-three directional prediction modes defined in the H.265/HEVC standard. It is to be noted that the plurality of directional prediction modes may further

include thirty-two directional prediction modes in addition to the thirty-three directional prediction modes (for a total of sixty-five directional prediction modes). FIG. 31 is a diagram illustrating sixty-seven intra prediction modes in total used in intra prediction (two non-directional prediction modes and sixty-five directional prediction modes). The solid arrows represent the thirty-three directions defined in the H.265/HEVC standard, and the dashed arrows represent the additional thirty-two directions (the two non-directional prediction modes are not illustrated in FIG. 31).

[0410] In various kinds of mounting examples, a luma block may be referred to in intra prediction of a chroma block. In other words, a chroma component of the current block may be predicted based on a luma component of the current block. Such intra prediction is also referred to as cross-component linear model (CCLM). The intra prediction mode for a chroma block in which such a luma block is referred to (also referred to as, for example, a CCLM mode) may be added as one of the intra prediction modes for chroma blocks.

[0411] Intra predictor 124 may correct intra-predicted pixel values based on horizontal/vertical reference pixel gradients. The intra prediction which accompanies this sort of correcting is also referred to as position dependent intra prediction combination (PDPC). Information indicating whether to apply PDPC (referred to as, for example, a PDPC flag) is normally signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, brick level, or CTU level).

[0412] FIG. 32 is a flow chart illustrating one example of a process performed by intra predictor 124.

[0413] Intra predictor 124 selects one intra prediction mode from a plurality of intra prediction modes (Step Sw_1). Intra predictor 124 then generates a prediction image according to the selected intra prediction mode (Step Sw_2). Next, intra predictor 124 determines most probable modes (MPMs) (Step Sw_3). MPMs include, for example, six intra prediction modes. Two modes among the six intra prediction modes may be planar mode and DC prediction mode, and the other four modes may be directional prediction modes. Intra predictor 124 determines whether the intra prediction mode selected in Step Sw_1 is included in the MPMs (Step Sw_4).

[0414] Here, when determining that the intra prediction mode selected in Step Sw_1 is included in the MPMs (Yes in Step Sw_4), intra predictor 124 sets an MPM flag to 1 (Step Sw_5), and generates information indicating the selected intra prediction mode among the MPMs (Step Sw_6). It is to be noted that the MPM flag set to 1 and the information indicating the intra prediction mode are encoded as prediction parameters by entropy encoder 110.

[0415] When determining that the selected intra prediction mode is not included in the MPMs (No in Step Sw_4), intra predictor 124 sets the MPM flag to 0 (Step Sw_7). Alternatively, intra predictor 124 does not set any MPM flag. Intra predictor 124 then generates information indicating the selected intra prediction mode among at least one intra prediction mode which is not included in the MPMs (Step Sw_8). It is to be noted that the MPM flag set to 0 and the information indicating the intra prediction mode are encoded

as prediction parameters by entropy encoder 110. The information indicating the intra prediction mode indicates, for example, any one of 0 to 60.

[Inter Predictor]

[0416] Inter predictor 126 generates a prediction image (inter prediction image) by performing inter prediction (also referred to as inter frame prediction) of the current block by referring to a block or blocks in a reference picture which is different from the current picture and is stored in frame memory 122. Inter prediction is performed in units of a current block or a current sub-block in the current block. The sub-block is included in the block and is a unit smaller than the block. The size of the sub-block may be 4×4 pixels, 8×8 pixels, or another size. The size of the sub-block may be switched for a unit such as slice, brick, picture, etc.

[0417] For example, inter predictor 126 performs motion estimation in a reference picture for a current block or a current sub-block, and finds out a reference block or a reference sub-block which best matches the current block or current sub-block. Inter predictor 126 then obtains motion information (for example, a motion vector) which compensates a motion or a change from the reference block or the reference sub-block to the current block or the current sub-block. Inter predictor 126 generates an inter prediction image of the current block or the current sub-block by performing motion compensation (or motion prediction) based on the motion information. Inter predictor 126 outputs the generated inter prediction image to prediction controller 128.

[0418] The motion information used in motion compensation may be signaled as inter prediction images in various forms. For example, a motion vector may be signaled. As another example, the difference between a motion vector and a motion vector predictor may be signaled.

[Reference Picture List]

[0419] FIG. 33 is a diagram illustrating examples of reference pictures. FIG. 34 is a conceptual diagram illustrating examples of reference picture lists. Each reference picture list is a list indicating at least one reference picture stored in frame memory 122. It is to be noted that, in FIG. 33, each of rectangles indicates a picture, each of arrows indicates a picture reference relationship, the horizontal axis indicates time, I, P, and B in the rectangles indicate an intra prediction picture, a uni-prediction picture, and a bi-prediction picture, respectively, and numerals in the rectangles indicate a decoding order. As illustrated in FIG. 33, the decoding order of the pictures is an order of I0, P1, B2, B3, and B4, and the display order of the pictures is an order of I0, B3, B2, B4, and P1. As illustrated in FIG. 34, the reference picture list is a list representing reference picture candidates. For example, one picture (or a slice) may include at least one reference picture list. For example, one reference picture list is used when a current picture is a uni-prediction picture, and two reference picture lists are used when a current picture is a bi-prediction picture. In the examples of FIGS. 33 and 34, picture B3 which is current picture currPic has two reference picture lists which are the L0 list and the L1 list. When current picture currPic is picture B3, reference picture candidates for current picture currPic are I0, P1, and B2, and the reference picture lists (which are the L0 list and the L1 list) indicate these pictures. Inter predictor 126 or

prediction controller **128** specifies which picture in each reference picture list is to be actually referred to in form of a reference picture index refIdxLx . In FIG. 34, reference pictures P1 and B2 are specified by reference picture indices refIdxL0 and refIdxL1 .

[0420] Such a reference picture list may be generated for each unit such as a sequence, picture, slice, brick, CTU, or CU. In addition, among reference pictures indicated in reference picture lists, a reference picture index indicating a reference picture to be referred to in inter prediction may be signaled at the sequence level, picture level, slice level, brick level, CTU level, or CU level. In addition, a common reference picture list may be used in a plurality of inter prediction modes.

[Basic Flow of Inter Prediction]

[0421] FIG. 35 is a flow chart illustrating a basic processing flow of inter prediction.

[0422] First, inter predictor **126** generates a prediction signal (Steps Se_1 to Se_3). Next, subtractor **104** generates the difference between a current block and a prediction image as a prediction residual (Step Se_4).

[0423] Here, in the generation of the prediction image, inter predictor **126** generates the prediction image through, for example, determination of a motion vector (MV) of the current block (Steps Se_1 and Se_2) and motion compensation (Step Se_3). Furthermore, in determination of an MV, inter predictor **126** determines the MV through, for example, selection of a motion vector candidate (MV candidate) (Step Se_1) and derivation of an MV (Step Se_2). The selection of the MV candidate is made by means of, for example, inter predictor **126** generating an MV candidate list and selecting at least one MV candidate from the MV candidate list. It is to be noted that MVs derived in the past may be added to the MV candidate list. Alternatively, in derivation of an MV, inter predictor **126** may further select at least one MV candidate from the at least one MV candidate, and determine the selected at least one MV candidate as the MV for the current block. Alternatively, inter predictor **126** may determine the MV for the current block by performing estimation in a reference picture region specified by each of the selected at least one MV candidate. It is to be noted that the estimation in the reference picture region may be referred to as motion estimation.

[0424] In addition, although Steps Se_1 to Se_3 are performed by inter predictor **126** in the above-described example, a process that is, for example, Step Se_1, Step Se_2, or the like may be performed by another constituent element included in encoder **100**.

[0425] It is to be noted that an MV candidate list may be generated for each process in inter prediction mode, or a common MV candidate list may be used in a plurality of inter prediction modes. The processes in Steps Se_3 and Se_4 correspond to Steps Sa_3 and Sa_4 illustrated in FIG. 9, respectively. The process in Step Se_3 corresponds to the process in Step Sd_1b in FIG. 30.

[MV Derivation Flow]

[0426] FIG. 36 is a flow chart illustrating one example of MV derivation.

[0427] Inter predictor **126** may derive an MV for a current block in a mode for encoding motion information (for example, an MV). In this case, for example, the motion

information may be encoded as a prediction parameter, and may be signaled. In other words, the encoded motion information is included in a stream.

[0428] Alternatively, inter predictor **126** may derive an MV in a mode in which motion information is not encoded. In this case, no motion information is included in the stream.

[0429] Here, MV derivation modes include a normal inter mode, a normal merge mode, a FRUC mode, an affine mode, etc. which are described later. Modes in which motion information is encoded among the modes include the normal inter mode, the normal merge mode, the affine mode (specifically, an affine inter mode and an affine merge mode), etc. It is to be noted that motion information may include not only an MV but also MV predictor selection information which is described later. Modes in which no motion information is encoded include the FRUC mode, etc. Inter predictor **126** selects a mode for deriving an MV of the current block from the plurality of modes, and derives the MV of the current block using the selected mode.

[0430] FIG. 37 is a flow chart illustrating another example of MV derivation.

[0431] Inter predictor **126** may derive an MV for a current block in a mode in which an MV difference is encoded. In this case, for example, the MV difference is encoded as a prediction parameter, and is signaled. In other words, the encoded MV difference is included in a stream. The MV difference is the difference between the MV of the current block and the MV predictor. It is to be noted that the MV predictor is a motion vector predictor.

[0432] Alternatively, inter predictor **126** may derive an MV in a mode in which no MV difference is encoded. In this case, no encoded MV difference is included in the stream.

[0433] Here, as described above, the MV derivation modes include the normal inter mode, the normal merge mode, the FRUC mode, the affine mode, etc. which are described later. Modes in which an MV difference is encoded among the modes include the normal inter mode, the affine mode (specifically, the affine inter mode), etc. Modes in which no MV difference is encoded include the FRUC mode, the normal merge mode, the affine mode (specifically, the affine merge mode), etc. Inter predictor **126** selects a mode for deriving an MV of the current block from the plurality of modes, and derives the MV for the current block using the selected mode.

[MV Derivation Modes]

[0434] FIGS. 38A and 38B are each a diagram illustrating one example of categorization of modes for MV derivation. For example, as illustrated in FIG. 38A, MV derivation modes are roughly categorized into three modes according to whether to encode motion information and whether to encode MV differences. The three modes are inter mode, merge mode, and frame rate up-conversion (FRUC) mode. The inter mode is a mode in which motion estimation is performed, and in which motion information and an MV difference are encoded. For example, as illustrated in FIG. 38B, the inter mode includes affine inter mode and normal inter mode. The merge mode is a mode in which no motion estimation is performed, and in which an MV is selected from an encoded surrounding block and an MV for the current block is derived using the MV. The merge mode is a mode in which, basically, motion information is encoded and no MV difference is encoded. For example, as illustrated in FIG. 38B, the merge modes include normal merge mode

(also referred to as normal merge mode or regular merge mode), merge with motion vector difference (MMVD) mode, combined inter merge/intra prediction (CIIP) mode, triangle mode, ATMVP mode, and affine merge mode. Here, an MV difference is encoded exceptionally in the MMVD mode among the modes included in the merge modes. It is to be noted that the affine merge mode and the affine inter mode are modes included in the affine modes. The affine mode is a mode for deriving, as an MV of a current block, an MV of each of a plurality of sub-blocks included in the current block, assuming affine transform. The FRUC mode is a mode which is for deriving an MV of the current block by performing estimation between encoded regions, and in which neither motion information nor any MV difference is encoded. It is to be noted that the respective modes will be described later in detail.

[0435] It is to be noted that the categorization of the modes illustrated in FIGS. 38A and 38B are examples, and categorization is not limited thereto. For example, when an MV difference is encoded in CIIP mode, the CIIP mode is categorized into inter modes.

[MV Derivation>Normal Inter Mode]

[0436] The normal inter mode is an inter prediction mode for deriving an MV of a current block by finding out a block similar to the image of the current block from a reference picture region specified by an MV candidate. In this normal inter mode, an MV difference is encoded.

[0437] FIG. 39 is a flow chart illustrating an example of inter prediction by normal inter mode.

[0438] First, inter predictor 126 obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of encoded blocks temporally or spatially surrounding the current block (Step Sg_1). In other words, inter predictor 126 generates an MV candidate list.

[0439] Next, inter predictor 126 extracts N (an integer of 2 or larger) MV candidates from the plurality of MV candidates obtained in Step Sg_1, as motion vector predictor candidates according to a predetermined priority order (Step Sg_2). It is to be noted that the priority order is determined in advance for each of the N MV candidates.

[0440] Next, inter predictor 126 selects one MV predictor candidate from the N MV predictor candidates as the MV predictor for the current block (Step Sg_3). At this time, inter predictor 126 encodes, in a stream, MV predictor selection information for identifying the selected MV predictor. In other words, inter predictor 126 outputs the MV predictor selection information as a prediction parameter to entropy encoder 110 through prediction parameter generator 130.

[0441] Next, inter predictor 126 derives an MV of a current block by referring to an encoded reference picture (Step Sg_4). At this time, inter predictor 126 further encodes, in the stream, the difference value between the derived MV and the MV predictor as an MV difference. In other words, inter predictor 126 outputs the MV difference as a prediction parameter to entropy encoder 110 through prediction parameter generator 130. It is to be noted that the encoded reference picture is a picture including a plurality of blocks which have been reconstructed after being encoded.

[0442] Lastly, inter predictor 126 generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the encoded reference picture (Step Sg_5). The processes in

Steps Sg_1 to Sg_5 are executed on each block. For example, when the processes in Steps Sg_1 to Sg_5 are executed on each of all the blocks in the slice, inter prediction of the slice using the normal inter mode finishes. For example, when the processes in Steps Sg_1 to Sg_5 are executed on each of all the blocks in the picture, inter prediction of the picture using the normal inter mode finishes. It is to be noted that not all the blocks included in the slice may be subjected to the processes in Steps Sg_1 to Sg_5, and inter prediction of the slice using the normal inter mode may finish when part of the blocks are subjected to the processes. Likewise, inter prediction of the picture using the normal inter mode may finish when the processes in Steps Sg_1 to Sg_5 are executed on part of the blocks in the picture.

[0443] It is to be noted that the prediction image is an inter prediction signal as described above. In addition, information indicating the inter prediction mode (normal inter mode in the above example) used to generate the prediction image is, for example, encoded as a prediction parameter in an encoded signal.

[0444] It is to be noted that the MV candidate list may be also used as a list for use in another mode. In addition, the processes related to the MV candidate list may be applied to processes related to the list for use in another mode. The processes related to the MV candidate list include, for example, extraction or selection of an MV candidate from the MV candidate list, reordering of MV candidates, or deletion of an MV candidate.

[MV Derivation>Normal Merge Mode]

[0445] The normal merge mode is an inter prediction mode for selecting an MV candidate from an MV candidate list as an MV for a current block, thereby deriving the MV. It is to be noted that the normal merge mode is a merge mode in a narrow meaning and is also simply referred to as a merge mode. In this embodiment, the normal merge mode and the merge mode are distinguished, and the merge mode is used in a broad meaning.

[0446] FIG. 40 is a flow chart illustrating an example of inter prediction by normal merge mode.

[0447] First, inter predictor 126 obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of encoded blocks temporally or spatially surrounding the current block (Step Sh_1). In other words, inter predictor 126 generates an MV candidate list.

[0448] Next, inter predictor 126 selects one MV candidate from the plurality of MV candidates obtained in Step Sh_1, thereby deriving an MV for the current block (Step Sh_2). At this time, inter predictor 126 encodes, in a stream, MV selection information for identifying the selected MV candidate. In other words, inter predictor 126 outputs the MV selection information as a prediction parameter to entropy encoder 110 through prediction parameter generator 130.

[0449] Lastly, inter predictor 126 generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the encoded reference picture (Step Sh_3). The processes in Steps Sh_1 to Sh_3 are executed, for example, on each block. For example, when the processes in Steps Sh_1 to Sh_3 are executed on each of all the blocks in the slice, inter prediction of the slice using the normal merge mode finishes. In addition, when the processes in Steps Sh_1 to Sh_3 are executed on each of all the blocks in the picture, inter

prediction of the picture using the normal merge mode finishes. It is to be noted that not all the blocks included in the slice may be subjected to the processes in Steps Sh_1 to Sh_3, and inter prediction of the slice using the normal merge mode may finish when part of the blocks are subjected to the processes. Likewise, inter prediction of the picture using the normal merge mode may finish when the processes in Steps Sh_1 to Sh_3 are executed on part of the blocks in the picture.

[0450] In addition, information indicating the inter prediction mode (normal merge mode in the above example) used to generate the prediction image is, for example, encoded as a prediction parameter in a stream.

[0451] FIG. 41 is a diagram for illustrating one example of an MV derivation process for a current picture by normal merge mode.

[0452] First, inter predictor 126 generates an MV candidate list in which MV candidates are registered. Examples of MV candidates include: spatially neighboring MV candidates which are MVs of a plurality of encoded blocks located spatially surrounding a current block; temporally neighboring MV candidates which are MVs of surrounding blocks on which the position of a current block in an encoded reference picture is projected; combined MV candidates which are MVs generated by combining the MV value of a spatially neighboring MV predictor and the MV value of a temporally neighboring MV predictor; and a zero MV candidate which is an MV having a zero value.

[0453] Next, inter predictor 126 selects one MV candidate from a plurality of MV candidates registered in an MV candidate list, and determines the MV candidate as the MV of the current block.

[0454] Furthermore, entropy encoder 110 writes and encodes, in a stream, merge_idx which is a signal indicating which MV candidate has been selected.

[0455] It is to be noted that the MV candidates registered in the MV candidate list described in FIG. 41 are examples. The number of MV candidates may be different from the number of MV candidates in the diagram, the MV candidate list may be configured in such a manner that some of the kinds of the MV candidates in the diagram may not be included, or that one or more MV candidates other than the kinds of MV candidates in the diagram are included.

[0456] A final MV may be determined by performing a dynamic motion vector refreshing (DMVR) to be described later using the MV of the current block derived by normal merge mode. It is to be noted that, in normal merge mode, no MV difference is encoded, but an MV difference is encoded. In MMVD mode, one MV candidate is selected from an MV candidate list as in the case of normal merge mode, an MV difference is encoded. As illustrated in FIG. 38B, MMVD may be categorized into merge modes together with normal merge mode. It is to be noted that the MV difference in MMVD mode does not always need to be the same as the MV difference for use in inter mode. For example, MV difference derivation in MMVD mode may be a process that requires a smaller amount of processing than the amount of processing required for MV difference derivation in inter mode.

[0457] In addition, a combined inter merge/intra prediction (CIIP) mode may be performed. The mode is for overlapping a prediction image generated in inter prediction and a prediction image generated in intra prediction to generate a prediction image for a current block.

[0458] It is to be noted that the MV candidate list may be referred to as a candidate list. In addition, merge_idx is MV selection information.

[MV Derivation>HMVP Mode]

[0459] FIG. 42 is a diagram for illustrating one example of an MV derivation process for a current picture by HMVP merge mode.

[0460] In normal merge mode, an MV for, for example, a CU which is a current block is determined by selecting one MV candidate from an MV candidate list generated by referring to an encoded block (for example, a CU). Here, another MV candidate may be registered in the MV candidate list. The mode in which such another MV candidate is registered is referred to as HMVP mode.

[0461] In HMVP mode, MV candidates are managed using a first-in first-out (FIFO) buffer for HMVP, separately from the MV candidate list for normal merge mode.

[0462] In FIFO buffer, motion information such as MVs of blocks processed in the past are stored newest first. In the management of the FIFO buffer, each time when one block is processed, the MV for the newest block (that is the CU processed immediately before) is stored in the FIFO buffer, and the MV of the oldest CU (that is, the CU processed earliest) is deleted from the FIFO buffer. In the example illustrated in FIG. 42, HMVP1 is the MV for the newest block, and HMVP5 is the MV for the oldest MV.

[0463] Inter predictor 126 then, for example, checks whether each MV managed in the FIFO buffer is an MV different from all the MV candidates which have been already registered in the MV candidate list for normal merge mode starting from HMVP1. When determining that the MV is different from all the MV candidates, inter predictor 126 may add the MV managed in the FIFO buffer in the MV candidate list for normal merge mode as an MV candidate. At this time, the MV candidate registered from the FIFO buffer may be one or more.

[0464] By using the HMVP mode in this way, it is possible to add not only the MV of a block which neighbors the current block spatially or temporally but also an MV for a block processed in the past. As a result, the variation of MV candidates for normal merge mode is expanded, which increases the probability that coding efficiency can be increased.

[0465] It is to be noted that the MV may be motion information. In other words, information stored in the MV candidate list and the FIFO buffer may include not only MV values but also reference picture information, reference directions, the numbers of pictures, etc. In addition, the block is, for example, a CU.

[0466] It is to be noted that the MV candidate list and the FIFO buffer illustrated in FIG. 42 are examples. The MV candidate list and FIFO buffer may be different in size from those in FIG. 42, or may be configured to register MV candidates in an order different from the one in FIG. 42. In addition, the process described here is common between encoder 100 and decoder 200.

[0467] It is to be noted that the HMVP mode can be applied for modes other than the normal merge mode. For example, it is also excellent that motion information such as MVs of blocks processed in affine mode in the past may be stored newest first, and may be used as MV candidates. The mode obtained by applying HMVP mode to affine mode may be referred to as history affine mode.

[MV Derivation>FRUC Mode]

[0468] Motion information may be derived at the decoder **200** side without being signaled from the encoder **100** side. For example, motion information may be derived by performing motion estimation at the decoder **200** side. At this time, at the decoder **200** side, motion estimation is performed without using any pixel value in a current block. Modes in which motion estimation is performed at the decoder **200** side in this way include a frame rate up-conversion (FRUC) mode, a pattern matched motion vector derivation (PMMVD) mode, etc.

[0469] One example of a FRUC process is illustrated in FIG. 43. First, a list which indicates, as MV candidates, MVs for encoded blocks each of which neighbors the current block spatially or temporally is generated by referring to the MVs (the list may be an MV candidate list, and be also used as the MV candidate list for normal merge mode) (Step Si_1). Next, a best MV candidate is selected from the plurality of MV candidates registered in the MV candidate list (Step Si_2). For example, the evaluation values of the respective MV candidates included in the MV candidate list are calculated, and one MV candidate is selected as the best MV candidate based on the evaluation values. Based on the selected best MV candidate, a motion vector for the current block is then derived (Step Si_4). More specifically, for example, the selected best MV candidate is directly derived as the MV for the current block. In addition, for example, the MV for the current block may be derived using pattern matching in a surrounding region of a position which is included in a reference picture and corresponds to the selected best MV candidate. In other words, estimation using the pattern matching in a reference picture and the evaluation values may be performed in the surrounding region of the best MV candidate, and when there is an MV that yields a better evaluation value, the best MV candidate may be updated to the MV that yields the better evaluation value, and the updated MV may be determined as the final MV for the current block. Update to the MV that yields the better evaluation value may not be performed.

[0470] Lastly, inter predictor **126** generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the encoded reference picture (Step Si_5). The processes in Steps Si_1 to Si_5 are executed, for example, on each block. For example, when the processes in Steps Si_1 to Si_5 are executed on each of all the blocks in the slice, inter prediction of the slice using the FRUC mode finishes. For example, when the processes in Steps Si_1 to Si_5 are executed on each of all the blocks in the picture, inter prediction of the picture using the FRUC mode finishes. It is to be noted that not all the blocks included in the slice may be subjected to the processes in Steps Si_1 to Si_5, and inter prediction of the slice using the FRUC mode may finish when part of the blocks are subjected to the processes. Likewise, inter prediction of the picture using the FRUC mode may finish when the processes in Steps Si_1 to Si_5 are executed on part of the blocks included in the picture.

[0471] Each sub-block may be processed similarly to the above-described case of processing each block.

[0472] Evaluation values may be calculated according to various kinds of methods. For example, a comparison is made between a reconstructed image in a region in a reference picture corresponding to an MV and a reconstructed image in a determined region (the region may be,

for example, a region in another reference picture or a region in a neighboring block of a current picture, as indicated below). The difference between the pixel values of the two reconstructed images may be used for an evaluation value of the MV. It is to be noted that an evaluation value may be calculated using information other than the value of the difference.

[0473] Next, pattern matching is described in detail. First, one MV candidate included in an MV candidate list (also referred to as a merge list) is selected as a starting point for estimation by pattern matching. As the pattern matching, either a first pattern matching or a second pattern matching may be used. The first pattern matching and the second pattern matching may be referred to as bilateral matching and template matching, respectively.

[MV Derivation>FRUC>Bilateral Matching]

[0474] In the first pattern matching, the pattern matching is performed between two blocks which are located along a motion trajectory of a current block and included in two different reference pictures. Accordingly, in the first pattern matching, a region in another reference picture located along the motion trajectory of the current block is used as a determined region for calculating the evaluation value of the above-described MV candidate.

[0475] FIG. 44 is a diagram for illustrating one example of the first pattern matching (bilateral matching) between the two blocks in the two reference pictures located along the motion trajectory. As illustrated in FIG. 44, in the first pattern matching, two motion vectors (MV0, MV1) are derived by estimating a pair which best matches among pairs of two blocks which are included in the two different reference pictures (Ref0, Ref1) and located along the motion trajectory of the current block (Cur block). More specifically, a difference between the reconstructed image at a specified position in the first encoded reference picture (Ref0) specified by an MV candidate and the reconstructed image at a specified position in the second encoded reference picture (Ref1) specified by a symmetrical MV obtained by scaling the MV candidate at a display time interval is derived for the current block, and an evaluation value is calculated using the value of the obtained difference. It is excellent to select, as the best MV, the MV candidate which yields the best evaluation value among the plurality of MV candidates.

[0476] In the assumption of a continuous motion trajectory, the motion vectors (MV0, MV1) specifying the two reference blocks are proportional to temporal distances (TD0, TD1) between the current picture (Cur Pic) and the two reference pictures (Ref0, Ref1). For example, when the current picture is temporally located between the two reference pictures and the temporal distances from the current picture to the respective two reference pictures are equal to each other, mirror-symmetrical bi-directional MVs are derived in the first pattern matching.

[MV Derivation>FRUC>Template Matching]

[0477] In the second pattern matching (template matching), pattern matching is performed between a block in a reference picture and a template in the current picture (the template is a block neighboring the current block in the current picture (the neighboring block is, for example, an upper and/or left neighboring block(s))). Accordingly, in the

second pattern matching, the block neighboring the current block in the current picture is used as the determined region for calculating the evaluation value of the above-described MV candidate.

[0478] FIG. 45 is a diagram for illustrating one example of pattern matching (template matching) between a template in a current picture and a block in a reference picture. As illustrated in FIG. 45, in the second pattern matching, the MV for the current block (Cur block) is derived by estimating, in the reference picture (Ref0), the block which best matches the block neighboring the current block in the current picture (Cur Pic). More specifically, the difference between a reconstructed image in an encoded region which neighbors both left and above or either left or above and a reconstructed image which is in a corresponding region in the encoded reference picture (Ref0) and is specified by an MV candidate is derived, and an evaluation value is calculated using the value of the obtained difference. It is excellent to select, as the best MV candidate, the MV candidate which yields the best evaluation value among the plurality of MV candidates.

[0479] Such information indicating whether to apply the FRUC mode (referred to as, for example, a FRUC flag) may be signaled at the CU level. In addition, when the FRUC mode is applied (for example, when a FRUC flag is true), information indicating an applicable pattern matching method (either the first pattern matching or the second pattern matching) may be signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, brick level, CTU level, or sub-block level).

[MV Derivation>Affine Mode]

[0480] The affine mode is a mode for generating an MV using affine transform. For example, an MV may be derived in units of a sub-block based on motion vectors of a plurality of neighboring blocks. This mode is also referred to as an affine motion compensation prediction mode.

[0481] FIG. 46A is a diagram for illustrating one example of MV derivation in units of a sub-block based on MVs of a plurality of neighboring blocks. In FIG. 46A, the current block includes sixteen 4×4 pixel sub-blocks. Here, motion vector v_0 at an upper-left corner control point in the current block is derived based on an MV of a neighboring block, and likewise, motion vector v_1 at an upper-right corner control point in the current block is derived based on an MV of a neighboring sub-block. Two motion vectors v_0 and v_1 are projected according to an expression (1A) indicated below, and motion vectors (v_x , v_y) for the respective sub-blocks in the current block are derived.

[MATH. 1]

$$\begin{cases} v_x = \frac{(v_{1x} - v_{0x})}{w}x - \frac{(v_{1y} - v_{0y})}{w}y + v_{0x} \\ v_y = \frac{(v_{1y} - v_{0y})}{w}x + \frac{(v_{1x} - v_{0x})}{w}y + v_{0y} \end{cases} \quad (1A)$$

[0482] Here, x and y indicate the horizontal position and the vertical position of the sub-block, respectively, and w indicates a predetermined weighting coefficient.

[0483] Such information indicating the affine mode (for example, referred to as an affine flag) may be signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, brick level, CTU level, or sub-block level).

[0484] In addition, the affine mode may include several modes for different methods for deriving MVs at the upper-left and upper-right corner control points. For example, the affine modes include two modes which are the affine inter mode (also referred to as an affine normal inter mode) and the affine merge mode.

[0485] FIG. 46B is a diagram for illustrating one example of MV derivation in units of a sub-block in affine mode in which three control points are used. In FIG. 46B, the current block includes, for example, sixteen 4×4 pixel sub-blocks. Here, motion vector v_0 at an upper-left corner control point in the current block is derived based on an MV of a neighboring block. Here, motion vector v_1 at an upper-right corner control point in the current block is derived based on an MV of a neighboring block, and likewise, motion vector v_2 at a lower-left corner control point for the current block is derived based on an MV of a neighboring block. Three motion vectors v_0 , v_1 , and v_2 are projected according to an expression (1B) indicated below, and motion vectors (v_x , v_y) for the respective sub-blocks in the current block are derived.

[MATH. 2]

$$\begin{cases} v_x = \frac{(v_{1x} - v_{0x})}{w}x + \frac{(v_{2x} - v_{0x})}{h}y + v_{0x} \\ v_y = \frac{(v_{1y} - v_{0y})}{w}x + \frac{(v_{2y} - v_{0y})}{h}y + v_{0y} \end{cases} \quad (1B)$$

[0486] Here, x and y indicate the horizontal position and the vertical position of the sub-block, respectively, and each of w and h indicates a predetermined weighting coefficient. Here, w may indicate the width of a current block, and h may indicate the height of the current block.

[0487] Affine modes in which different numbers of control points (for example, two and three control points) are used may be switched and signaled at the CU level. It is to be noted that information indicating the number of control points in affine mode used at the CU level may be signaled at another level (for example, the sequence level, picture level, slice level, brick level, CTU level, or sub-block level).

[0488] In addition, such an affine mode in which three control points are used may include different methods for deriving MVs at the upper-left, upper-right, and lower-left corner control points. For example, the affine modes in which three control points are used include two modes which are affine inter mode and affine merge mode, as in the case of affine modes in which two control points are used.

[0489] It is to be noted that, in the affine modes, the size of each sub-block included in the current block may not be limited to 4×4 pixels, and may be another size. For example, the size of each sub-block may be 8×8 pixels.

[MV Derivation>Affine Mode>Control Point]

[0490] FIGS. 47A, 47B, and 47C are each a conceptual diagram for illustrating one example of MV derivation at control points in an affine mode.

[0491] As illustrated in FIG. 47A, in the affine mode, for example, MV predictors at respective control points for a current block are calculated based on a plurality of MVs corresponding to blocks encoded according to the affine mode among encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left) which neighbor the current block. More specifically, encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left) are checked in the listed order, and the first effective block encoded according to the affine mode is identified. The MV at each control point for the current block is calculated based on the plurality of MVs corresponding to the identified block.

[0492] For example, as illustrated in FIG. 47B, when block A which neighbors to the left of the current block has been encoded according to an affine mode in which two control points are used, motion vectors v_3 and v_4 projected at the upper-left corner position and the upper-right corner position of the encoded block including block A are derived. Motion vector v_0 at the upper-left control point and motion vector v_1 at the upper-right control point for the current block are then calculated from derived motion vectors v_3 and v_4 .

[0493] For example, as illustrated in FIG. 47C, when block A which neighbors to the left of the current block has been encoded according to an affine mode in which three control points are used, motion vectors v_3 , v_4 , and v_5 projected at the upper-left corner position, the upper-right corner position, and the lower-left corner position of the encoded block including block A are derived. Motion vector v_0 at the upper-left control point for the current block, motion vector v_1 at the upper-right control point for the current block, and motion vector v_2 at the lower-left control point for the current block are then calculated from derived motion vectors v_3 , v_4 , and v_5 .

[0494] The MV derivation methods illustrated in FIGS. 47A to 47C may be used in the MV derivation at each control point for the current block in Step Sk_1 illustrated in FIG. 50 described later, or may be used for MV predictor derivation at each control point for the current block in Step Sj_1 illustrated in FIG. 51 described later.

[0495] FIGS. 48A and 48B are each a conceptual diagram for illustrating another example of MV derivation at control points in affine mode.

[0496] FIG. 48A is a diagram for illustrating an affine mode in which two control points are used.

[0497] In the affine mode, as illustrated in FIG. 48A, an MV selected from MVs at encoded block A, block B, and block C which neighbor the current block is used as motion vector v_0 at the upper-left corner control point for the current block. Likewise, an MV selected from MVs of encoded block D and block E which neighbor the current block is used as motion vector v_1 at the upper-right corner control point for the current block.

[0498] FIG. 48B is a diagram for illustrating an affine mode in which three control points are used.

[0499] In the affine mode, as illustrated in FIG. 48B, an MV selected from MVs at encoded block A, block B, and block C which neighbor the current block is used as motion vector v_0 at the upper-left corner control point for the current block. Likewise, an MV selected from MVs of encoded block D and block E which neighbor the current block is used as motion vector v_1 at the upper-right corner control

point for the current block. Furthermore, an MV selected from MVs of encoded block F and block G which neighbor the current block is used as motion vector v_2 at the lower-left corner control point for the current block.

[0500] It is to be noted that the MV derivation methods illustrated in FIGS. 48A and 48B may be used in the MV derivation at each control point for the current block in Step Sk_1 illustrated in FIG. 50 described later, or may be used for MV predictor derivation at each control point for the current block in Step Sj_1 illustrated in FIG. 51 described later.

[0501] Here, when affine modes in which different numbers of control points (for example, two and three control points) are used may be switched and signaled at the CU level, the number of control points for an encoded block and the number of control points for a current block may be different from each other.

[0502] FIGS. 49A and 49B are each a conceptual diagram for illustrating one example of a method for MV derivation at control points when the number of control points for an encoded block and the number of control points for a current block are different from each other.

[0503] For example, as illustrated in FIG. 49A, a current block has three control points at the upper-left corner, the upper-right corner, and the lower-left corner, and block A which neighbors to the left of the current block has been encoded according to an affine mode in which two control points are used. In this case, motion vectors v_3 and v_4 projected at the upper-left corner position and the upper-right corner position in the encoded block including block A are derived. Motion vector v_0 at the upper-left corner control point and motion vector v_1 at the upper-right corner control point for the current block are then calculated from derived motion vectors v_3 and v_4 . Furthermore, motion vector v_2 at the lower-left corner control point is calculated from derived motion vectors v_0 and v_1 .

[0504] For example, as illustrated in FIG. 49B, a current block has two control points at the upper-left corner and the upper-right corner, and block A which neighbors to the left of the current block has been encoded according to an affine mode in which three control points are used. In this case, motion vectors v_3 , v_4 , and v_5 projected at the upper-left corner position in the encoded block including block A, the upper-right corner position in the encoded block, and the lower-left corner position in the encoded block are derived. Motion vector v_0 at the upper-left corner control point for the current block and motion vector v_1 at the upper-right corner control point for the current block are then calculated from derived motion vectors v_3 , v_4 , and v_5 .

[0505] It is to be noted that the MV derivation methods illustrated in FIGS. 49A and 49B may be used in the MV derivation at each control point for the current block in Step Sk_1 illustrated in FIG. 50 described later, or may be used for MV predictor derivation at each control point for the current block in Step Sj_1 illustrated in FIG. 51 described later.

[MV Derivation>Affine Mode>Affine Merge Mode]

[0506] FIG. 50 is a flow chart illustrating one example of the affine merge mode.

[0507] In the affine merge mode, first, inter predictor 126 derives MVs at respective control points for a current block (Step Sk_1). The control points are an upper-left corner point of the current block and an upper-right corner point of

the current block as illustrated in FIG. 46A, or an upper-left corner point of the current block, an upper-right corner point of the current block, and a lower-left corner point of the current block as illustrated in FIG. 46B. At this time, inter predictor 126 may encode MV selection information for identifying two or three derived MVs in a stream.

[0508] For example, when MV derivation methods illustrated in FIGS. 47A to 47C are used, as illustrated in FIG. 47A, inter predictor 126 checks encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left) in the listed order, and identifies the first effective block encoded according to the affine mode.

[0509] Inter predictor 126 derives the MV at the control point using the identified first effective block encoded according to the identified affine mode. For example, when block A is identified and block A has two control points, as illustrated in FIG. 47B, inter predictor 126 calculates motion vector v_0 at the upper-left corner control point of the current block and motion vector v_1 at the upper-right corner control point of the current block from motion vectors v_3 and v_4 at the upper-left corner of the encoded block including block A and the upper-right corner of the encoded block. For example, inter predictor 126 calculates motion vector v_0 at the upper-left corner control point of the current block and motion vector v_1 at the upper-right corner control point of the current block by projecting motion vectors v_3 and v_4 at the upper-left corner and the upper-right corner of the encoded block onto the current block.

[0510] Alternatively, when block A is identified and block A has three control points, as illustrated in FIG. 47C, inter predictor 126 calculates motion vector v_0 at the upper-left corner control point of the current block, motion vector v_1 at the upper-right corner control point of the current block, and motion vector v_2 at the lower-left corner control point of the current block from motion vectors v_3 , v_4 , and v_5 at the upper-left corner of the encoded block including block A, the upper-right corner of the encoded block, and the lower-left corner of the encoded block. For example, inter predictor 126 calculates motion vector v_0 at the upper-left corner control point of the current block, motion vector v_1 at the upper-right corner control point of the current block, and motion vector v_2 at the lower-left corner control point of the current block by projecting motion vectors v_3 , v_4 , and v_5 at the upper-left corner, the upper-right corner, and the lower-left corner of the encoded block onto the current block.

[0511] It is to be noted that, as illustrated in FIG. 49A described above, MVs at three control points may be calculated when block A is identified and block A has two control points, and that, as illustrated in FIG. 49B described above, MVs at two control points may be calculated when block A is identified and block A has three control points.

[0512] Next, inter predictor 126 performs motion compensation of each of a plurality of sub-blocks included in the current block. In other words, inter predictor 126 calculates an MV for each of the plurality of sub-blocks as an affine MV, using either two motion vectors v_0 and v_1 and the above expression (1A) or three motion vectors v_0 , v_1 , and v_2 and the above expression (1B) (Step Sk_2). Inter predictor 126 then performs motion compensation of the sub-blocks using these affine MVs and encoded reference pictures (Step Sk_3). When the processes in Steps Sk_2 and Sk_3 are executed for each of all the sub-blocks included in the current block, the process for generating a prediction image using the affine merge mode for the current block finishes.

In other words, motion compensation of the current block is performed to generate a prediction image of the current block.

[0513] It is to be noted that the above-described MV candidate list may be generated in Step Sk_1. The MV candidate list may be, for example, a list including MV candidates derived using a plurality of MV derivation methods for each control point. The plurality of MV derivation methods may be any combination of the MV derivation methods illustrated in FIGS. 47A to 47C, the MV derivation methods illustrated in FIGS. 48A and 48B, the MV derivation methods illustrated in FIGS. 49A and 49B, and other MV derivation methods.

[0514] It is to be noted that MV candidate lists may include MV candidates in a mode in which prediction is performed in units of a sub-block, other than the affine mode.

[0515] It is to be noted that, for example, an MV candidate list including MV candidates in an affine merge mode in which two control points are used and an affine merge mode in which three control points are used may be generated as an MV candidate list. Alternatively, an MV candidate list including MV candidates in the affine merge mode in which two control points are used and an MV candidate list including MV candidates in the affine merge mode in which three control points are used may be generated separately. Alternatively, an MV candidate list including MV candidates in one of the affine merge mode in which two control points are used and the affine merge mode in which three control points are used may be generated. The MV candidate(s) may be, for example, MVs for encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left), or an MV for an effective block among the blocks.

[0516] It is to be noted that index indicating one of the MVs in an MV candidate list may be transmitted as MV selection information.

[MV Derivation>Affine Mode>Affine Inter Mode]

[0517] FIG. 51 is a flow chart illustrating one example of an affine inter mode.

[0518] In the affine inter mode, first, inter predictor 126 derives MV predictors (v_0 , v_1) or (v_0 , v_1 , v_2) of respective two or three control points for a current block (Step Sj_1). The control points are an upper-left corner point for the current block, an upper-right corner point of the current block, and a lower-left corner point for the current block as illustrated in FIG. 46A or FIG. 46B.

[0519] For example, when the MV derivation methods illustrated in FIGS. 48A and 48B are used, inter predictor 126 derives the MV predictors (v_0 , v_1) or (v_0 , v_1 , v_2) at respective two or three control points for the current block by selecting MVs of any of the blocks among encoded blocks in the vicinity of the respective control points for the current block illustrated in either FIG. 48A or FIG. 48B. At this time, inter predictor 126 encodes, in a stream, MV predictor selection information for identifying the selected two or three MV predictors.

[0520] For example, inter predictor 126 may determine, using a cost evaluation or the like, the block from which an MV as an MV predictor at a control point is selected from among encoded blocks neighboring the current block, and may write, in a bitstream, a flag indicating which MV predictor has been selected. In other words, inter predictor

126 outputs, as a prediction parameter, the MV predictor selection information such as a flag to entropy encoder 110 through prediction parameter generator 130.

[0521] Next, inter predictor 126 performs motion estimation (Steps Sj_3 and Sj_4) while updating the MV predictor selected or derived in Step Sj_1 (Step Sj_2). In other words, inter predictor 126 calculates, as an affine MV, an MV of each of sub-blocks which corresponds to an updated MV predictor, using either the expression (1A) or expression (1B) described above (Step Sj_3). Inter predictor 126 then performs motion compensation of the sub-blocks using these affine MVs and encoded reference pictures (Step Sj_4). The processes in Steps Sj_3 and Sj_4 are executed on all the blocks in the current block each time an MV predictor is updated in Step Sj_2. As a result, for example, inter predictor 126 determines the MV predictor which yields the smallest cost as the MV at a control point in a motion estimation loop (Step Sj_5). At this time, inter predictor 126 further encodes, in the stream, the difference value between the determined MV and the MV predictor as an MV difference. In other words, inter predictor 126 outputs the MV difference as a prediction parameter to entropy encoder 110 through prediction parameter generator 130.

[0522] Lastly, inter predictor 126 generates a prediction image for the current block by performing motion compensation of the current block using the determined MV and the encoded reference picture (Step Sj_6).

[0523] It is to be noted that the above-described MV candidate list may be generated in Step Sj_1. The MV candidate list may be, for example, a list including MV candidates derived using a plurality of MV derivation methods for each control point. The plurality of MV derivation methods may be any combination of the MV derivation methods illustrated in FIGS. 47A to 47C, the MV derivation methods illustrated in FIGS. 48A and 48B, the MV derivation methods illustrated in FIGS. 49A and 49B, and other MV derivation methods.

[0524] It is to be noted that the MV candidate list may include MV candidates in a mode in which prediction is performed in units of a sub-block, other than the affine mode.

[0525] It is to be noted that, for example, an MV candidate list including MV candidates in an affine inter mode in which two control points are used and an affine inter mode in which three control points are used may be generated as an MV candidate list. Alternatively, an MV candidate list including MV candidates in the affine inter mode in which two control points are used and an MV candidate list including MV candidates in the affine inter mode in which three control points are used may be generated separately. Alternatively, an MV candidate list including MV candidates in one of the affine inter mode in which two control points are used and the affine inter mode in which three control points are used may be generated. The MV candidate (s) may be, for example, MVs for encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left), or an MV for an effective block among the blocks.

[0526] It is to be noted that index indicating one of the MV candidates in an MV candidate list may be transmitted as MV predictor selection information.

[MV Derivation>Triangle Mode]

[0527] Inter predictor 126 generates one rectangular prediction image for a rectangular current block in the above example. However, inter predictor 126 may generate a plurality of prediction images each having a shape different from a rectangle for the rectangular current block, and may combine the plurality of prediction images to generate the final rectangular prediction image. The shape different from a rectangle may be, for example, a triangle.

[0528] FIG. 52A is a diagram for illustrating generation of two triangular prediction images.

[0529] Inter predictor 126 generates a triangular prediction image by performing motion compensation of a first partition having a triangular shape in a current block by using a first MV of the first partition, to generate a triangular prediction image. Likewise, inter predictor 126 generates a triangular prediction image by performing motion compensation of a second partition having a triangular shape in a current block by using a second MV of the second partition, to generate a triangular prediction image. Inter predictor 126 then generates a prediction image having the same rectangular shape as the rectangular shape of the current block by combining these prediction images.

[0530] It is to be noted that a first prediction image having a rectangular shape corresponding to a current block may be generated as a prediction image for a first partition, using a first MV. In addition, a second prediction image having a rectangular shape corresponding to a current block may be generated as a prediction image for a second partition, using a second MV. A prediction image for the current block may be generated by performing a weighted addition of the first prediction image and the second prediction image. It is to be noted that the part which is subjected to the weighted addition may be a partial region across the boundary between the first partition and the second partition.

[0531] FIG. 52B is a conceptual diagram for illustrating examples of a first portion of a first partition which overlaps with a second partition, and first and second sets of samples which may be weighted as part of a correction process. The first portion may be, for example, one fourth of the width or height of the first partition. In another example, the first portion may have a width corresponding to N samples adjacent to an edge of the first partition, where N is an integer greater than zero, and N may be, for example, the integer 2. As illustrated, the left example of FIG. 52B shows a rectangular partition having a rectangular portion with a width which is one fourth of the width of the first partition, with the first set of samples including samples outside of the first portion and samples inside of the first portion, and the second set of samples including samples within the first portion. The center example of FIG. 52B shows a rectangular partition having a rectangular portion with a height which is one fourth of the height of the first partition, with the first set of samples including samples outside of the first portion and samples inside of the first portion, and the second set of samples including samples within the first portion. The right example of FIG. 52B shows a triangular partition having a polygonal portion with a height which corresponds to two samples, with the first set of samples including samples outside of the first portion and samples inside of the first portion, and the second set of samples including samples within the first portion.

[0532] The first portion may be a portion of the first partition which overlaps with an adjacent partition. FIG.

52C is a conceptual diagram for illustrating a first portion of a first partition, which is a portion of the first partition that overlaps with a portion of an adjacent partition. For ease of illustration, a rectangular partition having an overlapping portion with a spatially adjacent rectangular partition is shown. Partitions having other shapes, such as triangular partitions, may be employed, and the overlapping portions may overlap with a spatially or temporally adjacent partition.

[0533] In addition, although an example is given in which a prediction image is generated for each of two partitions using inter prediction, a prediction image may be generated for at least one partition using intra prediction.

[0534] FIG. 53 is a flow chart illustrating one example of a triangle mode.

[0535] In the triangle mode, first, inter predictor **126** splits the current block into the first partition and the second partition (Step Sx_1). At this time, inter predictor **126** may encode, in a stream, partition information which is information related to the splitting into the partitions as a prediction parameter. In other words, inter predictor **126** may output the partition information as the prediction parameter to entropy encoder **110** through prediction parameter generator **130**.

[0536] First, inter predictor **126** obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of encoded blocks temporally or spatially surrounding the current block (Step Sx_2). In other words, inter predictor **126** generates an MV candidate list.

[0537] Inter predictor **126** then selects the MV candidate for the first partition and the MV candidate for the second partition as a first MV and a second MV, respectively, from the plurality of MV candidates obtained in Step Sx_2 (Step Sx_3). At this time, inter predictor **126** encodes, in a stream, MV selection information for identifying the selected MV candidate, as a prediction parameter. In other words, inter predictor **126** outputs the MV selection information as a prediction parameter to entropy encoder **110** through prediction parameter generator **130**.

[0538] Next, inter predictor **126** generates a first prediction image by performing motion compensation using the selected first MV and an encoded reference picture (Step Sx_4). Likewise, inter predictor **126** generates a second prediction image by performing motion compensation using the selected second MV and an encoded reference picture (Step Sx_5).

[0539] Lastly, inter predictor **126** generates a prediction image for the current block by performing a weighted addition of the first prediction image and the second prediction image (Step Sx_6).

[0540] It is to be noted that, although the first partition and the second partition are triangles in the example illustrated in FIG. 52A, the first partition and the second partition may be trapezoids, or other shapes different from each other. Furthermore, although the current block includes two partitions in the example illustrated in FIG. 52A, the current block may include three or more partitions.

[0541] In addition, the first partition and the second partition may overlap with each other. In other words, the first partition and the second partition may include the same pixel region. In this case, a prediction image for a current block may be generated using a prediction image in the first partition and a prediction image in the second partition.

[0542] In addition, although the example in which the prediction image is generated for each of the two partitions using inter prediction has been illustrated, a prediction image may be generated for at least one partition using intra prediction.

[0543] It is to be noted that the MV candidate list for selecting the first MV and the MV candidate list for selecting the second MV may be different from each other, or the MV candidate list for selecting the first MV may be also used as the MV candidate list for selecting the second MV.

[0544] It is to be noted that partition information may include an index indicating the splitting direction in which at least a current block is split into a plurality of partitions. The MV selection information may include an index indicating the selected first MV and an index indicating the selected second MV. One index may indicate a plurality of pieces of information. For example, one index collectively indicating a part or the entirety of partition information and a part or the entirety of MV selection information may be encoded.

[MV Derivation>ATMVP Mode]

[0545] FIG. 54 is a diagram illustrating one example of an ATMVP mode in which an MV is derived in units of a sub-block.

[0546] The ATMVP mode is a mode categorized into the merge mode. For example, in the ATMVP mode, an MV candidate for each sub-block is registered in an MV candidate list for use in normal merge mode.

[0547] More specifically, in the ATMVP mode, first, as illustrated in FIG. 54, a temporal MV reference block associated with a current block is identified in an encoded reference picture specified by an MV (MV0) of a neighboring block located at the lower-left position with respect to the current block. Next, in each sub-block in the current block, the MV used to encode the region corresponding to the sub-block in the temporal MV reference block is identified. The MV identified in this way is included in an MV candidate list as an MV candidate for the sub-block in the current block. When the MV candidate for each sub-block is selected from the MV candidate list, the sub-block is subjected to motion compensation in which the MV candidate is used as the MV for the sub-block. In this way, a prediction image for each sub-block is generated.

[0548] Although the block located at the lower-left position with respect to the current block is used as a surrounding MV reference block in the example illustrated in FIG. 54, it is to be noted that another block may be used. In addition, the size of the sub-block may be 4×4 pixels, 8×8 pixels, or another size. The size of the sub-block may be switched for a unit such as a slice, brick, picture, etc.

[Motion Estimation>DMVR]

[0549] FIG. 55 is a diagram illustrating a relationship between a merge mode and DMVR.

[0550] Inter predictor **126** derives an MV for a current block according to the merge mode (Step SI_1). Next, inter predictor **126** determines whether to perform estimation of an MV that is motion estimation (Step SI_2). Here, when determining not to perform motion estimation (No in Step SI_2), inter predictor **126** determines the MV derived in Step SI_1 as the final MV for the current block (Step SI_4). In

other words, in this case, the MV for the current block is determined according to the merge mode.

[0551] When determining to perform motion estimation in Step **Sl_1** (Yes in Step **Sl_2**), inter predictor **126** derives the final MV for the current block by estimating a surrounding region of the reference picture specified by the MV derived in Step **Sl_1** (Step **Sl_3**). In other words, in this case, the MV for the current block is determined according to the DMVR.

[0552] FIG. **56** is a conceptual diagram for illustrating another example of DMVR for determining an MV.

[0553] First, in the merge mode for example, MV candidates (L0 and L1) are selected for the current block. A reference pixel is identified from a first reference picture (L0) which is an encoded picture in the L0 list according to the MV candidate (L0). Likewise, a reference pixel is identified from a second reference picture (L1) which is an encoded picture in the L1 list according to the MV candidate (L1). A template is generated by calculating an average of these reference pixels.

[0554] Next, each of the surrounding regions of MV candidates of the first reference picture (L0) and the second reference picture (L1) are estimated using the template, and the MV which yields the smallest cost is determined to be the final MV. It is to be noted that the cost may be calculated, for example, using a difference value between each of the pixel values in the template and a corresponding one of the pixel values in the estimation region, the values of MV candidates, etc.

[0555] Exactly the same processes described here do not always need to be performed. Any process for enabling derivation of the final MV by estimation in surrounding regions of MV candidates may be used.

[0556] FIG. **57** is a conceptual diagram for illustrating another example of DMVR for determining an MV. Unlike the example of DMVR illustrated in FIG. **56**, in the example illustrated in FIG. **57**, costs are calculated without generating any template.

[0557] First, inter predictor **126** estimates a surrounding region of a reference block included in each of reference pictures in the L0 list and L1 list, based on an initial MV which is an MV candidate obtained from each MV candidate list. For example, as illustrated in FIG. **57**, the initial MV corresponding to the reference block in the L0 list is **InitMV_L0**, and the initial MV corresponding to the reference block in the L1 list is **InitMV_L1**. In motion estimation, inter predictor **126** firstly sets a search position for the reference picture in the L0 list. Based on the position indicated by the vector difference indicating the search position to be set, specifically, the initial MV (that is, **InitMV_L0**), the vector difference to the search position is **MVd_L0**. Inter predictor **126** then determines the estimation position in the reference picture in the L1 list. This search position is indicated by the vector difference to the search position from the position indicated by the initial MV (that is, **InitMV_L1**). More specifically, inter predictor **126** determines the vector difference as **MVd_L1** by mirroring of **MVd_L0**. In other words, inter predictor **126** determines the position which is symmetrical with respect to the position indicated by the initial MV to be the search position in each reference picture in the L0 list and the L1 list. Inter predictor **126** calculates, for each search position, the total sum of the absolute differences (SADs) between values of pixels at search positions in blocks as a cost, and finds out the search position that yields the smallest cost.

[0558] FIG. **58A** is a diagram illustrating one example of motion estimation in DMVR, and FIG. **58B** is a flow chart illustrating one example of the motion estimation.

[0559] First, in Step **1**, inter predictor **126** calculates the cost between the search position (also referred to as a starting point) indicated by the initial MV and eight surrounding search positions. Inter predictor **126** then determines whether the cost at each of the search positions other than the starting point is the smallest. Here, when determining that the cost at the search position other than the starting point is the smallest, inter predictor **126** changes a target to the search position at which the smallest cost is obtained, and performs the process in Step **2**. When the cost at the starting point is the smallest, inter predictor **126** skips the process in Step **2** and performs the process in Step **3**.

[0560] In Step **2**, inter predictor **126** performs the search similar to the process in Step **1**, regarding, as a new starting point, the search position after the target change according to the result of the process in Step **1**. Inter predictor **126** then determines whether the cost at each of the search positions other than the starting point is the smallest. Here, when determining that the cost at the search position other than the starting point is the smallest, inter predictor **126** performs the process in Step **4**. When the cost at the starting point is the smallest, inter predictor **126** performs the process in Step **3**.

[0561] In Step **4**, inter predictor **126** regards the search position at the starting point as the final search position, and determines the difference between the position indicated by the initial MV and the final search position to be a vector difference.

[0562] In Step **3**, inter predictor **126** determines the pixel position at sub-pixel accuracy at which the smallest cost is obtained, based on the costs at the four points located at upper, lower, left, and right positions with respect to the starting point in Step **1** or Step **2**, and regards the pixel position as the final search position. The pixel position at the sub-pixel accuracy is determined by performing weighted addition of each of the four upper, lower, left, and right vectors ((0, 1), (0, -1), (-1, 0), and (1, 0)), using, as a weight, the cost at a corresponding one of the four search positions. Inter predictor **126** then determines the difference between the position indicated by the initial MV and the final search position to be the vector difference.

[Motion Compensation>BIO/OBMC/LIC]

[0563] Motion compensation involves a mode for generating a prediction image, and correcting the prediction image. The mode is, for example, BIO, OBMC, and LIC to be described later.

[0564] FIG. **59** is a flow chart illustrating one example of generation of a prediction image.

[0565] Inter predictor **126** generates a prediction image (Step **Sm_1**), and corrects the prediction image according to any of the modes described above (Step **Sm_2**).

[0566] FIG. **60** is a flow chart illustrating another example of generation of a prediction image.

[0567] Inter predictor **126** derives an MV of a current block (Step **Sn_1**). Next, inter predictor **126** generates a prediction image using the MV (Step **Sn_2**), and determines whether to perform a correction process (Step **Sn_3**). Here, when determining to perform a correction process (Yes in Step **Sn_3**), inter predictor **126** generates the final prediction image by correcting the prediction image (Step **Sn_4**). It is

to be noted that, in LIC described later, luminance and chrominance may be corrected in Step Sn_4. When determining not to perform a correction process (No in Step Sn_3), inter predictor 126 outputs the prediction image as the final prediction image without correcting the prediction image (Step Sn_5).

[Motion Compensation>OBMC]

[0568] It is to be noted that an inter prediction image may be generated using motion information for a neighboring block in addition to motion information for the current block obtained by motion estimation. More specifically, an inter prediction image may be generated for each sub-block in a current block by performing weighted addition of a prediction image based on the motion information obtained by motion estimation (in a reference picture) and a prediction image based on the motion information of the neighboring block (in the current picture). Such inter prediction (motion compensation) is also referred to as overlapped block motion compensation (OBMC) or an OBMC mode.

[0569] In OBMC mode, information indicating a sub-block size for OBMC (referred to as, for example, an OBMC block size) may be signaled at the sequence level. Moreover, information indicating whether to apply the OBMC mode (referred to as, for example, an OBMC flag) may be signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the sequence level and CU level, and may be performed at another level (for example, at the picture level, slice level, brick level, CTU level, or sub-block level).

[0570] The OBMC mode will be described in further detail. FIGS. 61 and 62 are a flow chart and a conceptual diagram for illustrating an outline of a prediction image correction process performed by OBMC.

[0571] First, as illustrated in FIG. 62, a prediction image (Pred) by normal motion compensation is obtained using an MV assigned to a current block. In FIG. 62, the arrow “MV” points a reference picture, and indicates what the current block of the current picture refers to in order to obtain the prediction image.

[0572] Next, a prediction image (Pred_L) is obtained by applying a motion vector (MV_L) which has been already derived for the encoded block neighboring to the left of the current block to the current block (re-using the motion vector for the current block). The motion vector (MV_L) is indicated by an arrow “MV_L” indicating a reference picture from a current block. A first correction of a prediction image is performed by overlapping two prediction images Pred and Pred_L. This provides an effect of blending the boundary between neighboring blocks.

[0573] Likewise, a prediction image (Pred_U) is obtained by applying an MV (MV_U) which has been already derived for the encoded block neighboring above the current block to the current block (re-using the MV for the current block). The MV (MV_U) is indicated by an arrow “MV_U” indicating a reference picture from a current block. A second correction of a prediction image is performed by overlapping the prediction image Pred_U to the prediction images (for example, Pred and Pred_L) on which the first correction has been performed. This provides an effect of blending the boundary between neighboring blocks. The prediction image obtained by the second correction is the one in which

the boundary between the neighboring blocks has been blended (smoothed), and thus is the final prediction image of the current block.

[0574] Although the above example is a two-path correction method using left and upper neighboring blocks, it is to be noted that the correction method may be three- or more-path correction method using also the right neighboring block and/or the lower neighboring block.

[0575] It is to be noted that the region in which such overlapping is performed may be only part of a region near a block boundary instead of the pixel region of the entire block.

[0576] It is to be noted that the prediction image correction process according to OBMC for obtaining one prediction image Pred from one reference picture by overlapping additional prediction images Pred_L and Pred_U has been described above. However, when a prediction image is corrected based on a plurality of reference images, a similar process may be applied to each of the plurality of reference pictures. In such a case, after corrected prediction images are obtained from the respective reference pictures by performing OBMC image correction based on the plurality of reference pictures, the obtained corrected prediction images are further overlapped to obtain the final prediction image.

[0577] It is to be noted that, in OBMC, a current block unit may be a PU or a sub-block unit obtained by further splitting the PU.

[0578] One example of a method for determining whether to apply OBMC is a method for using an obmc_flag which is a signal indicating whether to apply OBMC. As one specific example, encoder 100 may determine whether the current block belongs to a region having complicated motion. Encoder 100 sets the obmc_flag to a value of “1” when the block belongs to a region having complicated motion and applies OBMC when encoding, and sets the obmc_flag to a value of “0” when the block does not belong to a region having complicated motion and encodes the block without applying OBMC. Decoder 200 switches between application and non-application of OBMC by decoding the obmc_flag written in a stream. [Motion Compensation>BIO]

[0579] Next, an MV derivation method is described. First, a mode for deriving an MV based on a model assuming uniform linear motion is described. This mode is also referred to as a bi-directional optical flow (BIO) mode. In addition, this bi-directional optical flow may be written as BDOF instead of BIO.

[0580] FIG. 63 is a diagram for illustrating a model assuming uniform linear motion. In FIG. 63, (vx, vy) indicates a velocity vector, and T0 and T1 indicate temporal distances between a current picture (Cur Pic) and two reference pictures (Ref₀, Ref₁). (MV_{x0}, MV_{y0}) indicates an MV corresponding to reference picture Ref₀, and (MV_{x1}, MV_{y1}) indicates an MV corresponding to reference picture Ref₁.

[0581] Here, under the assumption of uniform linear motion exhibited by a velocity vector (vx, vy), (MV_{x0}, MV_{y0}) and (MV_{x1}, MV_{y1}) are represented as (vxT0, vyT0) and (-vxT1, -vyT1), respectively, and the following optical flow equation (2) is given.

[MATH. 3]

$$\partial I^{(k)} / \partial t + v_x \partial I^{(k)} / \partial x + v_y \partial I^{(k)} / \partial y = 0 \quad (2)$$

[0582] Here, $I(k)$ denotes a luma value from reference image k ($k=0, 1$) after motion compensation. This optical flow equation shows that the sum of (i) the time derivative of the luma value, (ii) the product of the horizontal velocity and the horizontal component of the spatial gradient of a reference image, and (iii) the product of the vertical velocity and the vertical component of the spatial gradient of a reference image is equal to zero. A motion vector of each block obtained from, for example, an MV candidate list may be corrected in units of a pixel, based on a combination of the optical flow equation and Hermite interpolation.

[0583] It is to be noted that a motion vector may be derived on the decoder 200 side using a method other than deriving a motion vector based on a model assuming uniform linear motion. For example, a motion vector may be derived in units of a sub-block based on MVs of a plurality of neighboring blocks.

[0584] FIG. 64 is a flow chart illustrating one example of inter prediction according to BIO. FIG. 65 is a diagram illustrating one example of a configuration of inter predictor 126 which performs inter prediction according to BIO.

[0585] As illustrated in FIG. 65, inter predictor 126 includes, for example, memory 126a, interpolated image deriver 126b, gradient image deriver 126c, optical flow deriver 126d, correction value deriver 126e, and prediction image corrector 126f. It is to be noted that memory 126a may be frame memory 122.

[0586] Inter predictor 126 derives two motion vectors ($M0, M1$), using two reference pictures (Ref_0, Ref_1) different from the picture ($Cur Pic$) including a current block. Inter predictor 126 then derives a prediction image for the current block using the two motion vectors ($M0, M1$) (Step Sy_1). It is to be noted that motion vector $M0$ is motion vector (MVx_0, MVy_0) corresponding to reference picture Ref_0 , and motion vector $M1$ is motion vector (MVx_1, MVy_1) corresponding to reference picture Ref_1 .

[0587] Next, interpolated image deriver 126b derives interpolated image I^0 for the current block, using motion vector $M0$ and reference picture $L0$ by referring to memory 126a. Next, interpolated image deriver 126b derives interpolated image I^1 for the current block, using motion vector $M1$ and reference picture $L1$ by referring to memory 126a (Step Sy_2). Here, interpolated image I^0 is an image included in reference picture Ref_0 and to be derived for the current block, and interpolated image I^1 is an image included in reference picture Ref_1 and to be derived for the current block. Each of interpolated image I^0 and interpolated image I^1 may be the same in size as the current block. Alternatively, each of interpolated image I^0 and interpolated image I^1 may be an image larger than the current block. Furthermore, interpolated image I^0 and interpolated image I^1 may include a prediction image obtained by using motion vectors ($M0, M1$) and reference pictures ($L0, L1$) and applying a motion compensation filter.

[0588] In addition, gradient image deriver 126c derives gradient images (Ix^0, Ix^1, Iy^0, Iy^1) of the current block, from interpolated image I^0 and interpolated image I^1 . It is to be noted that the gradient images in the horizontal direction are (Ix^0, Ix^1), and the gradient images in the vertical direction

are (Iy^0, Iy^1). Gradient image deriver 126c may derive each gradient image by, for example, applying a gradient filter to the interpolated images. It is only necessary that a gradient image indicate the amount of spatial change in pixel value along the horizontal direction or the vertical direction.

[0589] Next, optical flow deriver 126d derives, for each sub-block of the current block, an optical flow (vx, vy) which is a velocity vector, using the interpolated images (I^0, I^1) and the gradient images (Ix^0, Ix^1, Iy^0, Iy^1). The optical flow indicates coefficients for correcting the amount of spatial pixel movement, and may be referred to as a local motion estimation value, a corrected motion vector, or a corrected weighting vector. As one example, a sub-block may be 4x4 pixel sub-CU. It is to be noted that the optical flow derivation may be performed for each pixel unit, or the like, instead of being performed for each sub-block.

[0590] Next, inter predictor 126 corrects a prediction image for the current block using the optical flow (vx, vy). For example, correction value deriver 126e derives a correction value for the value of a pixel included in a current block, using the optical flow (vx, vy) (Step Sy_5). Prediction image corrector 126f may then correct the prediction image for the current block using the correction value (Step Sy_6). It is to be noted that the correction value may be derived in units of a pixel, or may be derived in units of a plurality of pixels or in units of a sub-block.

[0591] It is to be noted that the BIO process flow is not limited to the process disclosed in FIG. 64. Only part of the processes disclosed in FIG. 64 may be performed, or a different process may be added or used as a replacement, or the processes may be executed in a different processing order.

[Motion Compensation>LIC]

[0592] Next, one example of a mode for generating a prediction image (prediction) using a local illumination compensation (LIC) is described.

[0593] FIG. 66A is a diagram for illustrating one example of a prediction image generation method using a luminance correction process performed by LIC. FIG. 66B is a flow chart illustrating one example of a prediction image generation method using the LIC.

[0594] First, inter predictor 126 derives an MV from an encoded reference picture, and obtains a reference image corresponding to the current block (Step Sz_1).

[0595] Next, inter predictor 126 extracts, for the current block, information indicating how the luma value has changed between the current block and the reference picture (Step Sz_2). This extraction is performed based on the luma pixel values of the encoded left neighboring reference region (surrounding reference region) and the encoded upper neighboring reference region (surrounding reference region) in the current picture, and the luma pixel values at the corresponding positions in the reference picture specified by the derived MVs. Inter predictor 126 calculates a luminance correction parameter, using the information indicating how the luma value has changed (Step Sz_3).

[0596] Inter predictor 126 generates a prediction image for the current block by performing a luminance correction process in which the luminance correction parameter is applied to the reference image in the reference picture specified by the MV (Step Sz_4). In other words, the prediction image which is the reference image in the reference picture specified by the MV is subjected to the correc-

tion based on the luminance correction parameter. In this correction, luminance may be corrected, or chrominance may be corrected. In other words, a chrominance correction parameter may be calculated using information indicating how chrominance has changed, and a chrominance correction process may be performed.

[0597] It is to be noted that the shape of the surrounding reference region illustrated in FIG. 66A is one example; another shape may be used.

[0598] Moreover, although the process in which a prediction image is generated from a single reference picture has been described here, cases in which a prediction image is generated from a plurality of reference pictures can be described in the same manner. The prediction image may be generated after performing a luminance correction process of the reference images obtained from the reference pictures in the same manner as described above.

[0599] One example of a method for determining whether to apply LIC is a method for using a `lic_flag` which is a signal indicating whether to apply the LIC. As one specific example, encoder 100 determines whether the current block belongs to a region having a luminance change. Encoder 100 sets the `lic_flag` to a value of "1" when the block belongs to a region having a luminance change and applies LIC when encoding, and sets the `lic_flag` to a value of "0" when the block does not belong to a region having a luminance change and performs encoding without applying LIC. Decoder 200 may decode the `lic_flag` written in the stream and decode the current block by switching between application and non-application of LIC in accordance with the flag value.

[0600] One example of a different method of determining whether to apply a LIC process is a determining method in accordance with whether a LIC process has been applied to a surrounding block. As one specific example, when a current block has been processed in merge mode, inter predictor 126 determines whether an encoded surrounding block selected in MV derivation in merge mode has been encoded using LIC. Inter predictor 126 performs encoding by switching between application and non-application of LIC according to the result. It is to be noted that, also in this example, the same processes are applied to processes at the decoder 200 side.

[0601] The luminance correction (LIC) process has been described with reference to FIGS. 66A and 66B, and is further described below.

[0602] First, inter predictor 126 derives an MV for obtaining a reference image corresponding to a current block from a reference picture which is an encoded picture.

[0603] Next, inter predictor 126 extracts information indicating how the luma value of the reference picture has been changed to the luma value of the current picture, using the luma pixel values of encoded surrounding reference regions which neighbor to the left of and above the current block and the luma pixel values in the corresponding positions in the reference pictures specified by MVs, and calculates a luminance correction parameter. For example, it is assumed that the luma pixel value of a given pixel in the surrounding reference region in the current picture is p_0 , and that the luma pixel value of the pixel corresponding to the given pixel in the surrounding reference region in the reference picture is p_1 . Inter predictor 126 calculates coefficients A and B for optimizing $A \times p_1 + B = p_0$ as the luminance correction parameter for a plurality of pixels in the surrounding reference region.

[0604] Next, inter predictor 126 performs a luminance correction process using the luminance correction parameter for the reference image in the reference picture specified by the MV, to generate a prediction image for the current block. For example, it is assumed that the luma pixel value in the reference image is p_2 , and that the luminance-corrected luma pixel value of the prediction image is p_3 . Inter predictor 126 generates the prediction image after being subjected to the luminance correction process by calculating $A \times p_2 + B = p_3$ for each of the pixels in the reference image.

[0605] For example, a region having a determined number of pixels extracted from each of an upper neighboring pixel and a left neighboring pixel may be used as a surrounding reference region. In addition, the surrounding reference region is not limited to a region which neighbors the current block, and may be a region which does not neighbor the current block. In the example illustrated in FIG. 66A, the surrounding reference region in the reference picture may be a region specified by another MV in a current picture, from a surrounding reference region in the current picture. For example, the other MV may be an MV in a surrounding reference region in the current picture.

[0606] Although operations performed by encoder 100 have been described here, it is to be noted that decoder 200 performs similar operations.

[0607] It is to be noted that LIC may be applied not only to luma but also to chroma. At this time, a correction parameter may be derived individually for each of Y, Cb, and Cr, or a common correction parameter may be used for any of Y, Cb, and Cr.

[0608] In addition, the LIC process may be applied in units of a sub-block. For example, a correction parameter may be derived using a surrounding reference region in a current sub-block and a surrounding reference region in a reference sub-block in a reference picture specified by an MV of the current sub-block.

[Prediction Controller]

[0609] Prediction controller 128 selects one of an intra prediction image (an image or a signal output from intra predictor 124) and an inter prediction image (an image or a signal output from inter predictor 126), and outputs the selected prediction image to subtractor 104 and adder 116.

[Prediction Parameter Generator]

[0610] Prediction parameter generator 130 may output information related to intra prediction, inter prediction, selection of a prediction image in prediction controller 128, etc. as a prediction parameter to entropy encoder 110. Entropy encoder 110 may generate a stream, based on the prediction parameter which is input from prediction parameter generator 130 and quantized coefficients which are input from quantizer 108. The prediction parameter may be used in decoder 200. Decoder 200 may receive and decode the stream, and perform the same processes as the prediction processes performed by intra predictor 124, inter predictor 126, and prediction controller 128. The prediction parameter may include (i) a selection prediction signal (for example, an MV, a prediction type, or a prediction mode used by intra predictor 124 or inter predictor 126), or (ii) an optional index, a flag, or a value which is based on a prediction

process performed in each of intra predictor **124**, inter predictor **126**, and prediction controller **128**, or which indicates the prediction process.

[Decoder]

[0611] Next, decoder **200** capable of decoding a stream output from encoder **100** described above is described. FIG. **67** is a block diagram illustrating a configuration of decoder **200** according to this embodiment. Decoder **200** is an apparatus which decodes a stream that is an encoded image in units of a block.

[0612] As illustrated in FIG. **67**, decoder **200** includes entropy decoder **202**, inverse quantizer **204**, inverse transformer **206**, adder **208**, block memory **210**, loop filter **212**, frame memory **214**, intra predictor **216**, inter predictor **218**, prediction controller **220**, prediction parameter generator **222**, and splitting determiner **224**. It is to be noted that intra predictor **216** and inter predictor **218** are configured as part of a prediction executor.

[Mounting Example of Decoder]

[0613] FIG. **68** is a block diagram illustrating a mounting example of decoder **200**. Decoder **200** includes processor b1 and memory b2. For example, the plurality of constituent elements of decoder **200** illustrated in FIG. **67** are mounted on processor b1 and memory b2 illustrated in FIG. **68**.

[0614] Processor b1 is circuitry which performs information processing and is accessible to memory b2. For example, processor b1 is a dedicated or general electronic circuit which decodes a stream. Processor b1 may be a processor such as a CPU. In addition, processor b1 may be an aggregate of a plurality of electronic circuits. In addition, for example, processor b1 may take the roles of two or more constituent elements other than a constituent element for storing information out of the plurality of constituent elements of decoder **200** illustrated in FIG. **67**, etc.

[0615] Memory b2 is dedicated or general memory for storing information that is used by processor b1 to decode a stream. Memory b2 may be electronic circuitry, and may be connected to processor b1. In addition, memory b2 may be included in processor b1. In addition, memory b2 may be an aggregate of a plurality of electronic circuits. In addition, memory b2 may be a magnetic disc, an optical disc, or the like, or may be represented as a storage, a medium, or the like. In addition, memory b2 may be non-volatile memory, or volatile memory.

[0616] For example, memory b2 may store an image or a stream. In addition, memory b2 may store a program for causing processor b1 to decode a stream.

[0617] In addition, for example, memory b2 may take the roles of two or more constituent elements for storing information out of the plurality of constituent elements of decoder **200** illustrated in FIG. **67**, etc. More specifically, memory b2 may take the roles of block memory **210** and frame memory **214** illustrated in FIG. **67**. More specifically, memory b2 may store a reconstructed image (specifically, a reconstructed block, a reconstructed picture, or the like).

[0618] It is to be noted that, in decoder **200**, not all of the plurality of constituent elements illustrated in FIG. **67**, etc. may be implemented, and not all the processes described above may be performed. Part of the constituent elements

indicated in FIG. **67**, etc. may be included in another device, or part of the processes described above may be performed by another device.

[0619] Hereinafter, an overall flow of the processes performed by decoder **200** is described, and then each of the constituent elements included in decoder **200** is described. It is to be noted that, some of the constituent elements included in decoder **200** perform the same processes as performed by some of the constituent elements included in encoder **100**, and thus the same processes are not repeatedly described in detail. For example, inverse quantizer **204**, inverse transformer **206**, adder **208**, block memory **210**, frame memory **214**, intra predictor **216**, inter predictor **218**, prediction controller **220**, and loop filter **212** included in decoder **200** perform similar processes as performed by inverse quantizer **112**, inverse transformer **114**, adder **116**, block memory **118**, frame memory **122**, intra predictor **124**, inter predictor **126**, prediction controller **128**, and loop filter **120** included in encoder **100**, respectively.

[Overall Flow of Decoding Process]

[0620] FIG. **69** is a flow chart illustrating one example of an overall decoding process performed by decoder **200**.

[0621] First, splitting determiner **224** in decoder **200** determines a splitting pattern of each of a plurality of fixed-size blocks (128×128 pixels) included in a picture, based on a parameter which is input from entropy decoder **202** (Step Sp_1). This splitting pattern is a splitting pattern selected by encoder **100**. Decoder **200** then performs processes of Steps Sp_2 to Sp_6 for each of a plurality of blocks of the splitting pattern.

[0622] Entropy decoder **202** decodes (specifically, entropy decodes) encoded quantized coefficients and a prediction parameter of a current block (Step Sp_2).

[0623] Next, inverse quantizer **204** performs inverse quantization of the plurality of quantized coefficients and inverse transformer **206** performs inverse transform of the result, to restore prediction residuals of the current block (Step Sp_3).

[0624] Next, the prediction executor including all or part of intra predictor **216**, inter predictor **218**, and prediction controller **220** generates a prediction image of the current block (Step Sp_4).

[0625] Next, adder **208** adds the prediction image to a prediction residual to generate a reconstructed image (also referred to as a decoded image block) of the current block (Step Sp_5).

[0626] When the reconstructed image is generated, loop filter **212** performs filtering of the reconstructed image (Step Sp_6). Decoder **200** then determines whether decoding of the entire picture has been finished (Step Sp_7). When determining that the decoding has not yet been finished (No in Step Sp_7), decoder **200** repeatedly executes the processes starting with Step Sp_1.

[0627] It is to be noted that the processes of these Steps Sp_1 to Sp_7 may be performed sequentially by decoder **200**, or two or more of the processes may be performed in parallel. The processing order of the two or more of the processes may be modified.

[Splitting Determiner]

[0628] FIG. **70** is a diagram illustrating a relationship between splitting determiner **224** and other constituent elements. Splitting determiner **224** may perform the following processes as examples.

[0629] For example, splitting determiner 224 collects block information from block memory 210 or frame memory 214, and furthermore obtains a parameter from entropy decoder 202. Splitting determiner 224 may then determine the splitting pattern of a fixed-size block, based on the block information and the parameter. Splitting determiner 224 may then output information indicating the determined splitting pattern to inverse transformer 206, intra predictor 216, and inter predictor 218. Inverse transformer 206 may perform inverse transform of transform coefficients, based on the splitting pattern indicated by the information from splitting determiner 224. Intra predictor 216 and inter predictor 218 may generate a prediction image, based on the splitting pattern indicated by the information from splitting determiner 224.

[Entropy Decoder]

[0630] FIG. 71 is a block diagram illustrating one example of a configuration of entropy decoder 202.

[0631] Entropy decoder 202 generates quantized coefficients, a prediction parameter, and a parameter related to a splitting pattern, by entropy decoding the stream. For example, CABAC is used in the entropy decoding. More specifically, entropy decoder 202 includes, for example, binary arithmetic decoder 202a, context controller 202b, and debinarizer 202c. Binary arithmetic decoder 202a arithmetically decodes the stream using a context value derived by context controller 202b to a binary signal. Context controller 202b derives a context value according to a feature or a surrounding state of a syntax element, that is, an occurrence probability of a binary signal, in the same manner as performed by context controller 110b of encoder 100. Debinarizer 202c performs debinarization for transforming the binary signal output from binary arithmetic decoder 202a to a multi-level signal indicating quantized coefficients as described above. This binarization is performed according to the binarization method described above.

[0632] With this, entropy decoder 202 outputs quantized coefficients of each block to inverse quantizer 204. Entropy decoder 202 may output a prediction parameter included in a stream (see FIG. 1) to intra predictor 216, inter predictor 218, and prediction controller 220. Intra predictor 216, inter predictor 218, and prediction controller 220 are capable of executing the same prediction processes as those performed by intra predictor 124, inter predictor 126, and prediction controller 128 at the encoder 100 side.

[Entropy Decoder]

[0633] FIG. 72 is a diagram illustrating a flow of CABAC in entropy decoder 202.

[0634] First, initialization is performed in CABAC in entropy decoder 202. In the initialization, initialization in binary arithmetic decoder 202a and setting of an initial context value are performed. Binary arithmetic decoder 202a and debinarizer 202c then execute arithmetic decoding and debinarization of, for example, encoded data of a CTU. At this time, context controller 202b updates the context value each time arithmetic decoding is performed. Context controller 202b then saves the context value as a post process. The saved context value is used, for example, to initialize the context value for the next CTU.

[Inverse Quantizer]

[0635] Inverse quantizer 204 inverse quantizes quantized coefficients of a current block which are inputs from entropy decoder 202. More specifically, inverse quantizer 204 inverse quantizes the quantized coefficients of the current block, based on quantization parameters corresponding to the quantized coefficients. Inverse quantizer 204 then outputs the inverse quantized transform coefficients (that are transform coefficients) of the current block to inverse transformer 206.

[0636] FIG. 73 is a block diagram illustrating one example of a configuration of inverse quantizer 204.

[0637] Inverse quantizer 204 includes, for example, quantization parameter generator 204a, predicted quantization parameter generator 204b, quantization parameter storage 204d, and inverse quantization executor 204e.

[0638] FIG. 74 is a flow chart illustrating one example of inverse quantization performed by inverse quantizer 204.

[0639] Inverse quantizer 204 may perform an inverse quantization process as one example for each CU based on the flow illustrated in FIG. 74. More specifically, quantization parameter generator 204a determines whether to perform inverse quantization (Step Sv_11). Here, when determining to perform inverse quantization (Yes in Step Sv_11), quantization parameter generator 204a obtains a difference quantization parameter for the current block from entropy decoder 202 (Step Sv_12).

[0640] Next, predicted quantization parameter generator 204b then obtains a quantization parameter for a processing unit different from the current block from quantization parameter storage 204d (Step Sv_13). Predicted quantization parameter generator 204b generates a predicted quantization parameter of the current block based on the obtained quantization parameter (Step Sv_14).

[0641] Quantization parameter generator 204a then adds the difference quantization parameter for the current block obtained from entropy decoder 202 and the predicted quantization parameter for the current block generated by predicted quantization parameter generator 204b (Step Sv_15). This addition generates a quantization parameter for the current block. In addition, quantization parameter generator 204a stores the quantization parameter for the current block in quantization parameter storage 204d (Step Sv_16).

[0642] Next, inverse quantization executor 204e inverse quantizes the quantized coefficients of the current block into transform coefficients, using the quantization parameter generated in Step Sv_15 (Step Sv_17).

[0643] It is to be noted that the difference quantization parameter may be decoded at the bit sequence level, picture level, slice level, brick level, or CTU level. In addition, the initial value of the quantization parameter may be decoded at the sequence level, picture level, slice level, brick level, or CTU level. At this time, the quantization parameter may be generated using the initial value of the quantization parameter and the difference quantization parameter.

[0644] It is to be noted that inverse quantizer 204 may include a plurality of inverse quantizers, and may inverse quantize the quantized coefficients using an inverse quantization method selected from a plurality of inverse quantization methods.

[Inverse Transformer]

[0645] Inverse transformer **206** restores prediction residuals by inverse transforming the transform coefficients which are inputs from inverse quantizer **204**.

[0646] For example, when information parsed from a stream indicates that EMT or AMT is to be applied (for example, when an AMT flag is true), inverse transformer **206** inverse transforms the transform coefficients of the current block based on information indicating the parsed transform type.

[0647] Moreover, for example, when information parsed from a stream indicates that NSST is to be applied, inverse transformer **206** applies a secondary inverse transform to the transform coefficients.

[0648] FIG. 75 is a flow chart illustrating one example of a process performed by inverse transformer **206**.

[0649] For example, inverse transformer **206** determines whether information indicating that no orthogonal transform is performed is present in a stream (Step St_11). Here, when determining that no such information is present (No in Step St_11), inverse transformer **206** obtains information indicating the transform type decoded by entropy decoder **202** (Step St_12). Next, based on the information, inverse transformer **206** determines the transform type used for the orthogonal transform in encoder **100** (Step St_13). Inverse transformer **206** then performs inverse orthogonal transform using the determined transform type (Step St_14).

[0650] FIG. 76 is a flow chart illustrating another example of a process performed by inverse transformer **206**.

[0651] For example, inverse transformer **206** determines whether a transform size is smaller than or equal to a predetermined value (Step Su_11). Here, when determining that the transform size is smaller than or equal to a predetermined value (Yes in Step Su_11), inverse transformer **206** obtains, from entropy decoder **202**, information indicating which transform type has been used by encoder **100** among at least one transform type included in the first transform type group (Step Su_12). It is to be noted that such information is decoded by entropy decoder **202** and output to inverse transformer **206**.

[0652] Based on the information, inverse transformer **206** determines the transform type used for the orthogonal transform in encoder **100** (Step Su_13). Inverse transformer **206** then inverse orthogonal transforms the transform coefficients of the current block using the determined transform type (Step Su_14). When determining that a transform size is not smaller than or equal to the predetermined value (No in Step Su_11), inverse transformer **206** inverse transforms the transform coefficients of the current block using the second transform type group (Step Su_15).

[0653] It is to be noted that the inverse orthogonal transform by inverse transformer **206** may be performed according to the flow illustrated in FIG. 75 or FIG. 76 for each TU as one example. In addition, inverse orthogonal transform may be performed by using a predefined transform type without decoding information indicating a transform type used for orthogonal transform. In addition, the transform type is specifically DST7, DCT8, or the like. In inverse orthogonal transform, an inverse transform basis function corresponding to the transform type is used.

[Adder]

[0654] Adder **208** reconstructs the current block by adding a prediction residual which is an input from inverse transformer **206** and a prediction image which is an input from prediction controller **220**. In other words, a reconstructed image of the current block is generated. Adder **208** then outputs the reconstructed image of the current block to block memory **210** and loop filter **212**.

[Block Memory]

[0655] Block memory **210** is storage for storing a block which is included in a current picture and is referred to in intra prediction. More specifically, block memory **210** stores a reconstructed image output from adder **208**.

[Loop Filter]

[0656] Loop filter **212** applies a loop filter to the reconstructed image generated by adder **208**, and outputs the filtered reconstructed image to frame memory **214** and a display device, etc.

[0657] When information indicating ON or OFF of an ALF parsed from a stream indicates that an ALF is ON, one filter from among a plurality of filters is selected based on the direction and activity of local gradients, and the selected filter is applied to the reconstructed image.

[0658] FIG. 77 is a block diagram illustrating one example of a configuration of loop filter **212**. It is to be noted that loop filter **212** has a configuration similar to the configuration of loop filter **120** of encoder **100**.

[0659] For example, as illustrated in FIG. 77, loop filter **212** includes deblocking filter executor **212a**, SAO executor **212b**, and ALF executor **212c**. Deblocking filter executor **212a** performs a deblocking filter process of the reconstructed image. SAO executor **212b** performs a SAO process of the reconstructed image after being subjected to the deblocking filter process. ALF executor **212c** performs an ALF process of the reconstructed image after being subjected to the SAO process. It is to be noted that loop filter **212** does not always need to include all the constituent elements disclosed in FIG. 77, and may include only part of the constituent elements. In addition, loop filter **212** may be configured to perform the above processes in a processing order different from the one disclosed in FIG. 77.

[Frame Memory]

[0660] Frame memory **214** is, for example, storage for storing reference pictures for use in inter prediction, and is also referred to as a frame buffer. More specifically, frame memory **214** stores a reconstructed image filtered by loop filter **212**.

[Predictor (Intra Predictor, Inter Predictor, Prediction Controller)]

[0661] FIG. 78 is a flow chart illustrating one example of a process performed by a predictor of decoder **200**. It is to be noted that the prediction executor includes all or part of the following constituent elements: intra predictor **216**; inter predictor **218**; and prediction controller **220**. The prediction executor includes, for example, intra predictor **216** and inter predictor **218**.

[0662] The predictor generates a prediction image of a current block (Step Sq_1). This prediction image is also

referred to as a prediction signal or a prediction block. It is to be noted that the prediction signal is, for example, an intra prediction signal or an inter prediction signal. More specifically, the predictor generates the prediction image of the current block using a reconstructed image which has been already obtained for another block through generation of a prediction image, restoration of a prediction residual, and addition of a prediction image. The predictor of decoder 200 generates the same prediction image as the prediction image generated by the predictor of encoder 100. In other words, the prediction images are generated according to a method common between the predictors or mutually corresponding methods.

[0663] The reconstructed image may be, for example, an image in a reference picture, or an image of a decoded block (that is, the other block described above) in a current picture which is the picture including the current block. The decoded block in the current picture is, for example, a neighboring block of the current block.

[0664] FIG. 79 is a flow chart illustrating another example of a process performed by the predictor of decoder 200.

[0665] The predictor determines either a method or a mode for generating a prediction image (Step Sr_1). For example, the method or mode may be determined based on, for example, a prediction parameter, etc.

[0666] When determining a first method as a mode for generating a prediction image, the predictor generates a prediction image according to the first method (Step Sr_2a). When determining a second method as a mode for generating a prediction image, the predictor generates a prediction image according to the second method (Step Sr_2b). When determining a third method as a mode for generating a prediction image, the predictor generates a prediction image according to the third method (Step Sr_2c).

[0667] The first method, the second method, and the third method may be mutually different methods for generating a prediction image. Each of the first to third methods may be an inter prediction method, an intra prediction method, or another prediction method. The above-described reconstructed image may be used in these prediction methods.

[0668] FIG. 80A and FIG. 80B illustrate a flow chart illustrating another example of a process performed by a predictor of decoder 200.

[0669] The predictor may perform a prediction process according to the flow illustrated in FIG. 80A and FIG. 80B as one example. It is to be noted that intra block copy illustrated in FIG. 80A and FIG. 80B is one mode which belongs to inter prediction, and in which a block included in a current picture is referred to as a reference image or a reference block. In other words, no picture different from the current picture is referred to in intra block copy. In addition, the PCM mode illustrated in FIG. 80A is one mode which belongs to intra prediction, and in which no transform and quantization is performed.

[Intra Predictor]

[0670] Intra predictor 216 performs intra prediction by referring to a block in a current picture stored in block memory 210, based on the intra prediction mode parsed from the stream, to generate a prediction image of a current block (that is, an intra prediction image). More specifically, intra predictor 216 performs intra prediction by referring to pixel values (for example, luma and/or chroma values) of a block or blocks neighboring the current block to generate an

intra prediction image, and then outputs the intra prediction image to prediction controller 220.

[0671] It is to be noted that when an intra prediction mode in which a luma block is referred to in intra prediction of a chroma block is selected, intra predictor 216 may predict the chroma component of the current block based on the luma component of the current block.

[0672] Moreover, when information parsed from a stream indicates that PDPC is to be applied, intra predictor 216 corrects intra predicted pixel values based on horizontal/vertical reference pixel gradients.

[0673] FIG. 81 is a diagram illustrating one example of a process performed by intra predictor 216 of decoder 200.

[0674] Intra predictor 216 firstly determines whether an MPM flag indicating 1 is present in the stream (Step Sw_11). Here, when determining that the MPM flag indicating 1 is present (Yes in Step Sw_11), intra predictor 216 obtains, from entropy decoder 202, information indicating the intra prediction mode selected in encoder 100 among MPMs (Step Sw_12). It is to be noted that such information is decoded by entropy decoder 202 and output to intra predictor 216. Next, intra predictor 216 determines an MPM (Step Sw_13). MPMs include, for example, six intra prediction modes. Intra predictor 216 then determines the intra prediction mode which is included in a plurality of intra prediction modes included in the MPMs and is indicated by the information obtained in Step Sw_12 (Step Sw_14).

[0675] When determining that no MPM flag indicating 1 is present (No in Step Sw_11), intra predictor 216 obtains information indicating the intra prediction mode selected in encoder 100 (Step Sw_15). In other words, intra predictor 216 obtains, from entropy decoder 202, information indicating the intra prediction mode selected in encoder 100 from among at least one intra prediction mode which is not included in the MPMs. It is to be noted that such information is decoded by entropy decoder 202 and output to intra predictor 216. Intra predictor 216 then determines the intra prediction mode which is not included in a plurality of intra prediction modes included in the MPMs and is indicated by the information obtained in Step Sw_15 (Step Sw_17).

[0676] Intra predictor 216 generates a prediction image according to the intra prediction mode determined in Step Sw_14 or Step Sw_17 (Step Sw_18).

[Inter Predictor]

[0677] Inter predictor 218 predicts the current block by referring to a reference picture stored in frame memory 214. Prediction is performed in units of a current block or a current sub-block in the current block. It is to be noted that the sub-block is included in the block and is a unit smaller than the block. The size of the sub-block may be 4x4 pixels, 8x8 pixels, or another size. The size of the sub-block may be switched for a unit such as a slice, brick, picture, etc.

[0678] For example, inter predictor 218 generates an inter prediction image of a current block or a current sub-block by performing motion compensation using motion information (for example, an MV) parsed from a stream (for example, a prediction parameter output from entropy decoder 202), and outputs the inter prediction image to prediction controller 220.

[0679] When the information parsed from the stream indicates that the OBMC mode is to be applied, inter predictor 218 generates the inter prediction image using

motion information of a neighboring block in addition to motion information of the current block obtained through motion estimation.

[0680] Moreover, when the information parsed from the stream indicates that the FRUC mode is to be applied, inter predictor **218** derives motion information by performing motion estimation in accordance with the pattern matching method (bilateral matching or template matching) parsed from the stream. Inter predictor **218** then performs motion compensation (prediction) using the derived motion information.

[0681] Moreover, when the BIO mode is to be applied, inter predictor **218** derives an MV based on a model assuming uniform linear motion. In addition, when the information parsed from the stream indicates that the affine mode is to be applied, inter predictor **218** derives an MV for each sub-block, based on the MVs of a plurality of neighboring blocks.

[MV Derivation Flow]

[0682] FIG. 82 is a flow chart illustrating one example of MV derivation in decoder **200**.

[0683] Inter predictor **218** determines, for example, whether to decode motion information (for example, an MV). For example, inter predictor **218** may make the determination according to the prediction mode included in the stream, or may make the determination based on other information included in the stream. Here, when determining to decode motion information, inter predictor **218** derives an MV for a current block in a mode in which the motion information is decoded. When determining not to decode motion information, inter predictor **218** derives an MV in a mode in which no motion information is decoded.

[0684] Here, MV derivation modes include a normal inter mode, a normal merge mode, a FRUC mode, an affine mode, etc. which are described later. Modes in which motion information is decoded among the modes include the normal inter mode, the normal merge mode, the affine mode (specifically, an affine inter mode and an affine merge mode), etc. It is to be noted that motion information may include not only an MV but also MV predictor selection information which is described later. Modes in which no motion information is decoded include the FRUC mode, etc. Inter predictor **218** selects a mode for deriving an MV for the current block from the plurality of modes, and derives the MV for the current block using the selected mode.

[0685] FIG. 83 is a flow chart illustrating another example of MV derivation in decoder **200**.

[0686] For example, inter predictor **218** may determine whether to decode an MV difference, that is for example, may make the determination according to the prediction mode included in the stream, or may make the determination based on other information included in the stream. Here, when determining to decode an MV difference, inter predictor **218** may derive an MV for a current block in a mode in which the MV difference is decoded. In this case, for example, the MV difference included in the stream is decoded as a prediction parameter.

[0687] When determining not to decode any MV difference, inter predictor **218** derives an MV in a mode in which no MV difference is decoded. In this case, no encoded MV difference is included in the stream.

[0688] Here, as described above, the MV derivation modes include the normal inter mode, the normal merge

mode, the FRUC mode, the affine mode, etc. which are described later. Modes in which an MV difference is encoded among the modes include the normal inter mode and the affine mode (specifically, the affine inter mode), etc. Modes in which no MV difference is encoded include the FRUC mode, the normal merge mode, the affine mode (specifically, the affine merge mode), etc. Inter predictor **218** selects a mode for deriving an MV for the current block from the plurality of modes, and derives the MV for the current block using the selected mode.

[MV Derivation>Normal Inter Mode]

[0689] For example, when information parsed from a stream indicates that the normal inter mode is to be applied, inter predictor **218** derives an MV based on the information parsed from the stream and performs motion compensation (prediction) using the MV.

[0690] FIG. 84 is a flow chart illustrating an example of inter prediction by normal inter mode in decoder **200**.

[0691] Inter predictor **218** of decoder **200** performs motion compensation for each block. At this time, first, inter predictor **218** obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of decoded blocks temporally or spatially surrounding the current block (Step Sg_11). In other words, inter predictor **218** generates an MV candidate list.

[0692] Next, inter predictor **218** extracts N (an integer of 2 or larger) MV candidates from the plurality of MV candidates obtained in Step Sg_11, as motion vector predictor candidates (also referred to as MV predictor candidates) according to the predetermined ranks in priority order (Step Sg_12). It is to be noted that the ranks in priority order are determined in advance for the respective N MV predictor candidates.

[0693] Next, inter predictor **218** decodes the MV predictor selection information from the input stream, and selects one MV predictor candidate from the N MV predictor candidates as the MV predictor for the current block using the decoded MV predictor selection information (Step Sg_13).

[0694] Next, inter predictor **218** decodes an MV difference from the input stream, and derives an MV for the current block by adding a difference value which is the decoded MV difference and the selected MV predictor (Step Sg_14).

[0695] Lastly, inter predictor **218** generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the decoded reference picture (Step Sg_15). The processes in Steps Sg_11 to Sg_15 are executed on each block. For example, when the processes in Steps Sg_11 to Sg_15 are executed on each of all the blocks in the slice, inter prediction of the slice using the normal inter mode finishes. For example, when the processes in Steps Sg_11 to Sg_15 are executed on each of all the blocks in the picture, inter prediction of the picture using the normal inter mode finishes. It is to be noted that not all the blocks included in the slice may be subjected to the processes in Steps Sg_11 to Sg_15, and inter prediction of the slice using the normal inter mode may finish when part of the blocks are subjected to the processes. Likewise, inter prediction of the picture using the normal inter mode may finish when the processes in Steps Sg_11 to Sg_15 are executed on part of the blocks in the picture.

[MV Derivation>Normal Merge Mode]

[0696] For example, when information parsed from a stream indicates that the normal merge mode is to be applied, inter predictor **218** derives an MV and performs motion compensation (prediction) using the MV.

[0697] FIG. **85** is a flow chart illustrating an example of inter prediction by normal merge mode in decoder **200**.

[0698] At this time, first, inter predictor **218** obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of decoded blocks temporally or spatially surrounding the current block (Step Sh_11). In other words, inter predictor **218** generates an MV candidate list.

[0699] Next, inter predictor **218** selects one MV candidate from the plurality of MV candidates obtained in Step Sh_11, thereby deriving an MV for the current block (Step Sh_12). More specifically, inter predictor **218** obtains MV selection information included as a prediction parameter in a stream, and selects the MV candidate identified by the MV selection information as the MV for the current block.

[0700] Lastly, inter predictor **218** generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the decoded reference picture (Step Sh_13). The processes in Steps Sh_11 to Sh_13 are executed, for example, on each block. For example, when the processes in Steps Sh_11 to Sh_13 are executed on each of all the blocks in the slice, inter prediction of the slice using the normal merge mode finishes. In addition, when the processes in Steps Sh_11 to Sh_13 are executed on each of all the blocks in the picture, inter prediction of the picture using the normal merge mode finishes. It is to be noted that not all the blocks included in the slice are subjected to the processes in Steps Sh_11 to Sh_13, and inter prediction of the slice using the normal merge mode may finish when part of the blocks are subjected to the processes. Likewise, inter prediction of the picture using the normal merge mode may finish when the processes in Steps Sh_11 to Sh_13 are executed on part of the blocks in the picture.

[MV Derivation>FRUC Mode]

[0701] For example, when information parsed from a stream indicates that the FRUC mode is to be applied, inter predictor **218** derives an MV in the FRUC mode and performs motion compensation (prediction) using the MV. In this case, the motion information is derived at the decoder **200** side without being signaled from the encoder **100** side. For example, decoder **200** may derive the motion information by performing motion estimation. In this case, decoder **200** performs motion estimation without using any pixel value in a current block.

[0702] FIG. **86** is a flow chart illustrating an example of inter prediction by FRUC mode in decoder **200**.

[0703] First, inter predictor **218** generates a list indicating MVs of decoded blocks spatially or temporally neighboring the current block by referring to the MVs as MV candidates (the list is an MV candidate list, and may be used also as an MV candidate list for normal merge mode (Step Si_11)). Next, a best MV candidate is selected from the plurality of MV candidates registered in the MV candidate list (Step Si_12). For example, inter predictor **218** calculates the evaluation value of each MV candidate included in the MV candidate list, and selects one of the MV candidates as the

best MV candidate based on the evaluation values. Based on the selected best MV candidate, inter predictor **218** then derives an MV for the current block (Step Si_14). More specifically, for example, the selected best MV candidate is directly derived as the MV for the current block. In addition, for example, the MV for the current block may be derived using pattern matching in a surrounding region of a position which is included in a reference picture and corresponds to the selected best MV candidate. In other words, estimation using the pattern matching in a reference picture and the evaluation values may be performed in the surrounding region of the best MV candidate, and when there is an MV that yields a better evaluation value, the best MV candidate may be updated to the MV that yields the better evaluation value, and the updated MV may be determined as the final MV for the current block. Update to the MV that yields the better evaluation value may not be performed.

[0704] Lastly, inter predictor **218** generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the decoded reference picture (Step Si_15). The processes in Steps Si_11 to Si_15 are executed, for example, on each block. For example, when the processes in Steps Si_11 to Si_15 are executed on each of all the blocks in the slice, inter prediction of the slice using the FRUC mode finishes. For example, when the processes in Steps Si_11 to Si_15 are executed on each of all the blocks in the picture, inter prediction of the picture using the FRUC mode finishes. Each sub-block may be processed similarly to the above-described case of processing each block.

[MV Derivation>Affine Merge Mode]

[0705] For example, when information parsed from a stream indicates that the affine merge mode is to be applied, inter predictor **218** derives an MV in the affine merge mode and performs motion compensation (prediction) using the MV.

[0706] FIG. **87** is a flow chart illustrating an example of inter prediction by the affine merge mode in decoder **200**.

[0707] In the affine merge mode, first, inter predictor **218** derives MVs at respective control points for a current block (Step Sk_11). The control points are an upper-left corner point of the current block and an upper-right corner point of the current block as illustrated in FIG. **46A**, or an upper-left corner point of the current block, an upper-right corner point of the current block, and a lower-left corner point of the current block as illustrated in FIG. **46B**.

[0708] For example, when the MV derivation methods illustrated in FIGS. **47A** to **47C** are used, as illustrated in FIG. **47A**, inter predictor **218** checks decoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left) in this order, and identifies the first effective block decoded according to the affine mode.

[0709] Inter predictor **218** derives the MV at the control point using the identified first effective block decoded according to the affine mode. For example, when block A is identified and block A has two control points, as illustrated in FIG. **47B**, inter predictor **218** calculates motion vector v_0 at the upper-left corner control point of the current block and motion vector v_1 at the upper-right corner control point of the current block by projecting motion vectors v_3 and v_4 at the upper-left corner and the upper-right corner of the decoded block including block A onto the current block. In this way, the MV at each control point is derived.

[0710] It is to be noted that, as illustrated in FIG. 49A, MVs at three control points may be calculated when block A is identified and block A has two control points, and that, as illustrated in FIG. 49B, MVs at two control points may be calculated when block A is identified and when block A has three control points.

[0711] In addition, when MV selection information is included as a prediction parameter in a stream, inter predictor 218 may derive the MV at each control point for the current block using the MV selection information.

[0712] Next, inter predictor 218 performs motion compensation of each of a plurality of sub-blocks included in the current block. In other words, inter predictor 218 calculates an MV for each of the plurality of sub-blocks as an affine MV, using either two motion vectors v_0 and v_1 and the above expression (1A) or three motion vectors v_0 , v_1 , and v_2 and the above expression (1B) (Step Sk_12). Inter predictor 218 then performs motion compensation of the sub-blocks using these affine MVs and decoded reference pictures (Step Sk_13). When the processes in Steps Sk_12 and Sk_13 are executed for each of all the sub-blocks included in the current block, the inter prediction using the affine merge mode for the current block finishes. In other words, motion compensation of the current block is performed to generate a prediction image of the current block.

[0713] It is to be noted that the above-described MV candidate list may be generated in Step Sk_11. The MV candidate list may be, for example, a list including MV candidates derived using a plurality of MV derivation methods for each control point. The plurality of MV derivation methods may be any combination of the MV derivation methods illustrated in FIGS. 47A to 47C, the MV derivation methods illustrated in FIGS. 48A and 48B, the MV derivation methods illustrated in FIGS. 49A and 49B, and other MV derivation methods.

[0714] It is to be noted that an MV candidate list may include MV candidates in a mode in which prediction is performed in units of a sub-block, other than the affine mode.

[0715] It is to be noted that, for example, an MV candidate list including MV candidates in an affine merge mode in which two control points are used and an affine merge mode in which three control points are used may be generated as an MV candidate list. Alternatively, an MV candidate list including MV candidates in the affine merge mode in which two control points are used and an MV candidate list including MV candidates in the affine merge mode in which three control points are used may be generated separately. Alternatively, an MV candidate list including MV candidates in one of the affine merge mode in which two control points are used and the affine merge mode in which three control points are used may be generated.

[MV Derivation>Affine Inter Mode]

[0716] For example, when information parsed from a stream indicates that the affine inter mode is to be applied, inter predictor 218 derives an MV in the affine inter mode and performs motion compensation (prediction) using the MV.

[0717] FIG. 88 is a flow chart illustrating an example of inter prediction by the affine inter mode in decoder 200.

[0718] In the affine inter mode, first, inter predictor 218 derives MV predictors (v_0 , v_1) or (v_0 , v_1 , v_2) of respective two or three control points for a current block (Step Sj_11).

The control points are an upper-left corner point of the current block, an upper-right corner point of the current block, and a lower-left corner point of the current block as illustrated in FIG. 46A or FIG. 46B.

[0719] Inter predictor 218 obtains MV predictor selection information included as a prediction parameter in the stream, and derives the MV predictor at each control point for the current block using the MV identified by the MV predictor selection information. For example, when the MV derivation methods illustrated in FIGS. 48A and 48B are used, inter predictor 218 derives the motion vector predictors (v_0 , v_1) or (v_0 , v_1 , v_2) at control points for the current block by selecting the MV of the block identified by the MV predictor selection information among decoded blocks in the vicinity of the respective control points for the current block illustrated in either FIG. 48A or FIG. 48B.

[0720] Next, inter predictor 218 obtains each MV difference included as a prediction parameter in the stream, and adds the MV predictor at each control point for the current block and the MV difference corresponding to the MV predictor (Step Sj_12). In this way, the MV at each control point for the current block is derived.

[0721] Next, inter predictor 218 performs motion compensation of each of a plurality of sub-blocks included in the current block. In other words, inter predictor 218 calculates an MV for each of the plurality of sub-blocks as an affine MV, using either two motion vectors v_0 and v_1 and the above expression (1A) or three motion vectors v_0 , v_1 , and v_2 and the above expression (1B) (Step Sj_13). Inter predictor 218 then performs motion compensation of the sub-blocks using these affine MVs and decoded reference pictures (Step Sj_14). When the processes in Steps Sj_13 and Sj_14 are executed for each of all the sub-blocks included in the current block, the inter prediction using the affine merge mode for the current block finishes. In other words, motion compensation of the current block is performed to generate a prediction image of the current block.

[0722] It is to be noted that the above-described MV candidate list may be generated in Step Sj_11 as in Step Sk_11.

[MV Derivation>Triangle Mode]

[0723] For example, when information parsed from a stream indicates that the triangle mode is to be applied, inter predictor 218 derives an MV in the triangle mode and performs motion compensation (prediction) using the MV.

[0724] FIG. 89 is a flow chart illustrating an example of inter prediction by the triangle mode in decoder 200.

[0725] In the triangle mode, first, inter predictor 218 splits the current block into a first partition and a second partition (Step Sx_11). At this time, inter predictor 218 may obtain, from the stream, partition information which is information related to the splitting as a prediction parameter. Inter predictor 218 may then split a current block into a first partition and a second partition according to the partition information.

[0726] Next, first, inter predictor 218 obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of decoded blocks temporally or spatially surrounding the current block (Step Sx_12). In other words, inter predictor 218 generates an MV candidate list.

[0727] Inter predictor 218 then selects the MV candidate for the first partition and the MV candidate for the second

partition as a first MV and a second MV, respectively, from the plurality of MV candidates obtained in Step Sx₁₁ (Step Sx₁₃). At this time, inter predictor 218 may obtain, from the stream, MV selection information for identifying each selected MV candidate, as a prediction parameter. Inter predictor 218 may then select the first MV and the second MV according to the MV selection information.

[0728] Next, inter predictor 218 generates a first prediction image by performing motion compensation using the selected first MV and a decoded reference picture (Step Sx₁₄). Likewise, inter predictor 218 generates a second prediction image by performing motion compensation using the selected second MV and a decoded reference picture (Step Sx₁₅).

[0729] Lastly, inter predictor 218 generates a prediction image for the current block by performing a weighted addition of the first prediction image and the second prediction image (Step Sx₁₆).

[Motion Estimation>DMVR]

[0730] For example, information parsed from a stream indicates that DMVR is to be applied, inter predictor 218 performs motion estimation using DMVR.

[0731] FIG. 90 is a flow chart illustrating an example of motion estimation by DMVR in decoder 200.

[0732] Inter predictor 218 derives an MV for a current block according to the merge mode (Step Sl₁₁). Next, inter predictor 218 derives the final MV for the current block by searching the region surrounding the reference picture indicated by the MV derived in Sl₁₁ (Step Sl₁₂). In other words, the MV of the current block is determined according to the DMVR.

[0733] FIG. 91 is a flow chart illustrating a specific example of motion estimation by DMVR in decoder 200.

[0734] First, in Step 1 illustrated in FIG. 58A, inter predictor 218 calculates the cost between the search position (also referred to as a starting point) indicated by the initial MV and eight surrounding search positions. Inter predictor 218 then determines whether the cost at each of the search positions other than the starting point is the smallest. Here, when determining that the cost at one of the search positions other than the starting point is the smallest, inter predictor 218 changes a target to the search position at which the smallest cost is obtained, and performs the process in Step 2 illustrated in FIG. 58A. When the cost at the starting point is the smallest, inter predictor 218 skips the process in Step 2 illustrated in FIG. 58A and performs the process in Step 3.

[0735] In Step 2 illustrated in FIG. 58A, inter predictor 218 performs search similar to the process in Step 1, regarding the search position after the target change as a new starting point according to the result of the process in Step 1. Inter predictor 218 then determines whether the cost at each of the search positions other than the starting point is the smallest. Here, when determining that the cost at one of the search positions other than the starting point is the smallest, inter predictor 218 performs the process in Step 4. When the cost at the starting point is the smallest, inter predictor 218 performs the process in Step 3.

[0736] In Step 4, inter predictor 218 regards the search position at the starting point as the final search position, and determines the difference between the position indicated by the initial MV and the final search position to be a vector difference.

[0737] In Step 3 illustrated in FIG. 58A, inter predictor 218 determines the pixel position at sub-pixel accuracy at which the smallest cost is obtained, based on the costs at the four points located at upper, lower, left, and right positions with respect to the starting point in Step 1 or Step 2, and regards the pixel position as the final search position. The pixel position at the sub-pixel accuracy is determined by performing weighted addition of each of the four upper, lower, left, and right vectors ((0, 1), (0, -1), (-1, 0), and (1, 0)), using, as a weight, the cost at a corresponding one of the four search positions. Inter predictor 218 then determines the difference between the position indicated by the initial MV and the final search position to be the vector difference.

[Motion Compensation>BIO/OBMC/LIC]

[0738] For example, when information parsed from a stream indicates that correction of a prediction image is to be performed, upon generating a prediction image, inter predictor 218 corrects the prediction image based on the mode for the correction. The mode is, for example, one of BIO, OBMC, and LIC described above.

[0739] FIG. 92 is a flow chart illustrating one example of generation of a prediction image in decoder 200.

[0740] Inter predictor 218 generates a prediction image (Step Sm₁₁), and corrects the prediction image according to any of the modes described above (Step Sm₁₂).

[0741] FIG. 93 is a flow chart illustrating another example of generation of a prediction image in decoder 200.

[0742] Inter predictor 218 derives an MV for a current block (Step Sn₁₁). Next, inter predictor 218 generates a prediction image using the MV (Step Sn₁₂), and determines whether to perform a correction process (Step Sn₁₃). For example, inter predictor 218 obtains a prediction parameter included in the stream, and determines whether to perform a correction process based on the prediction parameter. This prediction parameter is, for example, a flag indicating whether each of the above-described modes is to be applied. Here, when determining to perform a correction process (Yes in Step Sn₁₃), inter predictor 218 generates the final prediction image by correcting the prediction image (Step Sn₁₄). It is to be noted that, in LIC, the luminance and chrominance of the prediction image may be corrected in Step Sn₁₄. When determining not to perform a correction process (No in Step Sn₁₃), inter predictor 218 outputs the final prediction image without correcting the prediction image (Step Sn₁₅).

[Motion Compensation>OBMC]

[0743] For example, when information parsed from a stream indicates that OBMC is to be performed, upon generating a prediction image, inter predictor 218 corrects the prediction image according to the OBMC.

[0744] FIG. 94 is a flow chart illustrating an example of correction of a prediction image by OBMC in decoder 200. It is to be noted that the flow chart in FIG. 94 indicates the correction flow of a prediction image using the current picture and the reference picture illustrated in FIG. 62.

[0745] First, as illustrated in FIG. 62, inter predictor 218 obtains a prediction image (Pred) by normal motion compensation using an MV assigned to the current block.

[0746] Next, inter predictor 218 obtains a prediction image (Pred_L) by applying a motion vector (MV_L) which has been already derived for the decoded block neighboring

to the left of the current block to the current block (re-using the motion vector for the current block). Inter predictor **218** then performs a first correction of a prediction image by overlapping two prediction images Pred and Pred_L. This provides an effect of blending the boundary between neighboring blocks.

[0747] Likewise, inter predictor **218** obtains a prediction image (Pred_U) by applying an MV (MV_U) which has been already derived for the decoded block neighboring above the current block to the current block (re-using the motion vector for the current block). Inter predictor **218** then performs a second correction of the prediction image by overlapping the prediction image Pred_U to the prediction images (for example, Pred and Pred_L) on which the first correction has been performed. This provides an effect of blending the boundary between neighboring blocks. The prediction image obtained by the second correction is the one in which the boundary between the neighboring blocks has been blended (smoothed), and thus is the final prediction image of the current block.

[Motion Compensation>BIO]

[0748] For example, when information parsed from a stream indicates that BIO is to be performed, upon generating a prediction image, inter predictor **218** corrects the prediction image according to the BIO.

[0749] FIG. 95 is a flow chart illustrating an example of correction of a prediction image by the BIO in decoder **200**.

[0750] As illustrated in FIG. 63, inter predictor **218** derives two motion vectors (M0, M1), using two reference pictures (Ref₀, Ref₁) different from the picture (Cur Pic) including a current block. Inter predictor **218** then derives a prediction image for the current block using the two motion vectors (M0, M1) (Step Sy_11). It is to be noted that motion vector M0 is a motion vector (MVx₀, MVy₀) corresponding to reference picture Ref₀, and motion vector M1 is a motion vector (MVx₁, MVy₁) corresponding to reference picture Ref₁.

[0751] Next, inter predictor **218** derives interpolated image I⁰ for the current block using motion vector M0 and reference picture L0. In addition, inter predictor **218** derives interpolated image I¹ for the current block using motion vector M1 and reference picture L1 (Step Sy_12). Here, interpolated image I⁰ is an image included in reference picture Ref₀ and to be derived for the current block, and interpolated image I¹ is an image included in reference picture Ref₁ and to be derived for the current block. Each of interpolated image I⁰ and interpolated image I¹ may be the same in size as the current block. Alternatively, each of interpolated image I⁰ and interpolated image I¹ may be an image larger than the current block. Furthermore, interpolated image I⁰ and interpolated image I¹ may include a prediction image obtained by using motion vectors (M0, M1) and reference pictures (L0, L1) and applying a motion compensation filter.

[0752] In addition, inter predictor **218** derives gradient images (Ix⁰, Ix¹, Iy⁰, Iy¹) of the current block, from interpolated image I⁰ and interpolated image I¹ (Step Sy_13). It is to be noted that the gradient images in the horizontal direction are (Ix⁰, Ix¹), and the gradient images in the vertical direction are (Iy⁰, Iy¹). Inter predictor **218** may derive the gradient images by, for example, applying a gradient filter to the interpolated images. The gradient images may be the ones each of which indicates the amount

of spatial change in pixel value along the horizontal direction or the amount of spatial change in pixel value along the vertical direction.

[0753] Next, inter predictor **218** derives, for each sub-block of the current block, an optical flow (vx, vy) which is a velocity vector, using the interpolated images (I⁰, I¹) and the gradient images (Ix⁰, Ix¹, Iy⁰, Iy¹). As one example, a sub-block may be 4×4 pixel sub-CU.

[0754] Next, inter predictor **218** corrects a prediction image for the current block using the optical flow (vx, vy). For example, inter predictor **218** derives a correction value for the value of a pixel included in a current block, using the optical flow (vx, vy) (Step Sy_15). Inter predictor **218** may then correct the prediction image for the current block using the correction value (Step Sy_16). It is to be noted that the correction value may be derived in units of a pixel, or may be derived in units of a plurality of pixels or in units of a sub-block.

[0755] It is to be noted that the BIO process flow is not limited to the process disclosed in FIG. 95. Only part of the processes disclosed in FIG. 95 may be performed, or a different process may be added or used as a replacement, or the processes may be executed in a different processing order.

[Motion Compensation>LIC]

[0756] For example, when information parsed from a stream indicates that LIC is to be performed, upon generating a prediction image, inter predictor **218** corrects the prediction image according to the LIC.

[0757] FIG. 96 is a flow chart illustrating an example of correction of a prediction image by the LIC in decoder **200**.

[0758] First, inter predictor **218** obtains a reference image corresponding to a current block from a decoded reference picture using an MV (Step Sz_11).

[0759] Next, inter predictor **218** extracts, for the current block, information indicating how the luma value has changed between the current picture and the reference picture (Step Sz_12). This extraction is performed based on the luma pixel values for the decoded left neighboring reference region (surrounding reference region) and the decoded upper neighboring reference region (surrounding reference region), and the luma pixel values at the corresponding positions in the reference picture specified by the derived MVs. Inter predictor **218** calculates a luminance correction parameter, using the information indicating how the luma value changed (Step Sz_13).

[0760] Inter predictor **218** generates a prediction image for the current block by performing a luminance correction process in which the luminance correction parameter is applied to the reference image in the reference picture specified by the MV (Step Sz_14). In other words, the prediction image which is the reference image in the reference picture specified by the MV is subjected to the correction based on the luminance correction parameter. In this correction, luminance may be corrected, or chrominance may be corrected.

[Prediction Controller]

[0761] Prediction controller **220** selects either an intra prediction image or an inter prediction image, and outputs the selected image to adder **208**. As a whole, the configurations, functions, and processes of prediction controller

220, intra predictor 216, and inter predictor 218 at the decoder 200 side may correspond to the configurations, functions, and processes of prediction controller 128, intra predictor 124, and inter predictor 126 at the encoder 100 side.

[Aspect 1 Relating to Coefficient Encoding]

[0762] FIG. 97 is a flow chart indicating a basic coefficient encoding method according to Aspect 1. Specifically, FIG. 97 indicates a coefficient encoding method in an area in which prediction residuals have been obtained in either intra encoding or inter encoding. The following description indicates an operation that is performed by encoder 100. Decoder 200 may perform an operation corresponding to the operation that is performed by encoder 100. For example, decoder 200 may perform inverse orthogonal transform and decoding corresponding to orthogonal transform and encoding that are performed by encoder 100.

[0763] FIG. 97 indicates last_sig_coeff, subblock_flag, thres, and CCB. Here, last_sig_coeff is a parameter indicating a coordinate position at which a coefficient that is not zero (a non-zero coefficient) appears firstly when scanning is performed in a block. Here, subblock_flag is a flag indicating whether or not a non-zero coefficient is present in a 4×4 subblock (that is also referred to as a 16 transform coefficient level). The subblock_flag is also referred to as coded_subblock_flag, or a subblock flag.

[0764] Here, thres is a constant that is determined on a block-by-block basis. It is also excellent that thres is determined in advance. It is also excellent that thres is different depending on the size of a block, or may have the same value regardless of the size of a block. It is also excellent that thres takes a value that is different between when orthogonal transform is applied and when no orthogonal transform is applied. It is also excellent that thres is determined based on the coordinate position that is defined in the block by last_sig_coeff.

[0765] Here, CCB indicates a total number of bins encoded in a context mode in Context Adaptive Binary Arithmetic Coding (CABAC). In other words, CCB indicates a total number of encoding processes based on the context mode in CABAC. The context mode is also referred to as a regular mode. Here, the encoding based on the context mode in CABAC is referred to as CABAC coding or context adaptive coding. In addition, the encoding based on a bypass mode in CABAC is referred to as bypass coding. The processing by bypass coding is lighter than the processing by CABAC coding.

[0766] The CABAC coding is processing for transforming a bin string obtained by binarizing a coding-target signal into an encoded bitstream based on the occurrence probabilities of 0 and 1 for each bin. It is to be noted that the total number of all flags for use in residual coefficient encoding or the number of part of flags for use in residual coefficient encoding may be counted as CCB. The bypass coding is processing for encoding one of the bins in the bin string as one of the bits in the encoded bitstream, without using the variable occurrence probabilities of 0 and 1 for each bin (in other words, using fixed probabilities).

[0767] For example, encoder 100 determines the coefficient encoding scheme through comparison between a CCB value and a thres value.

[0768] Specifically, in FIG. 97, CCB is initialized to 0 (S101). Whether or not orthogonal transform is applied to a

block is then determined (S102). When orthogonal transform is applied to the block (Yes in S102), encoder 100 encodes last_sig_coeff (S131). Encoder 100 then performs a loop process for each subblock (S141 to S148).

[0769] In the subblock-based loop process (S141 to S148), encoder 100 encodes subblock_flag regarding the sub-block. When subblock_flag is different from 0 (Yes in S146), encoder 100 encodes sixteen coefficients in the subblock using a first encoding scheme to be described later (S147).

[0770] When no orthogonal transform is applied to the block (No in S102), encoder 100 performs the loop process for each subblock (S121 to S128).

[0771] In the subblock-based loop process (S121 to S128), encoder 100 determines whether or not CCB is smaller than or equal to thres (S122). When CCB is smaller than or equal to thres (Yes in S122), encoder 100 encodes subblock_flag by CABAC coding (S123). Encoder 100 then counts up CCB (S124). Otherwise (No in S122), encoder 100 encodes subblock_flag by bypass coding (S125).

[0772] When subblock_flag is different from 0 (Yes in S126), encoder 100 encodes sixteen coefficients in the subblock using a second encoding scheme to be described later (S127).

[0773] A case where no orthogonal transform is applied to the block may be, for example, a case where orthogonal transform has been skipped. CCB is also used in the first encoding scheme and the second encoding scheme. CCB may be initialized on a subblock-by-subblock basis. In the case, thres may be not a fixed value for a block but a value that is variable for each subblock.

[0774] Although whether or not CCB counted up from 0 has reached thres is determined here, but it is to be noted that whether or not CCB counted down from thres (or a particular value) has reached 0 may be determined.

[0775] FIG. 98 is a flow chart indicating details of the first encoding scheme indicated in FIG. 97. In the first encoding scheme, a plurality of coefficients in a subblock are encoded. At that time, a first loop process (S151 to S156) is performed for each coefficient information flag of a corresponding one of coefficients in the subblock, and a second loop process (S161 to S165) is performed for each coefficient in the subblock.

[0776] In the first loop process (S151 to S156), one or more coefficient information flags each indicating one or more attributes of the coefficient are sequentially encoded. The one or more coefficient information flags may include sig_flag, gt1_flag, parity_flag, and gt3_flag. The one or more coefficient information flags are sequentially encoded by CABAC coding within a range in which CCB does not exceed thres, and CCB is counted up one-by-one each time of encoding. No coefficient information flag is encoded after CCB exceeds thres,

[0777] In other words, in the first loop process (S151 to S156), encoder 100 determines whether or not CCB is smaller than or equal to thres (S152). When CCB is smaller than or equal to thres (Yes in S152), encoder 100 encodes coefficient information flag by CABAC coding (S153). Encoder 100 then counts up CCB (S154). When CCB is not smaller than or equal to thres (No in S152), encoder 100 ends the first loop process (S151 to S156).

[0778] In the second loop process (S161 to S165), remainder that is a remainder value which is not represented by a coefficient information flag (that is a remainder value for reconstructing the value of the coefficient using the coeffi-

cient information flag) is encoded by Golomb-Rice coding for the coefficient for which the coefficient information flag has been encoded. A coefficient for which no coefficient information flag has been encoded is directly encoded by Golomb-Rice coding. It is to be noted that remainder may be encoded using another encoding scheme instead of using Golomb-Rice coding.

[0779] In other words, in the second loop process (S161 to S165), encoder 100 determines whether a coefficient information flag corresponding to a processing-target coefficient has been encoded (S162). When a coefficient information flag has been encoded (Yes in S162), encoder 100 encodes remainder by Golomb-Rice coding (S163). When no coefficient information flag has been encoded (No in S162), encoder 100 encodes the value of a coefficient by Golomb-Rice coding (S164).

[0780] The total number of loop processes is two here, but it is to be noted that the total number of loop processes may be different from two.

[0781] The above-described sig_flag is a flag indicating whether or not AbsLevel is non-zero. AbsLevel is a value of a coefficient, and specifically, the absolute value of the coefficient. Here, gt1_flag is a flag indicating whether or not AbsLevel is greater than 1. Here, parity_flag is a flag indicating a first bit of AbsLevel and indicating whether AbsLevel is an odd number or an even number. Here, gt3_flag is a flag indicating whether or not AbsLevel is greater than 3.

[0782] There is a case where gt1_flag and gt3_flag are respectively referred to as abs_gt1_flag and abs_gt3_flag. In addition, for example, the value obtained according to $(\text{Abslevel}-4)/2$ may be encoded as the above-described remainder.

[0783] Other one or more coefficient information flags different from the one or more coefficient information flags may be encoded. For example, part of the coefficient information flags does not always need to be encoded. A coefficient information flag included in the one or more coefficient information flags may be replaced with a coefficient information flag or a parameter having another meaning.

[0784] FIG. 99 is a flow chart indicating details of the second encoding scheme indicated in FIG. 97. In the second encoding scheme, a plurality of coefficients in a subblock are encoded. At that time, a first loop process (S171 to S176) is performed for each coefficient information flag of a corresponding one of coefficients in the subblock, and a second loop process (S181 to S185) is performed for each coefficient in the subblock.

[0785] In the first loop process (S171 to S176), one or more coefficient information flags each indicating one or more attributes of the coefficient are sequentially encoded. The one or more coefficient information flags may include sig_flag, sign_flag, gt1_flag, parity_flag, gt3_flag, gt5_flag, gt7_flag, and gt9_flag.

[0786] Here, sign_flag is a flag indicating a plus or minus sign of a coefficient. Here, gt5_flag is a flag indicating whether or not AbsLevel is greater than 5. Here, gt7_flag is a flag indicating whether or not AbsLevel is greater than 7. Here, gt9_flag is a flag indicating whether or not AbsLevel is greater than 9. There is a case where gt5_flag, gt7_flag, and gt9_flag are respectively referred to as abs_gt5_flag, abs_gt7_flag, and abs_gt9_flag. Alternatively, flags each indicating whether or not AbsLevel is greater than x (x is an integer of 1 or more) may be collectively represented as

gtx_flag or abs_gtx_flag. AbsLevel is, for example, an absolute value at a transform coefficient level.

[0787] It is to be noted that other one or more coefficient information flags different from the one or more coefficient information flags may be encoded. For example, part of the coefficient information flags does not always need to be encoded. A coefficient information flag included in the one or more coefficient information flags may be replaced with a coefficient information flag or a parameter having another meaning.

[0788] The above-described one or more coefficient information flags are sequentially encoded by CABAC coding. Subsequently, CCB is counted up one-by-one each time of encoding. When CCB exceeds thres, the coefficient information flag is encoded by bypass coding.

[0789] In other words, in the first loop process (S171 to S176), encoder 100 determines whether or not CCB is smaller than or equal to thres (S172). When CCB is smaller than or equal to thres (Yes in S172), encoder 100 encodes a coefficient information flag by CABAC coding (S173). Encoder 100 then counts up CCB (S174). When CCB is not smaller than or equal to thres (No in S172), encoder 100 encodes the coefficient information flag by bypass coding (S175).

[0790] Syntaxes in the second loop process in FIG. 99 do not change before and after CCB exceeds thres. In other words, the same process is performed in the second loop process (S181 to S185) regardless of whether the coefficient information flag is encoded by CABAC coding or the coefficient information flag is encoded by bypass coding.

[0791] Specifically, in the second loop process (S181 to S185), encoder 100 encodes remainder that is a remainder value which is not represented by a coefficient information flag (that is a remainder value for reconstructing a value of the coefficient using the coefficient information flag) by Golomb-Rice coding (S183). It is to be noted that remainder may be encoded using another encoding scheme instead of using Golomb-Rice coding.

[0792] The total number of loop processes is two here, but it is to be noted that the total number of loop processes may be different from two.

[0793] As indicated in FIGS. 97, 98, and 99, in the basic operation according to this aspect, there is a flag that changes whether or not an encoding process of the flag is counted as the limited total number of CABAC coding processes, depending on whether or not orthogonal transform is applied. In addition, syntaxes in coefficient encoding are different between the case where orthogonal transform is applied and the case where no orthogonal transform is applied. For this reason, there is a possibility that different circuits need to be prepared for the respective cases. Accordingly, there is a possibility that the circuit configurations become complex. [A First Example in Aspect 1 relating to Coefficient Encoding]

[0794] FIG. 100 is a flow chart indicating a coefficient encoding method according to a first example in Aspect 1. In the example in FIG. 100, post-processing of last_sig_coeff (S132) and processing of subblock_flag (S142 to S145) are different from the processing in the example in FIG. 97.

[0795] In FIG. 97, when orthogonal transform is applied, CCB is counted up for sig_flag, parity_flag, and gtx_flag (X=1, 3). In the example in FIG. 100, CCB is counted up also for last_sig_coeff and subblock_flag. On the other hand,

the processing flow in the case where no orthogonal transform is applied is the same as the processing flow in the example in FIG. 97.

[0796] In other words, in the example in FIG. 100, encoder 100 encodes last_sig_coeff (S131), and then adds the total number of CABAC coding processes in encoding of last_sig_coeff to CCB (S132).

[0797] In addition, encoder 100 determines whether or not CCB is smaller than or equal to thres before encoding of subblock_flag (S142). When CCB is smaller than or equal to thres (Yes in S142), encoder 100 encodes subblock_flag by CABAC coding (S143). Encoder 100 then adds 1 to CCB (S144). On the other hand, when CCB is not smaller than or equal to thres (No in S142), encoder 100 encodes subblock_flag by bypass coding (S145).

[Effects of the First Example in Aspect 1 Relating to Coefficient Encoding]

[0798] According to the example in FIG. 100, there is a case where it is possible to commonalize and unify the processing flow of encoding of subblock_flag between when orthogonal transform is performed and when no orthogonal transform is performed. Accordingly, there are a possibility that part of the circuits are shared between when orthogonal transform is performed and when no orthogonal transform is performed and a possibility that the circuit scale is reduced. As a result, there is a possibility that a plurality of processing flows which are divided depending on the application or non-application of orthogonal transform are the same except for the presence or absence of last_sig_coeff.

[0799] For example, even when the total number of CABAC coding processes is limited at a block level, in the example in FIG. 97, subblock_flag is to be inevitably encoded by CABAC coding after CCB reaches thres. On the other hand, in the case of FIG. 100, subblock_flag is not to be encoded by CABAC coding after CCB reaches thres. In this way, there is a possibility that the total number of CABAC coding processes is appropriately limited to thres.

[0800] It is to be noted that the total number of CABAC coding processes in last_sig_coeff does not always need to be included in CCB. In addition, thres may be determined based on the coordinate position that is defined in the block by last_sig_coeff.

[0801] Alternatively, encoder 100 may determine the value of subblock_flag to be always 1 after CCB exceeds thres. When it is defined that the value of subblock_flag is always 1 after CCB exceeds thres, encoder 100 does not always need to encode subblock_flag after CCB exceeds thres.

[0802] In addition, even when no orthogonal transform is applied, encoder 100 may determine the value of subblock_flag to be always 1 after CCB exceeds thres. Also in this case, when it is defined that the value of subblock_flag is always 1 after CCB exceeds thres, encoder 100 does not always need to encode subblock_flag after CCB exceeds thres.

[A Second Example in Aspect 1 Relating to Coefficient Encoding]

[0803] FIG. 101 is a flow chart indicating a coefficient encoding method according to a second example in Aspect

1. In the example in FIG. 101, the processing of subblock_flag (S123) is different from the processing in the example in FIG. 97.

[0804] In FIG. 97, when no orthogonal transform is applied, CCB is counted up for sig_flag, parity_flag, gtX_flag (X=1, 3, 5, 7, 9), and subblock_flag. In the example in FIG. 101, CCB is not counted up for subblock_flag. On the other hand, the processing flow in the case where orthogonal transform is applied is the same as the processing flow in the example in FIG. 97.

[0805] In other words, in the example in FIG. 101, encoder 100 always encodes subblock_flag by CABAC coding without counting up CCB regardless of whether or not CCB exceeds thres (S123).

[Effects of the Second Example in Aspect 1 Relating to Coefficient Encoding]

[0806] According to the example in FIG. 101, there is a case where it is possible to commonalize and unify the processing flow of encoding of subblock_flag between when orthogonal transform is performed and when no orthogonal transform is performed. Accordingly, there are a possibility that part of the circuits are shared between when orthogonal transform is performed and when no orthogonal transform is performed and a possibility that the circuit scale is reduced. As a result, there is a possibility that a plurality of processing flows which are divided depending on the application or non-application of orthogonal transform are the same except for the presence or absence of last_sig_coeff.

[0807] In addition, the processing is simplified in FIG. 101 compared with the processing in FIG. 100. Accordingly, there is possibility that the circuit scale is reduced. In addition, it is assumed that appearance frequency of 0 or 1 regarding subblock_flag tends to be deviated according to surrounding circumstances. Accordingly, it is assumed that the amount of reduction in the amount of codes is larger than the amount of increase in processing delay in the CABAC coding of subblock_flag. Accordingly, it is useful to perform the CABAC coding of subblock_flag without including the CABAC coding in the limited total number of CABAC coding processes.

[0808] It is to be noted that the total number of CABAC coding processes on last_sig_coeff does not always need to be included in CCB. In addition, thres may be determined based on the coordinate position that is defined in the block by last_sig_coeff.

[Aspect 2 Relating to Coefficient Encoding]

[A First Example in Aspect 2 Relating to Coefficient Encoding]

[0809] FIG. 102 is a flow chart indicating a coefficient encoding method according to a first example in Aspect 2. In the example of FIG. 102, even when no orthogonal transform is applied to a block, sixteen coefficients in a subblock is encoded by a first encoding scheme (S127a). This is a difference from the example in FIG. 97.

[0810] Specifically, in the example in FIG. 102, encoder 100 encodes the sixteen coefficients in the subblock using not the second encoding scheme indicated in FIG. 99 but the first encoding scheme indicated in FIG. 98 when no orthogonal transform is applied to a block (S127a). In other words, encoder 100 encodes the sixteen coefficients in the subblock

using not the second encoding scheme indicated in FIG. 99 but the first encoding scheme indicated in FIG. 98 regardless of whether orthogonal transform is applied or no orthogonal transform is applied.

[0811] More specifically, encoder 100 skips encoding of coefficient information flag without using bypass coding when CCB exceeds thres in a first loop process according to the first encoding scheme indicated in FIG. 98 regardless of application or non-application of orthogonal transform. Encoder 100 then encodes a coefficient value by Golomb-Rice coding without using any coefficient information flag when no coefficient information flag corresponding to the processing-target coefficient has been encoded in a second loop process.

[0812] It is to be noted that the syntax for encoding the coefficient information flag in the first loop process in FIG. 98 may be different between when orthogonal transform is applied and when no orthogonal transform is applied. For example, part or all of the coefficient information flags may be different between one or more coefficient information flags in the case where orthogonal transform is applied and one or more coefficient information flags in the case where no orthogonal transform is applied.

[Effects of the First Example in Aspect 2 Relating to Coefficient Encoding]

[0813] According to the example in FIG. 102, there is a possibility that the syntaxes for use in encoding of the sixteen coefficients in the subblock are commonalized after CCB exceeds thres regardless of application or non-application of orthogonal transform even when the syntaxes for use in encoding of the coefficient information flags are different depending on whether or not orthogonal transform is applied. In this way, there is a possibility that part of the circuits is commonalized or the circuit scales is reduced.

[0814] Furthermore, after CCB exceeds thres, a coefficient is encoded without being divided to a coefficient information flag which is encoded by bypass coding and remainder value information which is encoded by Golomb-Rice coding. Accordingly, there are a possibility that increase in information amount is reduced and a possibility that increase in the amount of codes is reduced.

[a Second Example in Aspect 2 Relating to Coefficient Encoding]

[0815] FIG. 103 is a flow chart indicating a coefficient encoding method according to a second example in Aspect 2. In the example of FIG. 103, even when orthogonal transform is applied to a block, sixteen coefficients in a subblock is encoded by a second encoding scheme (S147a). This is a difference from the example in FIG. 97.

[0816] In other words, in the example in FIG. 103, encoder 100 encodes the sixteen coefficients in the subblock using not a first encoding scheme indicated in FIG. 98 but the second encoding scheme indicated in FIG. 99 when orthogonal transform is applied to a block (S147a). In other words, encoder 100 encodes the sixteen coefficients in the subblock using not the first encoding scheme indicated in FIG. 98 but the second encoding scheme indicated in FIG. 99 regardless of whether orthogonal transform is applied or no orthogonal transform is applied.

[0817] More specifically, encoder 100 encodes the coefficient information flags by bypass coding without skipping

encoding when CCB exceeds thres in the first loop process, according to the second encoding scheme indicated in FIG. 99 regardless of application or non-application of orthogonal transform. Encoder 100 then encodes remainder that depends on the coefficient information flag in the second loop process by Golomb-Rice coding.

[0818] It is to be noted that the syntax for encoding the coefficient information flag in the first loop process in FIG. 99 may be different between when orthogonal transform is applied and when no orthogonal transform is applied. For example, part or all of the coefficient information flags may be different between one or more coefficient information flags in the case where orthogonal transform is applied and one or more coefficient information flags in the case where no orthogonal transform is applied.

[Effects of the Second Example in Aspect 2 Relating to Coefficient Encoding]

[0819] According to the example in FIG. 103, there is a possibility that the syntaxes for use in encoding of the sixteen coefficients in the subblock are commonalized after CCB exceeds thres regardless of application or non-application of orthogonal transform even when the syntaxes for use in encoding of the coefficient information flags are different depending on whether or not orthogonal transform is applied. In this way, there is a possibility that part of the circuits is commonalized or the circuit scales are reduced.

[Aspect 3 Relating to Coefficient Encoding]

[0820] FIG. 104 is a flow chart indicating a basic first encoding scheme according to Aspect 3. The syntaxes indicated in FIG. 104 correspond to examples of syntaxes for the first encoding scheme indicated in FIG. 97. Basically, the first encoding scheme is used when orthogonal transform is applied.

[0821] Here, the coefficient information flags and parameters are the same as the coefficient information flags and the parameters indicated in Aspect 1. It is to be noted that the coefficient information flags indicated here are examples, and other coefficient information flags may be encoded. For example, part of the coefficient information flags does not always need to be encoded. A coefficient information flag indicated here may be replaced with a coefficient information flag or a parameter having another meaning.

[0822] A first for loop in the example in FIG. 104 corresponds to the first loop process in the example in FIG. 98. When a value of CCB remains in the first for loop, in other words, when CCB does not exceed a threshold value, a coefficient information flag such as sig_flag is encoded by CABAC coding. When a value of CCB does not remain, no coefficient information flag is encoded. It is to be noted that the second example in Aspect 2 may be applied to this example. In other words, when a value of CCB does not remain, a coefficient information flag may be encoded by bypass coding.

[0823] A second for loop from above and a third for loop from above correspond to the second loop process in the example in FIG. 98. In the second for loop from above, a remainder value regarding a coefficient for which a coefficient information flag has been encoded is encoded by Golomb-Rice coding. In the third for loop from above, a coefficient that is a coefficient for which no coefficient information flag has been encoded is encoded by Golomb-

Rice coding. It is to be noted that a remainder value may be always encoded by Golomb-Rice coding by the second example in Aspect 2 being applied to this example.

[0824] In a fourth for loop from above, `sign_flag` is encoded by bypass coding.

[0825] The syntaxes explained in this aspect may be applied to each of the examples in FIGS. 97, 98, 100, 101, and 102.

[0826] FIG. 105 is a syntax diagram indicating a basic second encoding scheme according to Aspect 3. The syntaxes indicated in FIG. 105 correspond to examples of syntaxes for the second encoding scheme indicated in FIG. 97. Basically, the second encoding scheme is used when no orthogonal transform is applied.

[0827] It is to be noted that the coefficient information flags indicated here are examples, and other coefficient information flags may be encoded. For example, part of the coefficient information flags does not always need to be encoded. A coefficient information flag indicated here may be replaced with a coefficient information flag or a parameter having another meaning.

[0828] Initial five for loops in the example in FIG. 105 corresponds to the first loop process in the example in FIG. 99. When a value of CCB remains in the initial five for loops, in other words, when CCB does not exceed a threshold value, coefficient information flags such as `sig_flag` are encoded by CABAC coding. In other words, when a value of CCB does not remain, coefficient information flags may be encoded by bypass coding. It is to be noted that the first example in Aspect 2 may be applied to this example. In other words, when a value of CCB does not remain, a coefficient information flag may not always be encoded.

[0829] A sixth for loop from above corresponds to the second loop process in the example in FIG. 99. In the sixth for loop from above, a remainder value is encoded by Golomb-Rice coding. It is to be noted that the first example in Aspect 2 may be applied to this example. Specifically, a remainder value regarding the coefficient for which a coefficient information flag has been encoded may be encoded by Golomb-Rice coding. Then, a coefficient that is a coefficient for which no coefficient information flag has been encoded may be encoded by Golomb-Rice coding.

[0830] The syntaxes explained in this aspect may be applied to each of the examples FIGS. 97, 99, 100, 101, and 103.

[0831] The total number of loop processes for encoding coefficient information flags is larger when no orthogonal transform is applied (FIG. 105) than when orthogonal transform is applied (FIG. 104). Thus, there is a case where the amount of processing by hardware inevitably increases when no orthogonal transform is applied more than the amount of processing when orthogonal transform is applied. Furthermore, since the syntaxes for coefficient encoding are different depending on whether or not orthogonal transform is applied, there is a possibility that different circuits need to be prepared for the respective cases. Accordingly, there is a possibility that the circuits become complex.

[a First Example in Aspect 3 Relating to Coefficient Encoding]

[0832] FIG. 106 is a flow chart indicating a second encoding scheme according to a first example in Aspect 3. The syntaxes indicated in FIG. 106 correspond to examples of syntaxes for the second encoding scheme indicated in FIG.

97. The syntaxes indicated in FIG. 104 may be used for the first encoding scheme in FIG. 97. It is to be noted that this example may be combined with another example in Aspect 3, or may be combined with another aspect.

[0833] A first for loop in the example in FIG. 106 corresponds to the first loop process in the example in FIG. 99. When a value of eight or more of CCB remains in the first for loop, in other words, when CCB with 8 added does not exceed a threshold value, coefficient information flags the total number of which is eight at most are encoded by CABAC coding according to the coefficients, and CCB is counted up eight times at most. When a value of eight or more of CCB does not remain, in other words, when CCB with 8 added exceeds the threshold value, eight coefficient information flags are encoded by bypass coding according to the coefficients.

[0834] In other words, it is collectively determined whether or not the eight coefficient information flags can be encoded by CABAC coding, before the eight coefficient information flags are encoded. When the eight coefficient information flags can be encoded by CABAC coding, the coefficient information flags the total number of which is eight at most are encoded by CABAC coding.

[0835] It is to be noted that the first example in Aspect 2 may be applied to this example. In other words, when CCB with 8 added does not exceed a threshold value, the eight coefficient information flags do not always need to be encoded. In other words, in this case, encoding of the eight coefficient information flags may be skipped without the eight coefficient information flags being encoded by bypass coding.

[0836] In addition, as in FIG. 106, encoding of one or more coefficient information flags among the eight coefficient information flags may be skipped according to the values of coefficients. For example, when `sig_flag` is 0, encoding of the remaining seven coefficient information flags may be skipped.

[0837] A second for loop from above the first line corresponds to the second loop process in the example in FIG. 99. In the second for loop from above, a remainder value is encoded by Golomb-Rice coding. It is to be noted that the first example in Aspect 2 may be applied to this example. Specifically, a remainder value regarding the coefficient for which eight coefficient information flags have been encoded is encoded by Golomb-Rice coding, and the coefficient that is the coefficient for which eight coefficient information flags have not been encoded is encoded by Golomb-Rice coding.

[0838] It is to be noted that the coefficient information flags indicated here are examples, and other coefficient information flags may be encoded. For example, part of the coefficient information flags does not always need to be encoded. A coefficient information flag indicated here may be replaced with a coefficient information flag or a parameter having another meaning.

[0839] In addition, the example in FIG. 105 and the example in FIG. 106 may be combined. For example, in the example in FIG. 105, it is also excellent that a collective determination is made as to whether or not the four coefficient information flags can be encoded by CABAC coding, before the four coefficient information flags such as `sig_flag` and `sign_flag` are encoded.

[Effects of the First Example in Aspect 3 Relating to Coefficient Encoding]

[0840] In the example in FIG. 106, all coefficient information flags which are encoded by CABAC coding are encoded in a single loop process. In other words, the total number of loop processes in the example in FIG. 106 is small compared with the one in the example in FIG. 105. Thus, there is a possibility that the processing amount is reduced.

[0841] The total number of loop processes for encoding coefficient information flags by CABAC coding in the example in FIG. 104 and the total number of loop processes for encoding coefficient information flags by CABAC coding in the example in FIG. 106 are equal to each other. Accordingly, there is a possibility that the circuitry in the combination of the example in FIG. 104 and the example in FIG. 106 needs to be less changed than the circuitry in the combination of the example in FIG. 104 and the example in FIG. 105.

[0842] Furthermore, since whether or not the coefficient information flags can be encoded by CABAC coding is collectively determined before the coefficient information flags are encoded, there is a possibility that the processing is simplified and the processing delay is reduced.

[0843] It is also excellent that a collective determination is made as to whether or not the coefficient information flags can be encoded by CABAC coding before the coefficient information flags are encoded in both the case where orthogonal transform is applied and the case where no orthogonal transform is applied. In this way, there is a possibility that the difference between the encoding scheme used for a block to which orthogonal transform is applied and the encoding scheme used for a block to which no orthogonal transform is applied is reduced, and the circuit scale is reduced.

[0844] In addition, in the example in FIG. 106, sig_flag to abs_gt9_flag are included in a single loop, encoding schemes are not limited to the encoding scheme. It is also excellent that a plurality of loops (for example two loops) are used, and that a collective determination is made as to whether or not the coefficient information flags can be encoded by CABAC coding for each of the loops. The processing increases more than the processing in the single loop, a processing reduction effect similar to the processing reduction effect in the example of FIG. 105 is obtained.

[a Second Example in Aspect 3 Relating to Coefficient Encoding]

[0845] FIG. 107 is a flow chart indicating a second encoding scheme according to a second example in Aspect 3. The syntaxes indicated in FIG. 107 correspond to examples of syntaxes for the second encoding scheme indicated in FIG. 97. The syntaxes indicated in FIG. 104 may be used for the first encoding scheme in FIG. 97. It is to be noted that this example may be combined with another example in Aspect 3, or may be combined with another aspect.

[0846] A first for loop in the example in FIG. 107 corresponds to the first loop process in the example in FIG. 99. When a value of seven or more of CCB remains in the first for loop, in other words, when CCB with 7 added does not exceed a threshold value, coefficient information flags the total number of which is seven at most are encoded by CABAC coding according to the coefficients, and CCB is

counted up seven times at most. When a value of seven or more of CCB does not remain, in other words, when CCB with 7 added exceeds the threshold value, seven coefficient information flags are encoded by bypass coding according to the coefficients.

[0847] In other words, it is collectively determined whether or not the seven coefficient information flags can be encoded by CABAC coding, before the seven coefficient information flags are encoded. When the seven coefficient information flags can be encoded by CABAC coding, the coefficient information flags the total number of which is seven at most are encoded by CABAC coding.

[0848] It is to be noted that the first example in Aspect 2 may be applied to this example. In other words, when CCB with 7 added does not exceed the threshold value, the seven coefficient information flags do not always need to be encoded. In other words, in this case, encoding of the seven coefficient information flags may be skipped without the seven coefficient information flags being encoded by bypass coding.

[0849] In addition, as in FIG. 107, encoding of one or more coefficient information flags among the seven coefficient information flags may be skipped according to the values of coefficients. For example, when sig_flag is 0, encoding of the remaining six coefficient information flags may be skipped.

[0850] A second for loop from above corresponds to the second loop process in the example in FIG. 99. In the second for loop from above, a remainder value is encoded by Golomb-Rice coding. It is to be noted that the first example in Aspect 2 may be applied to this example. Specifically, a remainder value regarding the coefficient for which seven coefficient information flags have been encoded is encoded by Golomb-Rice coding, and the coefficient that is the coefficient for which seven coefficient information flags have not been encoded is encoded by Golomb-Rice coding.

[0851] When a value of CCB remains in the third for loop from above, in other words, when CCB does not exceed a threshold value, sign_flag is encoded by CABAC coding, and CCB is counted up. When a value of CCB does not remain, sign_flag is encoded by bypass coding. It is to be noted that sign_flag may be always encoded by bypass coding, as in the example in FIG. 104.

[0852] It is to be noted that the coefficient information flags indicated here are examples, and other coefficient information flags may be encoded. For example, part of the coefficient information flags does not always need to be encoded. A coefficient information flag indicated here may be replaced with a coefficient information flag or a parameter having another meaning.

[0853] In addition, the example in FIG. 105 and the example in FIG. 107 may be combined. For example, in the example in FIG. 105, it is also excellent that a collective determination is made as to whether or not the four coefficient information flags can be encoded by CABAC coding, before the four coefficient information flags such as sig_flag and sign_flag are encoded.

[Effects of the Second Example in Aspect 3 Relating to Coefficient Encoding]

[0854] As in the example in FIG. 106, in the example in FIG. 107, a plurality of coefficient information flags each is for indicating the magnitude of a coefficient compared with a threshold value (specifically, abs_gt3_flag and abs_gt5_

flag, etc.) are encoded in a single loop process. Thus, the total number of loop processes in the example in FIG. 107 is small compared with the one in the example in FIG. 105. For this reason, there is a possibility that the processing amount is reduced.

[0855] Compared with the example in FIG. 106, in the example in FIG. 107, the total number of loop processes for encoding the coefficient information flags inevitably increases. However, the example in FIG. 107 is partly similar to the example in FIG. 104, compared with the example in FIG. 106. For example, sign_flag is encoded finally. Accordingly, there is a possibility that the circuitry in the combination of the example in FIG. 104 and the example in FIG. 107 needs to be less changed than the circuitry in the combination of the example in FIG. 104 and the example in FIG. 106.

[0856] Furthermore, since whether or not the coefficient information flags can be encoded by CABAC coding is collectively determined before the coefficient information flags are encoded, there is a possibility that the processing is simplified and the processing delay is reduced.

[0857] It is also excellent that a collective determination is made as to whether or not the coefficient information flags can be encoded by CABAC before the coefficient information flags are encoded in both the case where orthogonal transform is applied and the case where no orthogonal transform is applied. In addition, there is a possibility that the difference between the encoding scheme used for a block to which orthogonal transform is applied and the encoding scheme used for a block to which no orthogonal transform is applied is reduced, and the circuit scale is further reduced.

[0858] In addition, in the example in FIG. 107, sig_flag to abs_gt9_flag are included in a single loop, encoding schemes are not limited to the encoding scheme. It is also excellent that a plurality of loops (for example two loops) are used, and that a collective determination is made as to whether or not the coefficient information flags can be encoded by CABAC coding for each of the loops. The processing increases more than the processing in the single loop, a processing reduction effect similar to the processing reduction effect in the example of FIG. 105 is obtained.

[Aspect 4 Relating to Coefficient Encoding]

[0859] FIG. 108 is a diagram indicating basic correspondences between coefficient information flags and context numbers (indices) according to Aspect 4. Specifically, FIG. 108 illustrates an example of a method of determining a context number for encoding, by CABAC coding, a coefficient information flag to be encoded when no orthogonal transform is applied. A context number indicates an appearance probability of a value of 0 or 1. Encoder 100 performs arithmetic encoding of a coefficient information flag based on this appearance probability.

[0860] For example, when there is a high possibility that a current coefficient information flag to be encoded is 0, the amount of codes may be reduced by encoding the coefficient information flag by CABAC coding, based on the context number indicating the high appearance probability for the value of 0. In other words, the amount of codes may be reduced by performing arithmetic coding of the coefficient information flag based on the appropriate context number indicating the appropriate appearance probability.

[0861] FIG. 109 is a conceptual diagram indicating basic neighboring (surrounding) coefficients according to Aspect 4. In FIG. 109, x denotes a current coefficient to be encoded.

[0862] Coefficient scanning is performed diagonally starting with the 4×4 subblock located at the upper-left position illustrated in FIG. 109. Specifically, in a coordinate system in which the coordinates of the upper-left position, the coordinates of the upper-right position, the coordinates of the lower-left position, and the coordinates of the lower-right position are represented as (0, 0), (3, 0), (0, 3), and (3, 3), respectively, the coefficient scanning is performed in the following order: (0, 0), (0, 1), (1, 0), (0, 2), (1, 1), . . . (3, 3). L and U indicate two coefficients neighboring x. Each of the positions of coefficients L and U neighbors the position of coefficient x. When no orthogonal transform is applied, the three positions of coefficients x, L, and U correspond to three pixel positions, respectively.

[0863] In this case, in the example in FIG. 108, a sum of the sig_flag of coefficient L and the sig_flag of coefficient U is used as the context number for encoding the sig_flag of coefficient x. In other words, the three context numbers are used selectively. In addition, a single fixed context number is used for each of parity_flag and gtX_flag (X in gtX is, for example, 1, 3, 5, 7, or 9).

[0864] In other words, in FIG. 108, only one context number is used for the coefficient information flag other than sig_flag. In general, in a block in which no orthogonal transform is applied, there is a tendency that pixels arranged in a horizontal or vertical direction have a correlation. However, when only one context number is used, no context number corresponding to the tendency is selected.

[0865] For example, when gt3_flag is to be encoded, there is a case where a coefficient for which gt3_flag has been encoded among a plurality of encoded coefficients is not present in a horizontal line or a vertical line in which a current coefficient to be encoded is present. In this case, there is a high possibility that another value of gt3_flag encoded in the past and the value of gt3_flag of the current coefficient are different from each other. As a result, an appropriate appearance probability of the value of gt3_flag of the current coefficient is not used, which may reduce the compression efficiency.

[a First Example in Aspect 4 Relating to Coefficient Encoding]

[0866] FIG. 110 is a conceptual diagram indicating neighboring coefficients according to a first example in Aspect 4. In FIG. 110, x denotes a current coefficient to be encoded. Each of x0, x1, and x2 indicates a corresponding one of three encoded coefficients neighboring x. When no orthogonal transform is applied, the four positions of coefficients x, x0, x1, and x2 correspond to four pixel positions, respectively. Hereinafter, each of x, x0, x1, and x2 may indicate a coefficient absolute value which is the absolute value of a coefficient.

[0867] In this case, based on the three neighboring coefficient absolute values x0, x1, and x2, pred which is a prediction value of x is defined as indicated below.

[MATH. 4]

$$\text{pred} = \begin{cases} \max(x1, x2) & (x0 \leq \min(x1, x2)) & (3A) \\ \min(x1, x2) & (x0 \geq \max(x1, x2)) & (3B) \\ x1 + x2 - x0 & (\text{otherwise}) & (3C) \end{cases}$$

[0868] However, when there is no neighboring pixel because x is located at a block boundary, pred is calculated assuming that the coefficient of a neighboring pixel is 0.

[0869] When no orthogonal transform is applied to a block, there is a tendency that the block has an edge in a vertical or horizontal direction. Here, the edge is a group of pixels in a part in which each pixel has a large pixel value compared with pixels surrounding the part. It is expected that a prediction accuracy for pred described above is high in both the case where coefficient x is on an edge and the case where coefficient x is not on an edge. The reasons are indicated below.

[0870] FIG. 111A is a conceptual diagram indicating a current coefficient to be encoded that is present in a position in an edge in a horizontal direction. In FIG. 111A, the hatched part is an edge, and x is on the edge. In this case, $x1$ is larger than or equal to $x0$, and the possibility that a conditional expression (3A) is satisfied is high according to the calculation condition for pred . In this case, since $x1$ is larger than or equal to $x2$, $\text{pred}=x1$ is satisfied. In other words pred for x on the edge is calculated as $x1$ on the edge.

[0871] FIG. 111B is a conceptual diagram indicating the current coefficient that is present in a position in an edge in a vertical direction. According to the same reasoning that applies also in FIG. 111B, pred is calculated as $x2$ on the edge. Specifically, in FIG. 111B, the hatched part is an edge, and x is on the edge. In this case, $x2$ is larger than or equal to $x0$, and the possibility that a conditional expression (3A) is satisfied is high according to the calculation condition for pred . In this case, since $x2$ is larger than or equal to $x1$, $\text{pred}=x2$ is satisfied. In other words pred for x on the edge is calculated as $x2$ on the edge.

[0872] FIG. 112A is a conceptual diagram indicating a current coefficient to be encoded that is present neighboring a position in an edge in a horizontal direction. In FIG. 112A, the hatched part is an edge, and x is outside the edge. In this case, $x0$ is on the edge, and is larger than $x1$, and thus the possibility that a conditional expression (3B) is satisfied is high according to the calculation condition for pred . In this case, since $x2$ is larger than or equal to $x1$, $\text{pred}=x1$ is satisfied. In other words, pred for x outside the edge is calculated as $x1$ outside the edge.

[0873] FIG. 112B is a conceptual diagram indicating a current coefficient to be encoded that is present neighboring a position in an edge in a vertical direction. According to the same reasoning that applies also in FIG. 112B, pred is calculated as $x2$ outside the edge. Specifically, in FIG. 112B, the hatched part is an edge, and x is outside the edge. In this case, $x0$ is on the edge, and is larger than $x2$, and thus the possibility that the conditional expression (3B) is satisfied is high according to the calculation condition for pred . In this case, since $x1$ is larger than or equal to $x2$, $\text{pred}=x2$ is satisfied. In other words, pred for x outside the edge is calculated as $x2$ outside the edge.

[0874] It is to be noted that when no edge is present around x , it is assumed that x corresponds to a value that is variable smoothly from the pixels of $x0$, $x1$, and $x2$. Accordingly, as

in a conditional expression (3C), the value obtainable according to $x1+x2-x0$ corresponding to a plane prediction value is calculated as the value of pred .

[0875] In this example, the context number of sig_flag of coefficient x is determined based on pred described above. Although a syntax in the first example in Aspect 3 is used as a syntax for coefficient encoding, another syntax may be used.

[0876] The minimum value for the coefficient absolute values represented as $x0$, $x1$, and $x2$ neighboring x is estimated based only on a plurality of coefficient information flags processed in a first loop process. In other words, when $y0$, $y1$, and $y2$ are original coefficient absolute values, the estimated minimum value for $x0$, $x1$, and $x2$ are represented as indicated below. Here, $a \% b$ indicates a remainder obtainable by dividing a by b .

[MATH. 5]

$$\begin{aligned} x0 &= \min(y0, 11 - !(y0 \% 2)) \\ x1 &= \min(y1, 11 - !(y1 \% 2)) \\ x2 &= \min(y2, 11 - !(y2 \% 2)) \end{aligned} \quad (4)$$

[0877] Using these three values, pred is calculated. Here, pred is a prediction value of the minimum value for x estimated based only on the plurality of coefficient information flags processed in the first loop process, and is specifically a prediction value of a value according to $\min(x, 11 - !(x \% 2))$. In addition, sig_ctx which is the context number of sig_flag of coefficient x is defined as indicated below.

[MATH. 6]

$$\text{sig_ctx} = \begin{cases} 0 & (\text{pred} = 0) \\ 1 & (\text{pred} > 0) \end{cases} \quad (5)$$

[0878] In FIG. 108, the context number of sig_flag is determined, and thus three kinds of context numbers are used selectively using the two left and upper coefficients. In this example, two kinds of context numbers are used selectively using the three coefficients neighboring the current coefficient.

[0879] It is to be noted that, in this example, pred which is the prediction value for the current coefficient is calculated using the three coefficients neighboring the current coefficient. The number of neighboring coefficients for calculating pred may be another number of neighboring coefficients. Alternatively, three coefficients may be obtained from positions different from the positions in this example. Alternatively, the method for calculating pred may be changed. Alternatively, coefficient absolute values $x0$, $x1$, and $x2$ may be estimated using all the flags in the same loop process, or may be estimated using only a part of the flags.

[0880] In the example in FIG. 108, one context number is used for each of the coefficient information flags other than sig_flag . However, a plurality of context numbers may be selectively used for the coefficient information flags other than sig_flag , according to a method similar to the method in this example. For example, parity_ctx and gtX_ctx (X in gtX is, for example, 1, 3, 5, 7, or 9) which are context

numbers of parity_flag and gtX_flag of x which is a current coefficient to be encoded may be derived as indicated below.

[MATH. 7]

$$\text{par_ctx} = \begin{cases} 0 & (\text{pred} \% 2 = 0) \\ 1 & (\text{pred} \% 2 = 1) \end{cases} \quad (6)$$

[MATH. 8]

$$\text{gtX_ctx} = \begin{cases} 0 & (\text{pred} \leq X) \\ 1 & (\text{pred} > X) \end{cases} \quad (7)$$

[0881] In this example, a context number of each coefficient information flag is selected based on the value of pred, from two kinds of context numbers. The two kinds of context numbers may be commonalized by a plurality of coefficient information flags different from each other. Alternatively, coefficient information flags for which context numbers are determined according to another determination method may be present. For example, the method in FIG. 108 may be used as the method of determining the context number of sig_flag and the method in this example may be used as the method of determining the context number of each of the other coefficient information flags. Alternatively, the determination methods may be switched.

[Effects of the First Example in Aspect 4 Relating to Coefficient Encoding]

[0882] As described above, there is a possibility that the prediction accuracy of the coefficient regarding pred is high. Accordingly, for example, switching of context numbers based on whether or not pred which is the prediction value regarding gtX_ctx is larger than or smaller than X (X in gtX) may produce a possibility that an appropriate occurrence probability is used and a possibility that the compression rate is high. Furthermore, as for sig_flag, this example produces a possibility that the compression rate becomes high according to the high prediction accuracy of the coefficient, compared with the example in which two neighboring coefficients as in FIGS. 108 and 109 are used.

[a Second Example in Aspect 4 Relating to Coefficient Encoding]

[0883] In the first example in Aspect 4, prediction value pred is used in order to select the context number of the coefficient information flag relating to coefficient x. The prediction value pred is calculated using the value of the plurality of kinds of coefficient information flags which have been already encoded in the loop process in which the coefficient information flags are encoded.

[0884] In this example, values of coefficient information flags to be encoded for a current coefficient to be encoded are values of three coefficient information flags for three coefficients which neighbor the current coefficient. The values of the three coefficient information flags are predicted using values of same kinds of three coefficient information flags as the coefficient information flags to be encoded.

[0885] For example, when gt3_flag for current coefficient x to be encoded is encoded, gt3_flag_pred which is a prediction value of gt3_flag is calculated as indicated below. Here, x0, x1, and x2 are three coefficients neighboring x as illustrated in FIG. 110.

[MATH. 9]

$$\text{gt3_flag_pred} = \quad (8)$$

$$\begin{cases} \max(\text{gt3_flag}[x1], \text{gt3_flag}[x2]) & (\text{gt3_flag}[x0] \leq \min(\text{gt3_flag}[x1], \text{gt3_flag}[x2])) \\ \min(\text{gt3_flag}[x1], \text{gt3_flag}[x2]) & (\text{gt3_flag}[x0] \geq \max(\text{gt3_flag}[x1], \text{gt3_flag}[x2])) \end{cases}$$

[0886] Next, gt3_flag_pred is defined as gt3_ctx which is a context number of gt3_flag for coefficient x.

[MATH. 10]

$$\text{gt3_ctx} = \text{gt3_flag_pred} \quad (9)$$

[0887] It is to be noted that each of the values of the three flags gt3_flag for the three neighboring coefficients to be used for prediction and each of the prediction values gt3_flag_pred is 0 or 1. Accordingly, gt3_flag_pred is represented using only bit computation as indicated below.

[MATH. 11]

$$\text{gt3_flag_pred} = \quad (10)$$

$$(141 >> (4\text{gt3_flag}[x0] + 2\text{gt3_flag}[x1] + 4\text{gt3_flag}[x2])) \& 1$$

[0888] Likewise, a prediction value of each of coefficient information flags sig_flag, parity_flag, gt1_flag, gt5_flag, gt7_flag, and gt9_flag is also calculated according to the above-described method. Next, the context number is defined.

[0889] It is to be noted that the number of coefficients to be used for prediction of coefficient information flags may be changed, or positions of coefficients may be changed. In addition, the method of predicting coefficient information flags may be changed. In addition, the number of candidate context numbers for coefficient information flags may be increased.

[Effects of the Second Example in Aspect 4 Relating to Coefficient Encoding]

[0890] Likewise the prediction of the coefficient information flags in the first example in Aspect 4, also in the prediction of the coefficient information flags in the second example in Aspect 4, there is a possibility that the resulting prediction accuracy becomes high both in the case where a coefficient is on an edge and in the case where a coefficient is not on an edge. As a result, there is a possibility that the resulting cabac compression rate is high.

[0891] Furthermore, since the prediction values of the coefficient information flags are calculated only by the bit computation, there is a possibility that the resulting processing amount is reduced more than the processing amount required when performing the calculation of pred in the first example in Aspect 4.

[a Third Example in Aspect 4 Relating to Coefficient Encoding]

[0892] This example also relates to a processing flow in the case where no orthogonal transform is applied in the

example in FIG. 97. Although a syntax in the first example in Aspect 3, that is the example in FIG. 106 is used as a syntax for coefficient encoding, another syntax may be used. In the example in FIG. 106, remainder which is a remainder of a coefficient that is not represented by a coefficient information flag is encoded with Golomb-Rice code. The explanation for remainder is as described in Aspect 1 relating to coefficient encoding.

[0893] In this example, pred is used for selection of a Rice parameter to be used in Golomb-Rice coding. Here, pred is a value to be predicted using three coefficients which neighbor a current coefficient to be encoded as described in the first example in Aspect 4.

[0894] First, the outline of Golomb-Rice coding is explained. In the encoding of remainder, an approach what is called Golomb-Rice coding is used. In the Golomb-Rice coding, the value of remainder is binarized using a prefix and a suffix.

[0895] Rice code is used for the prefix. In addition, encoding methods are switched by Rice parameter g (for example, g is a ternary of 0, 1, or 2). In addition, unary code and exponential Golomb code are used for the suffix.

[0896] FIG. 113 is a conceptual diagram indicating correspondences between residual values and Golomb-Rice code according to the third example in Aspect 4. Specifically, FIG. 113 indicates the encoded bin of each of the prefix and the suffix in the case where remainder is encoded by Golomb-Rice coding, according to the value of a Rice parameter.

[0897] Here, x denotes a current coefficient to be encoded. Here, x may denote the absolute value of the coefficient. Here, remainder[x] denotes remainder of x in the processing flow in FIG. 99. Each of remainder[x0], remainder[x1], and remainder[x2] denotes a corresponding one of three remainders of three coefficients which neighbor x in FIG. 110.

[0898] In addition, when any one of these three coefficients neighboring the coefficient to be encoded is represented as one or more coefficient information flags, that is, when no remainder has occurred for the one neighboring coefficient, remainder of the coefficient may be represented as 0. Next, prediction value remainder_pred for remainder [x] is defined as indicated below.

[MATH. 12]

$$\text{remainder_pred} = \begin{cases} \max(\text{remainder}[x1], \text{remainder}[x2]) & (\text{remainder}[x0] \leq \min(\text{remainder}[x1], \text{remainder}[x2])) \\ \max(\text{remainder}[x1], \text{remainder}[x2]) & (\text{remainder}[x0] \geq \max(\text{remainder}[x1], \text{remainder}[x2])) \\ \text{remainder}[x1] + \text{remainder}[x2] - \text{remainder}[x0] & (\text{otherwise}) \end{cases} \quad (11)$$

[0899] Next, Rice parameter r to be used for Golomb-Rice coding of remainder [x] is derived as indicated below.

[MATH. 13]

$$r = \text{rice_par}[\text{remainder_pred}] \quad (12)$$

[0900] In this case, a Rice parameter table may be defined as indicated below, for example.

[MATH. 14]

$$\text{rice_par} = \{0, 0, 0, 1, 1, 1, 2, 2, 2\} \quad (13)$$

[0901] It is to be noted that the number of coefficients to be used for prediction of remainder [x] may be changed, or positions of coefficients to be used for prediction of remainder [x] may be changed. The method for calculating remainder_pred may be changed. In addition, the values of the Rice parameter table may be changed. A calculation method different from the methods for calculating the Rice parameter described above may be used. The third example in Aspect 4 may be used in combination with the first example in Aspect 4, the second example in Aspect 4, or any other aspect.

[0902] Parameter remainder_pred may be derived based not on remainder of coefficients neighboring a current coefficient to be encoded but on the absolute values of coefficients neighboring a current coefficient to be encoded as in the first example in Aspect 4.

[0903] Specifically, remainder_pred which is a prediction value of x may be derived based on three absolute values x0, x1, and x2 of three coefficients neighboring a current coefficient to be encoded, according to a method similar to the method explained with reference to FIG. 110. Next, a Rice parameter may be derived using remainder_pred according to the above-described method.

[0904] In the above case, remainder_pred is large because remainder_pred is based not on remainder of each of the coefficients and on the absolute values of the coefficients. Accordingly, the Rice parameter table may be designed in such a manner that the Rice parameter corresponding to remainder_pred based on the absolute values of the neighboring coefficients is smaller than the Rice parameter for remainder_pred based on remainder of each of the neighboring coefficients.

[0905] Alternatively, remainder_pred may be converted to remainder_pred' as described below, for example. Next, the same table as the table for deriving the Rice parameter for remainder_pred based on remainder may be used as the table for deriving the Rice parameter for remainder_pred'.

[MATH. 15]

$$\text{remainder_pred}' = (\text{remainder_pred} - 4)/2 \quad (14)$$

(if remainder_pred ≤ 3, remainder_pred' = 0)

[Effects of the Third Example in Aspect 4 Relating to Coefficient Encoding]

[0906] Likewise the prediction of the coefficient information flags in the first example in Aspect 4, also in the prediction of the remainder in the third example in Aspect 4, there is a possibility that prediction accuracy becomes high both in the case where a coefficient is on an edge and in the case where a coefficient is not on an edge.

[0907] Furthermore, for example, the Rice parameter table is designed in such a manner that the code length of remainder encoded by Golomb-Rice coding using the Rice parameter becomes shortest. The Rise parameter is obtainable according to remainder_pred which is substituted in the Rice parameter table. For this reason, actually, there is a possibility that the code length of remainder encoded by Golomb-Rice coding using the Rice parameter becomes shortest.

[Aspect 5 Relating to Coefficient Encoding]

[0908] FIG. 114 is a flow chart indicating a basic coefficient encoding method according to Aspect 5. Specifically, FIG. 114 indicates a coefficient encoding method in an area in which prediction residuals have been obtained in either intra encoding or inter encoding.

[0909] The example in FIG. 114 is different from the example in FIG. 97 in that sixteen coefficients in a subblock is encoded using a third encoding scheme in the case where orthogonal transform is applied (S147b). In other words, in the example in FIG. 114, encoder 100 encodes the sixteen coefficients in the subblock using not a first encoding scheme indicated in FIG. 98 but the third encoding scheme to be described later, when orthogonal transform is applied to a block (S147b).

[0910] FIG. 115 is a flow chart indicating details of the third encoding scheme indicated in FIG. 114. In the third encoding scheme, a plurality of coefficients in a subblock are encoded. At that time, a first loop process (S151 to S156) is performed for each coefficient information flag of a corresponding one of coefficients in the subblock, and a second loop process (S161 to S165) is performed for each coefficient in the subblock.

[0911] In the first loop process (S151 to S156), one or more coefficient information flags are sequentially encoded as in the first encoding scheme indicated in FIG. 98. The one or more coefficient information flags may include sig_flag, gt1_flag, parity_flag, and gt3_flag. The one or more coefficient information flags are sequentially encoded by CABAC coding within a range in which CCB does not exceed thres, and CCB is counted up one-by-one each time of encoding. No coefficient information flag is encoded after CCB exceeds thres,

[0912] In the second loop process (S161 to S165), remainder that is a remainder value which is not represented by a coefficient information flag (that is a remainder value for reconstructing the value of the coefficient using the coefficient information flag) is encoded by Golomb-Rice coding for the coefficient for which the coefficient information flag has been encoded. A coefficient for which no coefficient information flags has been encoded is to be subjected to a conversion process to be described later (S164a), and then encoded by Golomb-Rice coding (S164b). It is to be noted that remainder may be encoded using another encoding scheme instead of using Golomb-Rice coding.

[0913] In other words, in the second loop process (S161 to S165), encoder 100 determines whether a coefficient information flag corresponding to a processing-target coefficient has been encoded (S162), as in the first encoding scheme indicated in FIG. 98. When a coefficient information flag has been encoded (Yes in S162), encoder 100 encodes remainder by Golomb-Rice coding (S163).

[0914] When no coefficient information flag has been encoded (No in S162) in the third encoding scheme indi-

cated in FIG. 115, encoder 100 applies a to-be-described conversion process what is called a zeropos process or a poszero process to a value of a coefficient (S164a). Next, encoder 100 encodes the value of the coefficient by Golomb-Rice coding (S164b).

[0915] The total number of loop processes is two here, but it is to be noted that the total number of loop processes may be different from two.

[0916] Next, an example of the above-described conversion process (S164a) is described with reference to FIG. 116 and FIG. 117.

[0917] FIG. 116 is a conceptual diagram indicating a plurality of neighboring coefficients according to Aspect 5. Coefficient scanning is performed diagonally starting with the 4×4 subblock located at the lower-right position illustrated in FIG. 116. Specifically, in a coordinate system in which the coordinates of the upper-left position, the coordinates of the upper-right position, the coordinates of the lower-left position, and the coordinates of the lower-right position are represented as (0, 0), (3, 0), (0, 3), and (3, 3), respectively, the coefficient scanning is performed in, for example, the following order: (3, 3), (3, 2), (2, 3), (3, 1), (2, 2), . . . (0, 0).

[0918] In addition, a denotes a current coefficient to be encoded (that is, a processing target). In addition, a0, a1, a2, a3, and a4 denote five encoded (that is, processed) coefficients neighboring coefficient a. Hereinafter, each of a, a0, a1, a2, a3, and a4 may indicate a value of a coefficient, and more specifically, each of them may indicate an absolute value of the coefficient.

[0919] Encoder 100 obtains sum which is a total sum of the absolute values of five coefficients a0, a1, a2, a3, and a4. Next, encoder 100 determines a value what is called poszero according to sum. For example, encoder 100 may determine poszero corresponding to sum, with reference to a look-up table.

[0920] Next, encoder 100 compares value a with poszero. Next, encoder 100 converts value a into value b that is determined as indicated below.

[MATH. 16]

$$b = \begin{cases} \text{poszero} & (a = 0) \\ a - 1 & (0 < a \leq \text{poszero}) \\ a & (a > \text{poszero}) \end{cases} \quad (15)$$

[0921] Lastly, encoder 100 encodes value b obtained through conversion on value a, by Golomb-Rice coding.

[0922] FIG. 117 is a conceptual diagram indicating a conversion process according to Aspect 5. In the example in FIG. 117, poszero is 4. When value a is 0, value b is poszero (that is, 4). When value a is greater than 0 and less than or equal to poszero (that is, less than or equal to 4), value b is obtained according to a-1. When value a is greater than poszero (that is, greater than 4), value b equals to value a. In this way, value b is uniquely defined by value a.

[0923] In addition, as indicated in FIG. 117, value a is to be uniquely defined when value b is defined. Accordingly, decoder 200 may decode value b by Golomb-Rice decoding, and convert decoded value b into value a. For example, decoder 200 may derive value a by converting value b into value a that is determined as indicated below.

[MATH. 17]

$$a = \begin{cases} 0 & (b = \text{poszero}) \\ b + 1 & (0 \leq b < \text{poszero}) \\ b & (b > \text{poszero}) \end{cases} \quad (16)$$

[0924] It is to be noted that a plurality of neighboring coefficients for determining poszero are not limited to the above five coefficients. Coefficients located at positions different from the positions of the above five coefficients may be used as neighboring coefficients for determining poszero. In addition, in order to determine poszero, six or more neighboring coefficients may be used, four or less neighboring coefficients may be used, or only one neighboring coefficient may be used. For example, poszero may be determined according to a total sum of absolute values of two coefficients a3 and a4.

[0925] In addition, poszero may be determined according to sum and information other than sum.

[0926] In addition, in the example of FIG. 116, positions of neighboring coefficients for determining poszero are determined according to the scan order from the lower-right position. However, when the scan order is opposite, the positions of neighboring coefficients for determining poszero may be reversed.

[0927] In general, a rate of appearance of a coefficient having a value of 0 (a zero coefficient) decreases in an area in which orthogonal transform is applied when CCB exceeds a limit than when CCB does not exceed the limit. For this reason, when the value of the coefficient is greater than 0 and less than or equal to poszero, a process of subtracting 1 from the value is performed. In this way, the amount of codes generated by following Golomb-rice coding is reduced. As a result, the coding efficiency increases.

[0928] Next, one example of Golomb-rice coding (S164b) of a coefficient to which a conversion process (S164a) has been applied is described with reference to FIG. 116. For example, in Golomb-rice coding (S164b), five neighboring coefficients a0, a1, a2, a3, and a4 are used, as in the conversion process (S164a).

[0929] Specifically, encoder 100 obtains sum which is a total sum of the absolute values of five coefficients a0, a1, a2, a3, and a4. Next, encoder 100 determines Rice parameter r according to sum. For example, encoder 100 may determine Rice parameter r corresponding to sum, with reference to a look-up table.

[0930] Encoder 100 encodes value b by Golomb-rice coding described with reference to FIG. 113. At that time, encoder 100 uses Rice parameter r determined according to sum, for Golomb-rice coding.

[0931] It is to be noted that a plurality of neighboring coefficients for determining Rice parameter r are not limited to the above five coefficients. Coefficients located at positions different from the positions of the above five coefficients may be used as neighboring coefficients for determining Rice parameter r. In addition, in order to determine Rice parameter r, six or more neighboring coefficients may be used, four or less neighboring coefficients may be used, or only one neighboring coefficient may be used. For example, Rice parameter r may be determined according to a total sum of absolute values of two coefficients a3 and a4.

[0932] In addition, the plurality of neighboring coefficients for determining Rice parameter r may be the same as or different from the plurality of neighboring coefficients for determining poszero.

[0933] In addition, Rice parameter r may be determined according to sum and information other than sum.

[0934] In addition, in the example of FIG. 116, positions of neighboring coefficients for determining Rice parameter r are determined according to the scan order from the lower-right position. However, when the scan order is opposite, the positions of neighboring coefficients for determining Rice parameter r may be reversed.

[0935] In general, there is a tendency that the values of coefficients become greater from high-frequency side to low-frequency side in an area in which orthogonal transform has been applied. Accordingly, determining Rice parameter r for encoding a coefficient by Golomb-rice coding according to the values of neighboring coefficients produces a possibility that the encoding efficiency increases.

[a First Example in Aspect 5 Relating to Coefficient Encoding]

[0936] FIG. 118 is a flow chart indicating a coefficient encoding method according to a first example in Aspect 5. Specifically, FIG. 118 indicates a coefficient encoding method in an area in which prediction residuals have been obtained in either intra encoding or inter encoding.

[0937] The example in FIG. 118 is different from the example in FIG. 114 in that sixteen coefficients in a subblock is encoded using a third encoding method in the case where no orthogonal transform is applied (S127b). In other words, in the example of FIG. 118, encoder 100 encodes the sixteen coefficients in the subblock using not the second encoding scheme indicated in FIG. 99 but the third encoding scheme described above also in the case where no orthogonal transform is applied to the block (S127b).

[0938] It is to be noted that the syntax for encoding the coefficient information flag in the first loop process in FIG. 115 may be different between when orthogonal transform is applied and when no orthogonal transform is applied. For example, part or all of the coefficient information flags may be different between one or more coefficient information flags in the case where orthogonal transform is applied and one or more coefficient information flags in the case where no orthogonal transform is applied.

[Effects of the First Example in Aspect 5 Relating to Coefficient Encoding]

[0939] According to the example in FIG. 118, there is a possibility that the syntaxes for use in encoding of the sixteen coefficients in the subblock are commonalized after CCB exceeds thres regardless of application or non-application of orthogonal transform even when the syntaxes for use in encoding of the coefficient information flags are different depending on whether or not orthogonal transform is applied. In this way, there is a possibility that part of the circuits is commonalized, and thus the circuit scales are reduced.

[0940] It is to be noted that part or all of the encoding processes (S127b and S147b) of the sixteen coefficients in the subblock may be different between when orthogonal transform is applied and when no orthogonal transform is applied. For example, the number of neighboring coeffi-

cients to be used to calculate poszero and Rice parameter r in the encoding processes (S127b and S147b) of the sixteen coefficients in the subblock may be different between when orthogonal transform is applied and when no orthogonal transform is applied.

[0941] In addition, the number of neighboring coefficients to be used to calculate poszero may be different from the number of neighboring coefficients to be used to calculate Rice parameter r .

[0942] In addition, the positions of neighboring coefficients to be used to calculate poszero and Rice parameter r may be determined according to a scan order in the case where orthogonal transform is applied and a scan order in the case where no orthogonal transform is applied. For example, when the scan orders are different, the positions of neighboring coefficients to be used to calculate poszero and the positions of neighboring coefficients to be used to calculate Rice parameter r may be different from each other.

[a Second Example in Aspect 5 Relating to Coefficient Encoding]

[0943] FIG. 119 is a flow chart indicating a coefficient encoding method according to a second example in Aspect 5. Specifically, FIG. 119 indicates a coefficient encoding method in an area in which prediction residuals have been obtained in either intra encoding or inter encoding.

[0944] The example in FIG. 119 is different from the example in FIG. 114 in that sixteen coefficients in a subblock is encoded using a first encoding method in the case where no orthogonal transform is applied (S127a). Specifically, in the example in FIG. 119, encoder 100 encodes the sixteen coefficients in the subblock using either the second encoding scheme or the third encoding scheme but the first encoding scheme indicated in FIG. 98 when no orthogonal transform is applied to a block (S127a).

[0945] For example, in the third encoding scheme, the conversion process (S164a) indicated in FIG. 115 is performed. On the other hand, in the first encoding scheme, the conversion process (S164a) indicated in FIG. 115 is performed. In this point, the first encoding scheme is different from the third encoding scheme. Except for the point, the first encoding scheme may be the same as the third encoding scheme.

[0946] It is to be noted that the syntax for encoding the coefficient information flag in the first loop process in each of FIG. 98 and FIG. 115 may be different between when orthogonal transform is applied and when no orthogonal transform is applied. For example, part or all of the coefficient information flags may be different between one or more coefficient information flags in the case where orthogonal transform is applied and one or more coefficient information flags in the case where no orthogonal transform is applied.

[Effects of the Second Example in Aspect 5 Relating to Coefficient Encoding]

[0947] In the example in FIG. 119, the syntax for encoding the coefficient information flag may be different between when orthogonal transform is applied and when no orthogonal transform is applied. On the other hand, there is a possibility that the syntax for encoding the sixteen coefficients in the subblock is commonalized after CCB exceeds thres except for a poszero process. In this way, there is a

possibility that part of the circuits is commonalized between when orthogonal transform is applied and when no orthogonal transform is applied, and thus the circuit scales are reduced.

[0948] In general, a poszero process is effective for increasing the coding efficiency when orthogonal transform is applied. On the other hand, a poszero process is not effective so much for increasing the coding efficiency when no orthogonal transform is applied. For this reason, a poszero process does not always need to be performed when no orthogonal transform is applied. In this way, there is a possibility that the processing efficiency and the coding efficiency increase.

[0949] For example, the number of neighboring coefficients to be used to calculate poszero and Rice parameter r in the third encoding may be different between when orthogonal transform is applied and when no orthogonal transform is applied. In addition, the number of neighboring coefficients to be used to calculate poszero may be different from the number of neighboring coefficients to be used to calculate Rice parameter r .

[0950] In addition, the positions of neighboring coefficients to be used to calculate poszero and Rice parameter r may be determined according to a scan order in the case where orthogonal transform is applied and a scan order in the case where no orthogonal transform is applied. For example, when the scan orders are different, the positions of neighboring coefficients to be used to calculate poszero and the positions of neighboring coefficients to be used to calculate Rice parameter r may be different from each other.

[Aspect 6 Relating to Coefficient Encoding]

[0951] FIG. 120 is a flow chart indicating a basic coefficient encoding method according to Aspect 6. Specifically, FIG. 120 indicates a coefficient encoding method in an area in which prediction residuals have been obtained in either intra encoding or inter encoding.

[0952] The example in FIG. 120 is different from the example in FIG. 101 in that sixteen coefficients in a subblock is encoded using a fourth encoding method in the case where orthogonal transform is not applied (S127c). In other words, in the example in FIG. 120, encoder 100 encodes the sixteen coefficients in the subblock using not the second encoding scheme indicated in FIG. 99 but the fourth encoding scheme to be described later, when orthogonal transform is not applied to a block (S127c). Except for the point, the example in FIG. 120 may be the same as the example in FIG. 101.

[0953] FIG. 121 is a flow chart indicating a basic fourth encoding scheme indicated in FIG. 120. In the fourth encoding scheme, a plurality of coefficients in a subblock are encoded. For example, sixteen coefficients in a 4x4 subblock are encoded. In doing so, the first loop process (S201) is performed for each coefficient in the subblock, and then the second loop process (S202) is performed for each coefficient in the subblock. Finally, the third loop process (S203) is performed for each coefficient in the subblock.

[0954] FIG. 122 is a flow chart indicating details of the first loop process (S201) indicated in FIG. 121.

[0955] In the first loop process (S201), steps (S211 to S216) are performed for each coefficient in the subblock. In doing so, one or more coefficient information flags each indicating a different one of one or more attributes of the coefficient are sequentially encoded. The one or more coefficient information flags may include sig_flag, sign_flag, and

gtl_flag. Each coefficient information flag of sig_flag, sign_flag, and gtl_flag is the same as the coefficient information flag described in Aspect 1 of the coefficient encoding.

[0956] For example, in the first loop process (S211 to S216), firstly, encoder 100 encodes sig_flag by CABAC coding or bypass coding (S212). Next, encoder 100 encodes sign_flag by CABAC coding or bypass coding (S213). Next, encoder 100 applies level mapping, which is a conversion process, to the value of the coefficient (S214). Next, encoder 100 encodes gtl_flag by CABAC coding or bypass coding (S215).

[0957] It is to be noted that the coefficient information flags indicated here are examples, and other coefficient information flags may be encoded. For example, part of the coefficient information flags does not always need to be encoded. A coefficient information flag indicated here may be replaced with a coefficient information flag or a parameter having another meaning.

[0958] In the encoding of sig_flag (S212), when CCB is less than or equal to thres, encoder 100 encodes sig_flag by CABAC coding. Encoder 100 then adds 1 to CCB. When CCB is greater than thres, encoder 100 encodes sig_flag by bypass coding. Encoder 100 also performs the same process when another coefficient information flag is encoded by CABAC coding or bypass coding.

[0959] The following describes details of the level mapping described above (S214) with reference to FIG. 123 and FIG. 124.

[0960] FIG. 123 is a conceptual diagram indicating a plurality of neighboring coefficients according to Aspect 6. In this example, coefficient scanning is performed diagonally starting with the 4×4 subblock located at the upper-left position illustrated in FIG. 123. Specifically, in a coordinate system in which the coordinates of the upper-left position, the coordinates of the upper-right position, the coordinates of the lower-left position, and the coordinates of the lower-right position are represented as (0, 0), (3, 0), (0, 3), and (3, 3), respectively, the coefficient scanning is performed in the following order: (0, 0), (1, 0), (0, 1), (2, 0), (1, 1), . . . (3, 3).

[0961] In this example, x denotes a current coefficient to be encoded (that is, to be processed). In addition, x1 and x2 denote two encoded (that is, processed) coefficients neighboring coefficient x. Here, coefficient x1 is the left-neighboring coefficient of coefficient x, and coefficient x2 is the upper-neighboring coefficient of coefficient x.

[0962] Hereinafter, each of x, x0, and x1 may indicate the value of a coefficient, and more specifically, denote a coefficient absolute value which is the absolute value of a coefficient. For example, x, x0, and x1 may be replaced with abs(x), abs(x1), and abs(x2). Here, abs() is a function that returns the absolute value of an argument.

[0963] Firstly, encoder 100 determines the prediction value pred of x using pred=max(x1, x2). Encoder 100 then compares x and pred, and converts the value of x as described below.

[MATH. 18]

$$x = \begin{cases} x+1 & (0 < x < \text{pred}) \\ 1 & (x = \text{pred}) \\ x & (x > \text{pred} \parallel x = 0) \end{cases} \quad (17)$$

[0964] When the level mapping is applied to x, in the following processes, encoder 100 encodes information on x

converted by the level mapping (for example, a coefficient information flag indicating the attribution of x converted by the level mapping, etc.).

[0965] FIG. 124 is a conceptual diagram indicating level mapping according to Aspect 6. In the example in FIG. 124, pred is 6. When x is less than pred (that is, x is less than 6), 1 is added to x. When x is equal to pred (that is, x is equal to 6), x is replaced with 1. When x is greater than pred (that is, x is greater than 6) or x is equal to 0, x remains as it is.

[0966] When x is equal to pred, the above level mapping converts x to a smaller value.

[0967] As shown in FIG. 124, a pre-level-mapping coefficient value is uniquely determined from a post-level-mapping coefficient value. Decoder 200 may decode the post-level-mapping coefficient value, and convert the decoded post-level-mapping coefficient value to the pre-level-mapping coefficient value. For example, decoder 200 may derive a pre-level-mapping coefficient value by converting a post-level-mapping coefficient value to the pre-level-mapping coefficient value as follows.

[MATH. 19]

$$x = \begin{cases} x-1 & (1 < x \leq \text{pred}) \\ \text{pred} & (x = 1) \\ x & (x > \text{pred} \parallel x = 0) \end{cases} \quad (18)$$

[0968] Alternatively, another determination method may be used as the method of determining pred, and another conversion method may be used as the method of converting x.

[0969] FIG. 125 is a flow chart indicating details of the second loop process (S202) indicated in FIG. 121.

[0970] In the second loop process (S202), steps (S221 to S227) are performed for each coefficient in the subblock. In doing so, one or more coefficient information flags each indicating a different one of one or more attributes of the coefficient are sequentially encoded. The one or more coefficient information flags may include gt2_flag, gt3_flag, gt4_flag, gt5_flag, and gt6_flag.

[0971] gtX_flag (X=2, 3, 4, 5, 6) in the present aspect is related to a quantization parameter (QP) determined for a current block to be encoded. The quantization parameter (QP) is a parameter for determining a quantization width for use in coefficient quantization. A possible maximum value of the current coefficient is determined from the quantization parameter (QP). Here, the maximum value is denoted by T.

[0972] Specifically, in the present aspect, gt2_flag is a flag indicating whether the coefficient absolute value falls within the closed interval [2, T/32+1]. gt3_flag is a flag indicating whether the coefficient absolute value falls within the closed interval [T/32+2, T/16+T/32+1]. gt4_flag is a flag indicating whether the coefficient absolute value falls within the closed interval [T/16+T/32+2, T/8+T/16+T/32+1].

[0973] Moreover, gt5_flag is a flag indicating whether the coefficient absolute value falls within the closed interval [T/8+T/16+T/32+2, T/4+T/8+T/16+T/32+1]. gt6_flag is a flag indicating whether the coefficient absolute value falls within the closed interval [T/4+T/8+T/16+T/32+2, T/4+T/4+T/8+T/16+T/32+1].

[0974] For example, in the second loop process (S221 to S227), firstly, encoder 100 encodes gt2_flag by CABAC coding or bypass coding (S222).

[0975] Next, encoder 100 encodes $gt3_flag$ by CABAC coding or bypass coding (S223). Next, encoder 100 encodes $gt4_flag$ by CABAC coding or bypass coding (S224). Next, encoder 100 encodes $gt5_flag$ by CABAC coding or bypass coding (S225). Next, encoder 100 encodes $gt6_flag$ by CABAC coding or bypass coding (S226).

[0976] In the encoding of $gt2_flag$ (S222), when CCB is less than or equal to $thres$, encoder 100 encodes $gt2_flag$ by CABAC coding. Encoder 100 then adds 1 to CCB. When CCB is greater than $thres$, encoder 100 encodes $gt2_flag$ by bypass coding. Encoder 100 also performs the same process when another coefficient information flag is encoded by CABAC coding or bypass coding.

[0977] The above description of gtX_flag ($X=2, 3, 4, 5, 6$) is one example. gtX_flag ($X=2, 3, 4, 5, 6$) may have a meaning different from the above description. For example, gtX_flag ($X=2, 3, 4, 5, 6$) may indicate whether to fall within another interval which has a width different from that of the above description.

[0978] Moreover, the number of coefficient information flags denoted by gtX_flag ($X=2, 3, 4, 5, 6$) may be more or less than 5.

[0979] FIG. 126 is a flow chart indicating details of the third loop process (S203) indicated in FIG. 121. In the third loop process (S203), steps (S231 to S233) are performed for each coefficient in the subblock. In doing so, the position of the value of the coefficient in the interval determined by gtX_flag ($X=2, 3, 4, 5, 6$) is encoded (S232).

[0980] For example, in the second loop process shown in FIG. 125, the interval including the value of the coefficient in the subblock is encoded using gtX_flag ($X=2, 3, 4, 5, 6$). In encoding of the position (S232), the position of the value of the coefficient in the interval is encoded. For example, when C in the interval $[A, B]$ ($A \leq C \leq B$) is the coefficient absolute value, encoder 100 encodes the position of the value of the coefficient into $C-A$ bits of 1 and 1 bit of 0 by truncated-unary coding. Another method may be used as the method of encoding the position of the value in the interval.

[0981] In the example of FIG. 121, every time each coefficient information flag is encoded, it is checked whether the CCB is greater than $thres$. Accordingly, the processing load of hardware increases.

[Example in Aspect 6 Relating to Coefficient Encoding]

[0982] FIG. 127 is a flow chart indicating a fourth encoding scheme according to an example in Aspect 6. In the example of FIG. 127, the first loop process (S201a) is performed for each coefficient in the subblock, and then the second loop process (S202a) is performed for each coefficient in the subblock. Finally, the third loop process (S203) is performed for each coefficient in the subblock. In the example of FIG. 127, details of the first loop process (S201a) and details of the second loop process (S202a) are different from those of the example of FIG. 121.

[0983] FIG. 128 is a flow chart indicating details of the first loop process (S201a) indicated in FIG. 127.

[0984] In the first loop process (S201a), steps (S301 to S311) are performed for each coefficient in the subblock. In doing so, one or more coefficient information flags each indicating a different one of one or more attributes of the coefficient are sequentially encoded, as in the example in FIG. 122. The one or more coefficient information flags may include sig_flag , $sign_flag$, and $gt1_flag$. Each coefficient

information flag of sig_flag , $sign_flag$, and $gt1_flag$ is the same as the coefficient information flag described in Aspect 1 of the coefficient encoding.

[0985] Specifically, in the example of FIG. 128, firstly, encoder 100 checks whether $thres-CCB$ is greater than or equal to 3 (S302).

[0986] When $thres-CCB$ is greater than or equal to 3 (Yes in S302), encoder 100 encodes sig_flag by CABAC coding (S303). Next, encoder 100 encodes $sign_flag$ by CABAC coding (S304). Moreover, encoder 100 applies the level mapping to the coefficient (S305). Next, encoder 100 encodes $gt1_flag$ by CABAC coding (S306). Encoder 100 may add 1 to the CCB every time each coefficient information flag is encoded by CABAC coding.

[0987] When $thres-CCB$ is less than 3 (No in S302), encoder 100 encodes sig_flag by bypass coding (S307). Next, encoder 100 encodes $sign_flag$ by bypass coding (S308). Moreover, encoder 100 applies the level mapping to the coefficient (S309). Next, encoder 100 encodes $gt1_flag$ by bypass coding (S310).

[0988] The second loop process (S202a) also may be performed in the same manner as the first loop process (S201a). For example, in the second loop process (S202a), it may be checked whether $thres-CCB$ is greater than or equal to 5 in accordance with the number of coefficient information flags to be encoded.

[0989] It is to be noted that the above-mentioned numerical value (3 or 5) used to check the CCB is one example. Another numerical value is possible as long as the numerical value is more than or equal to the number of coefficient information flags to be encoded in each loop process.

[Effects of Example in Aspect 6 Relating to Coefficient Encoding]

[0990] In the example of FIG. 127, the number of times that the CCB is checked is reduced in comparison with the example of FIG. 121. Accordingly, the processing load of hardware decreases.

[0991] Moreover, the method of counting the CCB in this example is the same as that of the first encoding scheme in the case where orthogonal transform is applied in the example of FIG. 101. Accordingly, the processing flow may be shared between the case where orthogonal transform is applied and the case where orthogonal transform is not applied, and thus it is possible to reduce the circuit size.

[Aspect 7 Relating to Coefficient Encoding]

[0992] FIG. 129 is a flow chart indicating a basic coefficient encoding method according to Aspect 7. Specifically, FIG. 129 indicates a coefficient encoding method in an area in which prediction residuals have been obtained in either intra encoding or inter encoding.

[0993] The example in FIG. 129 is different from the example in FIG. 114 in that sixteen coefficients in a subblock is encoded using a fifth encoding method in the case where orthogonal transform is not applied (S127d). In other words, in the example in FIG. 129, encoder 100 encodes the sixteen coefficients in the subblock using not the second encoding scheme indicated in FIG. 99 but the fifth encoding scheme to be described later, when orthogonal transform is not applied to a block (S127d). Except for the point, the example in FIG. 129 may be the same as the example in FIG. 114.

[0994] FIG. 130 is a flow chart indicating a basic fifth encoding scheme indicated in FIG. 129. In the fifth encoding scheme, a plurality of coefficients in a subblock are encoded. For example, sixteen coefficients in a 4x4 subblock are encoded. In doing so, the first loop process (S401) is performed for each coefficient in the subblock. Next, the second loop process (S402) is performed for each coefficient in the subblock. Next, the third loop process (S403) is performed for each coefficient in the subblock. Next, the fourth loop process (S404) is performed for each coefficient in the subblock.

[0995] FIG. 131 is a flow chart indicating details of the first loop process (S401) indicated in FIG. 130.

[0996] In the first loop process (S401), steps (S411 to S415) are performed for each coefficient in the subblock. In doing so, one or more coefficient information flags each indicating a different one of one or more attributes of the coefficient are sequentially encoded. The one or more coefficient information flags may include sig_flag and sign_flag. Each coefficient information flag of sig_flag and sign_flag is the same as the coefficient information flag described in Aspect 1 of the coefficient encoding.

[0997] For example, in the first loop process (S411 to S415), firstly, encoder 100 checks whether thres-CCB is greater than or equal to 2 (S412). When thres-CCB is greater than or equal to 2 (Yes in S412), encoder 100 encodes sig_flag by CABAC coding (S413). Next, encoder 100 encodes sign_flag by CABAC coding (S414).

[0998] When thres-CCB is not greater than or equal to 2 (No in S412), encoder 100 does not encode sig_flag and sign_flag of the coefficient, and the first loop process is terminated.

[0999] It is to be noted that the coefficient information flags indicated here are examples, and other coefficient information flags may be encoded. For example, part of the coefficient information flags does not always need to be encoded. A coefficient information flag indicated here may be replaced with a coefficient information flag or a parameter having another meaning.

[1000] FIG. 132 is a flow chart indicating details of the second loop process (S402) indicated in FIG. 130.

[1001] In the second loop process (S402), steps (S421 to S430) are performed for each coefficient in the subblock. In doing so, one or more coefficient information flags each indicating a different one of one or more attributes of the coefficient are sequentially encoded. The one or more coefficient information flags may include gt1_flag, gt2_flag, gt3_flag, gt4_flag, gt5_flag, and gt6_flag. gtX_flag (X=1, 2, 3, 4, 5, 6) is the same as gtX_flag (X=1, 2, 3, 4, 5, 6) described in Aspect 6 of the coefficient encoding.

[1002] For example, in the second loop process (S421 to S430), firstly, encoder 100 checks whether thres-CCB is greater than or equal to 6 (S422).

[1003] When thres-CCB is greater than or equal to 6 (Yes in S422), encoder 100 applies the level mapping described in Aspect 6 to the coefficient (S423). Next, encoder 100 encodes gt1_flag by CABAC coding (S424).

[1004] Moreover, encoder 100 encodes gt2_flag by CABAC coding (S425). Moreover, encoder 100 encodes gt3_flag by CABAC coding (S426). Moreover, encoder 100 encodes gt4_flag by CABAC coding (S427). Moreover, encoder 100 encodes gt5_flag by CABAC coding (S428). Moreover, encoder 100 encodes gt6_flag by CABAC coding (S429).

[1005] When thres-CCB is not greater than or equal to 6 (No in S422), encoder 100 does not encode gtX_flag (X=1, 2, 3, 4, 5, 6) of the coefficient, and the second loop process is terminated.

[1006] FIG. 133 is a flow chart indicating details of the third loop process (S403) indicated in FIG. 130. In the third loop process (S403), steps (S441 to S444) are performed for each coefficient in the subblock.

[1007] In doing so, firstly, encoder 100 checks whether gtX_flag (X≥2) of the current coefficient has been encoded in the second loop process (S402) (S442). When gtX_flag (X≥2) has been encoded (Yes in S442), encoder 100 encodes the position of the value of the coefficient in the interval determined by gtX_flag (X≥2) (S443). When gtX_flag (X≥2) has not been encoded (No in S442), encoder 100 does not encode the position of the value of the coefficient.

[1008] The encoding of the position of the value of the coefficient (S443) in the example of FIG. 133 is the same as the encoding of the position of the value of the coefficient (S232) in the example of FIG. 126.

[1009] FIG. 134 is a flow chart indicating details of the fourth loop process (S404) indicated in FIG. 130. In the fourth loop process (S404), steps (S451 to S455) are performed for each coefficient in the subblock.

[1010] In doing so, firstly, encoder 100 checks whether gt1_flag of the current coefficient has been encoded in the second loop process (S402) (S452). When gt1_flag has been encoded (Yes in S452), encoder 100 does not encode the value of the coefficient in the fourth loop process.

[1011] When gt1_flag has not been encoded (No in S452), encoder 100 applies the level mapping to the coefficient (S453). Next, encoder 100 encodes the value of the coefficient by Golomb-Rice coding (S454). For example, the Golomb-Rice coding described with reference to FIG. 113 may be used.

[1012] It is to be noted that the process performed in the third loop process when gtX_flag (X≥2) has been encoded (S443) and the process performed in the fourth loop process when gt1_flag has not been encoded (S453 and S454) are performed exclusively. The third loop process and the fourth loop process may be integrated into one loop process.

[Example in Aspect 7 Relating to Coefficient Encoding]

[1013] The fifth encoding scheme according to an example in Aspect 7 is different in the fourth loop process from the basic fifth encoding scheme according to Aspect 7.

[1014] In the fourth loop process in the basic fifth encoding scheme according to Aspect 7 (FIG. 134), after the CCB exceeds thres, the level mapping is applied to the coefficient and the value of the coefficient is encoded by Golomb-Rice coding. On the other hand, in this example, after the CCB exceeds thres, the level mapping process is omitted.

[1015] FIG. 135 is a flow chart indicating a fourth loop process according to an example in Aspect 7. In the example of FIG. 135, steps (S451 to S455) are performed for each coefficient in the subblock, as in the example in FIG. 134.

[1016] In doing so, firstly, encoder 100 checks whether gt1_flag of the current coefficient has been encoded in the second loop process (S402) (S452). When gt1_flag has been encoded (Yes in S452), encoder 100 does not encode the value of the coefficient in the fourth loop process.

[1017] When gt1_flag has not been encoded (No in S452), the level mapping is not applied to the coefficient, and encoder 100 encodes the value of the non-converted coef-

ficient by Golomb-Rice coding (S454a). For example, the Golomb-Rice coding described with reference to FIG. 113 may be used.

[1018] In other words, in the fifth encoding scheme according to an example in Aspect 7, when the coefficient information flag of the current coefficient (for example, `gt1_flag`) is encoded, the level mapping is applied according to the second loop process described in FIG. 132. On the other hands, when the coefficient information flag of the current coefficient is not encoded, the level mapping is not applied according to the fourth loop process described in FIG. 135.

[1019] For example, when orthogonal transform is applied, the third encoding scheme is used, and when orthogonal transform is not applied, the fifth encoding scheme including the above-mentioned fourth loop process is used. In the third encoding scheme, when the coefficient information flag is not encoded, the poszero process is applied and the value of the coefficient is encoded by Golomb-Rice coding. In the fifth encoding scheme, when the coefficient information flag is not encoded, the value of the non-converted coefficient is encoded by Golomb-Rice coding.

[1020] In other words, except for the poszero process, a process after the CCB exceeds `thres` may be shared between the case where orthogonal transform is not applied and the case where orthogonal transform is applied.

[1021] It is to be noted that in both the third encoding scheme and the fifth encoding scheme, `sign_flag` may be encoded by bypass coding after the Golomb-Rice coding of the value of the coefficient.

[Effects of Example in Aspect 7 Relating to Coefficient Encoding]

[1022] The level mapping process is omitted, and thus except for the poszero process, the coefficient encoding syntax after the CCB exceeds `thres` may be shared regardless of whether to apply orthogonal transform. Accordingly, it is possible to reduce the circuit scale.

[1023] Moreover, the level mapping process is performed using neighboring coefficients, and thus the level mapping process is influenced by the neighboring coefficients. Accordingly, when using CABAC coding which is also influenced by the neighboring coefficients, the level mapping process may be effective to reduce the code amount. On the other hand, when not using CABAC coding, the level mapping process may not be so effective to reduce the code amount.

[1024] Accordingly, it is possible to reduce the increase in processing delay and reduce the increase in code amount by controlling the level mapping process according to whether the coefficient information flag is encoded by context-adaptive coding.

[Variation of Coefficient Encoding]

[1025] Any of the aspects and examples regarding coefficient encoding described above may be combined. Alternatively, any of the aspects, examples, and combinations regarding coefficient encoding described above may be applied to luminance blocks or may be applied to chrominance blocks. At the time, `thres` used for luminance blocks and `thres` used for chrominance blocks may be different.

[1026] Alternatively, any of the aspects, examples, and combinations regarding coefficient encoding described above may be used for a block to which no orthogonal transform is applied and Block-based Delta Pulse Code Modulation (BDPCM) is applied. In the block to which BDPCM is applied, a residual signal vertically or horizontally neighboring a residual signal in the block is subtracted from the residual signal, which reduces the information amount.

[1027] Furthermore, any of the aspects, examples, and combinations regarding coefficient encoding described above may be used for a chrominance block to which BDPCM is applied.

[1028] Alternatively, any of the aspects, examples, and combinations regarding coefficient encoding described above may be used for a chrominance block to which Intra Sub-Partitions (ISP) are applied. In ISP, an intra block is split vertically or horizontally, and intra prediction for each current subblock is performed using a pixel value of a subblock neighboring the current subblock.

[1029] Furthermore, any of the aspects, examples, and combinations regarding coefficient encoding described above may be used for a chrominance block to which ISP is applied.

[1030] Furthermore, any of the aspects, examples, and combinations regarding coefficient encoding described above may be used when Chroma Joint Coding is used as an encoding mode for a chrominance block. Here, when Chroma Joint Coding is an encoding method for deriving the value of `Cr` from the value of `Cb`.

[1031] In addition, the value of `thres` in the case where orthogonal transform is applied may be double the value of `thres` in the case where no orthogonal transform is applied. Alternatively, the value of `thres` in the case where no orthogonal transform is applied may be double the value of `thres` in the case where orthogonal transform is applied.

[1032] Alternatively, only in the case where Chroma Joint Coding is used, the value of `thres` for CCB in the case where orthogonal transform is applied may be double the value of `thres` for CCB in the case where no orthogonal transform is applied. Alternatively, only in the case where Chroma Joint Coding is used, the value of `thres` of a CCB in the case where no orthogonal transform is applied may be double the value of `thres` of a CCB in the case where orthogonal transform is applied.

[1033] In addition, in the aspects and examples regarding coefficient encoding described above, the scanning order of coefficients in a block to which no orthogonal transform is applied may be the same as the scanning order of coefficients in a block to which orthogonal transform is applied.

[1034] Furthermore, although some examples of syntaxes are indicated in Aspect 3 and the examples in Aspect 3, syntaxes to be applied are not limited to the examples. For example, in aspects different from Aspect 3 and examples in the aspects, syntaxes different from the syntaxes indicated in Aspect 3 and the examples may be used. Various syntaxes for encoding sixteen coefficients can be applied.

[1035] Although encoding processing flow is indicated in each of the aspects and examples regarding coefficient encoding, it is to be noted that a decoding processing flow is basically the same as the encoding processing flow except for the difference between transmission and reception of a bitstream. For example, decoder 200 may perform inverse

orthogonal transform and decoding corresponding to orthogonal transform and encoding that are performed by encoder **100**.

[1036] In addition, each of the flow charts relating to the aspects and examples regarding coefficient encoding are examples. New conditions or processes may be added to each of the flow charts, or some of the conditions or processes in each of the flow charts may be deleted or modified.

[1037] In addition, here, a coefficient is a value included in an image of a block or a subblock. Specifically, coefficients included in an image may be obtained from pixel values of the image through orthogonal transform. Specifically, coefficients included in an image may be obtained from pixel values of the image through orthogonal transform. In other words, the coefficients included in the image may be the pixel values themselves of the image. Alternatively, each pixel value may be a pixel value in an original image or may be a value of a prediction residual. In addition, the coefficient may be quantized.

[Representative Examples of Configuration and Processing]

[1038] Representative examples of configurations of encoder **100** and decoder **200** and processes performed thereby as described above are indicated below.

[1039] FIG. **136** is a flow chart indicating an operation performed by encoder **100**. For example, encoder **100** includes circuitry and memory coupled to the circuitry. The circuitry and memory included in encoder **100** may correspond to processor a1 and memory a2 illustrated in FIG. **8**. The circuitry of encoder **100** performs an operation indicated in FIG. **136**. More specifically, the circuitry of encoder **100** encodes a block in an image in operation (S511).

[1040] For example, the circuitry of encoder **100** may limit the total number of processes of context adaptive coding, and encode a block in an image. When the total number of processes is within a limited range of the total number of processes, the coefficient information flag may be encoded. Here, the coefficient information flag indicates an attribute of a coefficient included in a block.

[1041] Moreover, in a case where orthogonal transform is not applied to the block, when the coefficient information flag is encoded, a conversion process may be performed on a value of the coefficient, and a converted value of the coefficient may be encoded using the coefficient information flag to be encoded by the context adaptive coding. Here, the conversion process is a process for converting the value of the coefficient using a value determined using neighboring coefficients located around the coefficient in the block.

[1042] Moreover, in a case where orthogonal transform is not applied to the block, when the coefficient information flag is not encoded, the conversion process need not be performed, and the value of the coefficient may be encoded by Golomb-Rice coding.

[1043] With this, when the coefficient information flag is encoded by context-adaptive coding, the conversion process is performed, and when the coefficient information flag is not encoded by context-adaptive coding, the conversion process is not performed. This conversion process is performed using neighboring coefficients, and thus the conversion process is influenced by the neighboring coefficients. Accordingly, when using context-adaptive coding which is also influenced by the neighboring coefficients, this conversion process may be effective to reduce the code amount.

[1044] In other words, when the coefficient information flag is encoded by context-adaptive coding, the conversion process that may be effective to reduce the code amount is performed, and when the coefficient information flag is not encoded by context-adaptive coding, the conversion process is not performed. Accordingly, it is possible to reduce the increase in processing delay and reduce the increase in code amount.

[1045] Moreover, for example, the coefficient information flag may be a flag indicating whether or not the value of the coefficient is greater than 1.

[1046] With this, it is possible to appropriately determine whether the coefficient information flag indicating whether or not the value of the coefficient is greater than 1 is encoded by context-adaptive coding, according to the limitation of the total number of processes of context-adaptive coding. Accordingly, it is possible to reduce the increase in processing delay and reduce the increase in code amount.

[1047] Moreover, for example, in a case where the orthogonal transform is applied to the block, when the coefficient information flag is encoded, the conversion process need not be performed, and the value of the coefficient may be encoded using the coefficient information flag to be encoded by the context-adaptive coding. Moreover, in a case where the orthogonal transform is applied to the block, when the coefficient information flag is not encoded, the conversion process need not be performed, and the value of the coefficient may be encoded by Golomb-Rice coding.

[1048] With this, the difference decreases between the encoding scheme used for a block to which orthogonal transform is applied and the encoding scheme used for a block to which orthogonal transform is not applied, and thus it is possible to reduce the circuit scale.

[1049] Moreover, for example, the conversion process may include: when the value of the coefficient is equal to the value determined using the neighboring coefficients, converting the value of the coefficient to 1; and when the value of the coefficient is greater than 0 and less than the value determined using the neighboring coefficients, converting the value of the coefficient by adding 1 to the value of the coefficient. With this, the value of the coefficient is appropriately converted according to the neighboring coefficients, and thus it is possible to reduce the increase in code amount.

[1050] It is to be noted that the above-described operation that is performed by the circuitry of encoder **100** may be performed by entropy encoder **110** of encoder **100**.

[1051] FIG. **137** is a flow chart indicating an operation performed by decoder **200**. For example, decoder **200** includes circuitry and memory coupled to the circuitry. The circuitry and memory included in decoder **200** may correspond to processor b1 and memory b2 illustrated in FIG. **68**. The circuitry of decoder **200** performs an operation indicated in FIG. **137**. More specifically, the circuitry of decoder **200** decodes a block in an image in operation (S521).

[1052] For example, the circuitry of decoder **200** may limit the total number of processes of context adaptive decoding, and decode a block in an image. When the total number of processes is within a limited range of the total number of processes, the coefficient information flag may be decoded. Here, the coefficient information flag indicates an attribute of a coefficient included in a block.

[1053] Moreover, in a case where inverse orthogonal transform is not applied to the block, when the coefficient information flag is decoded, a value of the coefficient may be

decoded using the coefficient information flag to be decoded by the context adaptive decoding. Next, a conversion process may be performed on the value of the coefficient to derive a converted value of the coefficient. Here, the conversion process is a process for converting the value of the coefficient using a value determined using neighboring coefficients located around the coefficient in the block.

[1054] Moreover, in a case where inverse orthogonal transform is not applied to the block, when the coefficient information flag is not decoded, the value of the coefficient may be decoded by Golomb-Rice decoding, to derive the value of the coefficient without performing the conversion process.

[1055] With this, when the coefficient information flag is decoded by context-adaptive decoding, the conversion process is performed, and when the coefficient information flag is not decoded by context-adaptive decoding, the conversion process is not performed. This conversion process is performed using neighboring coefficients, and thus the conversion process is influenced by the neighboring coefficients. Accordingly, when using context-adaptive decoding which is also influenced by the neighboring coefficients, this conversion process may be effective to reduce the code amount.

[1056] In other words, when the coefficient information flag is decoded by context-adaptive decoding, the conversion process that may be effective to reduce the code amount is performed, and when the coefficient information flag is not decoded by context-adaptive decoding, the conversion process is not performed. Accordingly, it is possible to reduce the increase in processing delay and reduce the increase in code amount.

[1057] Moreover, for example, the coefficient information flag may be a flag indicating whether or not the value of the coefficient is greater than 1. With this, it is possible to appropriately determine whether the coefficient information flag indicating whether or not the value of the coefficient is greater than 1 is decoded by context-adaptive decoding, according to the limitation of the total number of processes of context-adaptive decoding. Accordingly, it is possible to reduce the increase in processing delay and reduce the increase in code amount.

[1058] Moreover, for example, in a case where inverse orthogonal transform is applied to the block, when the coefficient information flag is decoded, the value of the coefficient may be decoded using the coefficient information flag to be decoded by the context adaptive decoding. The value of the coefficient may be derived without performing the conversion process. Moreover, in a case where inverse orthogonal transform is applied to the block, when the coefficient information flag is not decoded, the value of the coefficient may be decoded by Golomb-Rice decoding, to derive the value of the coefficient without performing the conversion process.

[1059] With this, the difference decreases between the decoding scheme used for a block to which inverse orthogonal transform is applied and the decoding scheme used for a block to which inverse orthogonal transform is not applied, and thus it is possible to reduce the circuit scale.

[1060] Moreover, for example, the conversion process may include: when the value of the coefficient is equal to 1, converting the value of the coefficient to the value determined using the neighboring coefficients; and when the value of the coefficient is greater than 0 and less than or equal to the value determined using the neighboring coef-

ficients, converting the value of the coefficient by subtracting 1 from the value of the coefficient. With this, the value of the coefficient is appropriately converted according to the neighboring coefficients, and thus it is possible to reduce the increase in code amount.

[1061] It is to be noted that the above-described operation that is performed by the circuitry of decoder 200 may be performed by entropy decoder 202 of decoder 200.

OTHER EXAMPLES

[1062] Encoder 100 and decoder 200 in each of the above-described examples may be used as an image encoder and an image decoder, respectively, or may be used as a video encoder and a video decoder, respectively.

[1063] In addition, encoder 100 and decoder 200 may perform only parts of the above-described operations, and other devices may perform the other parts of the operations. In addition, encoder 100 and decoder 200 may include only constituent elements among the plurality of constituent elements described above, and other devices may include the other constituent elements.

[1064] In addition, at least a part of each example described above may be used as an encoding method or a decoding method, or may be used as another method.

[1065] In addition, each constituent element may be configured with dedicated hardware, or may be implemented by executing a software program suitable for the constituent element. Each constituent element may be implemented by a program executor such as a CPU or a processor reading and executing a software program recorded on a recording medium such as a hard disc or a semiconductor memory.

[1066] More specifically, each of encoder 100 and decoder 200 may include processing circuitry and storage which is electrically coupled to the processing circuitry and is accessible from the processing circuitry. For example, the processing circuitry corresponds to processor a1 or b1, and the storage corresponds to memory a2 or b2.

[1067] The processing circuitry includes at least one of the dedicated hardware and the program executor, and executes processing using the storage. In addition, the storage stores a software program which is executed by the program executor when the processing circuitry includes the program executor.

[1068] Here, the software which implements either encoder 100, decoder 200, or the like described above is a program as indicated below.

[1069] For example, this program may cause a computer to execute an encoding method including limiting a total number of processes of context adaptive coding, and encoding a block in an image. In the encoding of the block, when the total number of processes is within a limited range of the total number of processes, a coefficient information flag is encoded. The coefficient information flag indicates attribution of a coefficient included in the block. In the encoding of the block, in a case where orthogonal transformation is not applied to the block, when the coefficient information flag is encoded, a conversion process is performed on a value of the coefficient, and a converted value of the coefficient is encoded using the coefficient information flag to be encoded by the context adaptive coding. The conversion process is a process of converting the value of the coefficient using a value determined using neighboring coefficients located around the coefficient in the block. When the coefficient

information flag is not encoded, the conversion process is not performed, and the value of the coefficient is encoded by Golomb-Rice coding.

[1070] In addition, for example, this program may cause the computer to execute a decoding method including limiting a total number of processes of context adaptive decoding, and decoding a block in an image. In the decoding of the block, when the total number of processes is within a limited range of the total number of processes, a coefficient information flag is decoded. The coefficient information flag indicates attribution of a coefficient included in the block. In the decoding of the block, in a case where inverse orthogonal transformation is not applied to the block, when the coefficient information flag is decoded, a value of the coefficient is decoded using the coefficient information flag to be decoded by the context adaptive decoding, and a conversion process is performed on the value of the coefficient to derive a converted value of the coefficient. The conversion process is a process of converting the value of the coefficient using a value determined using neighboring coefficients located around the coefficient in the block. When the coefficient information flag is not decoded, the value of the coefficient is decoded by Golomb-Rice decoding, to derive the value of the coefficient without performing the conversion process.

[1071] In addition, each constituent element may be circuitry as described above. Circuits may compose circuitry as a whole, or may be separate circuits. Alternatively, each constituent element may be implemented as a general processor, or may be implemented as an exclusive processor.

[1072] In addition, the process that is executed by a particular constituent element may be executed by another constituent element. In addition, the processing execution order may be modified, or a plurality of processes may be executed in parallel. In addition, an encoder and decoder may include encoder 100 and decoder 200.

[1073] In addition, the ordinal numbers such as “first” and “second” used for explanation may be changed appropriately. A new ordinal number may be attached to a constituent element, or an ordinal number attached to a constituent element may be removed.

[1074] Although aspects of encoder 100 and decoder 200 have been described based on a plurality of examples, aspects of encoder 100 and decoder 200 are not limited to these examples. The scope of the aspects of encoder 100 and decoder 200 may encompass embodiments obtainable by adding, to any of these embodiments, various kinds of modifications that a person skilled in the art would arrive at without deviating from the scope of the present disclosure and embodiments configurable by combining constituent elements in different embodiments.

[1075] One or more of the aspects disclosed herein may be performed in combination with at least part of the other aspects in the present disclosure. In addition, part of the processes indicated in any of the flow charts, part of the configuration of any of the devices, part of syntaxes, etc. according to one or more of the aspects disclosed herein may be performed in combination with the other aspects.

IMPLEMENTATIONS AND APPLICATIONS

[1076] As described in each of the above embodiments, each functional or operational block may typically be realized as an MPU (micro processing unit) and memory, for example. Moreover, processes performed by each of the

functional blocks may be realized as a program execution unit, such as a processor which reads and executes software (a program) recorded on a medium such as ROM. The software may be distributed. The software may be recorded on a variety of media such as semiconductor memory. Note that each functional block can also be realized as hardware (dedicated circuit).

[1077] The processing described in each of the embodiments may be realized via integrated processing using a single apparatus (system), and, alternatively, may be realized via decentralized processing using a plurality of apparatuses. Moreover, the processor that executes the above-described program may be a single processor or a plurality of processors. In other words, integrated processing may be performed, and, alternatively, decentralized processing may be performed.

[1078] Embodiments of the present disclosure are not limited to the above exemplary embodiments; various modifications may be made to the exemplary embodiments, the results of which are also included within the scope of the embodiments of the present disclosure.

[1079] Next, application examples of the moving picture encoding method (image encoding method) and the moving picture decoding method (image decoding method) described in each of the above embodiments will be described, as well as various systems that implement the application examples. Such a system may be characterized as including an image encoder that employs the image encoding method, an image decoder that employs the image decoding method, or an image encoder-decoder that includes both the image encoder and the image decoder. Other configurations of such a system may be modified on a case-by-case basis.

USAGE EXAMPLES

[1080] FIG. 138 illustrates an overall configuration of content providing system ex100 suitable for implementing a content distribution service. The area in which the communication service is provided is divided into cells of desired sizes, and base stations ex106, ex107, ex108, ex109, and ex110, which are fixed wireless stations in the illustrated example, are located in respective cells.

[1081] In content providing system ex100, devices including computer ex111, gaming device ex112, camera ex113, home appliance ex114, and smartphone ex115 are connected to internet ex101 via internet service provider ex102 or communications network ex104 and base stations ex106 through ex110. Content providing system ex100 may combine and connect any of the above devices. In various implementations, the devices may be directly or indirectly connected together via a telephone network or near field communication, rather than via base stations ex106 through ex110. Further, streaming server ex103 may be connected to devices including computer ex111, gaming device ex112, camera ex113, home appliance ex114, and smartphone ex115 via, for example, internet ex101. Streaming server ex103 may also be connected to, for example, a terminal in a hotspot in airplane ex117 via satellite ex116.

[1082] Note that instead of base stations ex106 through ex110, wireless access points or hotspots may be used. Streaming server ex103 may be connected to communications network ex104 directly instead of via internet ex101 or internet service provider ex102, and may be connected to airplane ex117 directly instead of via satellite ex116.

[1083] Camera ex113 is a device capable of capturing still images and video, such as a digital camera. Smartphone ex115 is a smartphone device, cellular phone, or personal handyphone system (PHS) phone that can operate under the mobile communications system standards of the 2G, 3G, 3.9G, and 4G systems, as well as the next-generation 5G system.

[1084] Home appliance ex114 is, for example, a refrigerator or a device included in a home fuel cell cogeneration system.

[1085] In content providing system ex100, a terminal including an image and/or video capturing function is capable of, for example, live streaming by connecting to streaming server ex103 via, for example, base station ex106. When live streaming, a terminal (e.g., computer ex111, gaming device ex112, camera ex113, home appliance ex114, smartphone ex115, or a terminal in airplane ex117) may perform the encoding processing described in the above embodiments on still-image or video content captured by a user via the terminal, may multiplex video data obtained via the encoding and audio data obtained by encoding audio corresponding to the video, and may transmit the obtained data to streaming server ex103. In other words, the terminal functions as the image encoder according to one aspect of the present disclosure.

[1086] Streaming server ex103 streams transmitted content data to clients that request the stream. Client examples include computer ex111, gaming device ex112, camera ex113, home appliance ex114, smartphone ex115, and terminals inside airplane ex117, which are capable of decoding the above-described encoded data. Devices that receive the streamed data decode and reproduce the received data. In other words, the devices may each function as the image decoder, according to one aspect of the present disclosure.

[Decentralized Processing]

[1087] Streaming server ex103 may be realized as a plurality of servers or computers between which tasks such as the processing, recording, and streaming of data are divided. For example, streaming server ex103 may be realized as a content delivery network (CDN) that streams content via a network connecting multiple edge servers located throughout the world. In a CDN, an edge server physically near a client is dynamically assigned to the client. Content is cached and streamed to the edge server to reduce load times. In the event of, for example, some type of error or change in connectivity due, for example, to a spike in traffic, it is possible to stream data stably at high speeds, since it is possible to avoid affected parts of the network by, for example, dividing the processing between a plurality of edge servers, or switching the streaming duties to a different edge server and continuing streaming.

[1088] Decentralization is not limited to just the division of processing for streaming; the encoding of the captured data may be divided between and performed by the terminals, on the server side, or both. In one example, in typical encoding, the processing is performed in two loops. The first loop is for detecting how complicated the image is on a frame-by-frame or scene-by-scene basis, or detecting the encoding load. The second loop is for processing that maintains image quality and improves encoding efficiency. For example, it is possible to reduce the processing load of the terminals and improve the quality and encoding efficiency of the content by having the terminals perform the

first loop of the encoding and having the server side that received the content perform the second loop of the encoding. In such a case, upon receipt of a decoding request, it is possible for the encoded data resulting from the first loop performed by one terminal to be received and reproduced on another terminal in approximately real time. This makes it possible to realize smooth, real-time streaming.

[1089] In another example, camera ex113 or the like extracts a feature amount from an image, compresses data related to the feature amount as metadata, and transmits the compressed metadata to a server. For example, the server determines the significance of an object based on the feature amount and changes the quantization accuracy accordingly to perform compression suitable for the meaning (or content significance) of the image. Feature amount data is particularly effective in improving the precision and efficiency of motion vector prediction during the second compression pass performed by the server. Moreover, encoding that has a relatively low processing load, such as variable length coding (VLC), may be handled by the terminal, and encoding that has a relatively high processing load, such as context-adaptive binary arithmetic coding (CABAC), may be handled by the server.

[1090] In yet another example, there are instances in which a plurality of videos of approximately the same scene are captured by a plurality of terminals in, for example, a stadium, shopping mall, or factory. In such a case, for example, the encoding may be decentralized by dividing processing tasks between the plurality of terminals that captured the videos and, if necessary, other terminals that did not capture the videos, and the server, on a per-unit basis. The units may be, for example, groups of pictures (GOP), pictures, or tiles resulting from dividing a picture. This makes it possible to reduce load times and achieve streaming that is closer to real time.

[1091] Since the videos are of approximately the same scene, management and/or instructions may be carried out by the server so that the videos captured by the terminals can be cross-referenced. Moreover, the server may receive encoded data from the terminals, change the reference relationship between items of data, or correct or replace pictures themselves, and then perform the encoding. This makes it possible to generate a stream with increased quality and efficiency for the individual items of data.

[1092] Furthermore, the server may stream video data after performing transcoding to convert the encoding format of the video data. For example, the server may convert the encoding format from MPEG to VP (e.g., VP9), and may convert H.264 to H.265.

[1093] In this way, encoding can be performed by a terminal or one or more servers. Accordingly, although the device that performs the encoding is referred to as a “server” or “terminal” in the following description, some or all of the processes performed by the server may be performed by the terminal, and likewise some or all of the processes performed by the terminal may be performed by the server. This also applies to decoding processes.

[3D, Multi-Angle]

[1094] There has been an increase in usage of images or videos combined from images or videos of different scenes concurrently captured, or of the same scene captured from different angles, by a plurality of terminals such as camera ex113 and/or smartphone ex115. Videos captured by the

terminals are combined based on, for example, the separately obtained relative positional relationship between the terminals, or regions in a video having matching feature points.

[1095] In addition to the encoding of two-dimensional moving pictures, the server may encode a still image based on scene analysis of a moving picture, either automatically or at a point in time specified by the user, and transmit the encoded still image to a reception terminal. Furthermore, when the server can obtain the relative positional relationship between the video capturing terminals, in addition to two-dimensional moving pictures, the server can generate three-dimensional geometry of a scene based on video of the same scene captured from different angles. The server may separately encode three-dimensional data generated from, for example, a point cloud and, based on a result of recognizing or tracking a person or object using three-dimensional data, may select or reconstruct and generate a video to be transmitted to a reception terminal, from videos captured by a plurality of terminals.

[1096] This allows the user to enjoy a scene by freely selecting videos corresponding to the video capturing terminals, and allows the user to enjoy the content obtained by extracting a video at a selected viewpoint from three-dimensional data reconstructed from a plurality of images or videos. Furthermore, as with video, sound may be recorded from relatively different angles, and the server may multiplex audio from a specific angle or space with the corresponding video, and transmit the multiplexed video and audio.

[1097] In recent years, content that is a composite of the real world and a virtual world, such as virtual reality (VR) and augmented reality (AR) content, has also become popular. In the case of VR images, the server may create images from the viewpoints of both the left and right eyes, and perform encoding that tolerates reference between the two viewpoint images, such as multi-view coding (MVC), and, alternatively, may encode the images as separate streams without referencing. When the images are decoded as separate streams, the streams may be synchronized when reproduced, so as to recreate a virtual three-dimensional space in accordance with the viewpoint of the user.

[1098] In the case of AR images, the server superimposes virtual object information existing in a virtual space onto camera information representing a real-world space, based on a three-dimensional position or movement from the perspective of the user. The decoder may obtain or store virtual object information and three-dimensional data, generate two-dimensional images based on movement from the perspective of the user, and then generate superimposed data by seamlessly connecting the images. Alternatively, the decoder may transmit, to the server, motion from the perspective of the user in addition to a request for virtual object information. The server may generate superimposed data based on three-dimensional data stored in the server, in accordance with the received motion, and encode and stream the generated superimposed data to the decoder. Note that superimposed data includes, in addition to RGB values, a value indicating transparency, and the server sets the a value for sections other than the object generated from three-dimensional data to, for example, 0, and may perform the encoding while those sections are transparent. Alternatively, the server may set the background to a determined

RGB value, such as a chroma key, and generate data in which areas other than the object are set as the background.

[1099] Decoding of similarly streamed data may be performed by the client (i.e., the terminals), on the server side, or divided therebetween. In one example, one terminal may transmit a reception request to a server, the requested content may be received and decoded by another terminal, and a decoded signal may be transmitted to a device having a display. It is possible to reproduce high image quality data by decentralizing processing and appropriately selecting content regardless of the processing ability of the communications terminal itself. In yet another example, while a TV, for example, is receiving image data that is large in size, a region of a picture, such as a tile obtained by dividing the picture, may be decoded and displayed on a personal terminal or terminals of a viewer or viewers of the TV. This makes it possible for the viewers to share a big-picture view as well as for each viewer to check his or her assigned area, or inspect a region in further detail up close.

[1100] In situations in which a plurality of wireless connections are possible over near, mid, and far distances, indoors or outdoors, it may be possible to seamlessly receive content using a streaming system standard such as MPEG Dynamic Adaptive Streaming over HTTP (MPEG-DASH). The user may switch between data in real time while freely selecting a decoder or display apparatus including the user's terminal, displays arranged indoors or outdoors, etc. Moreover, using, for example, information on the position of the user, decoding can be performed while switching which terminal handles decoding and which terminal handles the displaying of content. This makes it possible to map and display information, while the user is on the move in route to a destination, on the wall of a nearby building in which a device capable of displaying content is embedded, or on part of the ground. Moreover, it is also possible to switch the bit rate of the received data based on the accessibility to the encoded data on a network, such as when encoded data is cached on a server quickly accessible from the reception terminal, or when encoded data is copied to an edge server in a content delivery service.

[Web Page Optimization]

[1101] FIG. 139 illustrates an example of a display screen of a web page on computer ex111, for example. FIG. 140 illustrates an example of a display screen of a web page on smartphone ex115, for example. As illustrated in FIG. 139 and FIG. 140, a web page may include a plurality of image links that are links to image content, and the appearance of the web page differs depending on the device used to view the web page. When a plurality of image links are viewable on the screen, until the user explicitly selects an image link, or until the image link is in the approximate center of the screen or the entire image link fits in the screen, the display apparatus (decoder) may display, as the image links, still images included in the content or I pictures; may display video such as an animated gif using a plurality of still images or I pictures; or may receive only the base layer, and decode and display the video.

[1102] When an image link is selected by the user, the display apparatus performs decoding while giving the highest priority to the base layer. Note that if there is information in the Hyper Text Markup Language (HTML) code of the web page indicating that the content is scalable, the display apparatus may decode up to the enhancement layer. Further,

in order to guarantee real-time reproduction, before a selection is made or when the bandwidth is severely limited, the display apparatus can reduce delay between the point in time at which the leading picture is decoded and the point in time at which the decoded picture is displayed (that is, the delay between the start of the decoding of the content to the displaying of the content) by decoding and displaying only forward reference pictures (I picture, P picture, forward reference B picture). Still further, the display apparatus may purposely ignore the reference relationship between pictures, and coarsely decode all B and P pictures as forward reference pictures, and then perform normal decoding as the number of pictures received over time increases.

[Autonomous Driving]

[1103] When transmitting and receiving still image or video data such as two- or three-dimensional map information for autonomous driving or assisted driving of an automobile, the reception terminal may receive, in addition to image data belonging to one or more layers, information on, for example, the weather or road construction as metadata, and associate the metadata with the image data upon decoding. Note that metadata may be assigned per layer and, alternatively, may simply be multiplexed with the image data.

[1104] In such a case, since the automobile, drone, airplane, etc., containing the reception terminal is mobile, the reception terminal may seamlessly receive and perform decoding while switching between base stations among base stations ex106 through ex110 by transmitting information indicating the position of the reception terminal. Moreover, in accordance with the selection made by the user, the situation of the user, and/or the bandwidth of the connection, the reception terminal may dynamically select to what extent the metadata is received, or to what extent the map information, for example, is updated.

[1105] In content providing system ex100, the client may receive, decode, and reproduce, in real time, encoded information transmitted by the user.

[Streaming of Individual Content]

[1106] In content providing system ex100, in addition to high image quality, long content distributed by a video distribution entity, unicast or multicast streaming of low image quality, and short content from an individual are also possible. Such content from individuals is likely to further increase in popularity. The server may first perform editing processing on the content before the encoding processing, in order to refine the individual content. This may be achieved using the following configuration, for example.

[1107] In real time while capturing video or image content, or after the content has been captured and accumulated, the server performs recognition processing based on the raw data or encoded data, such as capture error processing, scene search processing, meaning analysis, and/or object detection processing. Then, based on the result of the recognition processing, the server—either when prompted or automatically—edits the content, examples of which include: correction such as focus and/or motion blur correction; removing low-priority scenes such as scenes that are low in brightness compared to other pictures, or out of focus; object edge adjustment; and color tone adjustment. The server encodes the edited data based on the result of the editing. It

is known that excessively long videos tend to receive fewer views. Accordingly, in order to keep the content within a specific length that scales with the length of the original video, the server may, in addition to the low-priority scenes described above, automatically clip out scenes with low movement, based on an image processing result. Alternatively, the server may generate and encode a video digest based on a result of an analysis of the meaning of a scene.

[1108] There may be instances in which individual content may include content that infringes a copyright, moral right, portrait rights, etc. Such instance may lead to an unfavorable situation for the creator, such as when content is shared beyond the scope intended by the creator. Accordingly, before encoding, the server may, for example, edit images so as to blur faces of people in the periphery of the screen or blur the inside of a house, for example. Further, the server may be configured to recognize the faces of people other than a registered person in images to be encoded, and when such faces appear in an image, may apply a mosaic filter, for example, to the face of the person. Alternatively, as pre- or post-processing for encoding, the user may specify, for copyright reasons, a region of an image including a person or a region of the background to be processed. The server may process the specified region by, for example, replacing the region with a different image, or blurring the region. If the region includes a person, the person may be tracked in the moving picture, and the person's head region may be replaced with another image as the person moves.

[1109] Since there is a demand for real-time viewing of content produced by individuals, which tends to be small in data size, the decoder first receives the base layer as the highest priority, and performs decoding and reproduction, although this may differ depending on bandwidth. When the content is reproduced two or more times, such as when the decoder receives the enhancement layer during decoding and reproduction of the base layer, and loops the reproduction, the decoder may reproduce a high image quality video including the enhancement layer. If the stream is encoded using such scalable encoding, the video may be low quality when in an unselected state or at the start of the video, but it can offer an experience in which the image quality of the stream progressively increases in an intelligent manner. This is not limited to just scalable encoding; the same experience can be offered by configuring a single stream from a low quality stream reproduced for the first time and a second stream encoded using the first stream as a reference.

OTHER IMPLEMENTATION AND APPLICATION EXAMPLES

[1110] The encoding and decoding may be performed by LSI (large scale integration circuitry) ex500 (see FIG. 138), which is typically included in each terminal. LSI ex500 may be configured of a single chip or a plurality of chips. Software for encoding and decoding moving pictures may be integrated into some type of a medium (such as a CD-ROM, a flexible disk, or a hard disk) that is readable by, for example, computer ex111, and the encoding and decoding may be performed using the software. Furthermore, when smartphone ex115 is equipped with a camera, video data obtained by the camera may be transmitted. In this case, the video data is coded by LSI ex500 included in smartphone ex115.

[1111] Note that LSI ex500 may be configured to download and activate an application. In such a case, the terminal

first determines whether it is compatible with the scheme used to encode the content, or whether it is capable of executing a specific service. When the terminal is not compatible with the encoding scheme of the content, or when the terminal is not capable of executing a specific service, the terminal first downloads a codec or application software and then obtains and reproduces the content.

[1112] Aside from the example of content providing system ex100 that uses internet ex101, at least the moving picture encoder (image encoder) or the moving picture decoder (image decoder) described in the above embodiments may be implemented in a digital broadcasting system. The same encoding processing and decoding processing may be applied to transmit and receive broadcast radio waves superimposed with multiplexed audio and video data using, for example, a satellite, even though this is geared toward multicast, whereas unicast is easier with content providing system ex100.

[Hardware Configuration]

[1113] FIG. 141 illustrates further details of smartphone ex115 shown in FIG. 138. FIG. 142 illustrates a configuration example of smartphone ex115. Smartphone ex115 includes antenna ex450 for transmitting and receiving radio waves to and from base station ex110, camera ex465 capable of capturing video and still images, and display ex458 that displays decoded data, such as video captured by camera ex465 and video received by antenna ex450. Smartphone ex115 further includes user interface ex466 such as a touch panel, audio output unit ex457 such as a speaker for outputting speech or other audio, audio input unit ex456 such as a microphone for audio input, memory ex467 capable of storing decoded data such as captured video or still images, recorded audio, received video or still images, and mail, as well as decoded data, and slot ex464 which is an interface for Subscriber Identity Module (SIM) ex468 for authorizing access to a network and various data. Note that external memory may be used instead of memory ex467.

[1114] Main controller ex460, which comprehensively controls display ex458 and user interface ex466, power supply circuit ex461, user interface input controller ex462, video signal processor ex455, camera interface ex463, display controller ex459, modulator/demodulator ex452, multiplexer/demultiplexer ex453, audio signal processor ex454, slot ex464, and memory ex467 are connected via bus ex470.

[1115] When the user turns on the power button of power supply circuit ex461, smartphone ex115 is powered on into an operable state, and each component is supplied with power from a battery pack.

[1116] Smartphone ex115 performs processing for, for example, calling and data transmission, based on control performed by main controller ex460, which includes a CPU, ROM, and RAM. When making calls, an audio signal recorded by audio input unit ex456 is converted into a digital audio signal by audio signal processor ex454, to which spread spectrum processing is applied by modulator/demodulator ex452 and digital-analog conversion and frequency conversion processing are applied by transmitter/receiver ex451, and the resulting signal is transmitted via antenna ex450. The received data is amplified, frequency converted, and analog-digital converted, inverse spread spectrum processed by modulator/demodulator ex452, converted into an analog audio signal by audio signal processor ex454, and then output from audio output unit ex457. In data

transmission mode, text, still-image, or video data is transmitted by main controller ex460 via user interface input controller ex462 based on operation of user interface ex466 of the main body, for example. Similar transmission and reception processing is performed. In data transmission mode, when sending a video, still image, or video and audio, video signal processor ex455 compression encodes, by the moving picture encoding method described in the above embodiments, a video signal stored in memory ex467 or a video signal input from camera ex465, and transmits the encoded video data to multiplexer/demultiplexer ex453. Audio signal processor ex454 encodes an audio signal recorded by audio input unit ex456 while camera ex465 is capturing a video or still image, and transmits the encoded audio data to multiplexer/demultiplexer ex453. Multiplexer/demultiplexer ex453 multiplexes the encoded video data and encoded audio data using a determined scheme, modulates and converts the data using modulator/demodulator (modulator/demodulator circuit) ex452 and transmitter/receiver ex451, and transmits the result via antenna ex450.

[1117] When a video appended in an email or a chat, or a video linked from a web page, is received, for example, in order to decode the multiplexed data received via antenna ex450, multiplexer/demultiplexer ex453 demultiplexes the multiplexed data to divide the multiplexed data into a bitstream of video data and a bitstream of audio data, supplies the encoded video data to video signal processor ex455 via synchronous bus ex470, and supplies the encoded audio data to audio signal processor ex454 via synchronous bus ex470. Video signal processor ex455 decodes the video signal using a moving picture decoding method corresponding to the moving picture encoding method described in the above embodiments, and video or a still image included in the linked moving picture file is displayed on display ex458 via display controller ex459. Audio signal processor ex454 decodes the audio signal and outputs audio from audio output unit ex457. Since real-time streaming is becoming increasingly popular, there may be instances in which reproduction of the audio may be socially inappropriate, depending on the user's environment. Accordingly, as an initial value, a configuration in which only video data is reproduced, i.e., the audio signal is not reproduced, may be preferable; and audio may be synchronized and reproduced only when an input is received from the user clicking video data, for instance.

[1118] Although smartphone ex115 was used in the above example, three other implementations are conceivable: a transceiver terminal including both an encoder and a decoder; a transmitter terminal including only an encoder; and a receiver terminal including only a decoder. In the description of the digital broadcasting system, an example is given in which multiplexed data obtained as a result of video data being multiplexed with audio data is received or transmitted. The multiplexed data, however, may be video data multiplexed with data other than audio data, such as text data related to the video. Further, the video data itself rather than multiplexed data may be received or transmitted.

[1119] Although main controller ex460 including a CPU is described as controlling the encoding or decoding processes, various terminals often include Graphics Processing Units (GPUs). Accordingly, a configuration is acceptable in which a large area is processed at once by making use of the performance ability of the GPU via memory shared by the CPU and GPU, or memory including an address that is

managed so as to allow common usage by the CPU and GPU. This makes it possible to shorten encoding time, maintain the real-time nature of streaming, and reduce delay. In particular, processing relating to motion estimation, deblocking filtering, sample adaptive offset (SAO), and transformation/quantization can be effectively carried out by the GPU, instead of the CPU, in units of pictures, for example, all at once.

[1120] Although only some exemplary embodiments of the present disclosure have been described in detail above, those skilled in the art will readily appreciate that many modifications are possible in the exemplary embodiments without materially departing from the novel teachings and advantages of the present disclosure. Accordingly, all such modifications are intended to be included within the scope of the present disclosure.

INDUSTRIAL APPLICABILITY

[1121] The present disclosure is applicable to, for example, television receivers, digital video recorders, car navigation systems, mobile phones, digital cameras, digital video cameras, teleconferencing systems, electronic mirrors, etc.

What is claimed is:

1. An encoder comprising:
circuitry; and

memory coupled to the circuitry, wherein:

in residual coding of a current block to which an orthogonal transform is not applied, the circuitry:

when a plurality of coefficient information flags relating to a coefficient in the current block are encoded,

performs a level mapping process to transform the coefficient to a second coefficient by using a predicted value, wherein the predicted value is determined based on a plurality of neighboring coefficients of the coefficient within the current block, the plurality of neighboring coefficients comprising a first neighboring coefficient located to the left of the coefficient and a second neighboring coefficient located above the coefficient;

encodes a plurality of second coefficient information flags by Context-based Adaptive Binary Arithmetic Coding (CABAC), each of the plurality of second coefficient information flags relating to the second coefficient; and

encodes a remainder value of the coefficient with Golomb-Rice code; and

when the plurality of coefficient information flags are not encoded, the circuitry, in operation,
skips the level mapping process; and

encodes a value of the coefficient with the Golomb-Rice code.

2. A decoder comprising:
circuitry; and

memory coupled to the circuitry, wherein:

in residual decoding of a current block to which a reverse-orthogonal transform is not applied, the circuitry:

when a plurality of coefficient information flags relating to a coefficient in the current block are decoded,

decodes a plurality of second coefficient information flags by Context-based Adaptive Binary Arithmetic Coding (CABAC) decoding, each of the plurality of second coefficient information flags relating to the coefficient;

decodes a remainder value of the coefficient with Golomb-Rice decoding;

reconstructs a value of the coefficient by using the plurality of coefficient information flags, the plurality of second coefficient information flags, and the remainder value; and

performs a reverse-level mapping process to transform the reconstructed coefficient to a second coefficient by using a predicted value, wherein the predicted value is determined based on a plurality of neighboring coefficients of the coefficient within the current block, the plurality of neighboring coefficients comprising a first neighboring coefficient located to the left of the coefficient and a second neighboring coefficient located above the coefficient; and

when the plurality of coefficient information flags are not decoded, the circuitry, in operation,

decodes a value of the coefficient with the Golomb-Rice decoding; and

skips the reverse-level mapping process.

3. A non-transitory computer readable medium storing a bitstream, the bitstream including information according to which a decoder performs residual decoding, wherein

in residual decoding of a current block to which a reverse-orthogonal transform is not applied,

when a plurality of coefficient information flags relating to a coefficient in the current block are decoded,

a plurality of second coefficient information flags are decoded by Context-based Adaptive Binary Arithmetic Coding (CABAC) decoding, each of the plurality of second coefficient information flags relating to the coefficient;

a remainder value of the coefficient is decoded with Golomb-Rice decoding;

a value of the coefficient is reconstructed by using the plurality of coefficient information flags, the plurality of second coefficient information flags, and the remainder value; and

the reconstructed coefficient is transformed to a second coefficient by a reverse-level mapping process using a predicted value, wherein the predicted value is determined based on a plurality of neighboring coefficients of the coefficient within the current block, the plurality of neighboring coefficients comprising a first neighboring coefficient located to the left of the coefficient and a second neighboring coefficient located above the coefficient; and

when the plurality of coefficient information flags are not decoded,

a value of the coefficient is decoded with the Golomb-Rice decoding; and

the reverse-level mapping process is skipped.

* * * * *